TOSCA

Generated by Doxygen 1.13.2

1 TOSCA 1

1.0.1 *Welcome to the official repository of TOSCA* . . . 1

1.0.2 License . . . 1

1.0.3 User Guide . . . 1

1.0.4 Reporting Bugs . . . 2

1.0.5 How to Cite . . . 2

1.0.6 Recent Highlights/Additions . . . 2

1.0.7 Executables . . . 2

1.0.8 Future Implementations: . . . 2

2 Class Index 3

2.1 Class List . . . 3

3 File Index 7

3.1 File List . . . 7

4 Class Documentation 11

4.1 abl_ Struct Reference . . . 11

4.2 access_ Struct Reference . . . 18

4.2.1 Detailed Description . . . 18

4.3 Acell Struct Reference . . . 18

4.3.1 Detailed Description . . . 19

4.4 acquisition_ Struct Reference . . . 19

4.4.1 Detailed Description . . . 20

4.5 avgFields Struct Reference . . . 20

4.5.1 Detailed Description . . . 22

4.6 BinIndex Struct Reference . . . 22

4.7 boundingBox Struct Reference . . . 22

4.7.1 Detailed Description . . . 23

4.8 Cabot Struct Reference . . . 23

4.8.1 Detailed Description . . . 23

4.9 cellIds Struct Reference . . . 23

4.9.1 Detailed Description . . . 23

4.10 cellList Struct Reference . . . 24

4.10.1 Detailed Description . . . 24

4.11 cellNode Struct Reference . . . 24

4.12 clock_ Struct Reference . . . 24

4.13 Cmpnts Struct Reference . . . 25

4.13.1 Detailed Description . . . 25

4.14 constants_ Struct Reference . . . 26

4.15 Cpt2D Struct Reference . . . 26

4.15.1 Detailed Description . . . 26

4.16 data3LM Struct Reference . . . 26

4.16.1 Detailed Description . . . 27
4.17 dataABL Struct Reference . . . 28
4.17.1 Detailed Description . . . 29
4.18 Dcell Struct Reference . . . 29
4.18.1 Detailed Description . . . 29
4.19 domain_ Struct Reference . . . 30
4.19.1 Detailed Description . . . 30
4.20 elementBox Struct Reference . . . 31
4.21 flags_ Struct Reference . . . 31
4.21.1 Detailed Description . . . 32
4.22 std::hash< BinIndex > Struct Reference . . . 32
4.23 HoleObject Struct Reference . . . 32
4.24 ibm_ Struct Reference . . . 32
4.24.1 Detailed Description . . . 33
4.25 ibmFluidCell Struct Reference . . . 34
4.26 ibmMesh Struct Reference . . . 35
4.27 ibmObject Struct Reference . . . 35
4.28 ibmPitchMotion Struct Reference . . . 37
4.29 ibmRotation Struct Reference . . . 37
4.30 ibmSineMotion Struct Reference . . . 37
4.31 inflowData Struct Reference . . . 38
4.31.1 Detailed Description . . . 38
4.32 inletFunctions Struct Reference . . . 38
4.32.1 Detailed Description . . . 38
4.33 inletFunctionTypes Struct Reference . . . 39
4.33.1 Detailed Description . . . 41
4.34 io_ Struct Reference . . . 41
4.34.1 Detailed Description . . . 42
4.35 keBox Struct Reference . . . 42
4.35.1 Detailed Description . . . 43
4.36 keFields Struct Reference . . . 44
4.36.1 Detailed Description . . . 45
4.37 les_ Struct Reference . . . 45
4.37.1 Detailed Description . . . 46
4.38 level3LM Struct Reference . . . 46
4.38.1 Detailed Description . . . 47
4.39 list Struct Reference . . . 47
4.39.1 Detailed Description . . . 47
4.40 LogLawAPG Struct Reference . . . 47
4.41 mapInfo Struct Reference . . . 47
4.41.1 Detailed Description . . . 48
4.42 Matrix3 Struct Reference . . . 48

4.43 mesh_ Struct Reference . . . 48
4.43.1 Detailed Description . . . 50
4.44 ModeData Struct Reference . . . 50
4.45 node Struct Reference . . . 51
4.45.1 Detailed Description . . . 51
4.46 OctreeNode Struct Reference . . . 51
4.47 overset_ Struct Reference . . . 52
4.47.1 Detailed Description . . . 53
4.48 oversetMotion Struct Reference . . . 53
4.48.1 Detailed Description . . . 53
4.49 patchVectorField Struct Reference . . . 53
4.49.1 Detailed Description . . . 54
4.50 peqn_ Struct Reference . . . 54
4.50.1 Detailed Description . . . 55
4.51 perturbFields Struct Reference . . . 55
4.51.1 Detailed Description . . . 56
4.52 postProcess Struct Reference . . . 56
4.52.1 Detailed Description . . . 56
4.53 PowerLawAPG Struct Reference . . . 57
4.53.1 Detailed Description . . . 57
4.54 precursor_ Struct Reference . . . 57
4.54.1 Detailed Description . . . 57
4.55 probeRake Struct Reference . . . 58
4.55.1 Detailed Description . . . 58
4.56 rakes Struct Reference . . . 59
4.56.1 Detailed Description . . . 59
4.57 scalarBC Struct Reference . . . 59
4.57.1 Detailed Description . . . 60
4.58 searchBox Struct Reference . . . 60
4.59 sections Struct Reference . . . 60
4.59.1 Detailed Description . . . 61
4.60 Shumann Struct Reference . . . 61
4.60.1 Detailed Description . . . 62
4.61 simInfo_ Struct Reference . . . 62
4.62 skewSymmTensor Struct Reference . . . 63
4.63 surface Struct Reference . . . 63
4.64 symmTensor Struct Reference . . . 63
4.64.1 Detailed Description . . . 64
4.65 Tensor Struct Reference . . . 64
4.66 TensorInvariants Struct Reference . . . 64
4.67 teqn_ Struct Reference . . . 64
4.67.1 Detailed Description . . . 66

4.68 udSections Struct Reference . . . 66
4.69 ueqn_ Struct Reference . . . 67
4.69.1 Detailed Description . . . 69
4.70 uSections Struct Reference . . . 69
4.70.1 Detailed Description . . . 70
4.71 vectorBC Struct Reference . . . 70
4.71.1 Detailed Description . . . 71
4.72 wallModel Struct Reference . . . 71
4.72.1 Detailed Description . . . 71

5 File Documentation **73**
5.1 /zfs60/nobackup/stipa/software/TOSCA/src/abl.c File Reference . . . 73
5.1.1 Detailed Description . . . 74
5.2 /zfs60/nobackup/stipa/software/TOSCA/src/acquisition.c File Reference . . . 74
5.2.1 Detailed Description . . . 76
5.2.2 Function Documentation . . . 76
5.2.2.1 read3LMFields() . . . 76
5.3 /zfs60/nobackup/stipa/software/TOSCA/src/boundary.c File Reference . . . 76
5.3.1 Detailed Description . . . 77
5.4 /zfs60/nobackup/stipa/software/TOSCA/src/clock.c File Reference . . . 77
5.4.1 Detailed Description . . . 78
5.5 /zfs60/nobackup/stipa/software/TOSCA/src/ibm.c File Reference . . . 78
5.5.1 Detailed Description . . . 80
5.6 /zfs60/nobackup/stipa/software/TOSCA/src/ibmInput.c File Reference . . . 80
5.6.1 Detailed Description . . . 81
5.6.2 Function Documentation . . . 81
5.6.2.1 readIBMSurfaceFileAbaqusInp() . . . 81
5.7 /zfs60/nobackup/stipa/software/TOSCA/src/include/abl.h File Reference . . . 81
5.7.1 Detailed Description . . . 82
5.8 abl.h . . . 82
5.9 access.h . . . 86
5.10 /zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition.h File Reference . . . 86
5.10.1 Detailed Description . . . 89
5.10.2 Function Documentation . . . 89
5.10.2.1 read3LMFields() . . . 89
5.11 acquisition.h . . . 89
5.12 /zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/ablAcquisition.h File Reference . . . 91
5.12.1 Detailed Description . . . 92
5.13 ablAcquisition.h . . . 92
5.14 /zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/averageAcquisition.h File Reference 93
5.14.1 Detailed Description . . . 93
5.15 averageAcquisition.h . . . 93

5.16 /zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/keAcquisition.h File Reference . . . 94
5.16.1 Detailed Description . . . 94
5.17 keAcquisition.h . . . 94
5.18 /zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/perturbAcquisition.h File Reference . 96
5.18.1 Detailed Description . . . 96
5.19 perturbAcquisition.h . . . 96
5.20 /zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/probesAcquisition.h File Reference . 96
5.20.1 Detailed Description . . . 97
5.21 probesAcquisition.h . . . 97
5.22 /zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/sectionsAcquisition.h File Reference 97
5.22.1 Detailed Description . . . 98
5.23 sectionsAcquisition.h . . . 98
5.24 /zfs60/nobackup/stipa/software/TOSCA/src/include/base.h File Reference . . . 98
5.24.1 Detailed Description . . . 100
5.25 base.h . . . 100
5.26 /zfs60/nobackup/stipa/software/TOSCA/src/include/boundary.h File Reference . . . 101
5.26.1 Detailed Description . . . 102
5.27 boundary.h . . . 103
5.28 catalystAdaptor.h . . . 105
5.29 /zfs60/nobackup/stipa/software/TOSCA/src/include/clock.h File Reference . . . 109
5.29.1 Detailed Description . . . 110
5.30 clock.h . . . 110
5.31 /zfs60/nobackup/stipa/software/TOSCA/src/include/domain.h File Reference . . . 110
5.31.1 Detailed Description . . . 111
5.32 domain.h . . . 111
5.33 flags.h . . . 112
5.34 /zfs60/nobackup/stipa/software/TOSCA/src/include/ibm.h File Reference . . . 112
5.34.1 Detailed Description . . . 115
5.35 ibm.h . . . 115
5.36 ibmInput.h . . . 120
5.37 /zfs60/nobackup/stipa/software/TOSCA/src/include/inflow.h File Reference . . . 120
5.37.1 Detailed Description . . . 121
5.38 inflow.h . . . 121
5.39 /zfs60/nobackup/stipa/software/TOSCA/src/include/initialField.h File Reference . . . 121
5.39.1 Detailed Description . . . 122
5.39.2 Function Documentation . . . 122
5.39.2.1 SetInitialField() . . . 122
5.40 initialField.h . . . 123
5.41 /zfs60/nobackup/stipa/software/TOSCA/src/include/initialization.h File Reference . . . 123
5.41.1 Detailed Description . . . 124
5.41.2 Function Documentation . . . 124
5.41.2.1 SetInitialField() . . . 124

5.41.2.2 simulationInitialize() . . . 124
5.42 initialization.h . . . 125
5.43 /zfs60/nobackup/stipa/software/TOSCA/src/include/inline.h File Reference . . . 125
5.43.1 Detailed Description . . . 132
5.43.2 Function Documentation . . . 132
5.43.2.1 computeEm() . . . 132
5.44 inline.h . . . 132
5.45 /zfs60/nobackup/stipa/software/TOSCA/src/include/io.h File Reference . . . 230
5.45.1 Detailed Description . . . 232
5.46 io.h . . . 233
5.47 les.h . . . 234
5.48 /zfs60/nobackup/stipa/software/TOSCA/src/include/lesScalar.h File Reference . . . 235
5.48.1 Detailed Description . . . 236
5.48.2 Function Documentation . . . 236
5.48.2.1 UpdateCsk() . . . 236
5.49 lesScalar.h . . . 236
5.50 /zfs60/nobackup/stipa/software/TOSCA/src/include/mesh.h File Reference . . . 236
5.50.1 Detailed Description . . . 237
5.51 mesh.h . . . 237
5.52 /zfs60/nobackup/stipa/software/TOSCA/src/include/objects.h File Reference . . . 238
5.52.1 Detailed Description . . . 238
5.53 objects.h . . . 238
5.54 /zfs60/nobackup/stipa/software/TOSCA/src/include/overset.h File Reference . . . 239
5.54.1 Detailed Description . . . 240
5.55 overset.h . . . 240
5.56 /zfs60/nobackup/stipa/software/TOSCA/src/include/peqn.h File Reference . . . 243
5.56.1 Detailed Description . . . 244
5.57 peqn.h . . . 244
5.58 /zfs60/nobackup/stipa/software/TOSCA/src/include/precursor.h File Reference . . . 246
5.58.1 Detailed Description . . . 247
5.59 precursor.h . . . 247
5.60 /zfs60/nobackup/stipa/software/TOSCA/src/include/teqn.h File Reference . . . 247
5.60.1 Detailed Description . . . 248
5.61 teqn.h . . . 248
5.62 /zfs60/nobackup/stipa/software/TOSCA/src/include/tosca2PV.h File Reference . . . 249
5.62.1 Detailed Description . . . 252
5.62.2 Function Documentation . . . 253
5.62.2.1 getTimeList() . . . 253
5.63 tosca2PV.h . . . 253
5.64 /zfs60/nobackup/stipa/software/TOSCA/src/include/turbines.h File Reference . . . 258
5.64.1 Detailed Description . . . 258
5.65 turbines.h . . . 258

5.66 /zfs60/nobackup/stipa/software/TOSCA/src/include/ueqn.h File Reference . . . 265
5.66.1 Detailed Description . . . 265
5.67 ueqn.h . . . 265
5.68 /zfs60/nobackup/stipa/software/TOSCA/src/include/wallfunctions.h File Reference . . . 267
5.68.1 Detailed Description . . . 267
5.69 wallfunctions.h . . . 268
5.70 /zfs60/nobackup/stipa/software/TOSCA/src/include/wallmodel.h File Reference . . . 269
5.70.1 Detailed Description . . . 269
5.71 wallmodel.h . . . 269
5.72 /zfs60/nobackup/stipa/software/TOSCA/src/initialField.c File Reference . . . 270
5.72.1 Detailed Description . . . 271
5.72.2 Function Documentation . . . 271
5.72.2.1 SetInitialField() . . . 271
5.73 /zfs60/nobackup/stipa/software/TOSCA/src/initialization.c File Reference . . . 271
5.73.1 Detailed Description . . . 272
5.73.2 Function Documentation . . . 272
5.73.2.1 simulationInitialize() . . . 272
5.74 /zfs60/nobackup/stipa/software/TOSCA/src/io.c File Reference . . . 273
5.74.1 Detailed Description . . . 275
5.74.2 Function Documentation . . . 275
5.74.2.1 getTimeList() . . . 275
5.75 /zfs60/nobackup/stipa/software/TOSCA/src/les.c File Reference . . . 275
5.75.1 Detailed Description . . . 276
5.75.2 Function Documentation . . . 276
5.75.2.1 UpdateCs() . . . 276
5.76 /zfs60/nobackup/stipa/software/TOSCA/src/mesh.c File Reference . . . 276
5.76.1 Detailed Description . . . 277
5.77 /zfs60/nobackup/stipa/software/TOSCA/src/overset.c File Reference . . . 277
5.77.1 Detailed Description . . . 278
5.78 /zfs60/nobackup/stipa/software/TOSCA/src/peqn.c File Reference . . . 278
5.78.1 Detailed Description . . . 280
5.78.2 Macro Definition Documentation . . . 280
5.78.2.1 matID . . . 280
5.79 /zfs60/nobackup/stipa/software/TOSCA/src/precursor.c File Reference . . . 281
5.79.1 Detailed Description . . . 281
5.80 /zfs60/nobackup/stipa/software/TOSCA/src/solvers/teqn_solvers.c File Reference . . . 282
5.80.1 Detailed Description . . . 282
5.81 /zfs60/nobackup/stipa/software/TOSCA/src/solvers/teqn_solvers.h File Reference . . . 282
5.81.1 Detailed Description . . . 283
5.82 teqn_solvers.h . . . 283
5.83 /zfs60/nobackup/stipa/software/TOSCA/src/solvers/ueqn_solvers.c File Reference . . . 283
5.83.1 Detailed Description . . . 284

5.84 /zfs60/nobackup/stipa/software/TOSCA/src/solvers/ueqn_solvers.h File Reference 284
5.84.1 Detailed Description 284
5.85 ueqn_solvers.h 284
5.86 /zfs60/nobackup/stipa/software/TOSCA/src/sources/teqn_sources.c File Reference 285
5.86.1 Detailed Description 285
5.87 /zfs60/nobackup/stipa/software/TOSCA/src/sources/teqn_sources.h File Reference 285
5.87.1 Detailed Description 286
5.88 teqn_sources.h 286
5.89 /zfs60/nobackup/stipa/software/TOSCA/src/sources/ueqn_sources.c File Reference 286
5.89.1 Detailed Description 287
5.90 /zfs60/nobackup/stipa/software/TOSCA/src/sources/ueqn_sources.h File Reference 287
5.90.1 Detailed Description 287
5.91 ueqn_sources.h 287
5.92 /zfs60/nobackup/stipa/software/TOSCA/src/teqn.c File Reference 288
5.92.1 Detailed Description 288
5.93 /zfs60/nobackup/stipa/software/TOSCA/src/turbines.c File Reference 288
5.93.1 Detailed Description 289
5.94 /zfs60/nobackup/stipa/software/TOSCA/src/turbines/turbines_force.h File Reference 289
5.94.1 Detailed Description 289
5.95 turbines_force.h 290
5.96 /zfs60/nobackup/stipa/software/TOSCA/src/turbines/turbines_io.h File Reference 290
5.96.1 Detailed Description 292
5.97 turbines_io.h 292
5.98 /zfs60/nobackup/stipa/software/TOSCA/src/turbines/turbines_openfast.h File Reference 293
5.99 turbines_openfast.h 293
5.100 /zfs60/nobackup/stipa/software/TOSCA/src/turbines/turbines_parallel.h File Reference 294
5.100.1 Detailed Description 295
5.101 turbines_parallel.h 295
5.102 /zfs60/nobackup/stipa/software/TOSCA/src/turbines/turbines_phys.h File Reference 295
5.102.1 Detailed Description 296
5.103 turbines_phys.h 296
5.104 /zfs60/nobackup/stipa/software/TOSCA/src/turbines/turbines_sampling.h File Reference 296
5.104.1 Detailed Description 297
5.105 turbines_sampling.h 297
5.106 /zfs60/nobackup/stipa/software/TOSCA/src/ueqn.c File Reference 297
5.106.1 Detailed Description 298

Index **299**

Chapter 1

TOSCA

1.0.1 *Welcome to the official repository of TOSCA*

(Toolbox fOr Stratified Convective Atmospheres)

TOSCA is an incompressible finite-volume large-eddy simulation code formulated in curvilinear coordinates with Boussinesq approximation to account for buoyancy. It features a moving immersed boundary method and several actuator models to include bodies and wind turbines within the simulation domain. It has been successfully used to simulate atmospheric boundary layer and wind farm flows, including terrain and wind farm induced atmospheric gravity waves, cluster wakes, as well as wind farm control problems. TOSCA is currently developed at the `Computational Fluid Dynamics Laboratory of the University of British Columbia, Okanagan Campus`.

We kindly ask you to *please* fill up `this` survey, which allows us to keep track of new TOSCA users around the world.

1.0.2 License

The code is open-source and licensed under the BSD 2-Clause Simplified License.

1.0.3 User Guide

A large effort has been recently put in creating a comprehensive user guide, which is hosted on `GitHub Pages`. Although some chapters are still under construction, you can already find instructions on how to install the code, a complete list of all input files and variables, together with their explanation, as well as a step by step guide of some of the tutorials that are available on the repository under the `tests/` directory. We refer to these set of read-the-docs pages online as the main TOSCA user guide.

An additional, temporary, user guide (which will soon become outdated) with a *TOSCA Getting Started* can be found inside `docs/` (*TOSCA_UserGuide.pdf*). For programmers, the code follows the *Doxygen* commenting style, so that a programmer's documentation is automatically generated by Doxygen. This can be found both inside `docs/` (*ReferenceManual.pdf*), or opened as a web documentation upon downloading the TOSCA repository and opening `docs/html/index.html`

1.0.4 Reporting Bugs

For bugs, questions and to request modification to the TOSCA code, please open an issue on GitHub. You can also contribute to the TOSCA code as explained here. For different matters, please contact sebastiano.stipa@vki.ac.be or aajay100@mail.ubc.ca.

1.0.5 How to Cite

The TOSCA code can be cited using the following articles. For generic citations, please use the first article in the list while the remaining articles can be used to cite specific features of the TOSCA code.

- TOSCA - an open-source, finite-volume, large-eddy simulation (LES) environment for wind farm flows.
- The actuator farm model for large eddy simulation (LES) of wind-farm-induced atmospheric gravity waves and farm-farm interaction.
- A large-eddy simulation (LES) model for wind-farm-induced atmospheric gravity wave effects inside conventionally neutral boundary layers.
- Mixed subgrid-scale models in generalized curvilinear coordinates for large-eddy simulations of heterogeneous turbulent flows.

1.0.6 Recent Highlights/Additions

- OpenFAST coupling
- Direct/Indirect profile assimilation techniques to drive LES with observations/mesoscale models
- New LES models
- New IBM wall models
- Lateral fringe region.

1.0.7 Executables

- `tosca` : transient solver for stratified incompressible flows. Temperature stratification is accounted via Boussinesq approximation.
- `tosca2PV` : post processor for ParaView visualization. Writes data in XMF/HDF format.

1.0.8 Future Implementations:

- Coupling with the Weather Research and Forecasting Model
- Multiphase capabilities

Credits & Copyright: Sebastiano Stipa - Arjun Ajay - Mohammad Hadi - Joshua Brinkerhoff - The University of British Columbia

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

abl_ 11

access_
Access database 18

Acell
Overset acceptor cell (used for both overset and base mesh acceptor cells) 18

acquisition_
Struct containing all acquisition data structures 19

avgFields
Struct defining the average fields 20

BinIndex 22

boundingBox
Structure defining the domain bounding box (easy access to xmin, xmax, ymin, ymax, zmin, zmax) 22

Cabot
Structure storing the Shumann wall models information 23

cellIds
Cell indices 23

cellList
Cell Node list 24

cellNode 24

clock_ 24

Cmpnts
Defines the x, y, z components of a vector 25

constants_ 26

Cpt2D
Structure for defining the x, y components of a 2D vector 26

data3LM
Struct defining a gridded point mesh for three layer model data (used for its validation) 26

dataABL
Struct defining ABL simulation acquisition data 28

Dcell
Overset donor cell (contains donor cell indices, owner, and distance) 29

domain_
Domain data structure definition 30

elementBox 31

flags_
Solution access flags . . . 31
std::hash< BinIndex > . . . 32
HoleObject . . . 32
ibm_
Struct storing IBM model . . . 32
ibmFluidCell . . . 34
ibmMesh . . . 35
ibmObject . . . 35
ibmPitchMotion . . . 37
ibmRotation . . . 37
ibmSineMotion . . . 37
inflowData
Struct defining inflow information . . . 38
inletFunctions
Struct storing inlet functions data . . . 38
inletFunctionTypes
Struct storing the inlet function for a patch . . . 39
io_
Struct defining io settings (simulation checkpointing) . . . 41
keBox
Struct defining the ke fields . . . 42
keFields
Struct defining the ke fields . . . 44
les_
Struct storing LES model . . . 45
level3LM
Struct defining a gridded point mesh level for three layer model (used for its validation) . . . 46
list
Node list . . . 47
LogLawAPG . . . 47
mapInfo
Concurrent precursor mapping info . . . 47
Matrix3 . . . 48
mesh_
Mesh structure storing mesh and curvilinear coordinates . . . 48
ModeData . . . 50
node
Node struct . . . 51
OctreeNode . . . 51
overset_
Main overset structure.
This structure holds the parameters, connectivity, and interpolation data for both the overset (child) mesh and the base (background) mesh update . . . 52
oversetMotion
Overset motion structure (for dynamic overset motion) . . . 53
patchVectorField
Vector field on the patch (used to store wall models fields) . . . 53
peqn_
Struct storing pressure equation . . . 54
perturbFields
Struct defining the ke fields . . . 55
postProcess
Structure defining the variables for postProcessing . . . 56
PowerLawAPG
Structure storing the Shumann wall models information . . . 57
precursor_
Concurrent precursor database . . . 57

probeRake
Struct defining a probe rake . . . 58
rakes
Struct defining all rakes in the mesh . . . 59
scalarBC
Structure dafining the type of boundary conditions for a scalar . . . 59
searchBox . . . 60
sections
Struct defining a generic section to save . . . 60
Shumann
Structure storing the Shumann wall models information for U and T . . . 61
simInfo_ . . . 62
skewSymmTensor . . . 63
surface . . . 63
symmTensor
Defines the components of a symmetric tensor . . . 63
Tensor . . . 64
TensorInvariants . . . 64
teqn_
Struct storing temperature equation . . . 64
udSections . . . 66
ueqn_
Structure storing momentum equation . . . 67
uSections
Struct defining a user defined section not necessarily along curvilinear co-ordinates . . . 69
vectorBC
Structure dafining the type of boundary conditions for a vector . . . 70
wallModel
Wall models container . . . 71

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

/zfs60/nobackup/stipa/software/TOSCA/src/abl.c
Contains atmospheric boundary layer definition functions . . . 73
/zfs60/nobackup/stipa/software/TOSCA/src/acquisition.c
Acquisition functions definition . . . 74
/zfs60/nobackup/stipa/software/TOSCA/src/boundary.c
Contains boundary conditions . . . 76
/zfs60/nobackup/stipa/software/TOSCA/src/clock.c
Contains simulation time function definitions . . . 77
/zfs60/nobackup/stipa/software/TOSCA/src/ibm.c
Contains Immersed boundary method function definitions . . . 78
/zfs60/nobackup/stipa/software/TOSCA/src/ibmInput.c
Contains Immersed boundary method input and read function definitions . . . 80
/zfs60/nobackup/stipa/software/TOSCA/src/initialField.c
Contains initial field function definitions . . . 270
/zfs60/nobackup/stipa/software/TOSCA/src/initialization.c
Contains inflow boundary condition function definitions . . . 271
/zfs60/nobackup/stipa/software/TOSCA/src/io.c
Contains i/o operations function definitions . . . 273
/zfs60/nobackup/stipa/software/TOSCA/src/les.c
Contains LES model function definitions . . . 275
/zfs60/nobackup/stipa/software/TOSCA/src/mesh.c
Contains mesh definition functions . . . 276
/zfs60/nobackup/stipa/software/TOSCA/src/overset.c
Overset function definitions . . . 277
/zfs60/nobackup/stipa/software/TOSCA/src/peqn.c
Contains P equation function definitions . . . 278
/zfs60/nobackup/stipa/software/TOSCA/src/precursor.c
Contains top to bottom level routines for the concurrent precursor method . . . 281
/zfs60/nobackup/stipa/software/TOSCA/src/teqn.c
Contains T equation function definitions . . . 288
/zfs60/nobackup/stipa/software/TOSCA/src/turbines.c
Contains top to bottom level routines for the wind farm modeling . . . 288
/zfs60/nobackup/stipa/software/TOSCA/src/ueqn.c
Contains U equation function definitions . . . 297
/zfs60/nobackup/stipa/software/TOSCA/src/include/abl.h
ABL-related object definition . . . 81

/zfs60/nobackup/stipa/software/TOSCA/src/include/access.h . . . 86
/zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition.h
Acquisition object declaration . . . 86
/zfs60/nobackup/stipa/software/TOSCA/src/include/base.h
Base header file including c/c++ libraries and PETSc and HYPRE libs . . . 98
/zfs60/nobackup/stipa/software/TOSCA/src/include/boundary.h
Boundary conditions header file . . . 101
/zfs60/nobackup/stipa/software/TOSCA/src/include/catalystAdaptor.h . . . 105
/zfs60/nobackup/stipa/software/TOSCA/src/include/clock.h
Simulation time struct header file . . . 109
/zfs60/nobackup/stipa/software/TOSCA/src/include/domain.h
Domain data structure definition . . . 110
/zfs60/nobackup/stipa/software/TOSCA/src/include/flags.h . . . 112
/zfs60/nobackup/stipa/software/TOSCA/src/include/ibm.h
IBM model header file . . . 112
/zfs60/nobackup/stipa/software/TOSCA/src/include/ibmInput.h . . . 120
/zfs60/nobackup/stipa/software/TOSCA/src/include/inflow.h
Contains inflow boundary condition headers . . . 120
/zfs60/nobackup/stipa/software/TOSCA/src/include/initialField.h
Initial field header file . . . 121
/zfs60/nobackup/stipa/software/TOSCA/src/include/initialization.h
Contains simulation initialization function headers . . . 123
/zfs60/nobackup/stipa/software/TOSCA/src/include/inline.h
Inline functions . . . 125
/zfs60/nobackup/stipa/software/TOSCA/src/include/io.h
Contains i/o operations function headers . . . 230
/zfs60/nobackup/stipa/software/TOSCA/src/include/les.h . . . 234
/zfs60/nobackup/stipa/software/TOSCA/src/include/lesScalar.h
LES model header file for scalar quantities . . . 235
/zfs60/nobackup/stipa/software/TOSCA/src/include/mesh.h
Mesh header file . . . 236
/zfs60/nobackup/stipa/software/TOSCA/src/include/objects.h
Contains forward object declarations . . . 238
/zfs60/nobackup/stipa/software/TOSCA/src/include/overset.h
Definitions for overset objects and functions for two–way coupling . . . 239
/zfs60/nobackup/stipa/software/TOSCA/src/include/peqn.h
P equation solution header file . . . 243
/zfs60/nobackup/stipa/software/TOSCA/src/include/precursor.h
Concurrent precursor header file . . . 246
/zfs60/nobackup/stipa/software/TOSCA/src/include/teqn.h
T equation solution header file . . . 247
/zfs60/nobackup/stipa/software/TOSCA/src/include/tosca2PV.h
Post processing header file . . . 249
/zfs60/nobackup/stipa/software/TOSCA/src/include/turbines.h
Wind Farm Objects for the arrays of std::string and arrays of structs we use dynamic allocation through array of pointers to each std::string or struct respectively . . . 258
/zfs60/nobackup/stipa/software/TOSCA/src/include/ueqn.h
U equation solution header file . . . 265
/zfs60/nobackup/stipa/software/TOSCA/src/include/wallfunctions.h
Wallfunction functions . . . 267
/zfs60/nobackup/stipa/software/TOSCA/src/include/wallmodel.h
Wall models header file . . . 269
/zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/ablAcquisition.h
Three layer model validation test. Depth averaged fields are gathered on a gridded point mesh for subsequent post processing, namely velocity, pressure and wind farm body force. The mesh must be cartesian and the vertical direction is assumed to be the j-direction. The same hypothesis holds for the ABL data acquisition, which performes spatial averages as a function of time on the planes parallel to the wall defined by each cell's center . . . 91

/zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/averageAcquisition.h
Average fields object declaration . . . 93
/zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/keAcquisition.h
Mean kinetic energy budget object declaration . . . 94
/zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/perturbAcquisition.h
Perturbation fields w.r.t. an x-normal sample plane . . . 96
/zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/probesAcquisition.h
Probes and rakes object declaration . . . 96
/zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/sectionsAcquisition.h
Sections along curvilinear coordinates object declaration . . . 97
/zfs60/nobackup/stipa/software/TOSCA/src/solvers/teqn_solvers.c
Temperature-equation solver function implementations . . . 282
/zfs60/nobackup/stipa/software/TOSCA/src/solvers/teqn_solvers.h
Private temperature-equation solver function declarations . . . 282
/zfs60/nobackup/stipa/software/TOSCA/src/solvers/ueqn_solvers.c
Momentum-equation solver function implementations . . . 283
/zfs60/nobackup/stipa/software/TOSCA/src/solvers/ueqn_solvers.h
Private momentum-equation solver function declarations . . . 284
/zfs60/nobackup/stipa/software/TOSCA/src/sources/teqn_sources.c
Temperature-equation source-term function implementations . . . 285
/zfs60/nobackup/stipa/software/TOSCA/src/sources/teqn_sources.h
Private temperature-equation source-term function declarations . . . 285
/zfs60/nobackup/stipa/software/TOSCA/src/sources/ueqn_sources.c
Momentum-equation source-term and auxiliary-flux function implementations . . . 286
/zfs60/nobackup/stipa/software/TOSCA/src/sources/ueqn_sources.h
Private momentum-equation source-term function declarations . . . 287
/zfs60/nobackup/stipa/software/TOSCA/src/turbines/turbines_force.h
Turbine force computation and projection routines . . . 289
/zfs60/nobackup/stipa/software/TOSCA/src/turbines/turbines_io.h
Turbine IO routines . . . 290
/zfs60/nobackup/stipa/software/TOSCA/src/turbines/turbines_openfast.h . . . 293
/zfs60/nobackup/stipa/software/TOSCA/src/turbines/turbines_parallel.h
Turbine parallel computation routines . . . 294
/zfs60/nobackup/stipa/software/TOSCA/src/turbines/turbines_phys.h
Turbine physics module and control (substituted by OpenFAST if the latter is used) . . . 295
/zfs60/nobackup/stipa/software/TOSCA/src/turbines/turbines_sampling.h
Turbine velocity sampling routines . . . 296

Chapter 4

Class Documentation

4.1 abl_ Struct Reference

Public Attributes

- PetscInt **controllerActive**

  activate velocity controller
- PetscInt **controllerActiveT**

  activate temperature controller
- PetscInt **coriolisActive**

  activate coriolis force
- PetscReal **uTau**

  friction Velocity
- PetscReal **hRough**

  equivalent roughness length
- Cpt2D **uRef**

  reference velocity
- PetscReal **hRef**

  reference height
- PetscReal **hGeo**

  geostrophic height (required for geostrophic controller)
- PetscReal **hInv**

  inversion height
- PetscReal **dInv**

  inversion width
- PetscReal **gInv**

  delta T across inversion layer
- PetscReal **gABL**

  temperature gradient below the inversion layer
- PetscReal **tRef**

  reference potential temperature
- PetscReal **gTop**

  temperature gradient above the inversion layer
- PetscReal **vkConst**

  von Karman constant
- PetscReal **smear**

Rampanelli Zardi model parameter.

- PetscReal **fc**

  Coriolis parameter (omegaEarth ∗ sin(latitude) = 7.292115e-5 ∗ sin(latitude))
- PetscInt **perturbations**

  add turbulent perturbations (only if initialization is set to ABLFlow)
- PetscReal ∗ **cellLevels**

  heights of the averaging planes
- PetscReal ∗ **tDes**

  initial temperature to be maintained
- word **controllerTypeT**

  initial or directProfileAssimilation
- word **controllerType**

  velocity controller type: write/read (writes in postProcessing/momentumSource, reads from momentumSource)
- word **controllerAction**
- word **controllerActionT**
- PetscReal **relax**

  source term relaxation factor
- PetscReal **alpha**

  proportional over integral controller action ratio
- PetscReal **timeWindow**

  time window of the integral part
- PetscReal ∗ **totVolPerLevel**

  total volume at each cell level
- PetscInt ∗ **totCelPerLevel**

  total number of cells per level
- PetscInt ∗ **closestLabels**

  closest heights w.r.t. controller height
- PetscReal ∗ **levelWeights**

  weights for variables interpolated at closest heights w.r.t. controller height
- PetscReal **controllerMaxHeight**

  max height of influence of the velocity controller
- PetscInt **geostrophicDampingActive**

  geosptrophic oscillation damping
- PetscInt **mesoScaleInputActive**

  use mesoscale data for pressure controller uDes
- PetscReal **geoDampAvgDT**

  average time step from simulation start
- Cmpnts **geoDampAvgS**

  expected geostrophic velocity
- Cmpnts **geoDampUBar**

  expected geostrophic velocity
- Cmpnts ∗ **geoDampU**

  average horizontal velocity at current iteration
- PetscReal **geoDampAlpha**

  alpha = 1 critical damping (>1 over-damped, <1 under-damped)
- PetscReal **geoDampH**

  H at which damping begins to be applied.
- PetscReal **geoDampDelta**

  rising distance for the application function
- PetscReal **geoDampC**

  critical damping coefficient

- PetscReal **geoDampStart**

  starting time of geostrophic damping
- PetscReal **geoDampWindow**

  averaging window for the geostrophic velocity
- PetscInt ∗ **closestLabelsGeo**

  closest heights w.r.t. controller height
- PetscReal ∗ **levelWeightsGeo**

  weights for variables interpolated at closest heights w.r.t. controller height
- Cmpnts **uGeoBar**

  desired geostrophic wind speed magnitude
- PetscReal **refHubAngle**

  reference hub angle (typically 0 for canonical ABL cases)
- PetscReal **geoAngle**

  geostrophic wind angle
- PetscInt **windAngleController**

  flag to switch on wind angle controller - to control the wind direction to match the reference hub angle
- Cmpnts **a**
- Cmpnts **b**

  the two constant parts of the controller (a = geo forcing, b = wind angle controller)
- PetscInt **currentCloseIdx**

  save the current closest index at each iteration to speed up the interpolation search
- PetscInt **currentCloseIdxT**
- PetscReal **sourceAvgStartTime**

  if controllerType is 'average', average sources from this time value
- PetscReal ∗∗ **preCompSources**

  table of given sources [ntimesteps][time|sourceX|sourceY|sourceZ] for velocity controller type = timeSeries/time↩ AverageSeries
- PetscReal ∗∗∗ **timeHtSources**

  table of given timeheight sources [time|sourceX|sourceY|sourceZ] at each cell level, controller type = timeHeight↩ Series
- PetscReal ∗∗∗ **timeHtSourcesT**

  table of given timeheight sources [time|source|] at each cell level, controller type = timeHeightSeries
- PetscInt **nSourceTimes**

  number of times in the pre-computed sources
- Cmpnts **cumulatedSource**

  cumulated error of the velocity controller (equalt to gradP at steady state)
- Cmpnts ∗ **cumulatedSourceHt**

  cumulated error of the velocity controller for every mesh level
- PetscReal **avgTimeStep**

  average time step from the momentum source file
- PetscReal **zDampingStart**

  starting height of the Rayleigh damping layer
- PetscReal **zDampingEnd**

  ending height of the Rayleigh damping layer
- PetscReal **zDampingAlpha**

  damping paramter (it is equal to the Rayleigh viscosity at hEnd)
- Cmpnts ∗ **uBarMeanZ**

  array storing the reference velocity (averages along i at the k = 1 faces, size = my)
- PetscInt **avgWeight**
- PetscInt **zDampingAlsoXY**

  damp also x and y velocity components (if 0 only z component is damped)

- PetscInt **zDampingXYType**

  type 1 (default) averages at inlet, type (2) requires concurrent precursor and xDamping and uses planar averages
- PetscReal **kLeftPatchDist**

  width of the kLeft Rayleigh damping layer
- PetscReal **kLeftDampingAlpha**

  kLeft Rayleigh damping coefficient
- Cmpnts **kLeftDampingUBar**

  kLeft bar velocity with respect to which the flow is damped
- PetscReal **kLeftDampingFilterHeight**

  above this height damping is unity (transitions to zero at kLeftDampingFilterHeight - kLeftDampingFilterWidth)
- PetscReal **kLeftDampingFilterWidth**

  width of transition region from no damping to damping
- PetscReal **kRightPatchDist**

  width of the kRight Rayleigh damping layer
- PetscReal **kRightDampingAlpha**

  kRight Rayleigh damping coefficient
- Cmpnts **kRightDampingUBar**

  kRight bar velocity with respect to which the flow is damped
- PetscReal **kRightDampingFilterHeight**

  above this height damping is unity (transitions to zero at kRightDampingFilterHeight - kRightDampingFilterWidth)
- PetscReal **kRightDampingFilterWidth**

  width of transition region from no damping to damping
- PetscReal **xDampingStart**

  starting x of the fringe layer
- PetscReal **xDampingEnd**

  ending x of the fringe layer
- PetscReal **xDampingDelta**

  damping raise/decay distance (must be less than 0.5∗(xDampingEnd - xDampingStart))
- PetscReal **xDampingAlpha**

  damping paramter (see Inoue, Matheou, Teixeira 2014)
- word **xDampingControlType**

  type of controller: alphaFixed or alphaOptimized
- PetscReal **advDampingXStart**

  starting x of the adv damping layer
- PetscReal **advDampingXEnd**

  ending x of the adv damping layer
- PetscReal **advDampingXDeltaStart**

  damping raise/decay distance
- PetscReal **advDampingXDeltaEnd**

  damping raise/decay distance
- PetscReal **xDampingTimeWindow**

  time constant for velocity filtering
- PetscReal **xDampingVBar**

  desired y-velocity sampled from precursor
- PetscReal **vEnd**
- PetscReal **vStart**

  line-averaged start and end y-velocity at the fringe region extrema
- PetscReal **xDampingError**

  error on y-velocity at fringe end w.r.t precursor
- PetscReal **xDampingLineSamplingYmin**

  starting y of the sampling lines

- PetscReal **xDampingLineSamplingYmax**

  ending y of the sampling lines
- PetscReal **xDampingTimeStart**

  last time that alpha was modified
- PetscReal **xDampingDeltaV**

  y-velocity jump across the fringe
- PetscReal **xDampingCoeff**

  coeff = |Vend - Vstart| / alpha
- PetscInt ∗ **closestLabelsFringe**

  closest height w.r.t. fringe controller height
- PetscReal ∗ **levelWeightsFringe**

  weights for variables interpolated at closest heights w.r.t. fringe controller height
- PetscReal **advDampingYStart**

  starting y of the adv damping y layer
- PetscReal **advDampingYEnd**

  ending y of the adv damping y layer
- PetscReal **advDampingYDeltaStart**

  damping raise/decay distance
- PetscReal **advDampingYDeltaEnd**

  damping raise/decay distance
- PetscReal **yDampingStart**

  starting y of the fringe layer
- PetscReal **yDampingEnd**

  ending y of the fringe layer
- PetscReal **yDampingDelta**

  damping raise/decay distance (must be less than 0.5∗(yDampingEnd - yDampingStart))
- PetscReal **yDampingAlpha**

  damping paramter
- PetscInt **yDampingNumPeriods**

  number of periodizations in the streamwise direction of the x fringe region
- Vec **uBarInstY**

  instantaneous bar velocity for y-fringe region (only used for concurent precursor)
- Vec **tBarInstY**

  instantaneous bar temperature for y-fringe region (only used for concurent precursor)
- PetscInt **numSourceProc**

  global to all procs - number of processors within the source (part of x fringe region within the lateral fringe region)
- PetscInt ∗ **sourceProcList**

  global to all procs - list of processors in the source
- MPI_Comm ∗ **yDamp_comm**

  communicator that links each source processor to its corresponding periodization processors(destination) in the lateral fringe region - each source processor has a separate communicator for its set of source-destination processors
- PetscMPIInt ∗ **srcCommLocalRank**

  global to communicator procs - local rank of the source processor within the source-destination communicator
- cellIds ∗ **srcMinInd**

  global to communicator procs - minimum k,j,i index of each processor within the source domain
- cellIds ∗ **srcMaxInd**

  global to communicator procs - maximum k,j,i index of each processor within the source domain
- PetscInt ∗ **isdestProc**

  flag indicating if a given processor is within the destination region of a source processor
- cellIds ∗∗ **destMinInd**

  local to each proc - minimum k,j,i index of a processor in the destination domain

- cellIds ** **destMaxInd**

  local to each proc - maximum k,j,i index of a processor in the destination domain
- PetscInt * **srcNumI**

  global to communicator procs - number of i index for each processor in source
- PetscInt * **srcNumJ**

  global to communicator procs - number of j index for each processor in source
- PetscInt * **srcNumK**

  global to communicator procs - number of k index for each processor in source
- MPI_Request * **mapRequest**

  MPI variable to perform non blocking broadcast operation.
- Cmpnts ** **velMapped**

  one d array of the mapped velocity of each source processor
- PetscReal ** **tMapped**
- PetscInt ** **closestKCell**

  closest 2 k index of the fictitious mesh(mapped from source) to the k indexes of the sucessor domain
- PetscReal ** **wtsKCell**

  weights of the 2 closest k cells based on their distance
- PetscInt **xFringeUBarSelectionType**

  read type of fringe region in uBarSelectionType
- Cmpnts ** **uBarInstX**

  array storing the instantaneous velocity field for x damping layer
- PetscReal ** **tBarInstX**

  array storing the instantaneous temperature field for x damping layer
- PetscInt ** **nProcsKLine**

  number of processors in each k-line, used to average after MPI_Allreduce
- PetscReal **xStartCanopy**
- PetscReal **xEndCanopy**

  x start and ending coordinates of the region where the side force is applied
- PetscReal **yStartCanopy**
- PetscReal **yEndCanopy**

  x start and ending coordinates of the region where the side force is applied
- PetscReal **zStartCanopy**
- PetscReal **zEndCanopy**

  z start and ending coordinates of the region where the side force is applied
- PetscReal **cftCanopy**

  thrust coefficient of the entire wind canopy
- Cmpnts **diskDirCanopy**

  disk direction of turbines inside the canopy
- PetscReal **zPeak**
- PetscReal **deltaV**
- PetscReal **deltaU**
- PetscReal **Uperiods**
- PetscReal **Vperiods**
- PetscReal * **timeV**
- PetscReal * **hV**
- PetscReal * **timeT**
- PetscReal * **hT**
- Cmpnts ** **uMeso**
- Cmpnts * **uGeoPrev**
- PetscReal ** **tMeso**
- PetscInt **numhV**
- PetscInt **numhT**

- PetscInt **numtV**
- PetscInt **numtT**
- PetscInt ** **velInterpIdx**
- PetscReal ** **velInterpWts**
- PetscInt ** **tempInterpIdx**
- PetscReal ** **tempInterpWts**
- PetscInt **lowestIndV**
- PetscInt **lowestIndT**
- PetscInt **highestIndV**
- PetscInt **highestIndT**
- PetscInt **lMesoIndV**
- PetscInt **hMesoIndV**
- PetscReal **lowestSrcHt**
- Cmpnts * **luMean**
- Cmpnts * **guMean**
- Cmpnts * **srcPA**
- PetscInt **closestTimeIndV**

  closest index in time for the mesoscale timevarying data
- PetscReal **closestTimeWtV**
- PetscInt **closestTimeIndT**

  closest index in time for the mesoscale timevarying data
- PetscReal **closestTimeWtT**
- PetscInt **polyOrder**
- PetscInt **polyOrderT**
- word **wtDist**
- PetscReal ** **polyCoeffM**
- PetscReal ** **polyCoeffT**
- PetscInt **averageSource**
- PetscReal **currAvgtime**
- PetscReal **tAvgWindow**
- Cmpnts * **avgsrc**
- word **flType**
- word **flTypeT**
- PetscReal * **avgTotalStress**
- Cmpnts * **avgStress**
- PetscReal * **avgHeatFlux**
- PetscReal **hAvgTime**
- PetscReal **bottomSrcHtV**
- PetscReal **bottomSrcHtT**
- PetscReal **ablHt**
- PetscReal **ablHtStartTime**
- PetscReal **heatFluxSwitch**
- PetscInt **kernelRadius**
- PetscReal **sigma**
- PetscReal **omega**
- word **waveName**
- word **waveTMethod**
- word **waveExtn**
- word **waveConv**
- PetscInt **waveLevel**
- PetscInt **waveletBlend**
- precursor_ * **precursor**

  concurrent precursor data structure
- access_ * **access**

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/abl.h

4.2 access_ Struct Reference

access database

```
#include <access.h>
```

Public Attributes

- clock_ ∗ **clock**
- simInfo_ ∗ **info**
- constants_ ∗ **constants**
- flags_ ∗ **flags**
- mesh_ ∗ **mesh**
- io_ ∗ **io**
- ueqn_ ∗ **ueqn**
- peqn_ ∗ **peqn**
- teqn_ ∗ **teqn**
- les_ ∗ **les**
- farm_ ∗ **farm**
- overset_ ∗ **os**
- abl_ ∗ **abl**
- acquisition_ ∗ **acquisition**
- ibm_ ∗ **ibm**
- PetscInt ∗ **domainID**

4.2.1 Detailed Description

access database

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/access.h

4.3 Acell Struct Reference

Overset acceptor cell (used for both overset and base mesh acceptor cells)

```
#include <overset.h>
```

Public Attributes

- PetscInt **indi**
- PetscInt **indj**
- PetscInt **indk**

  cell indices
- PetscReal **coorx**
- PetscReal **coory**
- PetscReal **coorz**

  cell center coordinates
- PetscMPIInt **rank**

  owning processor
- PetscReal **cell_size**

  representative cell size
- PetscInt **face**

  face indicator (used for BCs etc.)
- PetscInt **donorId**
- PetscInt **parentCellId**

4.3.1 Detailed Description

Overset acceptor cell (used for both overset and base mesh acceptor cells)

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/overset.h

4.4 acquisition_ Struct Reference

Struct containing all acquisition data structures.

```
#include <acquisition.h>
```

Public Attributes

- rakes * **probes**

  probe rakes
- avgFields * **fields**

  average and turbulence fields
- keFields * **keBudFields**

  kinetic energy budjets
- sections * **kSections**

  information about the k-sections
- sections * **jSections**

  information about the j-sections
- sections * **iSections**

  information about the i-sections
- udSections * **userSections**

information about the user defined sections

- dataABL * **statisticsABL**

  ABL statistics.
- data3LM * **LM3**

  3LM statistics
- perturbFields * **perturbABL**

  ABL perturbation fields for gravity waves.
- PetscInt **isProbesActive**
- PetscInt **isSectionsActive**
- PetscInt **isAverageABLActive**
- PetscInt **isAverage3LMActive**
- PetscInt **isPerturbABLActive**
- access_ * **access**

  access database

4.4.1 Detailed Description

Struct containing all acquisition data structures.

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition.h

4.5 avgFields Struct Reference

Struct defining the average fields.

```
#include <averageAcquisition.h>
```

Public Attributes

- Vec **divU**

  velocity divergence
- Vec **Coriolis**

  coriolis source
- Vec **Driving**

  driving pressure gradient source
- Vec **xDamping**

  x damping layer (x fringe region source)
- Vec **CanopyForce**

  side force for fringe region testing
- Vec **windFarmForce**

  wind farm body force
- Vec **yDampMappedU**

  y damping mapped velocity
- Vec **Q**

  Q criterion field.
- Vec **L2**

L2 criterion field (not yet implemented)

- Vec **Qg**

  second invariant of VGT
- Vec **Rg**

  third invariant of VGT
- Vec **Qs**

  second invariant of strain
- Vec **Rs**

  third invariant of strain
- Vec **Qr**

  rotation invariant
- Vec **avgU**

  sum of u, v, w over time
- Vec **avgP**

  sum of p over time
- Vec **avgUU**

  sum of resolved R stresses over time
- Vec **avgDUU**

  sum of dispersive stresses over time
- Vec **avgP2**

  sum of p^2 over time
- Vec **avgNut**

  sun of nut over time
- Vec **avgCs**

  sum of cs over time
- Vec **avgOmega**

  sum of vorticity over time
- Vec **avgOmegaOmega**

  sum of resolved vorticity stresses over time
- Vec **avgUdotGradP**

  sum of scalar u∗dpdx + v∗dpdy + w∗dpdz over time
- Vec **avgMagGradU**

  sum of velocity grad magnitude over time
- Vec **avgMagUU**

  sum of vector (u^2+v^2+w^2)∗ui over time
- Vec **pAvgU**

  phase sum of u, v, w over time
- Vec **pAvgP**

  phase sum of p over time
- Vec **pAvgUU**

  phase sum of resolved R stresses over time
- Vec **pAvgP2**

  phase sum of p^2 over time
- Vec **pAvgNut**

  phase sun of nut over time
- Vec **pAvgCs**

  phase sum of cs over time
- Vec **pAvgOmega**

  phase sum of vorticity over time
- Vec **pAvgOmegaOmega**

  phase sum of resolved vorticity stresses over time

- Vec **pAvgUdotGradP**

  phase sum of scalar u∗dpdx + v∗dpdy + w∗dpdz over time
- Vec **pAvgMagGradU**

  phase sum of velocity grad magnitude over time
- Vec **pAvgMagUU**

  phase sum of vector (u^2+v^2+w^2)∗ui over time

4.5.1 Detailed Description

Struct defining the average fields.

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/averageAcquisition.h

4.6 BinIndex Struct Reference

Public Member Functions

- bool **operator==** (const BinIndex &other) const

Public Attributes

- int **ix**
- int **iy**
- int **iz**

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/overset.h

4.7 boundingBox Struct Reference

Structure defining the domain bounding box (easy access to xmin, xmax, ymin, ymax, zmin, zmax)

```
#include <mesh.h>
```

Public Attributes

- PetscReal **xmin**
- PetscReal **xmax**
- PetscReal **ymin**
- PetscReal **ymax**
- PetscReal **zmin**
- PetscReal **zmax**
- PetscReal **Lx**
- PetscReal **Ly**
- PetscReal **Lz**

4.7.1 Detailed Description

Structure defining the domain bounding box (easy access to xmin, xmax, ymin, ymax, zmin, zmax)

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/mesh.h

4.8 Cabot Struct Reference

structure storing the Shumann wall models information

```
#include <wallmodel.h>
```

Public Attributes

- PetscReal **roughness**
- PetscReal **kappa**

  von karman constant (usually 0.4)

4.8.1 Detailed Description

structure storing the Shumann wall models information

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/wallmodel.h

4.9 cellIds Struct Reference

Cell indices.

```
#include <base.h>
```

Public Attributes

- PetscInt **i**
- PetscInt **j**
- PetscInt **k**

4.9.1 Detailed Description

Cell indices.

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/base.h

4.10 cellList Struct Reference

cell Node list

```
#include <ibm.h>
```

Public Attributes

- cellNode * **head**

4.10.1 Detailed Description

cell Node list

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/ibm.h

4.11 cellNode Struct Reference

Public Attributes

- cellIds **Node**
- struct cellNode * **next**

  cell id

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/ibm.h

4.12 clock_ Struct Reference

Public Attributes

- PetscReal **time**

  current time value
- PetscReal **startTime**

  simulation start time
- PetscReal **endTime**

  simulation end time
- word **startFrom**

  choose if start from startTime value or latestTime (startTime is ignored)
- PetscReal **dt**

  time step
- PetscReal **startDt**

time step

- PetscReal **dtOld**

  old time step
- PetscReal **cfl**

  cfl number
- PetscReal **dxMin**

  min cell side size
- PetscReal **acquisitionDt**

  uniform dt due to acquistion if applied from the start
- PetscInt **it**

  current iteration value
- PetscInt **itStart**

  start iteration value (zero)
- PetscInt **timePrecision**

  time write precision

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/clock.h

4.13 Cmpnts Struct Reference

Defines the x, y, z components of a vector.

```
#include <base.h>
```

Public Attributes

- PetscScalar **x**
- PetscScalar **y**
- PetscScalar **z**

4.13.1 Detailed Description

Defines the x, y, z components of a vector.

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/base.h

4.14 constants_ Struct Reference

Public Attributes

- PetscReal **Pr**

  Prantl number.
- PetscReal **nu**

  kinematic viscosity
- PetscReal **rho**

  flow density [Kg/m3]
- PetscReal **tRef**

  reference T, required when ABL is not active

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/base.h

4.15 Cpt2D Struct Reference

Structure for defining the x, y components of a 2D vector.

```
#include <base.h>
```

Public Attributes

- PetscReal **x**
- PetscReal **y**

4.15.1 Detailed Description

Structure for defining the x, y components of a 2D vector.

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/base.h

4.16 data3LM Struct Reference

Struct defining a gridded point mesh for three layer model data (used for its validation)

```
#include <ablAcquisition.h>
```

Public Attributes

- word **checkPointTimeName**

  name of the time where checkpoint is contained
- PetscInt **nstw**

  number of points in streamwise direction
- PetscInt **nspw**

  number of points in the spanwise direction
- Cmpnts **upDir**

  vertical direction unit vector
- Cmpnts **streamDir**

  streamwise direction unit vector
- Cmpnts **spanDir**

  spanwise direction unit vector
- Cmpnts ** **points**

  array of [nstw, nspw] where point coordinates are stored
- cellIds ** **closestCells**

  array of [nstw, nspw] where i and k ids of the averaging line are stored (j is meaningless)
- PetscReal **avgStartTime**

  start time of acquisition system
- PetscReal **avgPrd**

  acquisition time interval (overrides simulation time step if smaller)
- PetscInt **avgWeight**

  number of averages taken up to this time step (snapshot weighting)
- level3LM ** **levels**

  array of pointers to the three levels of the 3LM model
- PetscReal ** **etaTBL**

  total boundary layer displacement
- PetscReal ** **etaLBL**

  total boundary layer displacement
- PetscReal ** **etaIBL**
- PetscReal ** **bkgU**
- PetscReal **xSample**
- PetscInt **kSample**
- Vec **avgU**

  average velocity needed to compute Tau
- Vec **avgdTdz**

  defined as the average in time of sqrt(uw^2 + vw^2), lowercase are fluctuations

4.16.1 Detailed Description

Struct defining a gridded point mesh for three layer model data (used for its validation)

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/ablAcquisition.h

4.17 dataABL Struct Reference

Struct defining ABL simulation acquisition data.

```
#include <ablAcquisition.h>
```

Public Attributes

- word **timeName**

  name of the time directory where ABL averages are written
- PetscReal **avgStartTime**

  start time of acquisition system
- PetscReal **avgPrd**

  acquisition time interval (overrides simulation time step if smaller) //!< number of averages taken up to this time step (snapshot weighting)
- PetscReal ∗ **cellLevels**

  heights of the averaging planes
- PetscReal ∗ **totVolPerLevel**

  total volume at each cell level
- PetscInt ∗ **totCelPerLevel**

  total number of cells per level
- PetscReal ∗ **UMean**

  mean x velocity at each level
- PetscReal ∗ **VMean**

  mean y velocity at each level
- PetscReal ∗ **WMean**

  mean z velocity at each level
- PetscReal ∗ **TMean**

  mean temperature at each level
- PetscReal ∗ **nutMean**

  mean turbulent viscosity at each level
- PetscReal ∗ **ksgsMean**

  mean subgrid scale kinetic energy at each level
- PetscReal ∗ **diffTMean**

  mean turbulent diffusivity at each level
- Vec **UPrime**

  fluctuating velocity field
- Vec **TPrime**

  fluctuating temperature field
- PetscReal ∗ **uuMean**
- PetscReal ∗ **uvMean**
- PetscReal ∗ **uwMean**
- PetscReal ∗ **vvMean**
- PetscReal ∗ **vwMean**
- PetscReal ∗ **wwMean**
- PetscReal ∗ **wuuMean**
- PetscReal ∗ **wuvMean**
- PetscReal ∗ **wuwMean**
- PetscReal ∗ **wvvMean**
- PetscReal ∗ **wvwMean**
- PetscReal ∗ **wwwMean**

- PetscReal * **R11Mean**
- PetscReal * **R12Mean**
- PetscReal * **R13Mean**
- PetscReal * **R22Mean**
- PetscReal * **R23Mean**
- PetscReal * **R33Mean**
- PetscReal * **TuMean**
- PetscReal * **TvMean**
- PetscReal * **TwMean**
- PetscReal * **q1Mean**
- PetscReal * **q2Mean**
- PetscReal * **q3Mean**

4.17.1 Detailed Description

Struct defining ABL simulation acquisition data.

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/ablAcquisition.h

4.18 Dcell Struct Reference

Overset donor cell (contains donor cell indices, owner, and distance)

```
#include <overset.h>
```

Public Attributes

- PetscInt **indi**
- PetscInt **indj**
- PetscInt **indk**

  cell indices
- PetscMPIInt **rank**

  donor cell owner
- PetscReal **dist2p**

  squared distance from acceptor cell center

4.18.1 Detailed Description

Overset donor cell (contains donor cell indices, owner, and distance)

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/overset.h

4.19 domain_ Struct Reference

Domain data structure definition.

```
#include <domain.h>
```

Public Attributes

- simInfo_ **info**
- constants_ **constants**
- PetscInt **domainID**
- clock_ ∗ **clock**

  time info
- mesh_ ∗ **mesh**

  mesh data structure
- io_ ∗ **io**

  io settings data structure
- ueqn_ ∗ **ueqn**

  momentum equation data structure
- peqn_ ∗ **peqn**

  pressure equation data structure
- teqn_ ∗ **teqn**

  potential temperature transport equation
- les_ ∗ **les**

  LES model data structure.
- overset_ ∗ **os**

  overset data structure
- farm_ ∗ **farm**

  wind farm data structure
- ibm_ ∗ **ibm**

  IBM data structure.
- abl_ ∗ **abl**

  atmospheric boundary layer data structure
- acquisition_ ∗ **acquisition**

  acquisition data structure
- flags_ **flags**

  solution flags data structure
- access_ **access**

  access database data structure

4.19.1 Detailed Description

Domain data structure definition.

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/domain.h

4.20 elementBox Struct Reference

Public Attributes

- PetscInt * **thisElemControlled**

  flag telling if a ibm element is controlled by this processor
- PetscInt * **thisElemTransfered**

  flag telling that the control of this point is being transferred to another processor
- boundingBox **innerZone**
- boundingBox **outerZone**

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/ibm.h

4.21 flags_ Struct Reference

solution access flags

```
#include <flags.h>
```

Public Attributes

- PetscInt **isOversetActive**
- PetscInt **isAdjustableTime**
- PetscInt **isLesActive**
- PetscInt **isTeqnActive**
- PetscInt **isAquisitionActive**
- PetscInt **isWindFarmActive**
- PetscInt **isAblActive**
- PetscInt **isIBMActive**
- PetscInt **isZDampingActive**
- PetscInt **isXDampingActive**
- PetscInt **isYDampingActive**
- PetscInt **isCanopyActive**
- PetscInt **isPrecursorSpinUp**
- PetscInt **isConcurrentPrecursorActive**
- PetscInt **isPvCatalystActive**
- PetscInt **isKLeftRayleighDampingActive**
- PetscInt **isKRightRayleighDampingActive**
- PetscInt **isAdvectionDampingXActive**
- PetscInt **isAdvectionDampingYActive**
- PetscInt **isSideForceActive**
- PetscInt **isNonInertialFrameActive**
- PetscInt **isGravityWaveModelingActive**
- PetscInt **isMeangradPForcingActive**

4.21.1 Detailed Description

solution access flags

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/flags.h

4.22 std::hash< BinIndex > Struct Reference

Public Member Functions

- std::size_t **operator()** (const BinIndex &b) const

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/overset.h

4.23 HoleObject Struct Reference

Public Attributes

- word **bodyName**
- PetscInt **ownerMesh**
- PetscInt **donorMesh**
- word **fileType**
- Cmpnts **baseLocation**
- PetscInt **searchCellRatio**

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/overset.h

4.24 ibm_ Struct Reference

struct storing IBM model

```
#include <ibm.h>
```

Public Attributes

- Vec **INvertFixed**
- PetscInt **numBodies**

  number of bodies
- word **IBInterpolationModel**

  interpolation methodology
- word **curvibType**
- word **curvibOrder**
- ibmObject ∗∗ **ibmBody**

  array of pointers to ibm objects
- searchBox ∗ **sBox**

  array of searchBox with number of search cells and their size for each ibm object
- ibmFluidCell ∗ **ibmFCells**

  cell ids and supports cells of the ibm fluid cells within each processor
- PetscInt **numIBMFluid**

  number of ibm fluid cells within each processor
- access_ ∗ **access**
- PetscInt **dbg**
- PetscInt **dynamic**
- PetscInt **computeForce**
- PetscInt **checkNormal**
- PetscInt **wallShearOn**
- PetscInt **writeSTL**
- PetscInt **ibmABL**
- PetscReal **interpDist**
- PetscReal **timeStart**

  start time of acquisition system
- word **intervalType**

  timeStep: sample at every (timeInterval) iter, adjustableTime sample at every (timeInterval) seconds
- PetscReal **timeInterval**

  acquisition time interval (overrides simulation time step if smaller and adjustableTime active)

4.24.1 Detailed Description

struct storing IBM model

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/ibm.h

4.25 ibmFluidCell Struct Reference

Public Attributes

- cellIds **cellId**

  cell id of the ibm fluid cell
- cellList **flNodes**

  list of the cell ids of the fluid cells as support to the ibm fluid cell
- list **slNodes**

  list of the ibm mesh nodes as support to the ibm fluid cell
- list **bID**

  list of the body id corresponding to the ibm mesh node in the slNodes list
- PetscInt **numFl**

  number of fluid cells in the support
- PetscInt **numSl**

  number of ibm mesh nodes in the support
- PetscReal **rad**

  support radius of the ibm fluid cell
- PetscInt **closestElem**

  closest ibm mesh element to the ibm fluid cell
- Cmpnts **pMin**

  projected point on the ibm mesh element from the ibm fluid cell
- PetscReal **minDist**

  dist to the ibm mesh element from the ibm fluid cell
- PetscInt **bodyID**
- Cmpnts **normal**
- PetscReal **cs1**
- PetscReal **cs2**
- PetscReal **cs3**

  ibm interpolation coefficient of the projected point from the ibm element nodes
- PetscReal **cr1**
- PetscReal **cr2**
- PetscReal **cr3**

  ibm interpolation coefficient of the background mesh point from the background plane triangle nodes
- PetscInt **i1**
- PetscInt **j1**
- PetscInt **k1**

  first node (cell center id of fluid mesh) of the background interpolation plane
- PetscInt **i2**
- PetscInt **j2**
- PetscInt **k2**

  second node (cell center id of fluid mesh) of the background interpolation plane
- PetscInt **i3**
- PetscInt **j3**
- PetscInt **k3**

  third node (cell center id of fluid mesh) of the background interpolation plane
- PetscInt **imode**

  indicates which triangle (out 0f 8) the interpolated point belongs in the background plane
- PetscReal **dB**

  distance to the background plane from ibm fluid node along the IBM element normal

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/ibm.h

4.26 ibmMesh Struct Reference

Public Attributes

- PetscInt **nodes**

  number of nodes in the IBM Body
- PetscInt **elems**

  number of elements in the IBM body
- Cmpnts ∗ **nCoor**

  pointer to the co-ordinates of the nodes
- PetscInt ∗ **nID1**
- PetscInt ∗ **nID2**
- PetscInt ∗ **nID3**

  pointer to the 3 node ids of a triangular mesh element
- Cmpnts ∗ **eN**

  pointers to the component of face normal in x, y and z direction of element
- Cmpnts ∗ **eT1**

  pointers to the component of face tangential1 (eT1 = eN x k, where k is unit normal along z, which is taken as a generic direction)
- Cmpnts ∗ **eT2**

  pointers to the component of face tangential2 (eT2 = eN x eT1)
- PetscInt ∗ **eSurface**

  pointer to the surfaceBody that this element belongs to
- PetscReal ∗ **eA**

  area of the element
- Cmpnts ∗ **eCent**

  coordinate of the element center
- Cmpnts ∗ **nU**

  velocity of the nodes
- Cmpnts ∗ **nUPrev**

  velocity of the node at the previous timestep
- Cmpnts ∗ **eQVec**

  center of the smallest bounding sphere of an ibm element
- PetscReal ∗ **eRVec**

  radius of the smallest bounding sphere of an ibm element

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/ibm.h

4.27 ibmObject Struct Reference

Public Attributes

- word **bodyName**
- word **bodyType**
- PetscInt **numSurfaces**
- PetscInt **bodyID**
- PetscInt **thinBody**

- word **elementSet**
- PetscInt **addSideFaces**
- ibmMesh ∗ **ibMsh**
- surface ∗∗ **ibmSurface**
- word **fileType**
- wallModel ∗ **ibmWallModelU**
- wallModel ∗ **ibmWallModelT**
- word **velocityBC**
- word **uBCSetType**
- PetscInt **wallFunctionTypeU**
- word **tempBC**
- word **tBCSetType**
- PetscInt **wallFunctionTypeT**
- PetscReal **fixedTemp**
- Cmpnts **baseLocation**
- word **bodyMotion**
- ibmRotation ∗ **ibmRot**
- ibmSineMotion ∗ **ibmSine**
- ibmPitchMotion ∗ **ibmPitch**
- PetscReal **searchCellRatio**
- boundingBox ∗ **bound**
- list ∗ **searchCellList**
- Cmpnts ∗ **ibmPForce**

  pressure force acting on each ibm element
- Cmpnts **procBoundCenter**

  center of the processor bounding box for the ibm body
- Cmpnts **procBoundSize**

  size of the processor bounding box in the x, y and z direction
- PetscInt ∗ **thisPtControlled**

  flag telling if a ibm mesh point is controlled by this processor based on normal projection
- PetscInt ∗ **thisPtControlTransfer**

  flag telling that the control of this point is being transferred to another processor
- cellIds ∗ **closestCells**

  closest cell id to the outward normal projection from the ibm mesh element
- PetscInt **ibmControlled**

  flag which tells if this proc controls this IBM body
- MPI_Comm **IBM_COMM**

  communicator for this IBM
- PetscInt ∗ **elementMapping**
- elementBox ∗ **eBox**

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/ibm.h

4.28 ibmPitchMotion Struct Reference

Public Attributes

- PetscReal **amplitude**
- PetscReal **frequency**
- PetscReal **initAngPosition**
- Cmpnts **pitchAxis**
- Cmpnts **pitchCenter**
- PetscReal **tPrev**

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/ibm.h

4.29 ibmRotation Struct Reference

Public Attributes

- PetscReal **angSpeed**
- PetscReal **angAcc**
- PetscReal **rotAngle**
- PetscReal **maxR**
- Cmpnts **rotAxis**
- Cmpnts **rotCenter**

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/ibm.h

4.30 ibmSineMotion Struct Reference

Public Attributes

- PetscReal **amplitude**
- PetscReal **frequency**
- Cmpnts **motionDir**
- PetscReal **tPrev**

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/ibm.h

4.31 inflowData Struct Reference

Struct defining inflow information.

```
#include <boundary.h>
```

Public Attributes

- PetscReal * **inflowTimes**

  array with the available times (stored at the beginning)
- PetscInt **nInflowTimes**

  number of times in the inflow database
- PetscInt **currentCloseIdx**

  save the current closest index at each iteration to speed up the interpolation search

4.31.1 Detailed Description

Struct defining inflow information.

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/boundary.h

4.32 inletFunctions Struct Reference

Struct storing inlet functions data.

```
#include <boundary.h>
```

Public Attributes

- inletFunctionTypes * **iLeft**
- inletFunctionTypes * **iRight**
- inletFunctionTypes * **jLeft**
- inletFunctionTypes * **jRight**
- inletFunctionTypes * **kLeft**
- inletFunctionTypes * **kRight**

4.32.1 Detailed Description

Struct storing inlet functions data.

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/boundary.h

4.33 inletFunctionTypes Struct Reference

Struct storing the inlet function for a patch.

```
#include <boundary.h>
```

Public Attributes

- PetscInt **typeU**
- PetscInt **typeT**
- PetscInt **typeNut**
- Cmpnts **Uref**
- PetscReal **Href**
- PetscReal **uPrimeRMS**
- Cmpnts **Udir**

  velocity direction vector
- PetscReal **roughness**

  equivalent roughness size
- PetscReal **hInv**

  height from bottom patch of inversion layer
- PetscReal **uTau**

  friction velocity used to compute log profile
- PetscReal **dInv**

  inversion width
- PetscReal **gInv**

  delta T across inversion layer
- PetscReal **tRef**

  reference potential temperature
- PetscReal **gTop**

  temperature gradient above the inversion layer
- PetscReal **gABL**

  temperature gradient below the inversion layer
- PetscReal **smear**

  Rampanelli Zardi model parameter.
- PetscReal **latitude**

  for Nieuwstadt model
- PetscReal **fc**

  Coriolis parameter.
- PetscInt **n1**

  number of inflow cells along the 1st index (ordered as accessed: k,j,i. Eg. for k-normal patch this is n cells along j)
- PetscInt **n2**

  number of inflow cells along the 2nd index (ordered as accessed: k,j,i. Eg. for k-normal patch this is n cells along i)
- PetscInt **n1wg**

  n1 with ghosts (n1 + 2)
- PetscInt **n2wg**

  n2 with ghosts (n2 + 2)
- PetscInt **prds1**

  indicates how many times inflow data should be periodized along 1st index
- PetscInt **prds2**

  indicates how many times inflow data should be periodized along 2nd index

- PetscInt **merge1**

  average data at 5 top cells
- PetscInt **shift2**

  apply shift on inflow slice data along direction 2
- PetscInt **mapT**

  flag telling if also T is mapped (if temperatureTransport is active is mandatory)
- PetscInt **mapNut**

  flag telling if also nut is mapped (optional)
- PetscReal $***$ **inflowWeights**

  array of 4 weights for each cell center (ordered as a 2D plane array)
- cellIds $***$ **closestCells**

  array of 4 closest cells for each cell center
- PetscReal $***$ **inflowWeights_1**

  array of 6 weights for each cell center (ordered as a 2D plane array) - j spline interp
- cellIds $***$ **closestCells_1**

  array of 6 closest cells for each cell center - j spline interp
- PetscReal $***$ **inflowWeights_2**

  array of 6 weights for each cell center (ordered as a 2D plane array) - i spline interp
- cellIds $***$ **closestCells_2**

  array of 6 closest cells for each cell center - i spline interp
- word **sourceType**

  source mesh type (uniform or grading)
- word **interpMethod**

  interpolation method (linear or nullDiv)
- PetscReal **width1**

  inflow cell width in the 1st direction
- PetscReal **width2**

  inflow cell width in the 2nd direction
- PetscReal **inflowHeigth**

  inflow height for gradient extrapolation aloft
- Cmpnts $**$ **ucat_plane**

  cartesian velocity inflow data
- PetscReal $**$ **t_plane**

  temperature inflow data
- PetscReal $**$ **nut_plane**

  nut inflow data
- inflowData **inflowU**

  velocity inflow database inflormation for unsteadyMappedInflow BC
- inflowData **inflowT**

  temperature inflow database inflormation for unsteadyMappedInflow BC
- inflowData **inflowNut**

  temperature inflow database inflormation for unsteadyMappedInflow BC
- Cmpnts $*$ **uBarAvgTopX**

  velocity average from inflow database at top 10 points
- PetscReal $*$ **tBarAvgTopX**

  temperature average from inflow database at top 10 points
- PetscReal $*$ **avgTopPointCoords**

  z coordinates of the 10 average top points
- PetscReal **avgTopDelta**

  length of the five top cells
- PetscReal **avgTopLength**

height of the inflow slices

- PetscReal **shiftSpeed**

  speed of the i-direction shift
- PetscReal ∗ **ycent**

  vector storing y cell center coordinates (assuming they don't vary vertically)
- PetscInt ∗ **yIDs**

  IDs for the right interpolation point (the left would be IDs[i]-1)
- PetscReal ∗ **yWeights**

  vector storing the right weight for the interpolation (left weight is 1.0 - right weight)
- MPI_Comm **IFFCN_COMM**

  communicator involving all processors touching k-boundaries
- PetscReal **amplitude**

  oscillation amplitude w.r.t. reference velocity magnitude
- PetscReal **periods**

  number of periods in the spanwise direction

4.33.1 Detailed Description

Struct storing the inlet function for a patch.

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/boundary.h

4.34 io_ Struct Reference

Struct defining io settings (simulation checkpointing)

```
#include <io.h>
```

Public Attributes

- word **intervalType**

  timeStep or adjustableTime
- PetscReal **timeInterval**

  in iterations if intervalType = timeStep, in seconds if intervalType = adjustableTime
- PetscInt **runTimeWrite**

  flag telling if must write at this iteration
- PetscInt **purgeWrite**

  deletes all other files after writing the current
- PetscInt **averaging**

  compute the time-averaged solution fields
- PetscReal **avgPrd**

  sampling period in seconds
- PetscReal **avgStartTime**

  start time of averaging procedure
- PetscInt **phaseAveraging**

  compute the phase-averaged solution fields

- PetscReal **phAvgPrd**

  sampling period in seconds
- PetscReal **phAvgStartTime**

  start time of phase averaging procedure
- PetscInt **keBudgets**

  compute kinetic energy budget terms
- PetscInt **writePForce**

  compute pressure on individual elements
- PetscInt **avgWeight**

  number of average snapshots (cumulated at runtime)
- PetscInt **pAvgWeight**

  number of phase average snapshots (cumulated at runtime)
- PetscInt **keAvgWeight**

  number of keBudget average snapshots (cumulated at runtime)
- PetscInt **qCrit**
- PetscInt **l2Crit**
- PetscInt **vgtQg**
- PetscInt **vgtRg**
- PetscInt **vgtQs**
- PetscInt **vgtRs**
- PetscInt **vgtQr**
- PetscInt **windFarmForce**
- PetscInt **sources**
- PetscInt **buoyancy**
- PetscInt **continuity**
- word **lastModification**
- PetscReal **timeIntervalCatalyst**
- PetscReal **startTimeCatalyst**
- word **outputTypeCatalyst**
- word **ioTypeCatalyst**

  type of catalyst output (generic/script)
- word **scriptNameCatalyst**

  name of input script
- access_ * **access**

4.34.1 Detailed Description

Struct defining io settings (simulation checkpointing)

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/io.h

4.35 keBox Struct Reference

Struct defining the ke fields.

```
#include <keAcquisition.h>
```

Public Attributes

- word * **name**
- Cmpnts **center**
- PetscReal **sizeX**
- PetscReal **sizeY**
- PetscReal **sizeZ**
- PetscInt **minKFace**
- PetscInt **maxKFace**
- PetscInt **minJFace**
- PetscInt **maxJFace**
- PetscInt **minIFace**
- PetscInt **maxIFace**
- PetscReal **avgDumK**
- PetscReal **avgDupK**
- PetscReal **avgDpmK**
- PetscReal **avgDppK**
- PetscReal **avgDumJ**
- PetscReal **avgDupJ**
- PetscReal **avgDpmJ**
- PetscReal **avgDppJ**
- PetscReal **avgDumI**
- PetscReal **avgDupI**
- PetscReal **avgDpmI**
- PetscReal **avgDppI**
- PetscReal **avgFK**
- PetscReal **avgFJ**
- PetscReal **avgFI**
- PetscReal **avgEps**
- PetscReal **avgPf**
- PetscReal **avgPtheta**
- PetscReal **avgPinf**
- PetscReal **avgErr**
- PetscReal **avgPturb**
- PetscReal **avgPsgs**
- PetscInt **thisBoxControlled**

  this processor controls this box
- MPI_Comm **KEBOX_COMM**

  communicator for this box
- PetscMPIInt **writerRank**

  label of master rank of the KEBOX_COMM communicator in the MPI_COMM_WORLD rank list
- PetscMPIInt **nProcsBox**

  size of the KEBOX_COMM communicator

4.35.1 Detailed Description

Struct defining the ke fields.

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/keAcquisition.h

4.36 keFields Struct Reference

Struct defining the ke fields.

```
#include <keAcquisition.h>
```

Public Attributes

- PetscReal **avgStartTime**
- PetscReal **avgPrd**
- PetscInt **cartesian**

  perform the budget in cartesian components (GCC otherwise)
- PetscInt **debug**
- PetscInt **nBox**

  number of boxes in the array
- keBox ** **box**

  array of boxes
- Vec **Error**

  error on the equation
- Vec **D**
- Vec **F**
- Vec **Pinf**

  mean background pressure power
- Vec **Pf**

  wind turbine power
- Vec **Ptheta**

  kinetic to potential energy conversion
- Vec **Eps**

  turbulent dissipation
- Vec **IEm**

  mechanical energy
- Vec **IDum**

  mean kinetic energy advection
- Vec **IDup**

  turbulent kinetic energy advection
- Vec **IF**

  turbulent fluxes
- Vec **IPturb**

  resolved turbulent production
- Vec **IPsgs**

  sgs turbulent production
- Vec **IVmTauSGS**

  advection of residual stresses
- Vec **IDpm**

  advecton of mean pressure work
- Vec **IDpp**

  advection of mean pressure fluct
- Vec **IUm**

  average of cartesian velocity
- Vec **IVmVpVp**

- Vec **IVpVpVp**
- Vec **IVDm**
- Vec **IVmPmG**
- Vec **IVpPpG**
- Vec **IDdVm**
- Vec **IVmCsi**
- Vec **IVmEta**
- Vec **IVmZet**
- Vec **IVm**
- Vec **IPm**
- Vec **IVpVpEta**
- Vec **IVpVpCsi**
- Vec **IVpVpZet**
- Vec **IVpVp**

4.36.1 Detailed Description

Struct defining the ke fields.

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/keAcquisition.h

4.37 les_ Struct Reference

struct storing LES model

```
#include <les.h>
```

Public Attributes

- LesModel **model**
- PetscReal **maxCs**
- PetscReal **amdCs**
- PetscInt **turbStat**
- Vec **ISx**
- Vec **ISy**
- Vec **ISz**
- Vec **IS**
- Vec **ILM**
- Vec **IMM**
- Vec **IQN**
- Vec **INN**
- Vec **ILM_old**
- Vec **IMM_old**
- Vec **IQN_old**
- Vec **INN_old**
- Vec **ICs**
- Vec **IN**
- Vec **ICh**

- Vec **L**

  length scale
- Vec **lNu_t**

  eddy viscosity
- Vec **lKsgs**
- Vec **lk_t**

  eddy diffusivity
- Vec **lCsk**
- Vec **ldTheta**
- Vec **lKX**
- Vec **lXX**
- Vec **lPY**
- Vec **lYY**
- Vec **lKX_old**
- Vec **lXX_old**
- Vec **lPY_old**
- Vec **lYY_old**
- Vec **lTau**

  tau sgs tensor for structural models
- Vec **lQ**

  q sgs vector for structural models
- word **initFieldType**
- access_ * **access**

  access database

4.37.1 Detailed Description

struct storing LES model

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/les.h

4.38 level3LM Struct Reference

Struct defining a gridded point mesh level for three layer model (used for its validation)

```
#include <ablAcquisition.h>
```

Public Attributes

- word **levelName**

  name of the mesh level
- Cmpnts **hStart**

  starting height for depth average
- Cmpnts **hEnd**

  ending height for depth average
- Cmpnts ** **U**

  depth average velocity averaged in time at the 3LM mesh points
- PetscReal ** **P**

  depth average pressure averaged in time at the 3LM mesh points

4.38.1 Detailed Description

Struct defining a gridded point mesh level for three layer model (used for its validation)

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/ablAcquisition.h

4.39 list Struct Reference

Node list.

```
#include <ibm.h>
```

Public Attributes

- node * **head**

4.39.1 Detailed Description

Node list.

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/ibm.h

4.40 LogLawAPG Struct Reference

Public Attributes

- PetscReal **roughness**
- PetscReal **kappa**

  von karman constant (usually 0.4)

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/wallmodel.h

4.41 mapInfo Struct Reference

concurrent precursor mapping info

```
#include <precursor.h>
```

Public Attributes

- PetscInt **kFringeStart**
- PetscInt **kFringeEnd**

  start and end ids of the fringe region in the successr indexing
- PetscInt **kStart**
- PetscInt **kEnd**

  start and end ids of this processor in the k direction in the successor indexing

4.41.1 Detailed Description

concurrent precursor mapping info

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/precursor.h

4.42 Matrix3 Struct Reference

Public Attributes

- PetscScalar **m** [3][3]

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/base.h

4.43 mesh_ Struct Reference

mesh structure storing mesh and curvilinear coordinates

```
#include <mesh.h>
```

Public Attributes

- word **meshName**

  name of the domain
- word **meshFileType**

  type of mesh file
- boundingBox **bounds**

  domain extensions and lengths information
- PetscReal **grndLevel**

  the ground level height - would be = bounds->zmin for normal simulation, but could change if using IBM
- PetscInt **IM**
- PetscInt **JM**
- PetscInt **KM**

  ncells in the GCC directions
- DM **da**

  data structure for scalars (include the grid geometry informaion, to obtain the mesh information, use DMDAGet↩ Coordinates)
- DM **fda**

  data Structure for vectors
- DM **sda**

  data Structure for symmetric tensors
- DM **tda**

  data Structure for generic tensors
- DMDALocalInfo **info**

  data struct that contains information about a the Distributed Array (essentially mesh information)
- Vec **Cent**
- Vec **ICent**

  cell centers coordinates in the cartesian frame
- Vec **ICsi**
- Vec **IEta**
- Vec **IZet**
- Vec **IAj**

  jacobian evaluated at cell centers (1 / cell volume)
- Vec **IICsi**
- Vec **IIEta**
- Vec **IIZet**
- Vec **IIAj**

  jacobian evaluated at the i-faces
- Vec **IJCsi**
- Vec **IJEta**
- Vec **IJZet**
- Vec **IJAj**

  jacobian evaluated at the j-faces
- Vec **IKCsi**
- Vec **IKEta**
- Vec **IKZet**
- Vec **IKAj**

  jacobian evaluated at the j-faces
- Vec **Nvert**
- Vec **Nvert_o**

  solid body field for IBM
- Vec **INvert**

- Vec **INvert_o**
- Vec **meshTag**
- Vec **lmeshTag**
- PetscInt **i_periodic**
- PetscInt **ii_periodic**
- PetscInt **j_periodic**
- PetscInt **jj_periodic**
- PetscInt **k_periodic**
- PetscInt **kk_periodic**
- scalarBC **boundaryNut**
- scalarBC **boundaryT**
- vectorBC **boundaryU**
- inletFunctions **inletF**
- Vec **fluxLimiter**
- MPI_Comm **MESH_COMM**
- access_ ∗ **access**

4.43.1 Detailed Description

mesh structure storing mesh and curvilinear coordinates

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/mesh.h

4.44 ModeData Struct Reference

Public Attributes

- PetscReal **kappaM**
- PetscReal **khatx**
- PetscReal **khaty**
- PetscReal **khatz**
- PetscReal **amp**
- PetscReal **phi**
- PetscReal **ktx**
- PetscReal **kty**
- PetscReal **ktz**
- PetscReal **sx**
- PetscReal **sy**
- PetscReal **sz**

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/initialField.h

4.45 node Struct Reference

Node struct.

```
#include <ibm.h>
```

Public Attributes

- PetscInt **Node**
- struct node ∗ **next**

  node id

4.45.1 Detailed Description

Node struct.

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/ibm.h

4.46 OctreeNode Struct Reference

Public Member Functions

- **OctreeNode** (Cmpnts minB, Cmpnts maxB)

Public Attributes

- PetscInt **hasCells**
- PetscInt **imin**
- PetscInt **imax**
- PetscInt **jmin**
- PetscInt **jmax**
- PetscInt **kmin**
- PetscInt **kmax**
- Cmpnts **minBounds**
- Cmpnts **maxBounds**
- OctreeNode ∗ **children** [8]

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/overset.c

4.47 overset_ Struct Reference

Main overset structure.
This structure holds the parameters, connectivity, and interpolation data for both the overset (child) mesh and the base (background) mesh update.

```
#include <overset.h>
```

Public Attributes

- labelList **parentMeshId**

  List of parent (background) domains.
- labelList **childMeshId**

  List of attached overset (child) domains.
- word **interpolationType**

  Interpolation method ("trilinear", "inverseDist", "LS1", etc.)
- PetscInt **dynamicOverset**

  Flag for dynamic overset (nonzero if active)
- PetscInt **procChange**

  Flag if background processor intersections have changed.
- oversetMotion * **oMotion**

  Pointer to overset motion data.
- std::vector< std::vector< PetscInt > > **AcellProcMatDb**

  Processor connectivity matrix for acceptor cells.
- std::vector< std::vector< PetscInt > > **AcellProcMatHc**

  Processor connectivity matrix for acceptor cells.
- std::vector< PetscInt > **NumAcellPerProcDb**

  Number of acceptor cells per processor.
- std::vector< PetscInt > **NumAcellPerProcHc**

  Number of acceptor cells per processor.
- std::vector< Acell > **aCellDb**

  List of acceptor cells.
- std::vector< Acell > **aCellHc**

  List of acceptor cells.
- std::vector< Dcell > **closestDonorDb**

  For trilinear interpolation: closest donor cell per acceptor at domain boundary.
- std::vector< Dcell > **closestDonorHc**

  For trilinear interpolation: closest donor cell per acceptor at hole cut boundary.
- std::vector< std::vector< PetscReal > > **DWeights**

  MLS weights for overset interpolation.
- std::vector< std::vector< Dcell > > **dCell**

  List of donor cell.
- std::vector< MPI_Comm > **oset_comm**
- PetscReal **cellAvg**

  Average cell size (optional)
- PetscReal **cellFactor**

  Factor to scale cell size for search radius.
- ibm_ * **oibm**
- access_ * **access**

4.47.1 Detailed Description

Main overset structure.
This structure holds the parameters, connectivity, and interpolation data for both the overset (child) mesh and the base (background) mesh update.

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/overset.h

4.48 oversetMotion Struct Reference

Overset motion structure (for dynamic overset motion)

```
#include <overset.h>
```

Public Attributes

- Cmpnts **prescribedVel**

  prescribed translational velocity
- word **motionType**

  type of motion ("Translation", "Rotation", etc.)
- PetscInt **setMotion**

  if nonzero, use the prescribed motion
- PetscInt **ibmAttached**

  if nonzero, use IBM–based motion instead

4.48.1 Detailed Description

Overset motion structure (for dynamic overset motion)

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/overset.h

4.49 patchVectorField Struct Reference

Vector field on the patch (used to store wall models fields)

```
#include <base.h>
```

Public Attributes

- PetscReal ** **x**
- PetscReal ** **y**
- PetscReal ** **z**

4.49.1 Detailed Description

Vector field on the patch (used to store wall models fields)

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/base.h

4.50 peqn_ Struct Reference

struct storing pressure equation

```
#include <peqn.h>
```

Public Attributes

- Vec **phi**

  phi is the actual solution of Poisson equation (then converted in Phi)
- Vec **Phi**
- Vec **lPhi**

  pressure correction for frac. step method
- Vec **P**
- Vec **lP**

  pressure at current and previous time step
- Vec **lLid**
- Vec **lGid**

  matrixFree - matrixBased connectivity (used to build the poisson coeff. matrix)
- HYPRE_Int **thisRankSize**

  number of cells owned by this processore (used to build the poisson coeff. matrix)
- HYPRE_Int **thisRankStart**

  first cell ID owned by this processor in global indexing (used to build the poisson coeff. matrix)
- HYPRE_Int **thisRankEnd**

  last cell ID owned by this processor in global indexing (thisRankStart + thisRankSize - 1)
- HYPRE_Int **totalSize**

  total number of cells (excluding physical ghost & IBM points, depending of the Hypre Poisson type, -1, -2, 1)
- PetscReal **initialPoissonRes**
- PetscReal **finalPoissonRes**

  initial and final residual of the Poisson iteration
- HYPRE_Int **poissonIterations**

  number of Poisson iterations
- PetscInt **hypreSolverType**

  1: GMRES, 2: PCG
- PetscInt **poissonIt**

  max number of poisson iterations per timestep
- PetscReal **poissonTol**

  relative exit tolerance
- PetscInt **amgAgg**

  aggresive coarsening is good for > 50mil grids
- PetscInt **amgCoarsenType**

- PetscReal **amgThresh**

  threshold value - 0.5 : Cartesian, 0.6 : Distorted
- word **solverType**

  HYPRE or PETSc.
- HYPRE_Solver **hypreSlvr**

  solver
- HYPRE_Solver **hyprePC**

  preconditioner
- HYPRE_IJMatrix **hypreA**

  coefficient matrix
- HYPRE_ParCSRMatrix **hypreParA**

  coefficient matrix
- HYPRE_IJVector **hypreP**
- HYPRE_IJVector **hypreRhs**

  unknwon and RHS
- HYPRE_ParVector **hypreParP**
- HYPRE_ParVector **hypreParRhs**

  unknown and RHS
- Mat **petscA**

  coefficient matrix
- KSP **ksp**

  linear krylov-subspace context
- PC **petscPC**

  preconditioner
- Vec **petscRhs**

  right hand side
- MatNullSpace **petscNs**

  null space
- access_ * **access**

  access database
- word **initFieldType**

  initial pressure field

4.50.1 Detailed Description

struct storing pressure equation

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/peqn.h

4.51 perturbFields Struct Reference

Struct defining the ke fields.

```
#include <perturbAcquisition.h>
```

Public Attributes

- PetscReal **avgStartTime**
- PetscReal **avgPrd**
- PetscReal **avgWindowTime**
- Cmpnts ** **bkgU**
- PetscReal ** **bkgP**
- PetscReal ** **bkgT**
- PetscReal **xSample**
- PetscInt **kSample**
- PetscInt **avgWeight**
- Vec **pertU**
- Vec **pertP**
- Vec **pertT**
- PetscInt **read**
- PetscInt **initialized**
- PetscInt **movAvgActive**

4.51.1 Detailed Description

Struct defining the ke fields.

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/perturbAcquisition.h

4.52 postProcess Struct Reference

Structure defining the variables for postProcessing.

```
#include <tosca2PV.h>
```

Public Attributes

- PetscInt **postProcessFields**
- PetscInt **writeRaster**
- PetscInt **samplingSections**
- PetscInt **postProcessPrecursor**
- precursor_ * **precursor**

4.52.1 Detailed Description

Structure defining the variables for postProcessing.

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/tosca2PV.h

4.53 PowerLawAPG Struct Reference

structure storing the Shumann wall models information

```
#include <wallmodel.h>
```

Public Attributes

- PetscReal **roughness**
- PetscReal **kappa**

  von karman constant (usually 0.4)

4.53.1 Detailed Description

structure storing the Shumann wall models information

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/wallmodel.h

4.54 precursor_ Struct Reference

concurrent precursor database

```
#include <precursor.h>
```

Public Attributes

- PetscInt **thisProcessorInFringe**

  1: this processors will solve, 0: this processor will idle
- mapInfo **map**

  precursor/successor mapping info
- domain_ * **domain**

4.54.1 Detailed Description

concurrent precursor database

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/precursor.h

4.55 probeRake Struct Reference

Struct defining a probe rake.

```
#include <probesAcquisition.h>
```

Public Attributes

- word **rakeName**

  name of the probes rake
- word **timeName**

  name of the time directory where rake data are written
- PetscInt **probesNumber**

  number of probes in this rake
- word **intervalType**

  can be timeStep or adjustableTime
- PetscReal **timeStart**

  start time of acquisition system
- PetscReal **timeInterval**

  acquisition time interval (overrides simulation time step if smaller)
- PetscInt **Uflag**

  flag telling if velocity field must be acquired
- PetscInt **Tflag**

  flag telling if fempearature field must be acquired
- PetscInt **Pflag**

  flag telling if pressure field must be acquired
- PetscReal $**$ **locations**

  array of [n, 3] where n is the number of probes in the rake and 3 are the probe coords
- PetscInt $**$ **cells**

  array of [n, 3] where n is the number of probes in the rake and 3 are cell indices of the computational pt closes to the probe
- PetscInt $*$ **owner**

  array of [n] where n is the number of probes in the rake and every element is the rank owning the probe
- PetscInt $*$ **domainID**

  number of the domain which controls this probe
- PetscInt **thisRakeControlled**

  this processor controls this rake (totally or partially)
- MPI_Comm **RAKE_COMM**

  communicator for this probe rake

4.55.1 Detailed Description

Struct defining a probe rake.

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/probesAcquisition.h

4.56 rakes Struct Reference

Struct defining all rakes in the mesh.

```
#include <probesAcquisition.h>
```

Public Attributes

- PetscInt **nRakes**

  number of probe rakes
- probeRake * **rakes**

  array storing all probe rakes in the domain
- PetscInt **allSameIO**

  accelerates the time-step check if all probes have the same IO settings (automatic)
- PetscInt **verbose**

  prints probe information

4.56.1 Detailed Description

Struct defining all rakes in the mesh.

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/probesAcquisition.h

4.57 scalarBC Struct Reference

Structure dafining the type of boundary conditions for a scalar.

```
#include <boundary.h>
```

Public Attributes

- word **iLeft**
- word **iRight**

  type of boundary condition
- word **jLeft**
- word **jRight**
- word **kLeft**
- word **kRight**
- PetscInt **iLeftPatchType**

  patch type (0 is open, 1 is wall)
- PetscInt **iRightPatchType**
- PetscInt **jLeftPatchType**
- PetscInt **jRightPatchType**
- PetscInt **kLeftPatchType**
- PetscInt **kRightPatchType**
- PetscReal **iLval**

- PetscReal **iRval**

  value (defined if BC prefix is 'fixed' only)
- PetscReal **jLval**
- PetscReal **jRval**
- PetscReal **kLval**
- PetscReal **kRval**
- PetscInt **iLWF**
- PetscInt **iRWF**

  wall function type (defined for velocity BC only)
- PetscInt **jLWF**
- PetscInt **jRWF**
- PetscInt **kLWF**
- PetscInt **kRWF**

4.57.1 Detailed Description

Structure dafining the type of boundary conditions for a scalar.

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/boundary.h

4.58 searchBox Struct Reference

Public Attributes

- PetscInt **ncx**
- PetscInt **ncy**
- PetscInt **ncz**

  number of search cells in x,y and z direction
- PetscReal **dcx**
- PetscReal **dcy**
- PetscReal **dcz**

  search cell size in x,y and z direction

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/ibm.h

4.59 sections Struct Reference

Struct defining a generic section to save.

```
#include <sectionsAcquisition.h>
```

Public Attributes

- PetscInt **available**

  does this struct contain data?
- PetscInt **nSections**

  number of sections in k,j,i-direction
- PetscReal **timeStart**

  start time of acquisition system
- word **intervalType**

  timeStep if sample at every time step, adjustableTime to enforce acquisition time interval
- PetscReal **timeInterval**

  acquisition time interval (overrides simulation time step if smaller), not used if intervalType = timeStep
- PetscInt $*$ **indices**
- PetscReal $*$ **coordinates**

  cutting coordinate of the section
- symmTensor $**$ **symmTensorSec**

  temporary saves symmetric tensor section data
- Cmpnts $**$ **vectorSec**

  temporarly saves vector section data
- PetscReal $**$ **scalarSec**

  temporarly saves scalar section data

4.59.1 Detailed Description

Struct defining a generic section to save.

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/sectionsAcquisition.h

4.60 Shumann Struct Reference

structure storing the Shumann wall models information for U and T

```
#include <wallmodel.h>
```

Public Attributes

- word **wfEvalType**

  type of uStar evaluation localized/averaged
- PetscReal **kappa**

  von karman constant (usually 0.4)
- PetscReal **thetaRef**

  reference potential temperature
- PetscReal **roughness**

  equivalent roughness height
- PetscReal **gammaM**

  momentum gammaM from Paulson 1970 (stable BL)

- PetscReal **betaM**

  momentum betaM from Pauslon 1970 (unstable BL)
- PetscReal **gammaH**

  pot. temp. gammaH from Paulson 1970 (stable BL)
- PetscReal **betaH**

  pot. temp. betaH from Paulson 1970 (unstable BL)
- PetscReal **alphaH**

  pot. temp. alphaH from Paulson 1970 (stable BL)
- PetscReal **tLast**

  time at which the last theta update was done
- PetscReal **heatingRate**

  surface heating rate
- PetscReal ∗∗ **surfaceTheta**

  surface temperature
- PetscInt **surfaceThetaSet**

  surface temperature has been initialized
- PetscReal **qWall**

  prescribed wall heat flux
- PetscReal ∗ **surfTemp**
- PetscReal ∗ **surfL**
- PetscReal ∗ **timeVec**
- PetscInt **numT**

4.60.1 Detailed Description

structure storing the Shumann wall models information for U and T

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/wallmodel.h

4.61 simInfo_ Struct Reference

Public Attributes

- PetscInt **nDomains**
- PetscInt **periodic**
- PetscInt **nHoleRegions**

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/base.h

4.62 skewSymmTensor Struct Reference

Public Attributes

- PetscScalar **xy**
- PetscScalar **xz**
- PetscScalar **yz**

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/base.h

4.63 surface Struct Reference

Public Attributes

- word **surfaceName**
- word **surfaceFileType**
- PetscInt **surfaceId**
- Cmpnts **baseLocation**
- word **elementSet**
- PetscInt ∗ **elementMapping**
- ibmMesh ∗ **ibMsh**

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/ibm.h

4.64 symmTensor Struct Reference

Defines the components of a symmetric tensor.

```
#include <base.h>
```

Public Attributes

- PetscScalar **xx**
- PetscScalar **xy**
- PetscScalar **xz**
- PetscScalar **yy**
- PetscScalar **yz**
- PetscScalar **zz**

4.64.1 Detailed Description

Defines the components of a symmetric tensor.

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/base.h

4.65 Tensor Struct Reference

Public Attributes

- PetscScalar **xx**
- PetscScalar **xy**
- PetscScalar **xz**
- PetscScalar **yx**
- PetscScalar **yy**
- PetscScalar **yz**
- PetscScalar **zx**
- PetscScalar **zy**
- PetscScalar **zz**

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/base.h

4.66 TensorInvariants Struct Reference

Public Attributes

- PetscScalar **I1**
- PetscScalar **I2**
- PetscScalar **I3**

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/base.h

4.67 teqn_ Struct Reference

struct storing temperature equation

```
#include <teqn.h>
```

Public Attributes

- word **solverType**

  solver type (kept for logging)
- SNES **snesT**

  non linear matrix free context
- Mat **JT**

  non linear matrix free preconditioner
- Mat **AT**
- Mat **CT**
- KSP **ksp**

  linear krylov-subspace context (backwardEuler/BDF2 SNES inner KSP)
- PC **pc**
- KSP **kspIMEX**

  standalone direct KSP for IMEX schemes
- Mat **JvIMEX**

  MatShell for IMEX operator A∗v.
- word **kspType**

  KSP solver type for SNES/IMEX.
- PetscInt **gmresRestart**

  GMRES restart parameter for SNES/IMEX.
- Vec **Rhs**

  rhs of the temperature equation
- Vec **bT**

  rhs of the linear system in IMEX schemes
- Vec **RhsConv**

  convective RHS at current time (for AB2 blending in ABBE)
- Vec **RhsConv_o**

  convective RHS at previous time (for AB2 blending in ABBE)
- Vec **TmprtTmp**

  temporary solution
- Vec **Tmprt**
- Vec **lTmprt**
- Vec **Tmprt_o**
- Vec **lTmprt_o**
- Vec **Tmprt_oo**

  previous solution for BDF2
- Vec **lDivT**
- Vec **lViscT**
- Vec **lViscIBMT**

  viscous and divergence temperature equation fluxes
- Vec **sourceT**

  temperature sources
- Vec **lRhoK**

  rhok / rho0 field
- Vec **ghGradRhok**

  buoyancy term for momentum equation (current step)
- Vec **ghGradRhok_o**

  buoyancy term for momentum equation (previous step, for AB2)
- PetscReal **absExitTol**

  absolute exit tolerance
- PetscReal **relExitTol**

relative exit tolerance

- word **ddtScheme**

  time derivative scheme
- PetscReal **hyperVisc4i**

  IMEX biharmonic hyperviscosity along i () index direction (-imexHyperVisc4U_i, default 0)
- PetscReal **hyperVisc4j**

  IMEX biharmonic hyperviscosity along j () index direction (-imexHyperVisc4U_j, default 0)
- PetscReal **hyperVisc4k**

  IMEX biharmonic hyperviscosity along k () index direction, typically vertical (-imexHyperVisc4U_k, default 0)
- wallModel ∗ **iLWM**

  wall model on the i-left patch
- wallModel ∗ **iRWM**

  wall model on the i-right patch
- wallModel ∗ **jLWM**

  wall model on the j-left patch
- wallModel ∗ **jRWM**

  wall model on the j-right patch
- PetscInt **pTildeFormulation**

  buoyancy term is expressed as gradient in momentum
- word **initFieldType**
- access_ ∗ **access**

  access database

4.67.1 Detailed Description

struct storing temperature equation

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/teqn.h

4.68 udSections Struct Reference

Public Attributes

- PetscInt **available**

  does this struct contain data?
- PetscInt **nSections**

  number of sections in k,j,i-direction
- uSections ∗∗ **uSection**

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/sectionsAcquisition.h

4.69 ueqn_ Struct Reference

structure storing momentum equation

```
#include <ueqn.h>
```

Public Attributes

- word **solverType**

  solver type (kept for logging)
- SNES **snesU**

  non linear matrix free context for momentum equation
- Mat **JU**

  non linear matrix free preconditioner
- Mat **A**
- Mat **C**
- KSP **ksp**

  linear krylov-subspace context (crankNicholson SNES inner KSP)
- PC **pc**
- KSP **kspIMEX**

  standalone direct KSP for IMEX scheme (no SNES wrapper)
- Mat **JvIMEX**

  MatShell for IMEX operator A∗v = v - dt∗scale∗Visc(v)
- PetscInt **snesMaxIter**

  max SNES outer iterations (-snesMaxItersU in control.dat)
- word **kspType**

  KSP solver type: BiCGStab or GMRES (-kspTypeU in control.dat)
- PetscInt **gmresRestart**

  GMRES restart parameter (-kspGMRESRestartU in control.dat, default 30)
- Vec **Utmp**

  temporary solution passed to the SNES/KSP evaluation function
- Vec **Rhs**

  rhs of the momentum equation (stores transport and viscous fluxes), low level use in FormU
- Vec **Rhs_o**

  rhs of the momentum equation at previous time step (used by many time schemes)
- Vec **lFp**

  rhs of the momentum equation prior to dotting with curv. coords basis (becomes Rhs)
- Vec **lDiv1**
- Vec **lDiv2**
- Vec **lDiv3**

  Components of the convective term in momentum equation.
- Vec **lVisc1**
- Vec **lVisc2**
- Vec **lVisc3**

  Components of the viscous term in the momentum equation.
- Vec **lViscIBM1**
- Vec **lViscIBM2**
- Vec **lViscIBM3**

  Components of the viscous term in the momentum equation for IBM faces.
- Vec **dP**

  pressure term of the momentum equation

- Vec **dPAGW**

  pressure term of the momentum equation as calculated from provided atmopsheric gravity waves pressure
- Vec **bTheta**

  buoyancy field at current time (used for AB2 buoyancy term extrapolation)
- Vec **bTheta_o**

  buoyancy field at previous time (used for AB2 buoyancy term extrapolation)
- Vec **sourceU**

  source term to drive Uref at Zref with periodic BCs
- Vec **gCont**

  gravity vector in cuvilinear coordinates
- Vec **Ucont**
- Vec **lUcont**

  contravariant fluxes (contravariant velocity / J)
- Vec **Ucat**
- Vec **lUcat**

  cartesian velocity
- Vec **Ucont_o**

  contravariant fluxes at the previous time step (old solution, contains BCs)
- Vec **bU**

  precomputed constant-RHS contributions for CN, RHS of the linear system for IMEX schemes
- Vec **RhsVisc**

  viscous-part of the RHS at current step (used in IMEX schemes)
- Vec **RhsConv**

  convective-only RHS at current step (used by ABCN IMEX scheme to build AB2 explicit part)
- Vec **RhsConv_o**

  convective-only RHS at previous step (used by ABCN IMEX scheme to build AB2 explicit part)
- Vec **lUstar**
- Cmpnts **meanGradP**
- Cmpnts **uBulk**
- word **ddtScheme**

  time derivative scheme
- word **divScheme**

  divergence scheme
- word **viscScheme**
- PetscReal **relExitTol**

  relative exit tolerance for SNES/KSP solvers
- PetscReal **absExitTol**

  absolute exit tolerance for SNES/KSP solvers
- PetscInt **inviscid**

  inviscid run
- PetscInt **buoyancy**

  buoyancy term
- PetscInt **coriolis**

  coriolis term
- PetscInt **fringe**

  fringe region term
- PetscInt **centralDiv**

  linear divergence scheme
- PetscInt **central4Div**

  4th order central scheme
- PetscInt **centralUpwindDiv**

blending between linear and upwind scheme

- PetscInt **centralUpwindWDiv**

  blending between linear and upwind scheme for non-uniform mesh
- PetscInt **quickDiv**

  3rd order QUICK scheme
- PetscInt **weno3Div**

  3rd order WENO scheme
- PetscReal **hyperVisc**

  hyperviscocity parameter to add artificial diffusion - (b = 0 is blend between 3rd order upwind and 4th order, b = 1 is central 4th order, b = 0.8 is hybrid 3-4 scheme)
- PetscReal **hyperVisc4i**

  biharmonic hyperviscosity along csi direction
- PetscReal **hyperVisc4j**

  biharmonic hyperviscosity along eta direction
- PetscReal **hyperVisc4k**

  biharmonic hyperviscosity along zeta direction
- wallModel ∗ **iLWM**

  wall model on the i-left patch
- wallModel ∗ **iRWM**

  wall model on the i-right patch
- wallModel ∗ **jLWM**

  wall model on the j-left patch
- wallModel ∗ **jRWM**

  wall model on the j-right patch
- word **initFieldType**
- access_ ∗ **access**

  access database

4.69.1 Detailed Description

structure storing momentum equation

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/ueqn.h

4.70 uSections Struct Reference

Struct defining a user defined section not necessarily along curvilinear co-ordinates.

```
#include <sectionsAcquisition.h>
```

Public Attributes

- PetscInt **nodes**
- word **sectionName**
- Cmpnts ** **coor**
- cellIds ** **closestId**
- PetscInt ** **hasCoor**

  flag which sets if a given processor has this coordinate
- PetscReal **timeStart**

  start time of acquisition system
- word **intervalType**

  timeStep if sample at every time step, adjustableTime to enforce acquisition time interval
- PetscReal **timeInterval**

  acquisition time interval (overrides simulation time step if smaller), not used if intervalType = timeStep
- PetscInt **nx**
- PetscInt **ny**
- Cmpnts ** **vectorSec**

  temporarly saves vector section data
- PetscReal ** **scalarSec**

  temporarly saves scalar section data

4.70.1 Detailed Description

Struct defining a user defined section not necessarily along curvilinear co-ordinates.

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/sectionsAcquisition.h

4.71 vectorBC Struct Reference

Structure dafining the type of boundary conditions for a vector.

```
#include <boundary.h>
```

Public Attributes

- word **iLeft**
- word **iRight**

  type of boundary condition
- word **jLeft**
- word **jRight**
- word **kLeft**
- word **kRight**
- PetscInt **iLeftPatchType**

  patch type (0 is open, 1 is wall)
- PetscInt **iRightPatchType**
- PetscInt **jLeftPatchType**
- PetscInt **jRightPatchType**

- PetscInt **kLeftPatchType**
- PetscInt **kRightPatchType**
- Cmpnts **iLval**
- Cmpnts **iRval**

  value (defined if BC prefix is 'fixed' only)
- Cmpnts **jLval**
- Cmpnts **jRval**
- Cmpnts **kLval**
- Cmpnts **kRval**
- PetscInt **iLWF**
- PetscInt **iRWF**

  wall function type (defined for velocity BC only)
- PetscInt **jLWF**
- PetscInt **jRWF**
- PetscInt **kLWF**
- PetscInt **kRWF**

4.71.1 Detailed Description

Structure dafining the type of boundary conditions for a vector.

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/boundary.h

4.72 wallModel Struct Reference

wall models container

```
#include <wallmodel.h>
```

Public Attributes

- Shumann * **wmShumann**
- Cabot * **wmCabot**
- PowerLawAPG * **wmPowerLawAPG**
- LogLawAPG * **wmLogLawAPG**
- patchVectorField **tauWall**
- patchVectorField **qWall**
- patchVectorField **uFilt**
- PetscReal ** **uMeanMag**

  average velocity to compute filtering time scale
- PetscInt **uFiltSet**

  filtered velocity has been initialized

4.72.1 Detailed Description

wall models container

The documentation for this struct was generated from the following file:

- /zfs60/nobackup/stipa/software/TOSCA/src/include/wallmodel.h

Chapter 5

File Documentation

5.1 /zfs60/nobackup/stipa/software/TOSCA/src/abl.c File Reference

Contains atmospheric boundary layer definition functions.

```
#include "include/base.h"
#include "include/domain.h"
#include "include/io.h"
#include "include/inline.h"
#include "include/inflow.h"
```

Functions

- PetscErrorCode **InitializeABL** (abl_ ∗abl)

  Read from ABLProperties.dat and initialize the ABL parameters.
- PetscReal **NieuwstadtGeostrophicWind** (abl_ ∗abl)

  Evaluate geostrophic speed using Nieuwstadt model.
- PetscErrorCode **readMesoScaleVelocityData** (abl_ ∗abl)
- PetscErrorCode **readMesoScaleTemperatureData** (abl_ ∗abl)

  read the mesoscale driving velocity and potential temperature profile
- PetscErrorCode **findTimeHeightSeriesInterpolationWts** (abl_ ∗abl)
- PetscErrorCode **findTimeHeightSeriesInterpolationWtsT** (abl_ ∗abl)
- PetscErrorCode **findVelocityInterpolationWeightsOnePt** (abl_ ∗abl)
- PetscErrorCode **findVelocityInterpolationWeights** (abl_ ∗abl)
- PetscErrorCode **findTemperatureInterpolationWeights** (abl_ ∗abl)
- PetscErrorCode **initializeYDampingMapping** (abl_ ∗abl)
- PetscErrorCode **setWeightsYDamping** (abl_ ∗abl)
- PetscErrorCode **computeLSqPolynomialCoefficientMatrix** (abl_ ∗abl)
- PetscErrorCode **computeLSqPolynomialCoefficientMatrixT** (abl_ ∗abl)
- PetscErrorCode **findABLHeight** (abl_ ∗abl)
- PetscErrorCode **waveletTransformContinuousVector** (abl_ ∗abl, Cmpnts ∗srcPAIn, Cmpnts ∗srcPAOut, PetscInt sigLength)
- PetscErrorCode **waveletTransformContinuousScalar** (abl_ ∗abl, PetscReal ∗srcPAIn, PetscReal ∗src↩PAOut, PetscInt sigLength)

5.1.1 Detailed Description

Contains atmospheric boundary layer definition functions.

5.2 /zfs60/nobackup/stipa/software/TOSCA/src/acquisition.c File Reference

Acquisition functions definition.

```
#include "include/base.h"
#include "include/domain.h"
#include "include/io.h"
#include "include/inline.h"
#include "sources/ueqn_sources.h"
```

Functions

- PetscErrorCode **InitializeAcquisition** (domain_ *domain)

  Reads acquisition settings and calls individual iniitialization functions.
- PetscErrorCode **InitializeAcquisitionPrecursor** (domain_ *domain)

  Reads acquisition settings and calls individual iniitialization functions for concurrent precursor.
- PetscErrorCode **WriteAcquisition** (domain_ *domain)

  Write acquisition data.
- PetscErrorCode **averageFieldsInitialize** (acquisition_ *acquisition)

  Initialize average fields acquisition.
- PetscErrorCode **averageKEBudgetsInitialize** (acquisition_ *acquisition)

  Initializes ke budgets fields acquisition.
- PetscErrorCode **setKeBoundsAndComms** (mesh_ *mesh, keFields *ke)

  Search box bounds and define communicators.
- PetscErrorCode **readKeBoxArray** (keFields *ke)

  Read box array.
- PetscErrorCode **averageFields** (acquisition_ *acquisition)

  Average fields.
- PetscErrorCode **averageKEBudgets** (acquisition_ *acquisition)

  Compute MKE budgets.
- PetscErrorCode **boxCumulateKEBudgets** (acquisition_ *acquisition)

  Do box cumulation for mesh. energy budgets.
- PetscErrorCode **averageKEBudgetsCat** (acquisition_ *acquisition)

  Compute MKE budgets in cartesian form.
- PetscErrorCode **averageKEBudgetsCont** (acquisition_ *acquisition)

  Compute MKE budgets in generalized curvilinaer form w/o mesh stretching.
- PetscErrorCode **sectionsInitialize** (acquisition_ *acquisition)

  Initializes sections.
- PetscErrorCode **writeSections** (acquisition_ *acquisition)

  Writes down section data.
- PetscErrorCode **iSectionSaveVector** (mesh_ *mesh, sections *sec, PetscInt iplane, Vec &V, const char *fieldName)

  Saves i-section vector data.

- PetscErrorCode **jSectionSaveVector** (mesh_ *mesh, sections *sec, PetscInt jplane, Vec &V, const char *fieldName)

  Saves j-section vector data.
- PetscErrorCode **kSectionSaveVector** (mesh_ *mesh, sections *sec, PetscInt kplane, Vec &V, const char *fieldName)

  Saves k-section vector data.
- PetscErrorCode **userSectionSaveVector** (mesh_ *mesh, uSections *uSection, Vec &V, const char *field↩ Name)

  Saves k-section vector data.
- PetscErrorCode **iSectionSaveScalar** (mesh_ *mesh, sections *sec, PetscInt iplane, Vec &V, const char *fieldName)

  Saves i-section scalar data.
- PetscErrorCode **jSectionSaveScalar** (mesh_ *mesh, sections *sec, PetscInt jplane, Vec &V, const char *fieldName)

  Saves j-section scalar data.
- PetscErrorCode **kSectionSaveScalar** (mesh_ *mesh, sections *sec, PetscInt kplane, Vec &V, const char *fieldName)

  Saves k-section scalar data.
- PetscErrorCode **userSectionSaveScalar** (mesh_ *mesh, uSections *uSection, Vec &V, const char *field↩ Name)
- PetscErrorCode **ProbesInitialize** (domain_ *domain, PetscInt postProcessing)

  Initializes the array of probe rakes.
- PetscErrorCode **InitRakeFile** (probeRake *rake, const char *fieldName)

  Initialize probe rake file.
- PetscErrorCode **writeProbes** (domain_ *domain)

  Writes probes to file.
- PetscErrorCode **computeQCritIO** (acquisition_ *acquisition)

  Compute Q criteria for I/O in cartesian form.
- PetscErrorCode **computeVgtSecondInvariantIO** (acquisition_ *acquisition)

  Compute Qg criteria for I/O in cartesian form.
- PetscErrorCode **computeVgtThirdInvariantIO** (acquisition_ *acquisition)

  Compute Rg criteria for I/O in cartesian form.
- PetscErrorCode **computeStrainRateSecondInvariantIO** (acquisition_ *acquisition)

  Compute Qs criteria for I/O in cartesian form.
- PetscErrorCode **computeStrainRateThirdInvariantIO** (acquisition_ *acquisition)

  Compute Rs criteria for I/O in cartesian form.
- PetscErrorCode **computeQrIO** (acquisition_ *acquisition)

  Compute Qr criteria for I/O in cartesian form.
- PetscErrorCode **computeCoriolisIO** (acquisition_ *acquisition)

  Compute coriolis force for I/O in cartesian form.
- PetscErrorCode **computeDrivingSourceIO** (acquisition_ *acquisition)

  Compute driving pressure force for I/O in cartesian form.
- PetscErrorCode **computeXDampingIO** (acquisition_ *acquisition)

  Compute x damping force for I/O in cartesian form.
- PetscErrorCode **computeCanopyForceIO** (acquisition_ *acquisition)

  Compute side force for I/O in cartesian form.
- PetscErrorCode **computeVelocityDivergence** (acquisition_ *acquisition)

  Compute velocity divergence field for I/O.
- PetscErrorCode **perturbationABLInitialize** (acquisition_ *acquisition)

  Initializes ABL perturbation fields w.r.t. given reference.
- PetscErrorCode **averagePerturbationABL** (acquisition_ *acquisition)

  Performs perturbation averaging and writes to memory.

- PetscErrorCode **averaging3LMInitialize** (domain_ *domain)

  Initialize 3LM data structure.
- PetscErrorCode read3LMFields (acquisition_ *acquisition)

  Reads velocity and pressure averages (they don't use auxiliary fields like TBL, LBL, IBL)
- PetscErrorCode **writeAveraging3LM** (domain_ *domain)

  Performs 3LM average and write 3LM fields.
- PetscErrorCode **write3LMPoints** (acquisition_ *acquisition)

  Write 3LM points to file inside postProcessing/3LM/points.
- PetscErrorCode **write3LMFields** (acquisition_ *acquisition)

  Write 3LM fields to file inside postProcessing/3LM/.
- PetscErrorCode **findAvgLineIds** (acquisition_ *acquisition)

  Inizializes the closest cells and the owner of the 3LM mesh points.
- PetscErrorCode **averagingABLInitialize** (domain_ *domain)

  Initialize ABL data structure.
- PetscErrorCode **writeAveragingABL** (domain_ *domain)

  Write spatial averaged ABL statistics.

5.2.1 Detailed Description

Acquisition functions definition.

5.2.2 Function Documentation

5.2.2.1 read3LMFields()

```
PetscErrorCode read3LMFields (
            acquisition_ * acquisition)
```

Reads velocity and pressure averages (they don't use auxiliary fields like TBL, LBL, IBL)

read field

5.3 /zfs60/nobackup/stipa/software/TOSCA/src/boundary.c File Reference

Contains boundary conditions.

```
#include "include/base.h"
#include "include/domain.h"
#include "include/io.h"
#include "include/inline.h"
#include "include/inflow.h"
#include "include/wallfunctions.h"
```

Functions

- PetscErrorCode **readScalarBC** (const word &location, const word &field, scalarBC ∗bc)

  Reads boundary conditions for a scalar field.
- PetscErrorCode **readVectorBC** (const word &location, const word &field, vectorBC ∗bc)

  Reads bundary conditions for a vector field.
- PetscErrorCode **SetBoundaryConditions** (mesh_ ∗mesh)

  Set boundary conditions.
- PetscErrorCode **checkBCsAndSetPatchTypes** (mesh_ ∗mesh)

  Checks available boundary conditions.
- PetscErrorCode **SetPeriodicConnectivity** (mesh_ ∗mesh, word &meshFileName)

  Set periodicconnectivity type.
- PetscErrorCode **UpdateContravariantBCs** (ueqn_ ∗ueqn)

  Update contravariant fluxes boundary conditions.
- PetscErrorCode **UpdateCartesianBCs** (ueqn_ ∗ueqn)

  Update cartesian boundary conditions.
- PetscErrorCode **UpdateTemperatureBCs** (teqn_ ∗teqn)

  Update temperature boundary conditions.
- PetscErrorCode **UpdateNutBCs** (les_ ∗les)

  Update effective viscosity boundary conditions.
- PetscErrorCode **UpdatektBCs** (les_ ∗les)

  Update effective diffusivity boundary conditions.
- PetscErrorCode **UpdatePressureBCs** (peqn_ ∗peqn)

  Update pressure boundary conditions.
- PetscErrorCode **UpdatePhiBCs** (peqn_ ∗peqn)

  Update pressure correction boundary conditions.
- PetscErrorCode **UpdateWallModelsU** (ueqn_ ∗ueqn)

  Update wall model for specified wall shear stress.
- PetscErrorCode **UpdateWallModelsT** (teqn_ ∗teqn)

  Update wall model for specified wall heat flux.
- PetscErrorCode **UpdateImmersedBCs** (ibm_ ∗ibm)

  Update ibm boundary conditions.
- PetscErrorCode **SetWallModels** (ueqn_ ∗ueqn)

  Set wall models.
- PetscErrorCode **readSurfaceTempData** (Shumann ∗wm)

  read surface temperature and obhukhov length data

5.3.1 Detailed Description

Contains boundary conditions.

5.4 /zfs60/nobackup/stipa/software/TOSCA/src/clock.c File Reference

Contains simulation time function definitions.

```
#include "include/base.h"
#include "include/domain.h"
#include "include/io.h"
#include "include/inline.h"
```

Functions

- PetscErrorCode **adjustTimeStep** (domain_ *domain)
- PetscErrorCode **timeStepInfo** (domain_ *domain, clock_ *clock, PetscReal &dxByU_min, PetscReal &maxU, cellIds &maxUCell)

5.4.1 Detailed Description

Contains simulation time function definitions.

5.5 /zfs60/nobackup/stipa/software/TOSCA/src/ibm.c File Reference

Contains Immersed boundary method function definitions.

```
#include "include/base.h"
#include "include/domain.h"
#include "include/io.h"
#include "include/inline.h"
#include "include/wallfunctions.h"
#include "include/ibmInput.h"
```

Functions

- PetscErrorCode **InitializeIBM** (ibm_ *ibm)

  initialize ibm: top level function
- PetscErrorCode **UpdateIBM** (ibm_ *ibm)

  update imb: top level function
- PetscErrorCode **setIBMWallModels** (ibm_ *ibm)

  set IBM wall model type and properties
- PetscErrorCode **ComputeForceMoment** (ibm_ *ibm)
- PetscErrorCode **writeIBMForceData** (ibm_ *ibm, PetscInt b, PetscReal *gElemPressure, PetscReal net↩Moment, PetscReal ibmPower, Cmpnts PresForce, Cmpnts ViscForce, Cmpnts momentVector)

  writes the IBM force data for element and net force
- PetscErrorCode **UpdateIBMesh** (ibm_ *ibm)

  update the ibm mesh when the ibm body is moving
- PetscErrorCode **recomputeIBMeshProperties** (ibm_ *ibm, PetscInt b)
- PetscErrorCode **pitchingMotion** (ibm_ *ibm, PetscInt b)

  prescribe pitching oscillation motion for IBM body
- PetscErrorCode **sineMotion** (ibm_ *ibm, PetscInt b)

  prescribe sinusoidal motion for IBM body
- PetscErrorCode **rotateIBMesh** (ibm_ *ibm, PetscInt b)

  rotate the ibm mesh based on the angular speed input
- PetscErrorCode **writeIBMData** (ibm_ *ibm, PetscInt b)

  writes the angular position when the body is rotating
- PetscErrorCode **findInterceptionPoint** (ibm_ *ibm)

  find the interceptionPoint on the background mesh plane for curvib interpolation
- PetscErrorCode **interceptionPt** (Cmpnts pCoor, Cmpnts pc[9], Cmpnts eNorm, ibmFluidCell *ibF)

  interception point algorithm based on dividing the plane into 8 triangles

- PetscErrorCode **findClosestIBMElement2Solid** (ibm_ *ibm)
- PetscErrorCode **findClosestIBMElement** (ibm_ *ibm)

  find the closest ibm mesh element to a IBM fluid node
- PetscErrorCode **CurvibInterpolationTriangular** (ibm_ *ibm)
- PetscErrorCode **CurvibInterpolationInternalCell** (ibm_ *ibm)
- PetscErrorCode **CurvibInterpolation** (ibm_ *ibm)

  CURVIB normal projection interpolation algorithm.
- PetscErrorCode **CurvibInterpolationQuadratic** (ibm_ *ibm)
- PetscErrorCode **findIBMWallShearChester** (ibm_ *ibm)

  compute shear stress at faces close to the IBM
- PetscErrorCode **findIBMWallShear** (ibm_ *ibm)

  compute shear stress at faces close to the IBM
- PetscErrorCode **ibmSearch** (ibm_ *ibm)

  ibm fluid node search performed using ray tracing algorithm
- PetscErrorCode **MLSInterpolation** (ibm_ *ibm)

  MLS interpolation algorithm.
- PetscErrorCode **checkIBMexists** (ibm_ *ibm)
- PetscErrorCode **computeIBMElementNormal** (ibm_ *ibm)
- PetscErrorCode **findIBMControlledProcs** (ibm_ *ibm)

  find in which processors the ibm body belongs
- PetscErrorCode **initElementProjectionProcs** (ibm_ *ibm)

  create list of ibm element processors and their closest ibm fluid cell to the ibm element normal projection
- PetscErrorCode **IBMProjectionProcessorTransfer** (ibm_ *ibm)

  transfer ibm mesh element across processors based on their movement - for dynamic case
- PetscErrorCode **createProcessorBufferZones** (ibm_ *ibm)

  create inner and outer buffer zones for ibm mesh element parallelization
- PetscErrorCode **initElementProcs** (ibm_ *ibm)

  create list of ibm element processors and their closest ibm fluid cell to the ibm element
- PetscErrorCode **IBMElementProcessorTransfer** (ibm_ *ibm)

  transfer ibm mesh element across processors based on their movement - for dynamic case
- PetscErrorCode **elementBoundingSphere** (ibmObject *ibmBody)

  find element bounding sphere
- PetscErrorCode **findBodyBoundingBox** (ibm_ *ibm)

  find the bounding box around an ibm body
- PetscErrorCode **findFluidSupportNodes** (ibm_ *ibm)

  find the fluid nodes that act as support nodes to the given ibm fluid node
- PetscErrorCode **findIBMFluidCells** (ibm_ *ibm)

  find list of ibm fluid nodes within each processor
- PetscErrorCode **findIBMMeshSupportNodes** (ibm_ *ibm)

  find the ibm mesh nodes that act as support node to the given ibm fluid node
- PetscErrorCode **findSearchCellDim** (ibm_ *ibm)

  find the search cell (coarser mesh around the ibm) dimension
- PetscErrorCode **createSearchCellList** (ibm_ *ibm)

  insert ibm mesh elements as a list into the search cell that they belong to
- PetscErrorCode **destroyLists** (ibm_ *ibm)

  destroy the ibm search cell list
- void **insertIBMSupportNodes** (Cmpnts pt, cellIds sCell, ibmFluidCell *ibF, ibmObject *ibBody, searchBox *sBox)

  insert ibm nodes into the list of support nodes around an ibm fluid node
- PetscErrorCode **rayCastLocal** (Cmpnts p, Cmpnts p1, Cmpnts p2, Cmpnts p3, Cmpnts p4, ibmMesh *ibMsh, cellIds sCell, searchBox *sBox, boundingBox *ibBox, list *searchCellList, PetscInt &intersect)

ray casting algorithm in local positive cell neighbourhood

- PetscReal **rayCastingTest** (Cmpnts p, ibmMesh ∗ibMsh, cellIds sCell, searchBox ∗sBox, boundingBox ∗ib↩Box, list ∗searchCellList)

  ray casting algorithm

- bool **isSearchCellSupport** (PetscReal rad, Cmpnts pt, cellIds cID, boundingBox ∗ibBox, searchBox ∗sBox)

  inline function to check if a search cell is a support to an ibm fluid node

- Cmpnts **randomdirection** (Cmpnts p, cellIds sCell, boundingBox ∗ibBox, searchBox ∗sBox, PetscInt seed)

  inline function to generate a random direction along the z index of the search cell for the ray casting algorithm

- PetscInt **intsectElement** (Cmpnts p, Cmpnts dir, Cmpnts node1, Cmpnts node2, Cmpnts node3, PetscReal ∗t, PetscReal ∗u, PetscReal ∗v)

  inline function to check if the ray intersects an ibm element

- PetscBool **isLineTriangleInt** (Cmpnts p1, Cmpnts p2, ibmMesh ∗ibMsh, PetscInt e)

  inline function to check if the line intersects an ibm element

- PetscInt **isPointInTriangle** (Cmpnts p, Cmpnts p1, Cmpnts p2, Cmpnts p3, Cmpnts norm)

  check whether point p is inside the triangle - 3D

- PetscInt **ISInsideTriangle2D** (Cpt2D p, Cpt2D pa, Cpt2D pb, Cpt2D pc)

  check whether point p is inside the triangle - 2D

- PetscInt **ISSameSide2D** (Cpt2D p, Cpt2D p1, Cpt2D p2, Cpt2D p3)

  Check whether 2D point p is located on the same side of line p1p2.

- void **disP2Line** (Cmpnts p, Cmpnts p1, Cmpnts p2, Cmpnts ∗po, PetscReal ∗d, PetscReal ∗t)

  find projection of point on a line and the distance to it

- void **triangleIntp** (Cpt2D p, Cpt2D p1, Cpt2D p2, Cpt2D p3, ibmFluidCell ∗ibF)

  find the interpolation weights of a point inside a triangle from its nodes

- void **triangleIntpBg** (Cpt2D p, Cpt2D p1, Cpt2D p2, Cpt2D p3, ibmFluidCell ∗ibF)

  find the interpolation weights of a point inside a triangle from its nodes used for background grid

5.5.1 Detailed Description

Contains Immersed boundary method function definitions.

5.6 /zfs60/nobackup/stipa/software/TOSCA/src/ibmInput.c File Reference

Contains Immersed boundary method input and read function definitions.

```
#include "include/base.h"
#include "include/domain.h"
#include "include/io.h"
#include "include/inline.h"
#include "include/ibmInput.h"
```

Functions

- PetscErrorCode **readIBMProperties** (ibm_ ∗ibm)

  read the IBMProperties.dat file , IBM mesh files and allocate memory for the objects
- PetscErrorCode **readIBMObjectMesh** (ibm_ ∗ibm, PetscInt b)
- PetscErrorCode **combineMesh** (ibmObject ∗ibmBody)
- PetscErrorCode **readIBMBodyFileAbaqusInp** (ibmObject ∗ibmBody)

  read the ibm mesh in abaqus inp format
- PetscErrorCode readIBMSurfaceFileAbaqusInp (surface ∗ibmSurface)

  read the ibm mesh in abaqus inp format
- PetscErrorCode **readIBMBodyFileASCII** (ibmObject ∗ibmBody)

  read the ibm mesh in ASCII format
- PetscErrorCode **readIBMBodyFileGRD** (ibmObject ∗ibmBody)

  read the ibm mesh in GRD format
- PetscErrorCode **readIBMSurfaceFileGRD** (surface ∗ibmSurface)

  read the ibm mesh in .grd format
- PetscErrorCode **writeSTLFile** (ibm_ ∗ibm, PetscInt b)

  write the STL mesh
- PetscErrorCode **readIBMBodyFileSTL** (ibmObject ∗ibmBody)

  read the ibm mesh in stl format
- PetscErrorCode **readIBMBodyFileUCD** (ibmObject ∗ibmBody)

  read the ibm mesh in ucd format
- PetscErrorCode **readIBMBodyFileUCD2** (ibmObject ∗ibmBody)

  read the ibm mesh in ucd2 format (without 0 tri entries)
- PetscErrorCode **readIBMSurfaceFileUCD** (surface ∗ibmSurface)

  read the ibm mesh in ucd format
- PetscErrorCode **readIBMSurfaceFileUCD2** (surface ∗ibmSurface)

  read the ibm mesh in ucd2 format (without 0 tri entries)

5.6.1 Detailed Description

Contains Immersed boundary method input and read function definitions.

5.6.2 Function Documentation

5.6.2.1 readIBMSurfaceFileAbaqusInp()

```
PetscErrorCode readIBMSurfaceFileAbaqusInp (
            surface * ibmSurface)
```

read the ibm mesh in abaqus inp format

combine the mesh nodes and elements of different surface bodies into one ibm body

5.7 /zfs60/nobackup/stipa/software/TOSCA/src/include/abl.h File Reference

ABL-related object definition.

Classes

- struct abl_

Functions

- PetscErrorCode **InitializeABL** (abl_ *abl)

  Read from ABLProperties.dat and initialize the ABL parameters.
- PetscReal **NieuwstadtGeostrophicWind** (abl_ *abl)

  Evaluate geostrophic speed using Nieuwstadt model.
- PetscErrorCode **readMesoScaleTemperatureData** (abl_ *abl)

  read the mesoscale driving velocity and potential temperature profile
- PetscErrorCode **readMesoScaleVelocityData** (abl_ *abl)
- PetscErrorCode **findVelocityInterpolationWeights** (abl_ *abl)
- PetscErrorCode **findVelocityInterpolationWeightsOnePt** (abl_ *abl)
- PetscErrorCode **findTemperatureInterpolationWeights** (abl_ *abl)
- PetscErrorCode **initializeYDampingMapping** (abl_ *abl)
- PetscErrorCode **setWeightsYDamping** (abl_ *abl)
- PetscErrorCode **computeLSqPolynomialCoefficientMatrix** (abl_ *abl)
- PetscErrorCode **computeLSqPolynomialCoefficientMatrixT** (abl_ *abl)
- PetscErrorCode **findTimeHeightSeriesInterpolationWts** (abl_ *abl)
- PetscErrorCode **findTimeHeightSeriesInterpolationWtsT** (abl_ *abl)
- PetscErrorCode **findABLHeight** (abl_ *abl)
- PetscErrorCode **waveletTransformContinuousVector** (abl_ *abl, Cmpnts *srcPAIn, Cmpnts *srcPAOut, PetscInt sigLength)
- PetscErrorCode **waveletTransformContinuousScalar** (abl_ *abl, PetscReal *srcPAIn, PetscReal *srcPAOut, PetscInt sigLength)

5.7.1 Detailed Description

ABL-related object definition.

5.8 abl.h

Go to the documentation of this file.

```
00001
00003
00004 #ifndef _ABL_H_
00005 #define _ABL_H_
00006
00007 struct abl_
00008 {
00009     // flags
00010     PetscInt     controllerActive;
00011     PetscInt     controllerActiveT;
00012     PetscInt     coriolisActive;
00013
00014     // physical quantities
00015     PetscReal    uTau;
00016     PetscReal    hRough;
00017     Cpt2D        uRef;
00018     PetscReal    hRef;
00019     PetscReal    hGeo;
00020     PetscReal    hInv;
00021     PetscReal    dInv;
00022     PetscReal    gInv;
00023     PetscReal    gABL;
00024     PetscReal    tRef;
```

```
PetscReal    gTop;
PetscReal    vkConst;
PetscReal    smear;
PetscReal    fc;
PetscInt     perturbations;

PetscReal   *cellLevels;

// temperature controller
PetscReal    *tDes;
word         controllerTypeT;

// velocity controller (common)
word         controllerType;
word         controllerAction;
word         controllerActionT;
PetscReal    relax;
PetscReal    alpha;
PetscReal    timeWindow;
PetscReal    *totVolPerLevel;
PetscInt     *totCelPerLevel;
PetscInt     *closestLabels;
PetscReal    *levelWeights;
PetscReal    controllerMaxHeight;

// geostrophic damping for pressure controller
PetscInt     geostrophicDampingActive;
PetscInt     mesoScaleInputActive;

PetscReal    geoDampAvgDT;
Cmpnts       geoDampAvgS;
Cmpnts       geoDampUBar;
Cmpnts       *geoDampU;
PetscReal    geoDampAlpha;
PetscReal    geoDampH;
PetscReal    geoDampDelta;
PetscReal    geoDampC;
PetscReal    geoDampStart;
PetscReal    geoDampWindow;

// geostrophic controller
PetscInt     *closestLabelsGeo;
PetscReal    *levelWeightsGeo;
Cmpnts       uGeoBar;
PetscReal    refHubAngle;
PetscReal    geoAngle;
PetscInt     windAngleController;
Cmpnts       a,b;

// read and average
PetscInt     currentCloseIdx;
PetscInt     currentCloseIdxT;
PetscReal    sourceAvgStartTime;
PetscReal    **preCompSources;
PetscReal    ***timeHtSources;
PetscReal    ***timeHtSourcesT;
PetscInt     nSourceTimes;
Cmpnts       cumulatedSource;
Cmpnts       *cumulatedSourceHt;
PetscReal    avgTimeStep;

// z damping layer (Rayleigh damping)
PetscReal    zDampingStart;
PetscReal    zDampingEnd;
PetscReal    zDampingAlpha;
Cmpnts       *uBarMeanZ;
PetscInt     avgWeight;
PetscInt     zDampingAlsoXY;
PetscInt     zDampingXYType;

// kLeft damping layer
PetscReal    kLeftPatchDist;
PetscReal    kLeftDampingAlpha;
Cmpnts       kLeftDampingUBar;
PetscReal    kLeftDampingFilterHeight;
PetscReal    kLeftDampingFilterWidth;

// kRight damping layer
PetscReal    kRightPatchDist;
PetscReal    kRightDampingAlpha;
Cmpnts       kRightDampingUBar;
PetscReal    kRightDampingFilterHeight;
PetscReal    kRightDampingFilterWidth;

// x damping layer (recycling fringe region)
PetscReal    xDampingStart;
PetscReal    xDampingEnd;
```

```
PetscReal    xDampingDelta;
PetscReal    xDampingAlpha;

// x damping layer controller parameters
word         xDampingControlType;
PetscReal    advDampingXStart;
PetscReal    advDampingXEnd;
PetscReal    advDampingXDeltaStart;
PetscReal    advDampingXDeltaEnd;
PetscReal    xDampingTimeWindow;
PetscReal    xDampingVBar;
PetscReal    vEnd, vStart;
PetscReal    xDampingError;
PetscReal    xDampingLineSamplingYmin;
PetscReal    xDampingLineSamplingYmax;
PetscReal    xDampingTimeStart;
PetscReal    xDampingDeltaV;
PetscReal    xDampingCoeff;
PetscInt    *closestLabelsFringe;
PetscReal   *levelWeightsFringe;
PetscReal    advDampingYStart;
PetscReal    advDampingYEnd;
PetscReal    advDampingYDeltaStart;
PetscReal    advDampingYDeltaEnd;

// y damping layer (can only be applied together with the x damping layer)
PetscReal    yDampingStart;
PetscReal    yDampingEnd;
PetscReal    yDampingDelta;
PetscReal    yDampingAlpha;
PetscInt     yDampingNumPeriods;
Vec          uBarInstY;
Vec          tBarInstY;

PetscInt     numSourceProc;
PetscInt     *sourceProcList;
MPI_Comm     *yDamp_comm;
PetscMPIInt  *srcCommLocalRank;
cellIds      *srcMinInd;
cellIds      *srcMaxInd;
PetscInt     *isdestProc;
cellIds      **destMinInd;
cellIds      **destMaxInd;
PetscInt      *srcNumI;
PetscInt      *srcNumJ;
PetscInt      *srcNumK;
MPI_Request  *mapRequest;
Cmpnts       **velMapped;
PetscReal    **tMapped;
PetscInt     **closestKCell;
PetscReal    **wtsKCell;

// type of uBar computation
PetscInt     xFringeUBarSelectionType;
Cmpnts       **uBarInstX;
PetscReal    **tBarInstX;
PetscInt     **nProcsKLine;

// side force for fringe region testing
PetscReal    xStartCanopy, xEndCanopy;
PetscReal    yStartCanopy, yEndCanopy;
PetscReal    zStartCanopy, zEndCanopy;
PetscReal    cftCanopy;
Cmpnts       diskDirCanopy;

// turbulent flow initialization
PetscReal    zPeak;
PetscReal    deltaV;
PetscReal    deltaU;
PetscReal    Uperiods;
PetscReal    Vperiods;

// mesoscale input parameters
PetscReal    *timeV;
PetscReal    *hV;
PetscReal    *timeT;
PetscReal    *hT;
Cmpnts       **uMeso;
Cmpnts       *uGeoPrev;
PetscReal    **tMeso;

PetscInt     numhV;
PetscInt     numhT;
PetscInt     numtV;
PetscInt     numtT;

PetscInt     **velInterpIdx;
```

```
    PetscReal    **velInterpWts;

    PetscInt     **tempInterpIdx;
    PetscReal    **tempInterpWts;

    PetscInt     lowestIndV;
    PetscInt     lowestIndT;
    PetscInt     highestIndV;
    PetscInt     highestIndT;
    PetscInt     lMesoIndV;
    PetscInt     hMesoIndV;
    PetscReal    lowestSrcHt;

    Cmpnts       *luMean;
    Cmpnts       *guMean;
    Cmpnts       *srcPA;

    PetscInt     closestTimeIndV;
    PetscReal    closestTimeWtV;
    PetscInt     closestTimeIndT;
    PetscReal    closestTimeWtT;

    PetscInt     polyOrder;
    PetscInt     polyOrderT;
    word         wtDist;
    PetscReal    **polyCoeffM;
    PetscReal    **polyCoeffT;

    //averaging
    PetscInt     averageSource;
    PetscReal    currAvgtime;
    PetscReal    tAvgWindow;
    Cmpnts       *avgsrc;

    word         flType;
    word         flTypeT;
    PetscReal    *avgTotalStress;
    Cmpnts       *avgStress;
    PetscReal    *avgHeatFlux;
    PetscReal    hAvgTime;
    PetscReal    bottomSrcHtV;
    PetscReal    bottomSrcHtT;

    PetscReal    ablHt;
    PetscReal    ablHtStartTime;
    PetscReal    heatFluxSwitch;

    //continuous wavelet transform parameters
    PetscInt     kernelRadius;
    PetscReal    sigma;
    PetscReal    omega;

    //discrete wavelet transform parameters
    word        waveName;
    word        waveTMethod;
    word        waveExtn;
    word        waveConv;
    PetscInt    waveLevel;
    PetscInt    waveletBlend;

    // concurrent precursor
    precursor_    *precursor;

    // access database
    access_       *access;
};

#endif

PetscErrorCode InitializeABL(abl_ *abl);

PetscReal NieuwstadtGeostrophicWind(abl_ *abl);

PetscErrorCode readMesoScaleTemperatureData(abl_ *abl);

PetscErrorCode readMesoScaleVelocityData(abl_ *abl);

PetscErrorCode findVelocityInterpolationWeights(abl_ *abl);

PetscErrorCode findVelocityInterpolationWeightsOnePt(abl_ *abl);

PetscErrorCode findTemperatureInterpolationWeights(abl_ *abl);

PetscErrorCode initializeYDampingMapping(abl_ *abl);

PetscErrorCode setWeightsYDamping(abl_ *abl);

```

```
PetscErrorCode computeLSqPolynomialCoefficientMatrix(abl_ *abl);

PetscErrorCode computeLSqPolynomialCoefficientMatrixT(abl_ *abl);

PetscErrorCode findTimeHeightSeriesInterpolationWts(abl_ *abl);

PetscErrorCode findTimeHeightSeriesInterpolationWtsT(abl_ *abl);

PetscErrorCode findABLHeight(abl_ *abl);

PetscErrorCode waveletTransformContinuousVector(abl_ *abl, Cmpnts *srcPAIn, Cmpnts *srcPAOut, PetscInt sigLength);

PetscErrorCode waveletTransformContinuousScalar(abl_ *abl, PetscReal *srcPAIn, PetscReal *srcPAOut, PetscInt sigLength);

#if USE_PYTHON
    #ifdef __cplusplus
    extern "C" {
    #endif
        PetscErrorCode pywavedecVector(abl_ *abl, Cmpnts *src, Cmpnts *dest, PetscInt nlevels);
        PetscErrorCode pywavedecScalar(abl_ *abl, PetscReal *srcPAIn, PetscReal *srcPAOut, PetscInt sigLength);
    #ifdef __cplusplus
    }
    #endif
#endif
```

5.9 access.h

```


#ifndef _DATABASE_H_
#define _DATABASE_H_

struct access_
{
    clock_       *clock;
    simInfo_     *info;
    constants_   *constants;
    flags_       *flags;
    mesh_        *mesh;
    io_          *io;
    ueqn_        *ueqn;
    peqn_        *peqn;
    teqn_        *teqn;
    les_         *les;
    farm_        *farm;
    overset_     *os;
    abl_         *abl;
    acquisition_ *acquisition;
    ibm_         *ibm;
    PetscInt     *domainID;
};

#endif
```

5.10 /zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition.h File Reference

Acquisition object declaration.

```
#include "acquisition/averageAcquisition.h"
#include "acquisition/probesAcquisition.h"
#include "acquisition/sectionsAcquisition.h"
#include "acquisition/ablAcquisition.h"
#include "acquisition/keAcquisition.h"
#include "acquisition/perturbAcquisition.h"
```

Classes

- struct acquisition_

  Struct containing all acquisition data structures.

Functions

- PetscErrorCode **InitializeAcquisition** (domain_ *domain)

  Reads acquisition settings and calls individual iniitialization functions.
- PetscErrorCode **InitializeAcquisitionPrecursor** (domain_ *domain)

  Reads acquisition settings and calls individual iniitialization functions for concurrent precursor.
- PetscErrorCode **WriteAcquisition** (domain_ *domain)

  Write acquisition data.
- PetscErrorCode **ProbesInitialize** (domain_ *domain, PetscInt postProcessing=0)

  Initializes the array of probe rakes.
- PetscErrorCode **InitRakeFile** (probeRake *rake, const char *fieldName)

  Initialize probe rake file.
- PetscErrorCode **writeProbes** (domain_ *domain)

  Writes probes to file.
- PetscErrorCode **sectionsInitialize** (acquisition_ *acquisition)

  Initializes sections.
- PetscErrorCode **writeSections** (acquisition_ *acquisition)

  Writes down section data.
- PetscErrorCode **iSectionSaveVector** (mesh_ *mesh, sections *sec, PetscInt iplane, Vec &V, const char *fieldName)

  Saves i-section vector data.
- PetscErrorCode **jSectionSaveVector** (mesh_ *mesh, sections *sec, PetscInt jplane, Vec &V, const char *fieldName)

  Saves j-section vector data.
- PetscErrorCode **kSectionSaveVector** (mesh_ *mesh, sections *sec, PetscInt kplane, Vec &V, const char *fieldName)

  Saves k-section vector data.
- PetscErrorCode **userSectionSaveVector** (mesh_ *mesh, uSections *uSection, Vec &V, const char *field↩Name)

  Saves k-section vector data.
- PetscErrorCode **iSectionSaveScalar** (mesh_ *mesh, sections *sec, PetscInt iplane, Vec &V, const char *fieldName)

  Saves i-section scalar data.
- PetscErrorCode **jSectionSaveScalar** (mesh_ *mesh, sections *sec, PetscInt jplane, Vec &V, const char *fieldName)

  Saves j-section scalar data.
- PetscErrorCode **kSectionSaveScalar** (mesh_ *mesh, sections *sec, PetscInt kplane, Vec &V, const char *fieldName)

  Saves k-section scalar data.
- PetscErrorCode **userSectionSaveScalar** (mesh_ *mesh, uSections *uSection, Vec &V, const char *field↩Name)
- PetscErrorCode **averageFieldsInitialize** (acquisition_ *acquisition)

  Initialize average fields acquisition.
- PetscErrorCode **averageFields** (acquisition_ *acquisition)

  Average fields.
- PetscErrorCode **computeQCritIO** (acquisition_ *acquisition)

Compute Q criteria for I/O in cartesian form.

- PetscErrorCode **computeVgtSecondInvariantIO** (acquisition_ *acquisition)

  Compute Qg criteria for I/O in cartesian form.
- PetscErrorCode **computeVgtThirdInvariantIO** (acquisition_ *acquisition)

  Compute Rg criteria for I/O in cartesian form.
- PetscErrorCode **computeStrainRateSecondInvariantIO** (acquisition_ *acquisition)

  Compute Qs criteria for I/O in cartesian form.
- PetscErrorCode **computeStrainRateThirdInvariantIO** (acquisition_ *acquisition)

  Compute Rs criteria for I/O in cartesian form.
- PetscErrorCode **computeQrIO** (acquisition_ *acquisition)

  Compute Qr criteria for I/O in cartesian form.
- PetscErrorCode **computeCoriolisIO** (acquisition_ *acquisition)

  Compute coriolis force for I/O in cartesian form.
- PetscErrorCode **computeDrivingSourceIO** (acquisition_ *acquisition)

  Compute driving pressure force for I/O in cartesian form.
- PetscErrorCode **computeXDampingIO** (acquisition_ *acquisition)

  Compute x damping force for I/O in cartesian form.
- PetscErrorCode **computeCanopyForceIO** (acquisition_ *acquisition)

  Compute side force for I/O in cartesian form.
- PetscErrorCode **computeVelocityDivergence** (acquisition_ *acquisition)

  Compute velocity divergence field for I/O.
- PetscErrorCode **averageKEBudgetsInitialize** (acquisition_ *acquisition)

  Initializes ke budgets fields acquisition.
- PetscErrorCode **readKeBoxArray** (keFields *ke)

  Read box array.
- PetscErrorCode **setKeBoundsAndComms** (mesh_ *mesh, keFields *ke)

  Search box bounds and define communicators.
- PetscErrorCode **boxCumulateKEBudgets** (acquisition_ *acquisition)

  Do box cumulation for mesh. energy budgets.
- PetscErrorCode **averageKEBudgets** (acquisition_ *acquisition)

  Compute MKE budgets.
- PetscErrorCode **averageKEBudgetsCat** (acquisition_ *acquisition)

  Compute MKE budgets in cartesian form.
- PetscErrorCode **averageKEBudgetsCont** (acquisition_ *acquisition)

  Compute MKE budgets in generalized curvilinaer form w/o mesh stretching.
- PetscErrorCode **averaging3LMInitialize** (domain_ *domain)

  Initialize 3LM data structure.
- PetscErrorCode **writeAveraging3LM** (domain_ *domain)

  Performs 3LM average and write 3LM fields.
- PetscErrorCode **write3LMPoints** (acquisition_ *acquisition)

  Write 3LM points to file inside postProcessing/3LM/points.
- PetscErrorCode **write3LMFields** (acquisition_ *acquisition)

  Write 3LM fields to file inside postProcessing/3LM/.
- PetscErrorCode **findAvgLineIds** (acquisition_ *acquisition)

  Inizializes the closest cells and the owner of the 3LM mesh points.
- PetscErrorCode read3LMFields (acquisition_ *acquisition)

  Reads velocity and pressure averages (they don't use auxiliary fields like TBL, LBL, IBL)
- PetscErrorCode **perturbationABLInitialize** (acquisition_ *acquisition)

  Initializes ABL perturbation fields w.r.t. given reference.
- PetscErrorCode **averagePerturbationABL** (acquisition_ *acquisition)

  Performs perturbation averaging and writes to memory.

- PetscErrorCode **averagingABLInitialize** (domain_ *domain)

  Initialize ABL data structure.
- PetscErrorCode **writeAveragingABL** (domain_ *domain)

  Write spatial averaged ABL statistics.

5.10.1 Detailed Description

Acquisition object declaration.

5.10.2 Function Documentation

5.10.2.1 read3LMFields()

```
PetscErrorCode read3LMFields (
            acquisition_ * acquisition)
```

Reads velocity and pressure averages (they don't use auxiliary fields like TBL, LBL, IBL)

read field

5.11 acquisition.h

Go to the documentation of this file.

```
00001
00003
00004 #ifndef _ACQUISITION_H_
00005 #define _ACQUISITION_H_
00006
00007 #include "acquisition/averageAcquisition.h"
00008 #include "acquisition/probesAcquisition.h"
00009 #include "acquisition/sectionsAcquisition.h"
00010 #include "acquisition/ablAcquisition.h"
00011 #include "acquisition/keAcquisition.h"
00012 #include "acquisition/perturbAcquisition.h"
00013
00015 struct acquisition_
00016 {
00017     rakes         *probes;
00018     avgFields     *fields;
00019     keFields      *keBudFields;
00020     sections      *kSections;
00021     sections      *jSections;
00022     sections      *iSections;
00023     udSections    *userSections;
00024     dataABL       *statisticsABL;
00025     data3LM       *LM3;
00026     perturbFields *perturbABL;
00027
00028     // acquisition flags
00029     PetscInt      isProbesActive;
00030     PetscInt      isSectionsActive;
00031     PetscInt      isAverageABLActive;
00032     PetscInt      isAverage3LMActive;
00033     PetscInt      isPerturbABLActive;
00034
00035     access_       *access;
00036 };
00037
00038 #endif
00039
00040 // GLOBAL ACQUISITION
00041 //
      ==========================================================================================
      //
```

```
PetscErrorCode InitializeAcquisition(domain_ *domain);

PetscErrorCode InitializeAcquisitionPrecursor(domain_ *domain);

PetscErrorCode WriteAcquisition(domain_ *domain);

// PROBES ACQUISITION
//
==========================================================================================================
//

PetscErrorCode ProbesInitialize(domain_ *domain, PetscInt postProcessing=0);

PetscErrorCode InitRakeFile(probeRake *rake, const char *fieldName);

PetscErrorCode writeProbes(domain_ *domain);

// SECTIONS ACQUISITION
//
==========================================================================================================
//

PetscErrorCode sectionsInitialize(acquisition_ *acquisition);

PetscErrorCode writeSections(acquisition_ *acquisition);

PetscErrorCode iSectionSaveVector(mesh_ *mesh, sections *sec, PetscInt iplane, Vec &V, const char*
fieldName);

PetscErrorCode jSectionSaveVector(mesh_ *mesh, sections *sec, PetscInt jplane, Vec &V, const char*
fieldName);

PetscErrorCode kSectionSaveVector(mesh_ *mesh, sections *sec, PetscInt kplane, Vec &V, const char*
fieldName);

PetscErrorCode userSectionSaveVector(mesh_ *mesh, uSections *uSection, Vec &V, const char* fieldName);

PetscErrorCode iSectionSaveScalar(mesh_ *mesh, sections *sec, PetscInt iplane, Vec &V, const char*
fieldName);

PetscErrorCode jSectionSaveScalar(mesh_ *mesh, sections *sec, PetscInt jplane, Vec &V, const char*
fieldName);

PetscErrorCode kSectionSaveScalar(mesh_ *mesh, sections *sec, PetscInt kplane, Vec &V, const char*
fieldName);

PetscErrorCode userSectionSaveScalar(mesh_ *mesh, uSections *uSection, Vec &V, const char* fieldName);

// FIELDS AVERAGES ACQUISITION
//
==========================================================================================================
//

PetscErrorCode averageFieldsInitialize(acquisition_ *acquisition);

PetscErrorCode averageFields(acquisition_ *acquisition);

PetscErrorCode computeQCritIO(acquisition_ *acquisition);

// Five invariants of VGT Qg, Rg, Qs, Rs, Qr
PetscErrorCode computeVgtSecondInvariantIO(acquisition_ *acquisition);

PetscErrorCode computeVgtThirdInvariantIO(acquisition_ *acquisition);

PetscErrorCode computeStrainRateSecondInvariantIO(acquisition_ *acquisition);

PetscErrorCode computeStrainRateThirdInvariantIO(acquisition_ *acquisition);

PetscErrorCode computeQrIO(acquisition_ *acquisition);

PetscErrorCode computeCoriolisIO(acquisition_ *acquisition);

PetscErrorCode computeDrivingSourceIO(acquisition_ *acquisition);

PetscErrorCode computeXDampingIO(acquisition_ *acquisition);

PetscErrorCode computeCanopyForceIO(acquisition_ *acquisition);

PetscErrorCode computeVelocityDivergence(acquisition_ *acquisition);

// MKE BUDGETS ACQUISITION
//
==========================================================================================================
//

PetscErrorCode averageKEBudgetsInitialize(acquisition_ *acquisition);
```

```
PetscErrorCode readKeBoxArray(keFields *ke);

PetscErrorCode setKeBoundsAndComms(mesh_ *mesh, keFields *ke);

PetscErrorCode boxCumulateKEBudgets(acquisition_ *acquisition);

PetscErrorCode averageKEBudgets(acquisition_ *acquisition);

PetscErrorCode averageKEBudgetsCat(acquisition_ *acquisition);

PetscErrorCode averageKEBudgetsCont(acquisition_ *acquisition);

// 3LM ACQUISITION
//
==========================================================================================================
//

PetscErrorCode averaging3LMInitialize(domain_ *domain);

PetscErrorCode writeAveraging3LM(domain_ *domain);

PetscErrorCode write3LMPoints(acquisition_ *acquisition);

PetscErrorCode write3LMFields(acquisition_ *acquisition);

PetscErrorCode findAvgLineIds(acquisition_ *acquisition);

PetscErrorCode read3LMFields(acquisition_ *acquisition);

// ABL PERTURBATION ACQUISITION
//
==========================================================================================================
//

PetscErrorCode perturbationABLInitialize(acquisition_ *acquisition);

PetscErrorCode averagePerturbationABL(acquisition_ *acquisition);

// ABL PLANAR AVERAGING ACQUISITION
//
==========================================================================================================
//

PetscErrorCode averagingABLInitialize(domain_ *domain);

PetscErrorCode writeAveragingABL(domain_ *domain);
```

5.12 /zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/abl↩ Acquisition.h File Reference

Three layer model validation test. Depth averaged fields are gathered on a gridded point mesh for subsequent post processing, namely velocity, pressure and wind farm body force. The mesh must be cartesian and the vertical direction is assumed to be the j-direction. The same hypothesis holds for the ABL data acquisition, which performes spatial averages as a function of time on the planes parallel to the wall defined by each cell's center.

Classes

- struct level3LM

  Struct defining a gridded point mesh level for three layer model (used for its validation)
- struct data3LM

  Struct defining a gridded point mesh for three layer model data (used for its validation)
- struct dataABL

  Struct defining ABL simulation acquisition data.

5.12.1 Detailed Description

Three layer model validation test. Depth averaged fields are gathered on a gridded point mesh for subsequent post processing, namely velocity, pressure and wind farm body force. The mesh must be cartesian and the vertical direction is assumed to be the j-direction. The same hypothesis holds for the ABL data acquisition, which performes spatial averages as a function of time on the planes parallel to the wall defined by each cell's center.

5.13 ablAcquisition.h

Go to the documentation of this file.

```
00001
00008
00009 #ifndef _ABLACQUISITION_H_
00010 #define _ABLACQUISITION_H_
00011
00013 typedef struct
00014 {
00015     word      levelName;
00016     Cmpnts    hStart;
00017     Cmpnts    hEnd;
00018
00019     Cmpnts     **U;
00020     PetscReal **P;
00021 } level3LM;
00022
00024 struct data3LM
00025 {
00026     word           checkPointTimeName;
00027     PetscInt       nstw;
00028     PetscInt       nspw;
00029     Cmpnts         upDir;
00030     Cmpnts         streamDir;
00031     Cmpnts         spanDir;
00032     Cmpnts         **points;
00033     cellIds        **closestCells;
00034
00035     PetscReal      avgStartTime;
00036     PetscReal      avgPrd;
00037
00038     PetscInt       avgWeight;
00039
00040     level3LM       **levels;
00041
00042     PetscReal      **etaTBL;
00043     PetscReal      **etaLBL;
00044     PetscReal      **etaIBL;
00045     PetscReal      **bkgU;
00046     PetscReal      xSample;
00047     PetscInt       kSample;
00048     Vec            avgU;
00049     Vec            avgdTdz;
00050 };
00051
00053 struct dataABL
00054 {
00055     word      timeName;
00056     PetscReal avgStartTime;
00057     PetscReal avgPrd;
00058
00059     PetscReal *cellLevels;
00060     PetscReal *totVolPerLevel;
00061     PetscInt  *totCelPerLevel;
00062     PetscReal *UMean;
00063     PetscReal *VMean;
00064     PetscReal *WMean;
00065     PetscReal *TMean;
00066     PetscReal *nutMean;
00067     PetscReal *ksgsMean;
00068     PetscReal *diffTMean;
00069     Vec       UPrime;
00070     Vec       TPrime;
00071
00072     PetscReal *uuMean;
00073     PetscReal *uvMean;
00074     PetscReal *uwMean;
00075     PetscReal *vvMean;
00076     PetscReal *vwMean;
```

```
    PetscReal *wwMean;

    PetscReal *wuuMean;
    PetscReal *wuvMean;
    PetscReal *wuwMean;
    PetscReal *wvvMean;
    PetscReal *wvwMean;
    PetscReal *wwwMean;

    PetscReal *R11Mean;
    PetscReal *R12Mean;
    PetscReal *R13Mean;
    PetscReal *R22Mean;
    PetscReal *R23Mean;
    PetscReal *R33Mean;

    PetscReal *TuMean;
    PetscReal *TvMean;
    PetscReal *TwMean;

    PetscReal *q1Mean;
    PetscReal *q2Mean;
    PetscReal *q3Mean;
};

#endif
```

5.14 /zfs60/nobackup/stipa/software/↩ TOSCA/src/include/acquisition/averageAcquisition.h File Reference

average fields object declaration

Classes

- struct avgFields

  Struct defining the average fields.

5.14.1 Detailed Description

average fields object declaration

5.15 averageAcquisition.h

Go to the documentation of this file.

```


#ifndef _AVERAGEACQUISITION_H
#define _AVERAGEACQUISITION_H

struct avgFields
{
    // numerical
    Vec           divU;

    // source terms
    Vec           Coriolis;
    Vec           Driving;
    Vec           xDamping;
    Vec           CanopyForce;

    Vec           windFarmForce;
    Vec           yDampMappedU;
    // turbulence criterions
    Vec           Q;
```

```
    Vec           L2;
    Vec           Qg;
    Vec           Rg;
    Vec           Qs;
    Vec           Rs;
    Vec           Qr;

    // averaging
    Vec           avgU;
    Vec           avgP;
    Vec           avgUU;
    Vec           avgDUU;
    Vec           avgP2;
    Vec           avgNut;
    Vec           avgCs;
    Vec           avgOmega;
    Vec           avgOmegaOmega;
    Vec           avgUdotGradP;
    Vec           avgMagGradU;
    Vec           avgMagUU;

    // phase averaging
    Vec           pAvgU;
    Vec           pAvgP;
    Vec           pAvgUU;
    Vec           pAvgP2;
    Vec           pAvgNut;
    Vec           pAvgCs;
    Vec           pAvgOmega;
    Vec           pAvgOmegaOmega;
    Vec           pAvgUdotGradP;
    Vec           pAvgMagGradU;
    Vec           pAvgMagUU;
};

#endif
```

5.16 /zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/ke↩ Acquisition.h File Reference

mean kinetic energy budget object declaration

Classes

- struct keBox

  Struct defining the ke fields.
- struct keFields

  Struct defining the ke fields.

5.16.1 Detailed Description

mean kinetic energy budget object declaration

5.17 keAcquisition.h

Go to the documentation of this file.

```

#ifndef _KEACQUISITION_H
#define _KEACQUISITION_H

struct keBox
```

```
{
    word          *name;
    Cmpnts        center;
    PetscReal     sizeX;
    PetscReal     sizeY;
    PetscReal     sizeZ;

    PetscInt      minKFace, maxKFace,
                  minJFace, maxJFace,
                  minIFace, maxIFace;

    PetscReal     avgDumK, avgDupK, avgDpmK, avgDppK,
                  avgDumJ, avgDupJ, avgDpmJ, avgDppJ,
                  avgDumI, avgDupI, avgDpmI, avgDppI,
                  avgFK, avgFJ, avgFI, avgEps, avgPf,
                  avgPtheta, avgPinf, avgErr,
                  avgPturb, avgPsgs;

    // communication color
    PetscInt      thisBoxControlled;
    MPI_Comm      KEBOX_COMM;
    PetscMPIInt   writerRank;
    PetscMPIInt   nProcsBox;

};

struct keFields
{
    // time controls
    PetscReal     avgStartTime;
    PetscReal     avgPrd;

    // flags
    PetscInt      cartesian;
    PetscInt      debug;

    PetscInt      nBox;
    keBox         **box;

    // fields
    Vec           Error;
    Vec           D;
    Vec           F;
    Vec           Pinf;
    Vec           Pf;
    Vec           Ptheta;
    Vec           Eps;
    Vec           lEm;
    Vec           lDum;
    Vec           lDup;
    Vec           lF;

    // fields which are not in the MKE but are useful for turbulence (only available for cartesian)
    Vec           lPturb;
    Vec           lPsgs;

    // working fields
    Vec           lVmTauSGS;
    Vec           lDpm;
    Vec           lDpp;
    Vec           lUm;

    // working fields at cell faces
    Vec           lVmVpVp;
    Vec           lVpVpVp;
    Vec           lVDm;
    Vec           lVmPmG;
    Vec           lVpPpG;
    Vec           lDdVm;
    Vec           lVmCsi;
    Vec           lVmEta;
    Vec           lVmZet;
    Vec           lVm;
    Vec           lPm;
    Vec           lVpVpEta;
    Vec           lVpVpCsi;
    Vec           lVpVpZet;
    Vec           lVpVp;

};

#endif
```

5.18 /zfs60/nobackup/stipa/software/↩ TOSCA/src/include/acquisition/perturbAcquisition.h File Reference

perturbation fields w.r.t. an x-normal sample plane

Classes

- struct perturbFields

  Struct defining the ke fields.

5.18.1 Detailed Description

perturbation fields w.r.t. an x-normal sample plane

5.19 perturbAcquisition.h

Go to the documentation of this file.

```
00001
00003
00004 #ifndef _PRTACQUISITION_H
00005 #define _PRTACQUISITION_H
00006
00008 struct perturbFields
00009 {
00010     // time controls
00011     PetscReal     avgStartTime;
00012     PetscReal     avgPrd;
00013     PetscReal     avgWindowTime;
00014
00015     Cmpnts        **bkgU;
00016     PetscReal     **bkgP;
00017     PetscReal     **bkgT;
00018     PetscReal     xSample;
00019     PetscInt      kSample;
00020     PetscInt      avgWeight;
00021
00022     // fields
00023     Vec           pertU;
00024     Vec           pertP;
00025     Vec           pertT;
00026
00027     // flags
00028     PetscInt      read;
00029     PetscInt      initialized;
00030     PetscInt      movAvgActive;
00031 };
00032
00033 #endif
```

5.20 /zfs60/nobackup/stipa/software/↩ TOSCA/src/include/acquisition/probesAcquisition.h File Reference

probes and rakes object declaration

Classes

- struct probeRake

  Struct defining a probe rake.
- struct rakes

  Struct defining all rakes in the mesh.

5.20.1 Detailed Description

probes and rakes object declaration

5.21 probesAcquisition.h

Go to the documentation of this file.

```
#ifndef _PROBESACQUISITION_H_
#define _PROBESACQUISITION_H_

struct probeRake
{
    word          rakeName;
    word          timeName;
    PetscInt      probesNumber;
    word          intervalType;
    PetscReal     timeStart;
    PetscReal     timeInterval;
    PetscInt      Uflag;
    PetscInt      Tflag;
    PetscInt      Pflag;

    PetscReal     **locations;
    PetscInt      **cells;
    PetscInt      *owner;

    PetscInt      *domainID;
    PetscInt      thisRakeControlled;
    MPI_Comm      RAKE_COMM;
};

struct rakes
{
    PetscInt      nRakes;
    probeRake     *rakes;
    PetscInt      allSameIO;
    PetscInt      verbose;
};

#endif
```

5.22 /zfs60/nobackup/stipa/software/↩ TOSCA/src/include/acquisition/sectionsAcquisition.h File Reference

Sections along curvilinear coordinates object declaration.

Classes

- struct sections

  Struct defining a generic section to save.
- struct uSections

  Struct defining a user defined section not necessarily along curvilinear co-ordinates.
- struct udSections

5.22.1 Detailed Description

Sections along curvilinear coordinates object declaration.

5.23 sectionsAcquisition.h

Go to the documentation of this file.

```
#ifndef _SECTIONSACQUISITION_H_
#define _SECTIONSACQUISITION_H_

struct sections
{
    PetscInt            available;
    PetscInt            nSections;
    PetscReal           timeStart;
    word                intervalType;
    PetscReal           timeInterval;
    PetscInt            *indices;
    PetscReal           *coordinates;
    symmTensor          **symmTensorSec;
    Cmpnts              **vectorSec;
    PetscReal           **scalarSec;
};

struct uSections
{
    PetscInt            nodes;
    word                sectionName;
    Cmpnts              **coor;
    cellIds             **closestId;
    PetscInt            **hasCoor;
    PetscReal           timeStart;
    word                intervalType;
    PetscReal           timeInterval;
    PetscInt            nx;
    PetscInt            ny;
    Cmpnts              **vectorSec;
    PetscReal           **scalarSec;
};


struct udSections
{
    PetscInt            available;
    PetscInt            nSections;
    uSections           **uSection;
};

#endif
```

5.24 /zfs60/nobackup/stipa/software/TOSCA/src/include/base.h File Reference

Base header file including c/c++ libraries and PETSc and HYPRE libs.

```
#include <random>
#include <vector>
#include <numeric>
#include <algorithm>
#include <assert.h>
#include <complex>
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>
```

```
#include <dirent.h>
#include <iostream>
#include <fstream>
#include <sstream>
#include <string.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <iomanip>
#include "petsctime.h"
#include "petscvec.h"
#include "petscdmda.h"
#include "petscksp.h"
#include "petscsnes.h"
#include "HYPRE_krylov.h"
#include "HYPRE.h"
#include "HYPRE_parcsr_ls.h"
#include "HYPRE_struct_ls.h"
#include "HYPRE_sstruct_ls.h"
#include "HYPRE_IJ_mv.h"
```

Classes

- struct simInfo_
- struct constants_
- struct cellIds

  Cell indices.
- struct Cmpnts

  Defines the x, y, z components of a vector.
- struct Cpt2D

  Structure for defining the x, y components of a 2D vector.
- struct symmTensor

  Defines the components of a symmetric tensor.
- struct skewSymmTensor
- struct Tensor
- struct TensorInvariants
- struct Matrix3
- struct patchVectorField

  Vector field on the patch (used to store wall models fields)

Macros

- #define **M_PI** 3.14159265358979323846264 3

Typedefs

- typedef std::string **word**
- typedef std::vector< PetscInt > **labelList**
- typedef std::vector< std::vector< PetscInt > > **labelListList**
- typedef std::complex< PetscReal > **complex**

5.24.1 Detailed Description

Base header file including c/c++ libraries and PETSc and HYPRE libs.

5.25 base.h

Go to the documentation of this file.

```
00001
00003
00004 #ifndef _BASE_H_
00005 #define _BASE_H_
00006
00007 // include base headers
00008 #include <random>
00009 #include <vector>
00010 #include <numeric>
00011 #include <algorithm>
00012 #include <assert.h>
00013 #include <complex>
00014 #include <sys/types.h>
00015 #include <sys/stat.h>
00016 #include <unistd.h>
00017 #include <dirent.h>
00018 #include <iostream>
00019 #include <fstream>
00020 #include <sstream>
00021 #include <string.h>
00022 #include <stdio.h>
00023 #include <stdlib.h>
00024 #include <time.h>
00025 #include <iomanip>
00026
00027 // include PETSc headers
00028 #include "petsctime.h"
00029 #include "petscvec.h"
00030 #include "petscdmda.h"
00031 #include "petscksp.h"
00032 #include "petscsnes.h"
00033
00034 // include HYPRE headers
00035 #include "HYPRE_krylov.h"
00036 #include "HYPRE.h"
00037 #include "HYPRE_parcsr_ls.h"
00038 #include "HYPRE_struct_ls.h"
00039 #include "HYPRE_sstruct_ls.h"
00040 #include "HYPRE_IJ_mv.h"
00041
00042 // typedefs
00043 typedef std::string                        word;
00044 typedef std::vector<PetscInt>              labelList;
00045 typedef std::vector<std::vector<PetscInt>> labelListList;
00046 typedef std::complex<PetscReal>            complex;
00047
00048 // constants
00049 #ifndef M_PI
00050     #define M_PI 3.14159265358979323846264 3
00051 #endif
00052
00053 struct simInfo_
00054 {
00055     PetscInt nDomains;
00056     PetscInt periodic;
00057     PetscInt nHoleRegions;
00058 };
00059
00060 struct constants_
00061 {
00062     // dimensionless numbers
00063     PetscReal        Pr;
00064
00065     // physical constants
00066     PetscReal        nu;
00067     PetscReal        rho;
00068
00069     // reference temperature
00070     PetscReal        tRef;
00071
00072 };
00073
```

```
typedef struct
{
   PetscInt i,j,k;
} cellIds;

typedef struct
{
    PetscScalar x, y, z;
} Cmpnts;

typedef struct
{
    PetscReal x, y;
} Cpt2D;

typedef struct
{
    PetscScalar xx, xy, xz, yy, yz, zz;
} symmTensor;

typedef struct
{
    PetscScalar xy, xz, yz;
} skewSymmTensor;

typedef struct
{
    PetscScalar xx, xy, xz,
                yx, yy, yz,
                zx, zy, zz;
} Tensor;

typedef struct
{
    PetscScalar I1;  // First invariant: trace(T)
    PetscScalar I2;  // Second invariant: 0.5*(trace(T)^2 - trace(T^2))
    PetscScalar I3;  // Third invariant: determinant of T
} TensorInvariants;

typedef struct
{
    PetscScalar m[3][3];

} Matrix3;

typedef struct
{
    PetscReal **x;
    PetscReal **y;
    PetscReal **z;
} patchVectorField;

#endif
```

5.26 /zfs60/nobackup/stipa/software/TOSCA/src/include/boundary.h File Reference

Boundary conditions header file.

Classes

- struct scalarBC

  Structure dafining the type of boundary conditions for a scalar.
- struct vectorBC

  Structure dafining the type of boundary conditions for a vector.
- struct inflowData

  Struct defining inflow information.
- struct inletFunctionTypes

  Struct storing the inlet function for a patch.
- struct inletFunctions

  Struct storing inlet functions data.

Functions

- PetscErrorCode **readScalarBC** (const word &location, const word &field, scalarBC ∗bc)

  Reads boundary conditions for a scalar field.
- PetscErrorCode **readVectorBC** (const word &location, const word &field, vectorBC ∗bc)

  Reads bundary conditions for a vector field.
- PetscErrorCode **SetBoundaryConditions** (mesh_ ∗mesh)

  Set boundary conditions.
- PetscErrorCode **checkBCsAndSetPatchTypes** (mesh_ ∗mesh)

  Checks available boundary conditions.
- PetscErrorCode **SetPeriodicConnectivity** (mesh_ ∗mesh, word &meshFileName)

  Set periodicconnectivity type.
- PetscErrorCode **SetWallModels** (ueqn_ ∗ueqn)

  Set wall models.
- PetscErrorCode **UpdateContravariantBCs** (ueqn_ ∗ueqn)

  Update contravariant fluxes boundary conditions.
- PetscErrorCode **UpdateCartesianBCs** (ueqn_ ∗ueqn)

  Update cartesian boundary conditions.
- PetscErrorCode **UpdateTemperatureBCs** (teqn_ ∗teqn)

  Update temperature boundary conditions.
- PetscErrorCode **UpdateNutBCs** (les_ ∗les)

  Update effective viscosity boundary conditions.
- PetscErrorCode **UpdatektBCs** (les_ ∗les)

  Update effective diffusivity boundary conditions.
- PetscErrorCode **UpdatePressureBCs** (peqn_ ∗peqn)

  Update pressure boundary conditions.
- PetscErrorCode **UpdatePhiBCs** (peqn_ ∗peqn)

  Update pressure correction boundary conditions.
- PetscErrorCode **UpdateImmersedBCs** (ibm_ ∗ibm)

  Update ibm boundary conditions.
- PetscErrorCode **UpdateWallModelsU** (ueqn_ ∗ueqn)

  Update wall model for specified wall shear stress.
- PetscErrorCode **UpdateWallModelsT** (teqn_ ∗teqn)

  Update wall model for specified wall heat flux.
- PetscErrorCode **readSurfaceTempData** (Shumann ∗wm)

  read surface temperature and obhukhov length data

5.26.1 Detailed Description

Boundary conditions header file.

5.27 boundary.h

Go to the documentation of this file.

```
#ifndef _BOUNDARY_H_
#define _BOUNDARY_H_

struct scalarBC
{
    word      iLeft, iRight,
              jLeft, jRight,
              kLeft, kRight;
    PetscInt  iLeftPatchType,
              iRightPatchType,
              jLeftPatchType,
              jRightPatchType,
              kLeftPatchType,
              kRightPatchType;
    PetscReal iLval, iRval ,
              jLval, jRval ,
              kLval, kRval ;
    PetscInt  iLWF , iRWF  ,
              jLWF , jRWF  ,
              kLWF , kRWF  ;
};


struct  vectorBC
{
    word     iLeft, iRight,
             jLeft, jRight,
             kLeft, kRight;
    PetscInt iLeftPatchType,
             iRightPatchType,
             jLeftPatchType,
             jRightPatchType,
             kLeftPatchType,
             kRightPatchType;
    Cmpnts   iLval, iRval ,
             jLval, jRval ,
             kLval, kRval ;
    PetscInt iLWF , iRWF  ,
             jLWF , jRWF  ,
             kLWF , kRWF  ;
};

struct inflowData
{
    PetscReal   *inflowTimes;
    PetscInt    nInflowTimes;
    PetscInt    currentCloseIdx;
};

struct inletFunctionTypes
{
    // type of inlet function
    PetscInt      typeU;
    PetscInt      typeT;
    PetscInt      typeNut;

    // type 1: power law (velocity only)
    Cmpnts        Uref;
    PetscReal     Href;
    PetscReal     uPrimeRMS;

    // type 2/5: log law/with veer + neutral stratification
    Cmpnts        Udir;
    PetscReal     roughness;
    PetscReal     hInv;
    PetscReal     uTau;
    PetscReal     dInv;
    PetscReal     gInv;
    PetscReal     tRef;
    PetscReal     gTop;
    PetscReal     gABL;
    PetscReal     smear;
    PetscReal     latitude;
    PetscReal     fc;

    // type 3/4: unsteady mapped/interpolated inflow (mapped assumes precursor and successor mesh are equal at the inflow location, interpolated assumes only periodicity)
    PetscInt      n1;
    PetscInt      n2;
    PetscInt      n1wg;
```

```
    PetscInt      n2wg;
    PetscInt      prds1;
    PetscInt      prds2;
    PetscInt      merge1;
    PetscInt      shift2;

    PetscInt      mapT;
    PetscInt      mapNut;

    PetscReal     ***inflowWeights;
    cellIds       ***closestCells;
    PetscReal     ***inflowWeights_1;
    cellIds       ***closestCells_1;
    PetscReal     ***inflowWeights_2;
    cellIds       ***closestCells_2;
    word          sourceType;
    word          interpMethod;
    PetscReal     width1;
    PetscReal     width2;
    PetscReal     inflowHeigth;

    Cmpnts        **ucat_plane;
    PetscReal     **t_plane;
    PetscReal     **nut_plane;
    inflowData    inflowU;
    inflowData    inflowT;
    inflowData    inflowNut;

    Cmpnts       *uBarAvgTopX;
    PetscReal    *tBarAvgTopX;
    PetscReal    *avgTopPointCoords;
    PetscReal    avgTopDelta;
    PetscReal    avgTopLength;

    PetscReal    shiftSpeed;
    PetscReal    *ycent;
    PetscInt     *yIDs;
    PetscReal    *yWeights;
    MPI_Comm     IFFCN_COMM;

    // type 6: i-dir sinusoidal inflow
    PetscReal    amplitude;
    PetscReal    periods;
};

struct inletFunctions
{
    inletFunctionTypes *iLeft;
    inletFunctionTypes *iRight;
    inletFunctionTypes *jLeft;
    inletFunctionTypes *jRight;
    inletFunctionTypes *kLeft;
    inletFunctionTypes *kRight;
};

#endif

PetscErrorCode readScalarBC(const word &location, const word &field, scalarBC *bc);

PetscErrorCode readVectorBC(const word &location, const word &field, vectorBC *bc);

PetscErrorCode SetBoundaryConditions(mesh_ *mesh);

PetscErrorCode checkBCsAndSetPatchTypes(mesh_ *mesh);

PetscErrorCode SetPeriodicConnectivity(mesh_ *mesh, word &meshFileName);

PetscErrorCode SetWallModels(ueqn_ *ueqn);

PetscErrorCode UpdateContravariantBCs(ueqn_ *ueqn);

PetscErrorCode UpdateCartesianBCs(ueqn_ *ueqn);

PetscErrorCode UpdateTemperatureBCs(teqn_ *teqn);

PetscErrorCode UpdateNutBCs(les_ *les);

PetscErrorCode UpdatektBCs(les_ *les);

PetscErrorCode UpdatePressureBCs(peqn_ *peqn);

PetscErrorCode UpdatePhiBCs(peqn_ *peqn);

PetscErrorCode UpdateImmersedBCs(ibm_ *ibm);

PetscErrorCode UpdateWallModelsU(ueqn_ *ueqn);

```

```
PetscErrorCode UpdateWallModelsT(teqn_ *teqn);

PetscErrorCode readSurfaceTempData(Shumann *wm);
```

5.28 catalystAdaptor.h

```
#ifndef CatalystAdaptor_h
#define CatalystAdaptor_h

#include <catalyst.h>
#include <stdio.h>
#include <string.h>


//*****************************************************************************************************************//

void catalystInitialize(domain_ *domain)
{
    // read input parameter
    io_   *io   = domain->io;
    mesh_ *mesh = domain->mesh;

    PetscPrintf(mesh->MESH_COMM, "Initializing TOSCA's catalyst-paraview interface...");

    readDictWord  ("sampling/catalystProperties", "ioType",      &(io->ioTypeCatalyst));
    readDictWord  ("sampling/catalystProperties", "outputType",  &(io->outputTypeCatalyst));
    readDictDouble("sampling/catalystProperties", "startTime",   &(io->startTimeCatalyst));
    readDictDouble("sampling/catalystProperties", "timeInterval",&(io->timeIntervalCatalyst));

    if(io->outputTypeCatalyst!="adjustableTime" && io->outputTypeCatalyst!="timeStep")
    {
        char error[512];
        sprintf(error, "unknown catalyst outputType. Available types are\n - adjustableTime\n - timeStep\n");
        fatalErrorInFunction("catalystInitialize",  error);
    }

    // create catalyst folder
    PetscMPIInt rank;
    MPI_Comm_rank(PETSC_COMM_WORLD, &rank);

    if(!rank)
    {
        errno = 0;
        PetscInt dirRes = mkdir("./catalyst", 0777);
        if(dirRes != 0 && errno != EEXIST)
        {
            char error[512];
            sprintf(error, "could not create catalyst directory\n");
            fatalErrorInFunction("catalystInitialize",  error);
        }
    }

    if(io->ioTypeCatalyst == "general") {}
    else if(io->ioTypeCatalyst == "script")
    {
        readDictWord("sampling/catalystProperties", "scriptName", &(io->scriptNameCatalyst));
    }
    else
    {
        char error[512];
        sprintf(error, "unknown catalyst ioType. Available types are\n - general\n - script\n");
        fatalErrorInFunction("catalystInitialize",  error);
    }

    // initialize catalyst
    conduit_node* catalyst_init_params = conduit_node_create();
    if(io->ioTypeCatalyst == "general")
    {
        std::string path2save = "catalyst/dataset-%04t.vtpd";
        conduit_node_set_path_char8_str(catalyst_init_params, "catalyst/pipelines/0/type", "io");
        conduit_node_set_path_char8_str(catalyst_init_params, "catalyst/pipelines/0/filename", path2save.c_str());
        conduit_node_set_path_char8_str(catalyst_init_params, "catalyst/pipelines/0/channel", "grid");
    }
    else if(io->ioTypeCatalyst == "script")
    {
        // pass scripts on command line
        word scriptName = "./sampling/" + io->scriptNameCatalyst;
        conduit_node_set_path_char8_str(catalyst_init_params, "catalyst/scripts/script0", scriptName.c_str());
    }

```

```
    conduit_node_set_path_char8_str(catalyst_init_params, "catalyst_load/implementation", "paraview");
    conduit_node_set_path_char8_str(catalyst_init_params, "catalyst_load/search_paths/paraview", PARAVIEW_IMPL_DIR);
    enum catalyst_status err = catalyst_initialize(catalyst_init_params);
    conduit_node_destroy(catalyst_init_params);

    if (err != catalyst_status_ok)
    {
        char error[512];
        sprintf(error, "\nTOSCA's catalyst-paraview interface initialization failed\n");
        fatalErrorInFunction("catalystInitialize",  error);
    }

    PetscPrintf(mesh->MESH_COMM, "done\n\n");
}


//*****************************************************************************************************************//

void catalystExecute(domain_ *domain)
{
    PetscInt  executeCatalystOperations = 0;

    word      intervalType = domain->io->outputTypeCatalyst;
    PetscReal timeInterval = domain->io->timeIntervalCatalyst;

    // write every "timeInterval" seconds
    if
    (
        intervalType == "adjustableTime" &&
        mustWrite(domain->clock->time, domain->io->startTimeCatalyst, timeInterval)
    )
    {
        executeCatalystOperations = 1;
    }
    // write every "timeInterval" iterations
    else if
    (
        (intervalType == "timeStep") &&
        (
            domain->clock->it / timeInterval -
            std::floor
            (
                domain->clock->it / timeInterval
            ) < 1e-10
        )
    )
    {
        executeCatalystOperations = 1;
    }

    if(executeCatalystOperations)
    {
        // add time/cycle information
        conduit_node* catalyst_exec_params = conduit_node_create();

        conduit_node_set_path_int64(catalyst_exec_params, "catalyst/state/timestep", domain->clock->it);
        conduit_node_set_path_float64(catalyst_exec_params, "catalyst/state/time", domain->clock->time);

        //create the mesh in "grid" channel, using Conduit Mesh Blueprint
        conduit_node_set_path_char8_str(catalyst_exec_params, "catalyst/channels/grid/type", "mesh");
        conduit_node* condMesh = conduit_node_create();

        mesh_         *mesh  = domain->mesh;
        DM            fda = mesh->fda, da = mesh->da;
        DMDALocalInfo info = mesh->info;
        PetscInt      i, j, k;
        PetscInt      xs = info.xs, xe = info.xs + info.xm;
        PetscInt      ys = info.ys, ye = info.ys + info.ym;
        PetscInt      zs = info.zs, ze = info.zs + info.zm;
        PetscInt      mx = info.mx, my = info.my, mz = info.mz;

        PetscInt      lxs, lxe, lys, lye, lzs, lze;

        Vec           Coor;
        Cmpnts        ***coor, ***ucat;
        PetscReal     ***qcrit, ***p;

        lxs = xs; lxe = xe; if (xs==0) lxs = xs+1; if (xe==mx) lxe = xe-1;
        lys = ys; lye = ye; if (ys==0) lys = ys+1; if (ye==my) lye = ye-1;
        lzs = zs; lze = ze; if (zs==0) lzs = zs+1; if (ze==mz) lze = ze-1;

        PetscInt  numberOfCells = (lxe-lxs) * (lye-lys) * (lze-lzs);
        PetscReal *cellVar; cellVar = new PetscReal[numberOfCells];

```

```
        PetscReal ts, te;
        PetscTime(&ts);

        PetscPrintf(mesh->MESH_COMM, "Executed paraview-catalyst actions in ");

        DMGetCoordinatesLocal(da, &Coor);  // get the local vector Coor linked to the coordinates
        DMDAVecGetArray(fda, Coor, &coor); // access this vector using the 3D array coor

        // store Qcrit inside acquisition
        computeQCritIO(domain->acquisition);

        DMDAVecGetArray(fda, domain->ueqn->Ucat, &ucat);
        DMDAVecGetArray(da, domain->peqn->P, &p);
        DMDAVecGetArray(da,  domain->acquisition->fields->Q, &qcrit);

        PetscInt cartesian    = 1,
                 rectilinear  = 2,
                 unstructured = 3;
        PetscInt gridType     = rectilinear;

        if (gridType == cartesian)
        {
            // add coordsets
            conduit_node_set_path_char8_str(condMesh, "coordsets/coords/type", "uniform");

            // add topology
            conduit_node_set_path_char8_str(condMesh, "topologies/mesh/type", "uniform");
            conduit_node_set_path_char8_str(condMesh, "topologies/mesh/coordset", "coords");

            conduit_node_set_path_int64(condMesh, "coordsets/coords/dims/i", lze-lzs+1); // number of points in each dimension
            conduit_node_set_path_int64(condMesh, "coordsets/coords/dims/j", lxe-lxs+1);
            conduit_node_set_path_int64(condMesh, "coordsets/coords/dims/k", lye-lys+1);

            conduit_node_set_path_float64(condMesh, "coordsets/coords/origin/x", lzs);
            conduit_node_set_path_float64(condMesh, "coordsets/coords/origin/y", lxs);
            conduit_node_set_path_float64(condMesh, "coordsets/coords/origin/z", lys);

            conduit_node_set_path_float64(condMesh, "coordsets/coords/spacing/dx", 0.1);
            conduit_node_set_path_float64(condMesh, "coordsets/coords/spacing/dy", 0.1);
            conduit_node_set_path_float64(condMesh, "coordsets/coords/spacing/dz", 0.1);

            // add cell-based q-criterion
            for (k=lzs; k<lze; k++)
            for (j=lys; j<lye; j++)
            for (i=lxs; i<lxe; i++)
            {
                cellVar[(k-lzs)*(lxe-lxs)*(lye-lys) + (j-lys)*(lxe-lxs) + (i-lxs)] = qcrit[k-lzs+zs][j-lys+ys][i-lxs+xs];
            }

            conduit_node_set_path_char8_str(condMesh, "fields/q/association", "element");
            conduit_node_set_path_char8_str(condMesh, "fields/q/topology", "mesh");
            conduit_node_set_path_char8_str(condMesh, "fields/q/volume_dependent", "false");
            conduit_node_set_path_external_float64_ptr(condMesh, "fields/q/values", cellVar, numberOfCells);

            // add cell-based pressure
            for (k=lzs; k<lze; k++)
            for (j=lys; j<lye; j++)
            for (i=lxs; i<lxe; i++)
            {
                cellVar[(k-lzs)*(lxe-lxs)*(lye-lys) + (j-lys)*(lxe-lxs) + (i-lxs)] = p[k-lzs+zs][j-lys+ys][i-lxs+xs];
            }

            conduit_node_set_path_char8_str(condMesh, "fields/p/association", "element");
            conduit_node_set_path_char8_str(condMesh, "fields/p/topology", "mesh");
            conduit_node_set_path_char8_str(condMesh, "fields/p/volume_dependent", "false");
            conduit_node_set_path_external_float64_ptr(condMesh, "fields/p/values", cellVar, numberOfCells);

            // add cell-based velocity magnitude
            for (k=lzs; k<lze; k++)
            for (j=lys; j<lye; j++)
            for (i=lxs; i<lxe; i++)
            {
                cellVar[(k-lzs)*(lxe-lxs)*(lye-lys) + (j-lys)*(lxe-lxs) + (i-lxs)] = nMag(ucat[k-lzs+zs][j-lys+ys][i-lxs+xs]);
            }

            conduit_node_set_path_char8_str(condMesh, "fields/uMag/association", "element");
            conduit_node_set_path_char8_str(condMesh, "fields/uMag/topology", "mesh");
            conduit_node_set_path_char8_str(condMesh, "fields/uMag/volume_dependent", "false");
            conduit_node_set_path_external_float64_ptr(condMesh, "fields/uMag/values", cellVar, numberOfCells);

```

```
            // add the mesh info (condMesh) to catalyst_exec_params
            conduit_node_set_path_external_node(catalyst_exec_params, "catalyst/channels/grid/data", condMesh);

            enum catalyst_status err = catalyst_execute(catalyst_exec_params);

            if (err != catalyst_status_ok)
            {
                char error[512];
                sprintf(error, "failed to execute catalyst-paraview interface\n");
                fatalErrorInFunction("catalystInitialize",  error);
            }
        }
        else if (gridType == rectilinear)
        {
            // add coordsets
            conduit_node_set_path_char8_str(condMesh, "coordsets/coords/type", "rectilinear");

            // add topology
            conduit_node_set_path_char8_str(condMesh, "topologies/mesh/type", "rectilinear");
            conduit_node_set_path_char8_str(condMesh, "topologies/mesh/coordset", "coords");

            PetscReal *xval; xval = new PetscReal[lze-lzs+1];
            PetscReal *yval; yval = new PetscReal[lxe-lxs+1];
            PetscReal *zval; zval = new PetscReal[lye-lys+1];

            for (i=lxs; i<lxe; i++) yval[i-lxs] = coor[lzs][lys][i].y;
            for (j=lys; j<lye; j++) zval[j-lys] = coor[lzs][j][lxs].z;
            for (k=lzs; k<lze; k++) xval[k-lzs] = coor[k][lys][lxs].x;

            // fill in the last point in each dimension
            PetscInt lastIndex;
            lastIndex = lxe-lxs; yval[lastIndex] = 2*yval[lastIndex-1] - yval[lastIndex-2];
            lastIndex = lye-lys; zval[lastIndex] = 2*zval[lastIndex-1] - zval[lastIndex-2];
            lastIndex = lze-lzs; xval[lastIndex] = 2*xval[lastIndex-1] - xval[lastIndex-2];

            conduit_node_set_path_float64_ptr(condMesh, "coordsets/coords/values/x", xval, lze-lzs+1);
            conduit_node_set_path_float64_ptr(condMesh, "coordsets/coords/values/y", yval, lxe-lxs+1);
            conduit_node_set_path_float64_ptr(condMesh, "coordsets/coords/values/z", zval, lye-lys+1);

            // add cell-based velocity mag
            /*
            for (k=lzs; k<lze; k++)
            for (j=lys; j<lye; j++)
            for (i=lxs; i<lxe; i++)
            {
                cellVar[(k-lzs)*(lxe-lxs)*(lye-lys) + (j-lys)*(lxe-lxs) + (i-lxs)] = nMag(ucat[k-lzs+zs][j-lys+ys][i-lxs+xs]);
            }
            */

            // add cell-based velocity mag
            for (j=lys; j<lye; j++)
            for (i=lxs; i<lxe; i++)
            for (k=lzs; k<lze; k++)
            {
                cellVar[(j-lys)*(lze-lzs)*(lxe-lxs) + (i-lxs)*(lze-lzs) + (k-lzs)] = nMag(ucat[k-lzs+zs][j-lys+ys][i-lxs+xs]);
            }

            conduit_node_set_path_char8_str(condMesh, "fields/uMag/association", "element");
            conduit_node_set_path_char8_str(condMesh, "fields/uMag/topology", "mesh");
            conduit_node_set_path_char8_str(condMesh, "fields/uMag/volume_dependent", "false");
            conduit_node_set_path_float64_ptr(condMesh, "fields/uMag/values", cellVar, numberOfCells);

            // add cell-based pressure
            for (j=lys; j<lye; j++)
            for (i=lxs; i<lxe; i++)
            for (k=lzs; k<lze; k++)
            {
                cellVar[(j-lys)*(lze-lzs)*(lxe-lxs) + (i-lxs)*(lze-lzs) + (k-lzs)] = p[k-lzs+zs][j-lys+ys][i-lxs+xs];
            }

            conduit_node_set_path_char8_str(condMesh, "fields/p/association", "element");
            conduit_node_set_path_char8_str(condMesh, "fields/p/topology", "mesh");
            conduit_node_set_path_char8_str(condMesh, "fields/p/volume_dependent", "false");
            conduit_node_set_path_float64_ptr(condMesh, "fields/p/values", cellVar, numberOfCells);

            // add cell-based qcrit
            for (j=lys; j<lye; j++)
            for (i=lxs; i<lxe; i++)
            for (k=lzs; k<lze; k++)
            {
                cellVar[(j-lys)*(lze-lzs)*(lxe-lxs) + (i-lxs)*(lze-lzs) + (k-lzs)] = qcrit[k-lzs+zs][j-lys+ys][i-lxs+xs];
            }
```

```

            conduit_node_set_path_char8_str(condMesh, "fields/q/association", "element");
            conduit_node_set_path_char8_str(condMesh, "fields/q/topology", "mesh");
            conduit_node_set_path_char8_str(condMesh, "fields/q/volume_dependent", "false");
            conduit_node_set_path_float64_ptr(condMesh, "fields/q/values", cellVar, numberOfCells);

            // for debugging
            // conduit_node_print(condMesh);

            // add the mesh info (condMesh) to catalyst_exec_params
            conduit_node_set_path_external_node(catalyst_exec_params, "catalyst/channels/grid/data", condMesh);

            enum catalyst_status err = catalyst_execute(catalyst_exec_params);

            if (err != catalyst_status_ok)
            {
                char error[512];
                sprintf(error, "failed to execute catalyst-paraview interface\n");
                fatalErrorInFunction("catalystInitialize",  error);
            }

            delete [] xval, yval, zval;
        }
        else if (gridType == unstructured)
        {
          // not implemented
        }

        DMDAVecRestoreArray(fda, Coor, &coor);
        DMDAVecRestoreArray(fda, domain->ueqn->Ucat, &ucat);
        DMDAVecRestoreArray(da,  domain->peqn->P, &p);
        DMDAVecRestoreArray(da,  domain->acquisition->fields->Q, &qcrit);

        delete [] cellVar;

        conduit_node_destroy(catalyst_exec_params);
        conduit_node_destroy(condMesh);

        PetscTime(&te);
        PetscPrintf(mesh->MESH_COMM, "%lf s\n",te-ts);
    }
}


//*****************************************************************************************************************//

void catalystFinalize()
{
    conduit_node* catalyst_fini_params = conduit_node_create();
    enum catalyst_status err = catalyst_finalize(catalyst_fini_params);

    if (err != catalyst_status_ok)
    {
        char error[512];
        sprintf(error, "failed to finalize catalyst-paraview interface\n");
        fatalErrorInFunction("catalystFinalize",  error);
    }

    conduit_node_destroy(catalyst_fini_params);
}

#endif
```

5.29 /zfs60/nobackup/stipa/software/TOSCA/src/include/clock.h File Reference

Simulation time struct header file.

Classes

- struct clock_

Functions

- PetscErrorCode **adjustTimeStep** (domain_ *domain)
- PetscErrorCode **timeStepInfo** (domain_ *domain, clock_ *clock, PetscReal &dxByU_min, PetscReal &maxU, cellIds &maxUCell)

5.29.1 Detailed Description

Simulation time struct header file.

5.30 clock.h

Go to the documentation of this file.

```
00001
00003
00004 #ifndef _CLOCK_H_
00005 #define _CLOCK_H_
00006
00007 struct clock_
00008 {
00009     PetscReal        time;
00010     PetscReal        startTime;
00011     PetscReal        endTime;
00012
00013     word             startFrom;
00014
00015     PetscReal        dt;
00016     PetscReal        startDt;
00017     PetscReal        dtOld;
00018     PetscReal        cfl;
00019     PetscReal        dxMin;
00020     PetscReal        acquisitionDt;
00021
00022     PetscInt         it;
00023     PetscInt         itStart;
00024
00025     PetscInt         timePrecision;
00026 };
00027
00028 #endif
00029
00030 PetscErrorCode adjustTimeStep(domain_ *domain);
00031
00032 PetscErrorCode timeStepInfo(domain_ *domain, clock_ *clock, PetscReal &dxByU_min, PetscReal &maxU,
      cellIds &maxUCell);
```

5.31 /zfs60/nobackup/stipa/software/TOSCA/src/include/domain.h File Reference

Domain data structure definition.

```
#include "objects.h"
#include "clock.h"
#include "access.h"
#include "flags.h"
#include "mesh.h"
#include "overset.h"
#include "acquisition.h"
#include "ibm.h"
#include "ueqn.h"
#include "peqn.h"
```

```
#include "teqn.h"
#include "les.h"
#include "lesScalar.h"
#include "precursor.h"
#include "turbines.h"
#include "abl.h"
#include "../sources/ueqn_sources.h"
#include "../sources/teqn_sources.h"
#include "../solvers/ueqn_solvers.h"
#include "../solvers/teqn_solvers.h"
```

Classes

- struct domain_

  Domain data structure definition.

5.31.1 Detailed Description

Domain data structure definition.

5.32 domain.h

Go to the documentation of this file.

```
00001
00003
00004 #ifndef _DOMAIN_H_
00005 #define _DOMAIN_H_
00006
00007 #include "objects.h"
00008 #include "clock.h"
00009 #include "access.h"
00010 #include "flags.h"
00011 #include "mesh.h"
00012 #include "overset.h"
00013 #include "acquisition.h"
00014 #include "ibm.h"
00015 #include "ueqn.h"
00016 #include "peqn.h"
00017 #include "teqn.h"
00018 #include "les.h"
00019 #include "lesScalar.h"
00020 #include "precursor.h"
00021 #include "turbines.h"
00022 #include "abl.h"
00023 #include "../sources/ueqn_sources.h"
00024 #include "../sources/teqn_sources.h"
00025 #include "../solvers/ueqn_solvers.h"
00026 #include "../solvers/teqn_solvers.h"
00027
00029 struct domain_
00030 {
00031     simInfo_      info;
00032     constants_    constants;
00033
00034     PetscInt      domainID;
00035     clock_        *clock;
00036
00037     mesh_         *mesh;
00038
00039     io_           *io;
00040
00041     ueqn_         *ueqn;
00042     peqn_         *peqn;
00043     teqn_         *teqn;
00044     les_          *les;
```

```

    overset_      *os;

    farm_         *farm;

    ibm_          *ibm;

    abl_          *abl;

    acquisition_  *acquisition;

    flags_        flags;
    access_       access;
};

#endif
```

5.33 flags.h

```


#ifndef _FLAGS_H_
#define _FLAGS_H_

struct flags_
{
    // simulation flags
    PetscInt isOversetActive;
    PetscInt isAdjustableTime;
    PetscInt isLesActive;
    PetscInt isTeqnActive;
    PetscInt isAquisitionActive;
    PetscInt isWindFarmActive;
    PetscInt isAblActive;
    PetscInt isIBMActive;
    PetscInt isZDampingActive;
    PetscInt isXDampingActive;
    PetscInt isYDampingActive;
    PetscInt isCanopyActive;
    PetscInt isPrecursorSpinUp;
    PetscInt isConcurrentPrecursorActive;
    PetscInt isPvCatalystActive;
    PetscInt isKLeftRayleighDampingActive;
    PetscInt isKRightRayleighDampingActive;
    PetscInt isAdvectionDampingXActive;
    PetscInt isAdvectionDampingYActive;
    PetscInt isSideForceActive;
    PetscInt isNonInertialFrameActive;
    PetscInt isGravityWaveModelingActive;
    PetscInt isMeangradPForcingActive;
};

#endif
```

5.34 /zfs60/nobackup/stipa/software/TOSCA/src/include/ibm.h File Reference

IBM model header file.

Classes

- struct node

  Node struct.
- struct cellNode
- struct list

  Node list.
- struct cellList

cell Node list

- struct searchBox
- struct ibmMesh
- struct ibmRotation
- struct ibmSineMotion
- struct ibmPitchMotion
- struct surface
- struct elementBox
- struct ibmObject
- struct ibmFluidCell
- struct ibm_

  struct storing IBM model

Macros

- #define **MAX_ELEMENTS_PER_NODE** 20

Typedefs

- typedef struct node **node**

  Node struct.
- typedef struct cellNode **cellNode**
- typedef struct list **list**

  Node list.
- typedef struct cellList **cellList**

  cell Node list

Functions

- PetscErrorCode **InitializeIBM** (ibm_ ∗ibm)

  initialize ibm: top level function
- PetscErrorCode **UpdateIBM** (ibm_ ∗ibm)

  update imb: top level function
- PetscErrorCode **writeIBMData** (ibm_ ∗ibm, PetscInt b)

  writes the angular position when the body is rotating
- PetscErrorCode **writeIBMForceData** (ibm_ ∗ibm, PetscInt b, PetscReal ∗gElemPressure, PetscReal net↩ Moment, PetscReal ibmPower, Cmpnts PresForce, Cmpnts ViscForce, Cmpnts momentVector)

  writes the IBM force data for element and net force
- PetscErrorCode **findIBMControlledProcs** (ibm_ ∗ibm)

  find in which processors the ibm body belongs
- PetscErrorCode **initElementProjectionProcs** (ibm_ ∗ibm)

  create list of ibm element processors and their closest ibm fluid cell to the ibm element normal projection
- PetscErrorCode **IBMProjectionProcessorTransfer** (ibm_ ∗ibm)

  transfer ibm mesh element across processors based on their movement - for dynamic case
- PetscErrorCode **createProcessorBufferZones** (ibm_ ∗ibm)

  create inner and outer buffer zones for ibm mesh element parallelization
- PetscErrorCode **initElementProcs** (ibm_ ∗ibm)

  create list of ibm element processors and their closest ibm fluid cell to the ibm element
- PetscErrorCode **IBMElementProcessorTransfer** (ibm_ ∗ibm)

  transfer ibm mesh element across processors based on their movement - for dynamic case

- PetscErrorCode **UpdateIBMesh** (ibm_ *ibm)

  update the ibm mesh when the ibm body is moving
- PetscErrorCode **rotateIBMesh** (ibm_ *ibm, PetscInt b)

  rotate the ibm mesh based on the angular speed input
- PetscErrorCode **sineMotion** (ibm_ *ibm, PetscInt b)

  prescribe sinusoidal motion for IBM body
- PetscErrorCode **pitchingMotion** (ibm_ *ibm, PetscInt b)

  prescribe pitching oscillation motion for IBM body
- PetscErrorCode **setIBMWallModels** (ibm_ *ibm)

  set IBM wall model type and properties
- PetscErrorCode **findBodyBoundingBox** (ibm_ *ibm)

  find the bounding box around an ibm body
- PetscErrorCode **findSearchCellDim** (ibm_ *ibm)

  find the search cell (coarser mesh around the ibm) dimension
- PetscErrorCode **ibmSearch** (ibm_ *ibm)

  ibm fluid node search performed using ray tracing algorithm
- PetscErrorCode **findClosestIBMElement** (ibm_ *ibm)

  find the closest ibm mesh element to a IBM fluid node
- PetscErrorCode **findClosestIBMElement2Solid** (ibm_ *ibm)
- PetscErrorCode **findInterceptionPoint** (ibm_ *ibm)

  find the interceptionPoint on the background mesh plane for curvib interpolation
- PetscErrorCode **interceptionPt** (Cmpnts pCoor, Cmpnts pc[9], Cmpnts eNorm, ibmFluidCell *ibF)

  interception point algorithm based on dividing the plane into 8 triangles
- PetscErrorCode **findIBMFluidCells** (ibm_ *ibm)

  find list of ibm fluid nodes within each processor
- PetscErrorCode **findFluidSupportNodes** (ibm_ *ibm)

  find the fluid nodes that act as support nodes to the given ibm fluid node
- PetscErrorCode **findIBMMeshSupportNodes** (ibm_ *ibm)

  find the ibm mesh nodes that act as support node to the given ibm fluid node
- PetscErrorCode **MLSInterpolation** (ibm_ *ibm)

  MLS interpolation algorithm.
- PetscErrorCode **CurvibInterpolation** (ibm_ *ibm)

  CURVIB normal projection interpolation algorithm.
- PetscErrorCode **CurvibInterpolationTriangular** (ibm_ *ibm)
- PetscErrorCode **CurvibInterpolationQuadratic** (ibm_ *ibm)
- PetscErrorCode **CurvibInterpolationInternalCell** (ibm_ *ibm)
- PetscErrorCode **ComputeForceMoment** (ibm_ *ibm)
- PetscErrorCode **findIBMWallShear** (ibm_ *ibm)

  compute shear stress at faces close to the IBM
- PetscErrorCode **findIBMWallShearChester** (ibm_ *ibm)

  compute shear stress at faces close to the IBM
- PetscErrorCode **checkIBMexists** (ibm_ *ibm)
- PetscErrorCode **createSearchCellList** (ibm_ *ibm)

  insert ibm mesh elements as a list into the search cell that they belong to
- PetscErrorCode **destroyLists** (ibm_ *ibm)

  destroy the ibm search cell list
- PetscErrorCode **computeIBMElementNormal** (ibm_ *ibm)
- PetscErrorCode **recomputeIBMeshProperties** (ibm_ *ibm, PetscInt b)
- PetscReal **rayCastingTest** (Cmpnts p, ibmMesh *ibMsh, cellIds sCell, searchBox *sBox, boundingBox *ib↩Box, list *searchCellList)

  ray casting algorithm

- PetscErrorCode **rayCastLocal** (Cmpnts p, Cmpnts p1, Cmpnts p2, Cmpnts p3, Cmpnts p4, ibmMesh *ibMsh, cellIds sCell, searchBox *sBox, boundingBox *ibBox, list *searchCellList, PetscInt &intersect)

  ray casting algorithm in local positive cell neighbourhood
- PetscErrorCode **elementBoundingSphere** (ibmObject *ibmBody)

  find element bounding sphere
- Cmpnts **randomdirection** (Cmpnts p, cellIds sCell, boundingBox *ibBox, searchBox *sBox, PetscInt seed)

  inline function to generate a random direction along the z index of the search cell for the ray casting algorithm
- PetscInt **intsectElement** (Cmpnts p, Cmpnts dir, Cmpnts node1, Cmpnts node2, Cmpnts node3, PetscReal *t, PetscReal *u, PetscReal *v)

  inline function to check if the ray intersects an ibm element
- PetscBool **isLineTriangleInt** (Cmpnts p1, Cmpnts p2, ibmMesh *ibMsh, PetscInt e)

  inline function to check if the line intersects an ibm element
- bool **isSearchCellSupport** (PetscReal rad, Cmpnts pt, cellIds cID, boundingBox *ibBox, searchBox *sBox)

  inline function to check if a search cell is a support to an ibm fluid node
- void **insertIBMSupportNodes** (Cmpnts pt, cellIds sCell, ibmFluidCell *ibF, ibmObject *ibBody, searchBox *sBox)

  insert ibm nodes into the list of support nodes around an ibm fluid node
- PetscInt **ISSameSide2D** (Cpt2D p, Cpt2D p1, Cpt2D p2, Cpt2D p3)

  Check whether 2D point p is located on the same side of line p1p2.
- PetscInt **ISInsideTriangle2D** (Cpt2D p, Cpt2D pa, Cpt2D pb, Cpt2D pc)

  check whether point p is inside the triangle - 2D
- PetscInt **isPointInTriangle** (Cmpnts p, Cmpnts p1, Cmpnts p2, Cmpnts p3, Cmpnts norm)

  check whether point p is inside the triangle - 3D
- void **disP2Line** (Cmpnts p, Cmpnts p1, Cmpnts p2, Cmpnts *po, PetscReal *d, PetscReal *t)

  find projection of point on a line and the distance to it
- void **triangleIntp** (Cpt2D p, Cpt2D p1, Cpt2D p2, Cpt2D p3, ibmFluidCell *ibF)

  find the interpolation weights of a point inside a triangle from its nodes
- void **triangleIntpBg** (Cpt2D p, Cpt2D p1, Cpt2D p2, Cpt2D p3, ibmFluidCell *ibF)

  find the interpolation weights of a point inside a triangle from its nodes used for background grid

5.34.1 Detailed Description

IBM model header file.

IBM input header file.

5.35 ibm.h

Go to the documentation of this file.

```
00001
00003
00004 #ifndef IBM_H
00005 #define IBM_H
00006
00007 #define MAX_ELEMENTS_PER_NODE 20
00008
00010 typedef struct node
00011 {
00012     PetscInt              Node;
00013     struct node           *next;
00014 } node;
00015
00016 typedef struct cellNode
00017 {
00018     cellIds         Node;
```

```
    struct cellNode *next;
} cellNode;

typedef struct list
{
    node         *head;
} list;

typedef struct cellList
{
    cellNode     *head;
} cellList;

typedef struct
{
    PetscInt     ncx, ncy, ncz;
    PetscReal    dcx, dcy, dcz;
} searchBox;


typedef struct
{
    PetscInt     nodes;
    PetscInt     elems;

    Cmpnts       *nCoor;
    PetscInt     *nID1 , *nID2 , *nID3 ;
    Cmpnts         *eN;
    Cmpnts       *eT1;
    Cmpnts       *eT2;

    PetscInt     *eSurface;
    PetscReal    *eA;
    Cmpnts       *eCent;
    Cmpnts       *nU;
    Cmpnts       *nUPrev;

    Cmpnts       *eQVec;
    PetscReal    *eRVec;


}ibmMesh;

typedef struct
{
    PetscReal     angSpeed;
    PetscReal     angAcc;
    PetscReal     rotAngle;
    PetscReal     maxR;

    Cmpnts        rotAxis;
    Cmpnts        rotCenter;

}ibmRotation;

typedef struct
{
    PetscReal     amplitude;
    PetscReal     frequency;
    Cmpnts        motionDir;
    PetscReal     tPrev;

}ibmSineMotion;

typedef struct
{
    PetscReal     amplitude;
    PetscReal     frequency;
    PetscReal     initAngPosition;
    Cmpnts        pitchAxis;
    Cmpnts        pitchCenter;
    PetscReal     tPrev;

}ibmPitchMotion;

typedef struct
{
    word     surfaceName;

    word     surfaceFileType;

    PetscInt surfaceId;

    Cmpnts   baseLocation;

    word     elementSet;

```

```
    // element localToGlobalMapping
    PetscInt      *elementMapping;

    ibmMesh  *ibMsh;
}surface;

// struct to define the bounding boxes around each process that define the local ibm mesh elements within each processor
typedef struct
{
    PetscInt      *thisElemControlled;
    PetscInt      *thisElemTransfered;

    boundingBox   innerZone;
    boundingBox   outerZone;

}elementBox;

typedef struct
{
    //ibm body parameters
    word          bodyName;
    word          bodyType;

    PetscInt      numSurfaces;
    PetscInt      bodyID;
    PetscInt      thinBody;

    word          elementSet;
    PetscInt      addSideFaces;

    ibmMesh       *ibMsh;
    surface       **ibmSurface;

    word          fileType;

    //ibm wall model properties
    wallModel     *ibmWallModelU;
    wallModel     *ibmWallModelT;

    //ibm velocity bc
    word          velocityBC;
    word          uBCSetType;
    PetscInt      wallFunctionTypeU;

    //ibm temp bc
    word          tempBC;
    word          tBCSetType;
    PetscInt      wallFunctionTypeT;
    PetscReal     fixedTemp;


    //ibm motion variables
    Cmpnts         baseLocation;
    word           bodyMotion;
    ibmRotation    *ibmRot;
    ibmSineMotion  *ibmSine;
    ibmPitchMotion *ibmPitch;

    //ibm search parameters
    PetscReal     searchCellRatio;
    boundingBox   *bound;
    list          *searchCellList;

    //ibm force parameters
    Cmpnts        *ibmPForce;

    Cmpnts        procBoundCenter;
    Cmpnts        procBoundSize;

    //element processor mapping for element normal projection - used in force calculation
    PetscInt      *thisPtControlled;
    PetscInt      *thisPtControlTransfer;
    cellIds       *closestCells;

    //flags
    PetscInt       ibmControlled;

    // communication color
    MPI_Comm            IBM_COMM;

    // element localToGlobalMapping
    PetscInt      *elementMapping;

    //local ibm elements box
    elementBox    *eBox;

```

```
}ibmObject;

typedef struct
{
  cellIds         cellId;
  cellList        flNodes;
  list            slNodes;
  list            bID;

  PetscInt        numFl;
  PetscInt        numSl;

  PetscReal       rad;

  PetscInt        closestElem;
  Cmpnts          pMin;
  PetscReal       minDist;
  PetscInt        bodyID;
  Cmpnts          normal;

  PetscReal       cs1, cs2, cs3;
  PetscReal       cr1, cr2, cr3;
  PetscInt        i1, j1, k1;
  PetscInt        i2, j2, k2;
  PetscInt        i3, j3, k3;
  PetscInt        imode;

  PetscReal       dB;
} ibmFluidCell;


struct ibm_
{
    Vec                lNvertFixed;
    PetscInt           numBodies;
    word               IBInterpolationModel;
    word               curvibType;
    word               curvibOrder;
    ibmObject          **ibmBody;

    searchBox          *sBox;

    ibmFluidCell       *ibmFCells;

    PetscInt           numIBMFluid;

    // access database
    access_              *access;

    // flags
    PetscInt                 dbg;
    PetscInt             dynamic;
    PetscInt        computeForce;
    PetscInt         checkNormal;
    PetscInt         wallShearOn;
    PetscInt            writeSTL;
    PetscInt              ibmABL;

    PetscReal         interpDist;
    // output parameters
    PetscReal          timeStart;
    word            intervalType;
    PetscReal       timeInterval;
};

#endif

// ============================================================================
// FUNCTIONS
// ============================================================================

PetscErrorCode InitializeIBM(ibm_ *ibm);

PetscErrorCode UpdateIBM(ibm_ *ibm);

PetscErrorCode writeIBMData(ibm_ *ibm, PetscInt b);

PetscErrorCode  writeIBMForceData(ibm_ *ibm, PetscInt b, PetscReal *gElemPressure, PetscReal
netMoment, PetscReal ibmPower, Cmpnts PresForce, Cmpnts ViscForce, Cmpnts momentVector);

PetscErrorCode findIBMControlledProcs(ibm_ *ibm);

PetscErrorCode initElementProjectionProcs(ibm_ *ibm);

PetscErrorCode IBMProjectionProcessorTransfer(ibm_ *ibm);

PetscErrorCode createProcessorBufferZones(ibm_ *ibm);
```

```
PetscErrorCode initElementProcs(ibm_ *ibm);

PetscErrorCode IBMElementProcessorTransfer(ibm_ *ibm);

PetscErrorCode UpdateIBMesh(ibm_ *ibm);

PetscErrorCode rotateIBMesh(ibm_ *ibm, PetscInt b);

PetscErrorCode sineMotion(ibm_ *ibm, PetscInt b);

PetscErrorCode pitchingMotion(ibm_ *ibm, PetscInt b);

PetscErrorCode setIBMWallModels(ibm_ *ibm);

PetscErrorCode findBodyBoundingBox(ibm_ *ibm);

PetscErrorCode findSearchCellDim(ibm_ *ibm);

PetscErrorCode ibmSearch(ibm_ *ibm);

PetscErrorCode findClosestIBMElement(ibm_ *ibm);

PetscErrorCode findClosestIBMElement2Solid(ibm_ *ibm);

PetscErrorCode findInterceptionPoint(ibm_ *ibm);

PetscErrorCode interceptionPt(Cmpnts pCoor, Cmpnts pc[9], Cmpnts eNorm, ibmFluidCell *ibF);

PetscErrorCode findIBMFluidCells(ibm_ *ibm);

PetscErrorCode findFluidSupportNodes(ibm_ *ibm);

PetscErrorCode findIBMMeshSupportNodes(ibm_ *ibm);

PetscErrorCode MLSInterpolation(ibm_ *ibm);

PetscErrorCode CurvibInterpolation(ibm_ *ibm);

PetscErrorCode CurvibInterpolationTriangular(ibm_ *ibm);

PetscErrorCode CurvibInterpolationQuadratic(ibm_ *ibm);

PetscErrorCode CurvibInterpolationInternalCell(ibm_ *ibm);

PetscErrorCode ComputeForceMoment(ibm_ *ibm);

PetscErrorCode findIBMWallShear(ibm_ *ibm);

PetscErrorCode findIBMWallShearChester(ibm_ *ibm);

// check if a given body exists in the mesh based on the IBMProperties.dat file
PetscErrorCode checkIBMexists(ibm_ *ibm);

PetscErrorCode createSearchCellList(ibm_ *ibm);

PetscErrorCode destroyLists(ibm_ *ibm);

PetscErrorCode computeIBMElementNormal(ibm_ *ibm);

PetscErrorCode recomputeIBMeshProperties(ibm_ *ibm, PetscInt b);

PetscReal rayCastingTest(Cmpnts p, ibmMesh *ibMsh, cellIds sCell, searchBox *sBox, boundingBox *ibBox,
    list *searchCellList);

PetscErrorCode rayCastLocal(Cmpnts p, Cmpnts p1, Cmpnts p2, Cmpnts p3, Cmpnts p4, ibmMesh *ibMsh,
    cellIds sCell, searchBox *sBox, boundingBox *ibBox, list *searchCellList, PetscInt &intersect);
PetscErrorCode elementBoundingSphere(ibmObject *ibmBody);

inline Cmpnts randomdirection(Cmpnts p, cellIds sCell, boundingBox *ibBox, searchBox *sBox, PetscInt
    seed);

inline PetscInt intsectElement(Cmpnts p, Cmpnts dir, Cmpnts node1, Cmpnts node2, Cmpnts node3,
    PetscReal *t, PetscReal *u, PetscReal *v);

inline PetscBool isLineTriangleInt(Cmpnts p1, Cmpnts p2, ibmMesh *ibMsh, PetscInt e);

inline bool isSearchCellSupport(PetscReal rad, Cmpnts pt, cellIds cID, boundingBox *ibBox, searchBox
    *sBox);

inline void insertIBMSupportNodes(Cmpnts pt, cellIds sCell, ibmFluidCell *ibF, ibmObject *ibBody,
    searchBox *sBox);

inline PetscInt ISSameSide2D(Cpt2D p, Cpt2D p1, Cpt2D p2, Cpt2D p3);

inline PetscInt ISInsideTriangle2D(Cpt2D p, Cpt2D pa, Cpt2D pb, Cpt2D pc);
```

```
inline PetscInt isPointInTriangle(Cmpnts p, Cmpnts p1, Cmpnts p2, Cmpnts p3, Cmpnts norm);

inline void disP2Line(Cmpnts p, Cmpnts p1, Cmpnts p2, Cmpnts *po, PetscReal *d, PetscReal *t);
inline void triangleIntp(Cpt2D p, Cpt2D p1, Cpt2D p2, Cpt2D p3, ibmFluidCell *ibF);

inline void triangleIntpBg(Cpt2D p, Cpt2D p1, Cpt2D p2, Cpt2D p3, ibmFluidCell *ibF);
```

5.36 ibmInput.h

```


// ==============================================================================
// FUNCTIONS
// ==============================================================================

PetscErrorCode readIBMProperties(ibm_ *ibm);

// \brief read the IBM mesh
PetscErrorCode readIBMObjectMesh(ibm_ *ibm, PetscInt b);

PetscErrorCode readIBMBodyFileGRD(ibmObject *ibmBody);

PetscErrorCode readIBMBodyFileASCII(ibmObject *ibmBody);

PetscErrorCode readIBMBodyFileUCD(ibmObject *ibmBody);

PetscErrorCode readIBMBodyFileSTL(ibmObject *ibmBody);

PetscErrorCode readIBMBodyFileUCD2(ibmObject *ibmBody);

PetscErrorCode readIBMBodyFileAbaqusInp(ibmObject *ibmBody);

PetscErrorCode readIBMSurfaceFileGRD(surface *ibmSurface);

PetscErrorCode readIBMSurfaceFileUCD(surface *ibmSurface);

PetscErrorCode readIBMSurfaceFileUCD2(surface *ibmSurface);

PetscErrorCode readIBMSurfaceFileAbaqusInp(surface *ibmSurface);

PetscErrorCode combineMesh(ibmObject *ibmBody);

PetscErrorCode writeSTLFile(ibm_ *ibm, PetscInt b);

```

5.37 /zfs60/nobackup/stipa/software/TOSCA/src/include/inflow.h File Reference

Contains inflow boundary condition headers.

Functions

- PetscErrorCode **SetInflowFunctions** (mesh_ ∗mesh)

  Initialize the inlet functions for inflow.
- PetscErrorCode **printInflowMappingAction** (mesh_ ∗mesh, inletFunctionTypes ∗ifPtr)

  print information regarding the inflow boundary mapping
- PetscErrorCode **mappedInflowInitialize** (inletFunctionTypes ∗ifPtr)

  allocate and set the inflow section fields
- PetscErrorCode **SetInflowWeights** (mesh_ ∗mesh, inletFunctionTypes ∗ifPtr)

  Calculate interpolation weights for the mapped interpolated inflow boundary condition.
- PetscErrorCode **readInflowU** (inletFunctionTypes ∗ifPtr, clock_ ∗clock)

  Reads inflow velocity from database and stores in inletFunction data.
- PetscErrorCode **readInflowT** (inletFunctionTypes ∗ifPtr, clock_ ∗clock)

Reads inflow tempereature from database and stores in inletFunction data.

- PetscErrorCode **readInflowNut** (inletFunctionTypes ∗ifPtr, clock_ ∗clock)

  Reads inflow sgs viscosity from database and stores in inletFunction data.
- PetscErrorCode **setShiftedInflowU** (inletFunctionTypes ∗ifPtr, ueqn_ ∗ueqn)

  Sets the un-shifted boundary field to be used with shifted periodic BC (inflow function type 7)
- PetscErrorCode **setShiftedInflowT** (inletFunctionTypes ∗ifPtr, teqn_ ∗teqn)

  Sets the un-shifted boundary field to be used with shifted periodic BC (inflow function type 7)
- PetscErrorCode **setShiftedInflowNut** (inletFunctionTypes ∗ifPtr, les_ ∗les)

  Sets the un-shifted boundary field to be used with shifted periodic BC (inflow function type 7)
- Cmpnts **NieuwstadtInflowEvaluate** (inletFunctionTypes ∗ifPtr, PetscReal h)

  Nieuwstadt model for velocity inflow type 5.

5.37.1 Detailed Description

Contains inflow boundary condition headers.

5.38 inflow.h

Go to the documentation of this file.

```
00001
00003
00004 // ==========================================================================
00005 // FUNCTIONS
00006 // ==========================================================================
00007
00009 PetscErrorCode SetInflowFunctions(mesh_ *mesh);
00010
00012 PetscErrorCode printInflowMappingAction(mesh_ *mesh, inletFunctionTypes *ifPtr);
00013
00015 PetscErrorCode mappedInflowInitialize(inletFunctionTypes *ifPtr);
00016
00018 PetscErrorCode SetInflowWeights(mesh_ *mesh, inletFunctionTypes *ifPtr);
00019
00021 PetscErrorCode readInflowU(inletFunctionTypes *ifPtr, clock_ *clock);
00022
00024 PetscErrorCode readInflowT(inletFunctionTypes *ifPtr, clock_ *clock);
00025
00027 PetscErrorCode readInflowNut(inletFunctionTypes *ifPtr, clock_ *clock);
00028
00030 PetscErrorCode setShiftedInflowU(inletFunctionTypes *ifPtr, ueqn_ *ueqn);
00031
00033 PetscErrorCode setShiftedInflowT(inletFunctionTypes *ifPtr, teqn_ *teqn);
00034
00036 PetscErrorCode setShiftedInflowNut(inletFunctionTypes *ifPtr, les_ *les);
00037
00039 Cmpnts NieuwstadtInflowEvaluate(inletFunctionTypes *ifPtr, PetscReal h);
```

5.39 /zfs60/nobackup/stipa/software/TOSCA/src/include/initialField.h File Reference

initial field header file.

Classes

- struct ModeData

Functions

- PetscErrorCode SetInitialField (domain_ *domain)

  <
- PetscErrorCode **SetInitialFieldPrecursor** (abl_ *abl)

  set the initial internal contravariant and cartesian velocity field
- PetscErrorCode **SetInitialFieldU** (ueqn_ *ueqn)

  set the initial temperature field
- PetscErrorCode **SetInitialFieldT** (teqn_ *teqn)

  set the initial pressure field
- PetscErrorCode **SetInitialFieldP** (peqn_ *peqn)

  set the initial variables for LES
- PetscErrorCode **SetInitialFieldLES** (les_ *les)

  set uniform value for the cartesian velocity (add perturbations if applicable)
- PetscErrorCode **SetUniformFieldU** (ueqn_ *ueqn, Cmpnts &uRef, PetscInt &addPerturbations)

  set initial ABL flow U
- PetscErrorCode **SetABLInitialFlowU** (ueqn_ *ueqn)
- PetscErrorCode **SetABLInitialFlowUZilitinkevich** (ueqn_ *ueqn)

  set initial field Taylor green vortex problem
- PetscErrorCode **SetTaylorGreenFieldU** (ueqn_ *ueqn, PetscReal &u0, PetscReal &freq)

  set the internal field as the spreaded inlet flow condition
- PetscErrorCode **SpreadInletFlowU** (ueqn_ *ueqn)

  set uniform value for the temperature
- PetscErrorCode **SetUniformFieldT** (teqn_ *teqn, PetscReal &tRef)

  set linear profile for the temperature
- PetscErrorCode **SetLinearFieldT** (teqn_ *teqn, PetscReal &tRef, PetscReal &tLapse)

  set const tRef + linear profile for the temperature above it like CNBL
- PetscErrorCode **SetLinearCNBLFieldT** (teqn_ *teqn, PetscReal &tRef, PetscReal &tLapse, PetscReal &h↩Ref)

  set initial ABL flow T
- PetscErrorCode **SetABLInitialFlowT** (teqn_ *teqn)

  set the internal field as the spreaded inlet flow condition
- PetscErrorCode **SpreadInletFlowT** (teqn_ *teqn)

  set the internal pressure field as the TGV flow
- PetscErrorCode **SetTaylorGreenFieldP** (peqn_ *peqn)

5.39.1 Detailed Description

initial field header file.

5.39.2 Function Documentation

5.39.2.1 SetInitialField()

```
PetscErrorCode SetInitialField (
            domain_ * domain)
```

<

set the initial internal fields

set the initial internal fields for the concurrent precursor simulation

5.40 initialField.h

Go to the documentation of this file.

```
#ifndef INITIALFIELD_H
#define INITIALFIELD_H

struct ModeData
{
    PetscReal   kappaM;
    PetscReal   khatx, khaty, khatz;
    PetscReal   amp;
    PetscReal   phi;
    PetscReal   ktx, kty, ktz;
    PetscReal   sx, sy, sz;
};

#endif

// ==============================================================================
// FUNCTIONS
// ==============================================================================

PetscErrorCode SetInitialField(domain_ *domain);

PetscErrorCode SetInitialFieldPrecursor(abl_ *abl);

PetscErrorCode SetInitialFieldU(ueqn_ *ueqn);

PetscErrorCode SetInitialFieldT(teqn_ *teqn);

PetscErrorCode SetInitialFieldP(peqn_ *peqn);

PetscErrorCode SetInitialFieldLES(les_ *les);

PetscErrorCode SetUniformFieldU(ueqn_ *ueqn, Cmpnts &uRef, PetscInt &addPerturbations);

PetscErrorCode SetABLInitialFlowU(ueqn_ *ueqn);

PetscErrorCode SetABLInitialFlowUZilitinkevich(ueqn_ *ueqn);

PetscErrorCode SetTaylorGreenFieldU(ueqn_ *ueqn, PetscReal &u0, PetscReal &freq);

PetscErrorCode SpreadInletFlowU(ueqn_ *ueqn);

PetscErrorCode SetUniformFieldT(teqn_ *teqn, PetscReal &tRef);

PetscErrorCode SetLinearFieldT(teqn_ *teqn, PetscReal &tRef, PetscReal &tLapse);

PetscErrorCode SetLinearCNBLFieldT(teqn_ *teqn, PetscReal &tRef, PetscReal &tLapse, PetscReal &hRef);

PetscErrorCode SetABLInitialFlowT(teqn_ *teqn);

PetscErrorCode SpreadInletFlowT(teqn_ *teqn);

PetscErrorCode SetTaylorGreenFieldP(peqn_ *peqn);
```

5.41 /zfs60/nobackup/stipa/software/TOSCA/src/include/initialization.h File Reference

Contains simulation initialization function headers.

Functions

- PetscErrorCode **PrintOkWindLogo** ()

  Print OkWind logo.
- PetscErrorCode **PrintNumberOfProcs** ()

  Print number of processors.
- PetscErrorCode **SetSimulationFlags** (flags_ *flags)

Set simulation flags.

- PetscErrorCode **SetSimulationInfo** (simInfo_ ∗info)

  Set simulation global info.
- PetscErrorCode **SetDomainsAndAllocate** (domain_ ∗∗domain, flags_ ∗flags, simInfo_ ∗info)

  Set number of domains (reads from OversetInput.dat file if overset is active)
- PetscErrorCode **ReadTimeControls** (clock_ ∗clock)

  Read time controls.
- PetscErrorCode **SetStartTime** (clock_ ∗clock, domain_ ∗domain, simInfo_ ∗info)

  Get startFrom parameter and set initial time (check multiple domains cosistency)
- PetscErrorCode **ReadPhysicalConstants** (domain_ ∗domain)

  Read physical constants.
- PetscErrorCode **SetDomainMemory** (domain_ ∗domain)

  Allocate memory for domain pointers.
- PetscErrorCode **SetAccessPointers** (domain_ ∗domain)

  Set access database pointers.
- PetscErrorCode simulationInitialize (domain_ ∗∗domainAddr, clock_ ∗clock, simInfo_ ∗info, flags_ ∗flags)

  Initialize the simulation parameters.
- PetscErrorCode SetInitialField (domain_ ∗domain)

  set the initial internal contravariant and cartesian velocity field
- PetscErrorCode **SetInitialField** (ueqn_ ∗ueqn)

5.41.1 Detailed Description

Contains simulation initialization function headers.

5.41.2 Function Documentation

5.41.2.1 SetInitialField()

```
PetscErrorCode SetInitialField (
            domain_ * domain)
```

set the initial internal contravariant and cartesian velocity field

set the initial internal contravariant and cartesian velocity field

set the initial internal fields

set the initial internal fields for the concurrent precursor simulation

5.41.2.2 simulationInitialize()

```
PetscErrorCode simulationInitialize (
            domain_ ** domainAddr,
            clock_ * clock,
            simInfo_ * info,
            flags_ * flags)
```

Initialize the simulation parameters.

set the initial internal fields

5.42 initialization.h

Go to the documentation of this file.

```
00001
00003
00004 // ==========================================================================
00005 // FUNCTIONS
00006 // ==========================================================================
00007
00009 PetscErrorCode PrintOkWindLogo();
00010
00012 PetscErrorCode PrintNumberOfProcs();
00013
00015 PetscErrorCode SetSimulationFlags(flags_ *flags);
00016
00018 PetscErrorCode SetSimulationInfo(simInfo_ *info);
00019
00021 PetscErrorCode SetDomainsAndAllocate(domain_ **domain, flags_ *flags, simInfo_ *info);
00022
00024 PetscErrorCode ReadTimeControls(clock_ *clock);
00025
00027 PetscErrorCode SetStartTime(clock_ *clock, domain_ *domain, simInfo_ *info);
00028
00030 PetscErrorCode ReadPhysicalConstants(domain_ *domain);
00031
00033 PetscErrorCode SetDomainMemory(domain_ *domain);
00034
00036 PetscErrorCode SetAccessPointers(domain_ *domain);
00037
00039 PetscErrorCode simulationInitialize(domain_ **domainAddr, clock_ *clock, simInfo_ *info, flags_
      *flags);
00040
00042 PetscErrorCode SetInitialField(domain_ *domain);
00043
00045 PetscErrorCode SetInitialField(ueqn_ *ueqn);
```

5.43 /zfs60/nobackup/stipa/software/TOSCA/src/include/inline.h File Reference

Inline functions.

```
#include "io.h"
#include "ibm.h"
#include <random>
```

Functions

- PetscReal **currentDistanceToWriteTime** (clock_ ∗clock, PetscReal timeStart, PetscReal timeInterval)
- void **timeStepSet** (clock_ ∗clock, PetscReal timeStart, PetscReal timeInterval, PetscReal dByU, PetscInt &flag, PetscReal &cfl)
- PetscInt **mustWrite** (PetscReal time, PetscReal startTime, PetscReal timeInterval)
- void **matMatProduct** (PetscReal ∗∗A, PetscReal ∗∗B, PetscReal ∗∗C, PetscInt numRowA, PetscInt num↩ ColA, PetscInt numRowB, PetscInt numColB)
- void **matVecProduct** (PetscReal ∗∗A, PetscReal ∗b, PetscReal ∗c, PetscInt numRowA, PetscInt numColA, PetscInt numRowB)
- void **gaussianSmooth1D** (PetscReal ∗input, PetscReal ∗output, PetscInt n, PetscInt window)
- PetscInt **isPresent** (PetscInt arr[ ], PetscInt n, PetscInt elem)
- PetscReal **sign** (PetscReal a)
- PetscReal **signNonZero** (PetscReal a)
- PetscReal **gcd** (PetscReal a, PetscReal b)
- PetscReal **gcdN** (PetscReal ∗v, PetscInt n)
- complex **gamma** (complex x)

- complex **digamma** (complex x)
- complex **hypergeom** (complex a, complex b, complex c, PetscReal z)
- PetscReal **polyLog2** (PetscReal x)
- PetscReal **splineB1** (PetscReal a)
- PetscReal **splineB2** (PetscReal a)
- PetscReal **angleDiff** (PetscReal a, PetscReal b)
- Cmpnts **nMax** (Cmpnts v, PetscReal c)
- Cmpnts **nAbs** (Cmpnts v)
- Cmpnts **nInv** (Cmpnts v)
- Cmpnts **nPow** (Cmpnts v, PetscReal c)
- Cmpnts **nScale** (PetscReal c, Cmpnts v)
- Cmpnts **nMultElWise** (Cmpnts v1, Cmpnts v2)
- Cmpnts **nDivElWise** (Cmpnts v1, Cmpnts v2)
- Cmpnts **nScaleX** (PetscReal c, Cmpnts v)
- Cmpnts **nScaleY** (PetscReal c, Cmpnts v)
- Cmpnts **nScaleZ** (PetscReal c, Cmpnts v)
- PetscReal **nDot** (Cmpnts v1, Cmpnts v2)
- PetscReal **nMag** (Cmpnts v)
- Cmpnts **nCross** (Cmpnts v1, Cmpnts v2)
- Cmpnts **nRot** (Cmpnts axis, Cmpnts vStart, PetscReal theta)
- Cmpnts **nTra** (Cmpnts point, Cmpnts translation)
- Cmpnts **nUnit** (Cmpnts v)
- Cmpnts **nSum** (Cmpnts v1, Cmpnts v2)
- Cmpnts **nSub** (Cmpnts v1, Cmpnts v2)
- Cmpnts **nSet** (Cmpnts value)
- Cmpnts **nSetFromComponents** (PetscReal vx, PetscReal vy, PetscReal vz)
- Cmpnts **nSetZero** ()
- void **mUnit** (Cmpnts &base)
- void **mSum** (Cmpnts &base, Cmpnts add)
- void **mSub** (Cmpnts &base, Cmpnts sub)
- void **mScale** (PetscReal c, Cmpnts &v)
- void **mScaleX** (PetscReal c, Cmpnts &v)
- void **mScaleY** (PetscReal c, Cmpnts &v)
- void **mScaleZ** (PetscReal c, Cmpnts &v)
- void **mSetScale** (PetscReal scale, Cmpnts &a, Cmpnts b)
- void **mRot** (Cmpnts axis, Cmpnts &v, PetscReal theta)
- void **mTra** (Cmpnts &point, Cmpnts translation)
- void **mSet** (Cmpnts &base, Cmpnts value)
- void **mSetValue** (Cmpnts &base, PetscReal value)
- void **AxByC** (PetscReal a, Cmpnts &X, PetscReal b, Cmpnts &Y, Cmpnts *C)

  C=aX+bY.
- bool **isInsideBoundingBox** (Cmpnts pt, const boundingBox &simBox)
- std::vector< PetscReal > **jPlaneScalarMean** (mesh_ *mesh, PetscReal ***s, PetscInt nLevels)
- void **Calculate_Covariant_metrics** (PetscReal g[3][3], PetscReal G[3][3])
- void **calculateNormal** (Cmpnts &csi, Cmpnts &eta, Cmpnts &zet, PetscReal ni[3], PetscReal nj[3], PetscReal nk[3])
- bool **isBoxIBMCell** (int k, int j, int i, PetscReal ***nvert)

  check if ibm cell within the neighbouring box of i,j,k
- bool **isFluidCell** (PetscInt k, PetscInt j, PetscInt i, PetscReal ***nvert)

  check if purely a fluid cell
- bool **isIBMCell** (PetscInt k, PetscInt j, PetscInt i, PetscReal ***nvert)

  check if IBM cell - this includes IBM fluid cells (cell next to IBM body) and IBM solid cells
- bool **isIBMFluidCell** (PetscInt k, PetscInt j, PetscInt i, PetscReal ***nvert)

  check if IBM Fluid cell (cell next to IBM body)

- bool **isIBMSolidCell** (PetscInt k, PetscInt j, PetscInt i, PetscReal ∗∗∗nvert)

  check if IBM solid cell
- bool **isFluidKFace** (PetscInt kL, PetscInt j, PetscInt i, PetscInt kR, PetscReal ∗∗∗nvert)

  check is a fluid k face - shared by 2 fluid cells
- bool **isFluidJFace** (PetscInt k, PetscInt jL, PetscInt i, PetscInt jR, PetscReal ∗∗∗nvert)

  check is a fluid j face - shared by 2 fluid cells
- bool **isFluidIFace** (PetscInt k, PetscInt j, PetscInt iL, PetscInt iR, PetscReal ∗∗∗nvert)

  check is a fluid i face - shared by 2 fluid cells
- bool **isIBMKFace** (PetscInt kL, PetscInt j, PetscInt i, PetscInt kR, PetscReal ∗∗∗nvert)

  check is a IBM k face - shared by atleast one IBM cell
- bool **isIBMJFace** (PetscInt k, PetscInt jL, PetscInt i, PetscInt jR, PetscReal ∗∗∗nvert)

  check is a IBM j face - shared by atleast one IBM cell
- bool **isIBMIFace** (PetscInt k, PetscInt j, PetscInt iL, PetscInt iR, PetscReal ∗∗∗nvert)

  check is a IBM i face - shared by atleast one IBM cell
- bool **isIBMFluidKFace** (PetscInt kL, PetscInt j, PetscInt i, PetscInt kR, PetscReal ∗∗∗nvert)

  check if IBM fluid k face - atleast one of the shared cells is IBM fluid
- bool **isIBMFluidJFace** (PetscInt k, PetscInt jL, PetscInt i, PetscInt jR, PetscReal ∗∗∗nvert)

  check if IBM fluid j face - at least one of the shared cells is IBM fluid
- bool **isIBMFluidIFace** (PetscInt k, PetscInt j, PetscInt iL, PetscInt iR, PetscReal ∗∗∗nvert)

  check if IBM fluid i face - at least one of the shared cells is IBM fluid
- bool **isIBMSolidKFace** (PetscInt kL, PetscInt j, PetscInt i, PetscInt kR, PetscReal ∗∗∗nvert)
- bool **isIBMSolidJFace** (PetscInt k, PetscInt jL, PetscInt i, PetscInt jR, PetscReal ∗∗∗nvert)

  check if IBM fluid j face - at least one of the shared cells is IBM fluid
- bool **isIBMSolidIFace** (PetscInt k, PetscInt j, PetscInt iL, PetscInt iR, PetscReal ∗∗∗nvert)

  check if IBM fluid i face - at least one of the shared cells is IBM fluid
- bool **isBoxOversetCell** (int k, int j, int i, PetscReal ∗∗∗meshTag)
- bool **isInterpolatedCell** (int k, int j, int i, PetscReal ∗∗∗meshTag)

  check if the given cell is interpolated
- bool **isZeroedCell** (int k, int j, int i, PetscReal ∗∗∗meshTag)

  check if the given cell is set to 0 - interior of overset mesh
- bool **isOversetCell** (int k, int j, int i, PetscReal ∗∗∗meshTag)
- bool **isCalculatedCell** (int k, int j, int i, PetscReal ∗∗∗meshTag)
- bool **isInterpolatedKFace** (int kL, int j, int i, int kR, PetscReal ∗∗∗meshTag)

  check if the given face is interpolated
- bool **isInterpolatedJFace** (int k, int jL, int i, int jR, PetscReal ∗∗∗meshTag)
- bool **isInterpolatedIFace** (int k, int j, int iL, int iR, PetscReal ∗∗∗meshTag)
- bool **isZeroedKFace** (int kL, int j, int i, int kR, PetscReal ∗∗∗meshTag)

  check if the given face is shared by a zeroed cell
- bool **isZeroedJFace** (int k, int jL, int i, int jR, PetscReal ∗∗∗meshTag)
- bool **isZeroedIFace** (int k, int j, int iL, int iR, PetscReal ∗∗∗meshTag)
- bool **isOversetKFace** (int kL, int j, int i, int kR, PetscReal ∗∗∗meshTag)

  check if the given face is shared by an overset cell
- bool **isOversetJFace** (int k, int jL, int i, int jR, PetscReal ∗∗∗meshTag)
- bool **isOversetIFace** (int k, int j, int iL, int iR, PetscReal ∗∗∗meshTag)
- bool **isCalculatedKFace** (int kL, int j, int i, int kR, PetscReal ∗∗∗meshTag)

  check if the given face is shared by only calculated cells
- bool **isCalculatedJFace** (int k, int jL, int i, int jR, PetscReal ∗∗∗meshTag)
- bool **isCalculatedIFace** (int k, int j, int iL, int iR, PetscReal ∗∗∗meshTag)
- void **resetNonResolvedCellCentersScalar** (mesh_ ∗mesh, Vec &V)

  Resets the value at the non-solved centers to zero.
- void **scaleNonResolvedCellCentersScalar** (mesh_ ∗mesh, Vec &V, PetscReal scale)

Resets the value at the non-solved centers to zero.

- void **ContravariantToCartesianPoint** (Cmpnts &csi, Cmpnts &eta, Cmpnts &zet, Cmpnts &ucont, Cmpnts *ucat)

  Interpolates cartesian velocity at cell center from the contravariant flux around a cell.
- void **resetNonResolvedCellFaces** (mesh_ *mesh, Vec &V)

  Resets the value at the non-solved faces to zero except for zeroGradient, outgoing flux and periodic.
- bool **isOnNonResolvedCellCenters** (PetscInt i, PetscInt j, PetscInt k, DMDALocalInfo info)
- bool **isOnCornerCellCenters** (PetscInt i, PetscInt j, PetscInt k, DMDALocalInfo info)
- void **getCell2Cell3StencilCsiNoGhost** (mesh_ *mesh, PetscInt i, PetscInt mx, PetscInt *iL, PetscInt *iR)

  Get cell's idxs of a 3 cell-sencil in csi direction given the cell idx. NEVER USE GHOSTS.
- void **getCell2Cell3StencilEtaNoGhost** (mesh_ *mesh, PetscInt j, PetscInt my, PetscInt *jL, PetscInt *jR)

  Get cell's idxs of a 3 cell-sencil in eta direction given the cell idx. NEVER USE GHOSTS.
- void **getCell2Cell3StencilZetNoGhost** (mesh_ *mesh, PetscInt k, PetscInt mz, PetscInt *kL, PetscInt *kR)

  Get cell's idxs of a 3 cell-sencil in zet direction given the cell idx. NEVER USE GHOSTS.
- void **getCell2Cell3StencilCsi** (mesh_ *mesh, PetscInt i, PetscInt mx, PetscInt *iL, PetscInt *iR)

  Get cell's idxs of a 3 cell-sencil in csi direction given the cell idx.
- void **getCell2Cell3StencilEta** (mesh_ *mesh, PetscInt j, PetscInt my, PetscInt *jL, PetscInt *jR)

  Get cell's idxs of a 3 cell-sencil in eta direction given the cell idx.
- void **getCell2Cell3StencilZet** (mesh_ *mesh, PetscInt k, PetscInt mz, PetscInt *kL, PetscInt *kR)

  Get cell's idxs of a 3 cell-sencil in zet direction given the cell idx.
- void **getFace2Cell4StencilCsi** (mesh_ *mesh, PetscInt k, PetscInt j, PetscInt i, PetscInt mx, PetscInt *iL, PetscInt *iR, PetscReal *denom, PetscReal ***nvert, PetscReal ***meshTag)

  Get outmost cell's idxs of a 4 cell-sencil in csi direction given the face idx.
- void **getFace2Cell4StencilEta** (mesh_ *mesh, PetscInt k, PetscInt j, PetscInt i, PetscInt my, PetscInt *jL, PetscInt *jR, PetscReal *denom, PetscReal ***nvert, PetscReal ***meshTag)

  Get outmost cell's idxs of a 4 cell-sencil in eta direction given the face idx.
- void **getFace2Cell4StencilZet** (mesh_ *mesh, PetscInt k, PetscInt j, PetscInt i, PetscInt mz, PetscInt *kL, PetscInt *kR, PetscReal *denom, PetscReal ***nvert, PetscReal ***meshTag)

  Get outmost cell's idxs of a 4 cell-sencil in zet direction given the face idx.
- void **getFace2Face3StencilCsi** (mesh_ *mesh, PetscInt i, PetscInt mx, PetscInt *iL, PetscInt *iR)

  Get neighboring face idxs of a 3 face-sencil in csi direction given the central face idx.
- void **getFace2Face3StencilEta** (mesh_ *mesh, PetscInt j, PetscInt my, PetscInt *jL, PetscInt *jR)

  Get neighboring face idxs of a 3 face-sencil in eta direction given the central face idx.
- void **getFace2Face3StencilZet** (mesh_ *mesh, PetscInt k, PetscInt mz, PetscInt *kL, PetscInt *kR)

  Get neighboring face idxs of a 3 face-sencil in zeta direction given the central face idx.
- void **getFace2Face7StencilCsi** (mesh_ *mesh, PetscInt i, PetscInt mx, PetscInt *m3, PetscInt *m2, PetscInt *m1, PetscInt *p1, PetscInt *p2, PetscInt *p3)

  Get neighboring face idxs of a 7 face-sencil in csi direction given the central face idx.
- void **getFace2Face7StencilEta** (mesh_ *mesh, PetscInt j, PetscInt my, PetscInt *m3, PetscInt *m2, PetscInt *m1, PetscInt *p1, PetscInt *p2, PetscInt *p3)

  Get neighboring face idxs of a 7 face-sencil in eta direction given the central face idx.
- void **getFace2Face7StencilZet** (mesh_ *mesh, PetscInt k, PetscInt mz, PetscInt *m3, PetscInt *m2, PetscInt *m1, PetscInt *p1, PetscInt *p2, PetscInt *p3)

  Get neighboring face idxs of a 7 face-sencil in zeta direction given the central face idx.
- void **getCell2Cell7StencilCsi** (mesh_ *mesh, PetscInt i, PetscInt mx, PetscInt *m3, PetscInt *m2, PetscInt *m1, PetscInt *p1, PetscInt *p2, PetscInt *p3)

  Get neighboring cell idxs of a 7 face-sencil in csi direction given the central cell idx.
- void **getCell2Cell7StencilEta** (mesh_ *mesh, PetscInt j, PetscInt my, PetscInt *m3, PetscInt *m2, PetscInt *m1, PetscInt *p1, PetscInt *p2, PetscInt *p3)

  Get neighboring cell idxs of a 7 face-sencil in eta direction given the central cell idx.
- void **getCell2Cell7StencilZet** (mesh_ *mesh, PetscInt k, PetscInt mz, PetscInt *m3, PetscInt *m2, PetscInt *m1, PetscInt *p1, PetscInt *p2, PetscInt *p3)

Get neighboring cell idxs of a 7 face-sencil in zeta direction given the central cell idx.

- void **Compute_du_i** (mesh_ *mesh, PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz, Cmpnts ***ucat, PetscReal ***nvert, PetscReal ***meshTag, PetscReal *dudc, PetscReal *dvdc, PetscReal *dwdc, PetscReal *dude, PetscReal *dvde, PetscReal *dwde, PetscReal *dudz, PetscReal *dvdz, PetscReal *dwdz)
- void **Compute_du_j** (mesh_ *mesh, PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz, Cmpnts ***ucat, PetscReal ***nvert, PetscReal ***meshTag, PetscReal *dudc, PetscReal *dvdc, PetscReal *dwdc, PetscReal *dude, PetscReal *dvde, PetscReal *dwde, PetscReal *dudz, PetscReal *dvdz, PetscReal *dwdz)
- void **Compute_du_k** (mesh_ *mesh, PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz, Cmpnts ***ucat, PetscReal ***nvert, PetscReal ***meshTag, PetscReal *dudc, PetscReal *dvdc, PetscReal *dwdc, PetscReal *dude, PetscReal *dvde, PetscReal *dwde, PetscReal *dudz, PetscReal *dvdz, PetscReal *dwdz)
- void **Compute_du_i_4** (mesh_ *mesh, PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz, Cmpnts ***ucat, PetscReal ***nvert, PetscReal ***meshTag, PetscReal *dudc, PetscReal *dvdc, PetscReal *dwdc, PetscReal *dude, PetscReal *dvde, PetscReal *dwde, PetscReal *dudz, PetscReal *dvdz, PetscReal *dwdz)
- void **Compute_du_j_4** (mesh_ *mesh, PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz, Cmpnts ***ucat, PetscReal ***nvert, PetscReal ***meshTag, PetscReal *dudc, PetscReal *dvdc, PetscReal *dwdc, PetscReal *dude, PetscReal *dvde, PetscReal *dwde, PetscReal *dudz, PetscReal *dvdz, PetscReal *dwdz)
- void **Compute_du_k_4** (mesh_ *mesh, PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz, Cmpnts ***ucat, PetscReal ***nvert, PetscReal ***meshTag, PetscReal *dudc, PetscReal *dvdc, PetscReal *dwdc, PetscReal *dude, PetscReal *dvde, PetscReal *dwde, PetscReal *dudz, PetscReal *dvdz, PetscReal *dwdz)
- void **Compute_dscalar_i** (mesh_ *mesh, PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz, PetscReal ***K, PetscReal ***nvert, PetscReal ***meshTag, PetscReal *dkdc, PetscReal *dkde, PetscReal *dkdz)
- void **Compute_dscalar_j** (mesh_ *mesh, PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz, PetscReal ***K, PetscReal ***nvert, PetscReal ***meshTag, PetscReal *dkdc, PetscReal *dkde, PetscReal *dkdz)
- void **Compute_dscalar_k** (mesh_ *mesh, PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz, PetscReal ***K, PetscReal ***nvert, PetscReal ***meshTag, PetscReal *dkdc, PetscReal *dkde, PetscReal *dkdz)
- void **Compute_du_center** (mesh_ *mesh, PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz, Cmpnts ***ucat, PetscReal ***nvert, PetscReal ***meshTag, PetscReal *dudc, PetscReal *dvdc, PetscReal *dwdc, PetscReal *dude, PetscReal *dvde, PetscReal *dwde, PetscReal *dudz, PetscReal *dvdz, PetscReal *dwdz)
- void **Compute_du_center4th** (mesh_ *mesh, PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz, Cmpnts ***ucat, PetscReal ***nvert, PetscReal ***meshTag, PetscReal *dudc, PetscReal *dvdc, PetscReal *dwdc, PetscReal *dude, PetscReal *dvde, PetscReal *dwde, PetscReal *dudz, PetscReal *dvdz, PetscReal *dwdz)
- void **Compute_dscalar_center** (mesh_ *mesh, PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz, PetscReal ***K, PetscReal ***nvert, PetscReal ***meshTag, PetscReal *dkdc, PetscReal *dkde, PetscReal *dkdz)
- PetscReal **integrateTestfilterSimpson** (PetscReal val[3][3][3], PetscReal w[3][3][3])
- PetscReal **integrateTestfilterSimpson5x5** (PetscReal val[5][5][5], PetscReal w[5][5][5])
- void **Compute_du_dxyz** (mesh_ *mesh, PetscReal csi0, PetscReal csi1, PetscReal csi2, PetscReal eta0, PetscReal eta1, PetscReal eta2, PetscReal zet0, PetscReal zet1, PetscReal zet2, PetscReal ajc, PetscReal dudc, PetscReal dvdc, PetscReal dwdc, PetscReal dude, PetscReal dvde, PetscReal dwde, PetscReal dudz, PetscReal dvdz, PetscReal dwdz, PetscReal *du_dx, PetscReal *dv_dx, PetscReal *dw_dx, PetscReal *du_dy, PetscReal *dv_dy, PetscReal *dw_dy, PetscReal *du_dz, PetscReal *dv_dz, PetscReal *dw_dz)
- void **Compute_dscalar_dxyz** (mesh_ *mesh, PetscReal csi0, PetscReal csi1, PetscReal csi2, PetscReal eta0, PetscReal eta1, PetscReal eta2, PetscReal zet0, PetscReal zet1, PetscReal zet2, PetscReal ajc, PetscReal dkdc, PetscReal dkde, PetscReal dkdz, PetscReal *dk_dx, PetscReal *dk_dy, PetscReal *dk_dz)
- void **Compute_du_wmLocal** (mesh_ *mesh, Cmpnts eN, Cmpnts eT1, Cmpnts eT2, PetscReal du_dx, PetscReal dv_dx, PetscReal dw_dx, PetscReal du_dy, PetscReal dv_dy, PetscReal dw_dy, PetscReal du_dz,

PetscReal dv_dz, PetscReal dw_dz, PetscReal *dut1dn, PetscReal *dut2dn, PetscReal *dundn, PetscReal *dut1dt1, PetscReal *dut2dt1, PetscReal *dundt1, PetscReal *dut1dt2, PetscReal *dut2dt2, PetscReal *dundt2)

- void **Comput_JacobTensor_i** (PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz, Cmpnts ***coor, PetscReal *dxdc, PetscReal *dxde, PetscReal *dxdz, PetscReal *dydc, PetscReal *dyde, PetscReal *dydz, PetscReal *dzdc, PetscReal *dzde, PetscReal *dzdz)
- void **Comput_JacobTensor_j** (PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz, Cmpnts ***coor, PetscReal *dxdc, PetscReal *dxde, PetscReal *dxdz, PetscReal *dydc, PetscReal *dyde, PetscReal *dydz, PetscReal *dzdc, PetscReal *dzde, PetscReal *dzdz)
- void **Comput_JacobTensor_k** (PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz, Cmpnts ***coor, PetscReal *dxdc, PetscReal *dxde, PetscReal *dxdz, PetscReal *dydc, PetscReal *dyde, PetscReal *dydz, PetscReal *dzdc, PetscReal *dzde, PetscReal *dzdz)
- void **Compute_du_Compgrid** (PetscReal dxdc, PetscReal dxde, PetscReal dxdz, PetscReal dydc, PetscReal dyde, PetscReal dydz, PetscReal dzdc, PetscReal dzde, PetscReal dzdz, PetscReal nx, PetscReal ny, PetscReal nz, PetscReal t1x, PetscReal t1y, PetscReal t1z, PetscReal t2x, PetscReal t2y, PetscReal t2z, PetscReal dut1dn, PetscReal dut2dn, PetscReal dundn, PetscReal dut1dt1, PetscReal dut2dt1, PetscReal dundt1, PetscReal dut1dt2, PetscReal dut2dt2, PetscReal dundt2, PetscReal *dudc, PetscReal *dvdc, PetscReal *dwdc, PetscReal *dude, PetscReal *dvde, PetscReal *dwde, PetscReal *dudz, PetscReal *dvdz, PetscReal *dwdz)
- PetscReal **minMod** (PetscReal m1, PetscReal m2)
- PetscReal **vanLeer** (PetscReal f0, PetscReal f1, PetscReal f2)
- PetscReal **central** (PetscReal f0, PetscReal f1)
- Cmpnts **centralVec** (Cmpnts f0, Cmpnts f1)
- symmTensor **centralSymmT** (symmTensor f0, symmTensor f1)
- PetscReal **wCentral** (PetscReal f0, PetscReal f1, PetscReal d0, PetscReal d1)
- Cmpnts **wCentralVec** (Cmpnts f0, Cmpnts f1, PetscReal d0, PetscReal d1)
- Cmpnts **centralVec4thCsi** (mesh_ *mesh, PetscInt k, PetscInt j, PetscInt i, PetscInt mx, PetscReal ***nvert, PetscReal ***meshTag, Cmpnts ***ucat, PetscReal wavespeed, PetscReal hyperVisc)
- Cmpnts **centralVec4thEta** (mesh_ *mesh, PetscInt k, PetscInt j, PetscInt i, PetscInt my, PetscReal ***nvert, PetscReal ***meshTag, Cmpnts ***ucat, PetscReal wavespeed, PetscReal hyperVisc)
- Cmpnts **centralVec4thZet** (mesh_ *mesh, PetscInt k, PetscInt j, PetscInt i, PetscInt mz, PetscReal ***nvert, PetscReal ***meshTag, Cmpnts ***ucat, PetscReal wavespeed, PetscReal hyperVisc)
- PetscReal **central4** (PetscReal f0, PetscReal f1, PetscReal f2, PetscReal f3)
- PetscReal **centralUpwind** (PetscReal f0, PetscReal f1, PetscReal f2, PetscReal f3, PetscReal wavespeed)
- Cmpnts **centralUpwindVec** (Cmpnts f0, Cmpnts f1, Cmpnts f2, Cmpnts f3, PetscReal wavespeed)
- PetscReal **centralUpwind** (PetscReal f0, PetscReal f1, PetscReal f2, PetscReal f3, PetscReal wavespeed, PetscReal limiter)
- PetscReal **upwind** (PetscReal f1, PetscReal f2, PetscReal wavespeed)
- Cmpnts **centralUpwindVec** (Cmpnts f0, Cmpnts f1, Cmpnts f2, Cmpnts f3, PetscReal wavespeed, PetscReal limiter)
- PetscReal **wCentralUpwind** (PetscReal f0, PetscReal f1, PetscReal f2, PetscReal f3, PetscReal d0, PetscReal d1, PetscReal d2, PetscReal d3, PetscReal wavespeed, PetscReal limiter)
- Cmpnts **wCentralUpwindVec** (Cmpnts f0, Cmpnts f1, Cmpnts f2, Cmpnts f3, PetscReal d0, PetscReal d1, PetscReal d2, PetscReal d3, PetscReal wavespeed, PetscReal limiter)
- PetscReal **quadraticUpwind** (PetscReal f0, PetscReal f1, PetscReal f2, PetscReal f3, PetscReal wavespeed)
- Cmpnts **quadraticUpwindVec** (Cmpnts f0, Cmpnts f1, Cmpnts f2, Cmpnts f3, PetscReal wavespeed)
- PetscReal **wQuadraticUpwind** (PetscReal f0, PetscReal f1, PetscReal f2, PetscReal f3, PetscReal d0, PetscReal d1, PetscReal d2, PetscReal d3, PetscReal wavespeed)
- PetscReal **weno3** (PetscReal f0, PetscReal f1, PetscReal f2, PetscReal f3, PetscReal wavespeed)
- Cmpnts **weno3Vec** (Cmpnts f0, Cmpnts f1, Cmpnts f2, Cmpnts f3, PetscReal wavespeed)
- PetscReal **weno5** (PetscReal f0, PetscReal f1, PetscReal f2, PetscReal f3, PetscReal f4, PetscReal f5, PetscReal wavespeed)
- void **resetNoPenetrationFluxes** (ueqn_ *ueqn)
- void **resetFacePeriodicFluxesVector** (mesh_ *mesh, Vec V, Vec lV, const char *scatterType)
- void **resetCellPeriodicFluxes** (mesh_ *mesh, Vec &V, Vec &lV, const char *type, const char *scatterType)

  Reset periodic values and scatter.

- PetscReal **viscRayleigh** (PetscReal &alpha, PetscReal &hS, PetscReal &hE, PetscReal &h)
- PetscReal **viscCosAscending** (PetscReal &alpha, PetscReal &hS, PetscReal &hE, PetscReal &h)
- PetscReal **viscCosDescending** (PetscReal &alpha, PetscReal &hS, PetscReal &hE, PetscReal &h)
- PetscReal **viscNordstrom** (PetscReal &alpha, PetscReal &hS, PetscReal &hE, PetscReal &delta, PetscReal &h)
- double **viscNordstromNoVertFilter** (double hS, double hE, double &delta, double &h)
- double **viscStipa** (double &hS, double &hE, double &delta, double &h, double &z, double &H)
- double **viscStipaDelta** (double &hS, double &hE, double &deltaS, double &deltaE, double &h, double &z, double &H)
- void **findInterpolationWeights** (PetscReal ∗weights, PetscInt ∗labels, PetscReal ∗pvec, PetscInt npts, PetscReal pval)
- void **findInterpolationWeightsWithExtrap** (PetscReal ∗weights, PetscInt ∗labels, PetscReal ∗pvec, PetscInt npts, PetscReal pval)
- PetscReal **scaleHyperTangBot** (PetscReal h, PetscReal H, PetscReal delta)
- PetscReal **scaleHyperTangTop** (PetscReal h, PetscReal H, PetscReal delta)
- PetscReal computeEm (Cmpnts &avgU, symmTensor &avgUprimeUprime, PetscReal avgP)

  Computes mechanical energy (Em = + <v><v> + <w><w> + <u'u'> + <v'v'> + <w'w'> +.
- PetscReal **computeEmTilde** (Cmpnts &avgU, symmTensor &avgUprimeUprime)

  Computes modified mechanical energy (Em = + <V><V> + <W><W> + <U'U'> + <V'V'> + <W'W'>) using contrav. fluxes.
- PetscReal **computeMKE** (Cmpnts &avgU)
- PetscReal **computeTKE** (symmTensor &avgUprimeUprime)
- void **scalarPointLocalVolumeInterpolation** (mesh_ ∗mesh, PetscReal px, PetscReal py, PetscReal pz, PetscInt ic, PetscInt jc, PetscInt kc, Cmpnts ∗∗∗cent, PetscReal ∗∗∗v, PetscReal &result)

  Trilinear volume interpolation function for scalars.
- void **PointInterpolationWeights** (mesh_ ∗mesh, PetscReal px, PetscReal py, PetscReal pz, PetscInt ic, PetscInt jc, PetscInt kc, Cmpnts ∗∗∗cent, PetscReal ∗intWts, PetscInt ∗intId)

  Trilinear volume interpolation function for scalars to return interpolation Weights.
- void **PointInterpolationCells** (mesh_ ∗mesh, PetscReal px, PetscReal py, PetscReal pz, PetscInt ic, PetscInt jc, PetscInt kc, Cmpnts ∗∗∗cent, PetscInt ∗intId)

  Trilinear volume interpolation function for scalars to return interpolation cells.
- void **vectorPointLocalVolumeInterpolation** (mesh_ ∗mesh, PetscReal px, PetscReal py, PetscReal pz, PetscInt ic, PetscInt jc, PetscInt kc, Cmpnts ∗∗∗cent, Cmpnts ∗∗∗v, Cmpnts &result)

  Trilinear volume interpolation function for vectors.
- void **scalarPointB2SplineVolumeInterpolation** (mesh_ ∗mesh, PetscReal px, PetscReal py, PetscReal pz, PetscInt ic, PetscInt jc, PetscInt kc, Cmpnts ∗∗∗cent, PetscReal ∗∗∗v, PetscReal &result)

  Quadratic B-spline interpolation for a scalar field.
- void **vectorPointB2SplineVolumeInterpolation** (mesh_ ∗mesh, PetscReal px, PetscReal py, PetscReal pz, PetscInt ic, PetscInt jc, PetscInt kc, Cmpnts ∗∗∗cent, Cmpnts ∗∗∗v, Cmpnts &result)

  Quadratic B-spline interpolation for a vector field.
- void **initlist** (list ∗ilist)
- void **initCellList** (cellList ∗ilist)
- bool **insertnode** (list ∗ilist, PetscInt Node)
- void **insertnode1** (list ∗ilist, PetscInt Node)
- bool **insertCellNode** (cellList ∗ilist, cellIds Node)
- void **insertCellNode1** (cellList ∗ilist, cellIds Node)
- void **destroy** (list ∗ilist)
- void **destroyCellList** (cellList ∗ilist)
- void **reorderMatrix** (PetscReal ∗∗A, PetscReal ∗∗B, PetscInt N, PetscInt pivot)
- void **elimJordanAlg** (PetscReal ∗∗A, PetscReal ∗∗B, PetscInt N, PetscInt pivot)
- void **inverseMatrix** (PetscReal ∗∗A, PetscReal ∗∗inv_A, PetscInt size)
- void **inv_4by4** (PetscReal ∗∗A, PetscReal ∗∗inv_A, PetscInt size)
- void **inv_3by3** (PetscReal ∗∗A, PetscReal ∗∗inv_A, PetscInt size)
- void **reorder_20** (PetscReal A[ ][20], PetscReal B[ ][20], PetscInt N, PetscInt pivot)
- void **elim_jordan_20** (PetscReal A[ ][20], PetscReal B[ ][20], PetscInt N, PetscInt pivot)

- void **inv_20** (PetscReal A[20][20], PetscReal inv_A[ ][20], PetscInt N)
- void **reorder_10** (PetscReal A[ ][10], PetscReal B[ ][10], PetscInt N, PetscInt pivot)
- void **elim_jordan_10** (PetscReal A[ ][10], PetscReal B[ ][10], PetscInt N, PetscInt pivot)
- void **inv_10** (PetscReal A[10][10], PetscReal inv_A[ ][10], PetscInt N)
- void **mult_mats3_lin** (PetscReal ∗∗inv_A, PetscReal ∗∗B, PetscInt nsupport, PetscReal ∗PHI)

5.43.1 Detailed Description

Inline functions.

5.43.2 Function Documentation

5.43.2.1 computeEm()

```
PetscReal computeEm (
            Cmpnts & avgU,
            symmTensor & avgUprimeUprime,
            PetscReal avgP)  [inline]
```

Computes mechanical energy (Em = + <v><v> + <w><w> + <u'u'> + <v'v'> + <w'w'> +.

/rho)

5.44 inline.h

Go to the documentation of this file.

```
#ifndef _INLINE_
#define _INLINE_

#include "io.h"
#include "ibm.h"
#include <random>

// DYNAMIC TIME STEPPING
//
//

inline PetscReal currentDistanceToWriteTime(clock_ *clock, PetscReal timeStart, PetscReal timeInterval)
{
    PetscReal currentAcquistionPeriodFraction = ((clock->time - timeStart) / timeInterval - std::floor((clock->time - timeStart) / timeInterval + 1e-10));
    return((1.0-currentAcquistionPeriodFraction)*timeInterval);
}

inline void timeStepSet(clock_ *clock, PetscReal timeStart, PetscReal timeInterval, PetscReal dByU, PetscInt &flag, PetscReal &cfl)
{
    PetscReal initialTimeStep  = clock->dt;

    // compute current fraction of the acquisition period
    PetscReal currentAcquistionPeriodFraction = (clock->time - timeStart) / timeInterval - std::floor((clock->time - timeStart) / timeInterval + 1e-10);

    // compute current distance to write time
    PetscReal currentDistanceToWriteTime = (1.0-currentAcquistionPeriodFraction)*timeInterval;

    // compute what will be the two CFL if adjusting with current or next distance
    PetscReal currentCFL = currentDistanceToWriteTime / dByU;

```

```
    if(currentCFL < clock->cfl)
    {
        if(clock->time + initialTimeStep >= timeStart)
        {
            if(currentDistanceToWriteTime < clock->dt)
            {
                clock->dt = currentDistanceToWriteTime;
                flag = 1;
                cfl  = currentCFL;
            }

        }
    }

    return;
}

inline PetscInt mustWrite(PetscReal time, PetscReal startTime, PetscReal timeInterval)
{
    return
    (
        (
            (time - startTime) / timeInterval -
            std::floor
            (
                (time - startTime) / timeInterval + 1e-10
            ) < 1e-10
        ) &&
        (time >= startTime)
    );
}

// MATH FUNCTIONS
//
//

//find the matrix product of two matrices

inline void matMatProduct(PetscReal **A, PetscReal **B, PetscReal **C, PetscInt numRowA, PetscInt numColA, PetscInt numRowB, PetscInt numColB)
{
    if( numColA != numRowB )
    {
        char error[512];
        sprintf(error, "number of columns of matrix 1 not equal to number of rows of matrix 2\n");
        fatalErrorInFunction("matMatProduct",  error);
    }

        for (PetscInt i = 0; i < numRowA; i++)
        {
            for (PetscInt j = 0; j < numColB; j++)
            {
                PetscReal dotProduct = 0.0;

                for (PetscInt k = 0; k < numColA; k++)
                {
                    dotProduct += A[i][k] * B[k][j];
                }
                C[i][j] = dotProduct;
            }
        }

    return;
}

inline void matVecProduct(PetscReal **A, PetscReal *b, PetscReal *c, PetscInt numRowA, PetscInt numColA, PetscInt numRowB)
{
    if( numColA != numRowB )
    {
        char error[512];
        sprintf(error, "number of columns of matrix 1 not equal to number of rows of vector 2\n");
        fatalErrorInFunction("matVecProduct",  error);
    }

    for (PetscInt i = 0; i < numRowA; i++)
    {
        PetscReal dotProduct = 0.0;

        for (PetscInt k = 0; k < numColA; k++)
        {
            dotProduct += A[i][k] * b[k];
        }
        c[i] = dotProduct;
    }

```

```
    return;
}

static inline void zeroTensor(Tensor* T) {
    T->xx = 0.0;
    T->xy = 0.0;
    T->xz = 0.0;
    T->yx = 0.0;
    T->yy = 0.0;
    T->yz = 0.0;
    T->zx = 0.0;
    T->zy = 0.0;
    T->zz = 0.0;
}

static inline Tensor symm(const Tensor T)
{
    Tensor S;
    S.xx = T.xx;
    S.xy = 0.5 * (T.xy + T.yx);
    S.xz = 0.5 * (T.xz + T.zx);
    S.yx = S.xy;
    S.yy = T.yy;
    S.yz = 0.5 * (T.yz + T.zy);
    S.zx = S.xz;
    S.zy = S.yz;
    S.zz = T.zz;
    return S;
}

static inline Tensor skewsymm(const Tensor T)
{
    Tensor K;
    K.xx = 0.0;
    K.xy = 0.5 * (T.xy - T.yx);
    K.xz = 0.5 * (T.xz - T.zx);
    K.yx = -K.xy;
    K.yy = 0.0;
    K.yz = 0.5 * (T.yz - T.zy);
    K.zx = -K.xz;
    K.zy = -K.yz;
    K.zz = 0.0;
    return K;
}

static inline Tensor dev(const Tensor T)
{
    Tensor D;

    PetscScalar tr = T.xx + T.yy + T.zz;

    PetscScalar oneThirdTr = tr / 3.0;

    D.xx = T.xx - oneThirdTr;
    D.xy = T.xy;
    D.xz = T.xz;
    D.yx = T.yx;
    D.yy = T.yy - oneThirdTr;
    D.yz = T.yz;
    D.zx = T.zx;
    D.zy = T.zy;
    D.zz = T.zz - oneThirdTr;
    return D;
}

static inline PetscReal trace(const Tensor T)
{
    return T.xx + T.yy + T.zz;
}
//compute the trace of T^2
static inline PetscReal traceSquare(const Tensor T)
{
    return T.xx * T.xx + T.yy * T.yy + T.zz * T.zz +
        2.0 * (T.xy * T.yx + T.xz * T.zx + T.yz * T.zy);
}


static inline TensorInvariants invariants(const Tensor T)
{
    TensorInvariants inv;

    inv.I1 = trace(T);

    inv.I2 = 0.5 * (inv.I1 * inv.I1 - traceSquare(T));

    inv.I3 = T.xx * (T.yy * T.zz - T.yz * T.zy)
        - T.xy * (T.yx * T.zz - T.yz * T.zx)
```

```
        + T.xz * (T.yx * T.zy - T.yy * T.zx);

    return inv;
}
//computes the inner Product of Tensors
static inline PetscReal tensorInnerProduct(const Tensor T)
{
    return (T.xx*T.xx + T.xy*T.xy + T.xz*T.xz +
            T.yx*T.yx + T.yy*T.yy + T.yz*T.yz +
            T.zx*T.zx + T.zy*T.zy + T.zz*T.zz);
}

// Computes the turbulence-model norm, defined as sqrt(2 * T_ij*T_ij).
static inline PetscReal TensorMagnitude(const Tensor T)
{
    return PetscSqrtReal(2.0 * tensorInnerProduct(T));
}

static inline Tensor multTensor(const Tensor A, const Tensor B) {
    Tensor C;
    zeroTensor(&C);

    C.xx = A.xx * B.xx + A.xy * B.yx + A.xz * B.zx;
    C.xy = A.xx * B.xy + A.xy * B.yy + A.xz * B.zy;
    C.xz = A.xx * B.xz + A.xy * B.yz + A.xz * B.zz;

    C.yx = A.yx * B.xx + A.yy * B.yx + A.yz * B.zx;
    C.yy = A.yx * B.xy + A.yy * B.yy + A.yz * B.zy;
    C.yz = A.yx * B.xz + A.yy * B.yz + A.yz * B.zz;

    C.zx = A.zx * B.xx + A.zy * B.yx + A.zz * B.zx;
    C.zy = A.zx * B.xy + A.zy * B.yy + A.zz * B.zy;
    C.zz = A.zx * B.xz + A.zy * B.yz + A.zz * B.zz;

    return C;
}

static inline PetscReal doubleContract(const Tensor A, const Tensor B)
{
    // return sum_{i,j} [A_ij * B_ij]
    return  A.xx * B.xx + A.xy * B.xy + A.xz * B.xz
          + A.yx * B.yx + A.yy * B.yy + A.yz * B.yz
          + A.zx * B.zx + A.zy * B.zy + A.zz * B.zz;
}

static inline Cmpnts multTensorVector(const Tensor T, const Cmpnts v)
{
    // Standard 3×3 matrix times 3×1 vector = 3×1 result
    Cmpnts out;
    out.x = T.xx*v.x + T.xy*v.y + T.xz*v.z;
    out.y = T.yx*v.x + T.yy*v.y + T.yz*v.z;
    out.z = T.zx*v.x + T.zy*v.y + T.zz*v.z;
    return out;
}

// Computes the standard Frobenius norm of the tensor: sqrt(T_ij*T_ij)
static inline PetscReal frobeniusNorm(const Tensor T)
{
    return PetscSqrtReal(tensorInnerProduct(T));
}

static inline Cmpnts vorticity(const Tensor T)
{
    Cmpnts omega;

    omega.x = (T.zy - T.yz);
    omega.y = (T.xz - T.zx);
    omega.z = (T.yx - T.xy);

    return omega;
}


//==================================================================================================

inline void gaussianSmooth1D(PetscReal *input, PetscReal *output, PetscInt n, PetscInt window)
{
    PetscReal weights[window];
    PetscReal sigma = window/3.0;
    PetscReal sum = 0;

    // Create Gaussian kernel
    for(PetscInt i=0; i<window; i++)
    {
        weights[i] = exp(-0.5*pow((i-window/2)/sigma, 2));
        sum += weights[i];
    }
```

```

    //normalize
    for(PetscInt i=0; i<window; i++) weights[i] /= sum;

    // Apply convolution
    for(PetscInt i=0; i<n; i++)
    {
        output[i] = 0;
        for(PetscInt j=-window/2; j<=window/2; j++)
        {
            PetscInt idx = i + j;
            if(idx < 0) idx = 0;
            if(idx >= n) idx = n-1;
            output[i] += input[idx] * weights[j + window/2];
        }
    }
    return;
}


//==========================================================================================
//check for duplicates in an integer array
inline PetscInt isPresent(PetscInt arr[], PetscInt n, PetscInt elem)
{
    for (PetscInt i = 0; i < n; i++)
    {
        if (arr[i] == elem)
        {
            return 1;
        }
    }

    return 0;
}

//==========================================================================================
//

inline PetscReal sign(PetscReal a)
{
    if (a > 0)      return  1.0;
    else if (a < 0) return -1.0;
    else            return  0.0;

}

//==========================================================================================
//

inline PetscReal signNonZero(PetscReal a)
{
    if (a > 0)      return  1.0;
    else if (a < 0) return -1.0;
    else            return  1.0;

}

//==========================================================================================
//

inline PetscReal gcd(PetscReal a, PetscReal b)
{
    if (a < b) return gcd(b, a);

    // base case
    if (fabs(b) < 1e-8) return a;

    else return (gcd(b, a - floor(a / b) * b));

}

//==========================================================================================
//

inline PetscReal gcdN(PetscReal *v, PetscInt n)
{
    PetscReal result = v[0];

    for (PetscInt i = 1; i < n; i++)
    {
        result = gcd(v[i], result);
    }

```

```
    return result;
}

//
==========================================================================================
//

inline complex gamma(complex x)
{
    complex f, z = x;

    PetscReal c[11] =
    {
        1.00000000000000174663,
        5716.400188274341379136,
        -14815.30426768413909044,
        14291.49277657478554025,
        -6348.160217641458813289,
        1301.608286058321874105,
        -108.1767053514369634679,
        2.605696505611755827729,
       -0.7423452510201416151527e-2,
        0.5384136432509564062961e-7,
       -0.4023533141268236372067e-8
    };

    if(x.real() < 0.0) z = -1.0*z;

    complex   ss, g = complex(9.0,0.0), t = z + g, s = complex(0.0, 0.0);

    for(PetscInt k=10; k>0; k--)
    {
        s = s + (complex)c[k] / t;
        t = t - 1.0;
    }

    s  = s + (complex)c[0];
    ss = (z + g - 0.5);
    s  = std::log(s * std::sqrt(2.0*M_PI)) + (z - 0.5) * std::log(ss) - ss;
    f  = std::exp(s);

    // recursive formula
    if(x.real() < 0.0) f = -1.0 * M_PI / (x * f * std::sin(M_PI*x));

    // negative integers
    if(std::round(x.real()) == x.real() && x.imag() == 0.0 && x.real() <= 0.0) f = 1e20;

    // exact result for integers arguments
    if(std::round(x.real()) == x.real() && x.imag() == 0.0 && x.real() > 0.0)
    {
        complex pp = complex(1.0, 0.0);

        for(PetscInt m = 1; m<(PetscInt)x.real(); m++)
        {
            pp = pp*complex((PetscReal)m,0.0);
        }
        f = pp;
    }

    return(f);
}

//
==========================================================================================
//

inline complex digamma(complex x)
{
    complex f, z = x;

    PetscReal c[15] =
    {
        0.99999999999999709182,
       57.156235665862923517,
       -59.597960355475491248,
       14.136097974741747174,
       -0.49191381609762019978,
       .33994649984811888699e-4,
       .46523628927048575665e-4,
       -.98374475304879564677e-4,
       .15808870322491248884e-3,
       -.21026444172410488319e-3,
       .21743961811521264320e-3,
       -.16431810653676389022e-3,
       .84418223983852743293e-4,
       -.26190838401581408670e-4,
       .36899182659531622704e-5
```

```
    };

    if(x.real() < 0.5) z = 1.0 - z;

    complex dz, dd, gg, g = complex(607.0/128.0, 0.0), n = complex(0.0,0.0), d = n;

    for(PetscInt k=14; k>0; k--)
    {
        dz = 1.0 / (z + (PetscReal)(k+1) - 2.0);
        dd = c[k] * dz;
        d  = d + dd;
        n  = n - dd * dz;
    }

    d  = d + c[0];
    gg = z + g - 0.5;

    f = std::log(gg) + (n/d - g/gg);

    if(x.real() < 0.5) f = f - M_PI * (1.0 / tan(M_PI * x));

    if(std::round(x.real()) == x.real() && x.imag() == 0.0 && x.real() <= 0.0) f = 1e20;

    return(f);
}

//
==========================================================================================================
//

inline complex hypergeom(complex a, complex b, complex c, PetscReal z)
{
    if(std::round(c.real()) == c.real() && c.imag() == 0 && c.real() <= 0)
    {
        return(complex(1e20,0.0));
    }

    PetscReal epsilon = 1e-15;
    complex   term    = complex(1.0,0.0),
              result  = complex(1.0,0.0);
    PetscInt  termination = 0, k;

    for(k=0; k<30000; k++)
    {
        term = term * ((a + (PetscReal)k) * (b + (PetscReal)k) * z / ( (c + (PetscReal)k) *
((PetscReal)k + 1.0)));

        if(std::abs(term) < abs(result*epsilon))
        {
            termination ++;
            if(termination >= 3)
            {
                return(result);
            }
        }
        else
        {
            termination = 0;
        }

        if(term == 0.0) break;

        result = result + term;

        if(std::isnan(result.real()) || std::isnan(result.imag()))
        {
            char warning[256];
            sprintf(warning, "NaN value detected in hypergeometric function at z = %.5f\n", z);
            warningInFunction("hypergeom",  warning);

            return(complex(std::nan(""),std::nan("")));
        }
    }

    return(result);
}

//
==========================================================================================================
//

inline PetscReal polyLog2(PetscReal x)
{
    const PetscReal PI = 3.1415926535897932;
    const PetscReal P[] =
    {
      0.9999999999999999502e+0,
```

```
     -2.6883926818565423430e+0,
      2.6477222699473109692e+0,
     -1.1538559607887416355e+0,
      2.0886077795020607837e-1,
     -1.0859777134152463084e-2
    };
    const PetscReal Q[] =
    {
      1.0000000000000000000e+0,
     -2.9383926818565635485e+0,
      3.2712093293018635389e+0,
     -1.7076702173954289421e+0,
      4.1596017228400603836e-1,
     -3.9801343754084482956e-2,
      8.2743668974466659035e-4
    };

    PetscReal y = 0, r = 0, s = 1;

    // transform to [0, 1/2]
    if (x < -1) {
      const PetscReal l = log(1 - x);
      y = 1/(1 - x);
      r = -PI*PI/6 + l*(0.5*l - log(-x));
      s = 1;
    } else if (x == -1) {
      return -PI*PI/12;
    } else if (x < 0) {
      const PetscReal l = log1p(-x);
      y = x/(x - 1);
      r = -0.5*l*l;
      s = -1;
    } else if (x == 0) {
      return 0;
    } else if (x < 0.5) {
      y = x;
      r = 0;
      s = 1;
    } else if (x < 1) {
      y = 1 - x;
      r = PI*PI/6 - log(x)*log1p(-x);
      s = -1;
    } else if (x == 1) {
      return PI*PI/6;
    } else if (x < 2) {
      const PetscReal l = log(x);
      y = 1 - 1/x;
      r = PI*PI/6 - l*(log(y) + 0.5*l);
      s = 1;
    } else {
      const PetscReal l = log(x);
      y = 1/x;
      r = PI*PI/3 - 0.5*l*l;
      s = -1;
    }

    const PetscReal y2 = y*y;
    const PetscReal y4 = y2*y2;
    const PetscReal p = P[0] + y * P[1] + y2 * (P[2] + y * P[3]) +
                        y4 * (P[4] + y * P[5]);
    const PetscReal q = Q[0] + y * Q[1] + y2 * (Q[2] + y * Q[3]) +
                        y4 * (Q[4] + y * Q[5] + y2 * Q[6]);

    return r + s*y*p/q;
}

//
==========================================================================================
//

inline PetscReal splineB1(PetscReal a)
{
    if (fabs(a) <= 1)       return 1.0 - fabs(a);
    else                    return 0.0;
}

//
==========================================================================================
//

inline PetscReal splineB2(PetscReal a)
{
    if      (fabs(a) <= 0.5)       return 3.0/4.0 - a*a;
    else if (fabs(a) <= 1.5 &&
             fabs(a) > 0.5)        return 1.0/8.0 * (2.0*fabs(a) - 3.0)*(2.0*fabs(a) - 3.0);
    else                           return 0.0;

```

```
}

// Computes the smallest angle difference between two angles (in radians), accounting for periodicity.
// Inputs: a, b - angles in radians.
// Returns: The smallest difference (a - b) in radians, adjusted to be within [-, ].
inline PetscReal angleDiff(PetscReal a, PetscReal b)
{
    PetscReal diff = a - b;
    while (diff > M_PI) diff -= 2.0 * M_PI;
    while (diff < -M_PI) diff += 2.0 * M_PI;
    return diff;
}

// VECTOR ALGEBRA
//
==========================================================================================================
//

inline Cmpnts nMax(Cmpnts v, PetscReal c)
{
    Cmpnts vMax;

    vMax.x = PetscMax(v.x, c);
    vMax.y = PetscMax(v.y, c);
    vMax.z = PetscMax(v.z, c);

    return(vMax);
}


//*****************************************************************************************************************//

inline Cmpnts nAbs(Cmpnts v)
{
    Cmpnts vR;

    vR.x = fabs(v.x);
    vR.y = fabs(v.y);
    vR.z = fabs(v.z);

    return(vR);
}


//*****************************************************************************************************************//

inline Cmpnts nInv(Cmpnts v)
{
    Cmpnts vR;

    vR.x = 1.0 / v.x;
    vR.y = 1.0 / v.y;
    vR.z = 1.0 / v.z;

    return(vR);
}


//*****************************************************************************************************************//

inline Cmpnts nPow(Cmpnts v, PetscReal c)
{
    Cmpnts vR;

    vR.x = pow(v.x, c);
    vR.y = pow(v.y, c);
    vR.z = pow(v.z, c);

    return(vR);
}


//*****************************************************************************************************************//

inline Cmpnts nScale(PetscReal c, Cmpnts v)
{
    Cmpnts vScaled;

    vScaled.x = c * v.x;
    vScaled.y = c * v.y;
    vScaled.z = c * v.z;

    return(vScaled);
}


//*****************************************************************************************************************//
```

```
inline Cmpnts nMultElWise(Cmpnts v1, Cmpnts v2)
{
    Cmpnts vR;

    vR.x = v1.x * v2.x;
    vR.y = v1.y * v2.y;
    vR.z = v1.z * v2.z;

    return(vR);
}


//*****************************************************************************************************//

inline Cmpnts nDivElWise(Cmpnts v1, Cmpnts v2)
{
    Cmpnts vR;

    vR.x = v1.x / v2.x;
    vR.y = v1.y / v2.y;
    vR.z = v1.z / v2.z;

    return(vR);
}


//*****************************************************************************************************//

inline Cmpnts nScaleX(PetscReal c, Cmpnts v)
{
    Cmpnts vScaled;

    vScaled.x = c * v.x;
    vScaled.y = v.y;
    vScaled.z = v.z;

    return(vScaled);
}


//*****************************************************************************************************//

inline Cmpnts nScaleY(PetscReal c, Cmpnts v)
{
    Cmpnts vScaled;

    vScaled.x = v.x;
    vScaled.y = c * v.y;
    vScaled.z = v.z;

    return(vScaled);
}


//*****************************************************************************************************//

inline Cmpnts nScaleZ(PetscReal c, Cmpnts v)
{
    Cmpnts vScaled;

    vScaled.x = v.x;
    vScaled.y = v.y;
    vScaled.z = c * v.z;

    return(vScaled);
}


//*****************************************************************************************************//

inline PetscReal nDot(Cmpnts v1, Cmpnts v2)
{
    return
    (
        v1.x*v2.x +
        v1.y*v2.y +
        v1.z*v2.z
    );
}


//*****************************************************************************************************//

inline PetscReal nMag(Cmpnts v)
{
    return
```

```
    (
        sqrt
        (
            v.x * v.x +
            v.y * v.y +
            v.z * v.z
        )
    );
}


//*****************************************************************************************************//

inline Cmpnts nCross(Cmpnts v1, Cmpnts v2)
{
    Cmpnts vCross;

    vCross.x = v1.y * v2.z - v1.z * v2.y;
    vCross.y = v1.z * v2.x - v1.x * v2.z;
    vCross.z = v1.x * v2.y - v1.y * v2.x;

    // if(nMag(vCross)<1e-20)
    // {
    //     char error[512];
    //     sprintf(error, "vectors are parallel...was told to exit\n");
    //     fatalErrorInFunction("nCross",  error);
    // }

    return(vCross);
}


//*****************************************************************************************************//

inline Cmpnts nRot(Cmpnts axis, Cmpnts vStart, PetscReal theta)
{
    Cmpnts vRotated = {0.0,0.0,0.0};
    PetscReal costheta,sintheta;

    // normalize rotation axis vector (add a small value for division)
    PetscReal axisMag
    =
    sqrt
    (
        axis.x * axis.x +
        axis.y * axis.y +
        axis.z * axis.z
    );

    if(axisMag < 1.0e-20)
    {
        char error[512];
        sprintf(error, "provided rotation axis too small to working precision\n");
        fatalErrorInFunction("nRot",  error);
    }

    axis.x = axis.x / axisMag;
    axis.y = axis.y / axisMag;
    axis.z = axis.z / axisMag;

    // compute sine and cosine
    costheta = cos(theta);
    sintheta = sin(theta);

    // explicitly compute rotation
    vRotated.x += (costheta + (1 - costheta) * axis.x * axis.x         ) * vStart.x;
    vRotated.x += ((1 - costheta) * axis.x * axis.y - axis.z * sintheta) * vStart.y;
    vRotated.x += ((1 - costheta) * axis.x * axis.z + axis.y * sintheta) * vStart.z;

    vRotated.y += ((1 - costheta) * axis.x * axis.y + axis.z * sintheta) * vStart.x;
    vRotated.y += (costheta + (1 - costheta) * axis.y * axis.y         ) * vStart.y;
    vRotated.y += ((1 - costheta) * axis.y * axis.z - axis.x * sintheta) * vStart.z;

    vRotated.z += ((1 - costheta) * axis.x * axis.z - axis.y * sintheta) * vStart.x;
    vRotated.z += ((1 - costheta) * axis.y * axis.z + axis.x * sintheta) * vStart.y;
    vRotated.z += (costheta + (1 - costheta) * axis.z * axis.z         ) * vStart.z;

    return(vRotated);

}


//*****************************************************************************************************//

inline Cmpnts nTra(Cmpnts point, Cmpnts translation)
{
    Cmpnts vTranslated;
```

```
    vTranslated.x = point.x + translation.x;
    vTranslated.y = point.y + translation.y;
    vTranslated.z = point.z + translation.z;

    return(vTranslated);
}


//*****************************************************************************************************//

inline Cmpnts nUnit(Cmpnts v)
{
    PetscReal eps=1.e-12;

    PetscReal vMag
    =
    sqrt
    (
        v.x * v.x +
        v.y * v.y +
        v.z * v.z
    );

    Cmpnts vUnit;

    vUnit.x = v.x / (vMag + eps);
    vUnit.y = v.y / (vMag + eps);
    vUnit.z = v.z / (vMag + eps);

    return(vUnit);
}


//*****************************************************************************************************//

inline Cmpnts nSum(Cmpnts v1, Cmpnts v2)
{
    Cmpnts vSum;

    vSum.x = v1.x + v2.x;
    vSum.y = v1.y + v2.y;
    vSum.z = v1.z + v2.z;

    return(vSum);
}


//*****************************************************************************************************//

inline Cmpnts nSub(Cmpnts v1, Cmpnts v2)
{
    Cmpnts vSub;

    vSub.x = v1.x - v2.x;
    vSub.y = v1.y - v2.y;
    vSub.z = v1.z - v2.z;

    return(vSub);
}


//*****************************************************************************************************//

inline Cmpnts nSet(Cmpnts value)
{
    Cmpnts newVector;

    newVector.x = value.x;
    newVector.y = value.y;
    newVector.z = value.z;

    return(newVector);
}


//*****************************************************************************************************//

inline Cmpnts nSetFromComponents(PetscReal vx, PetscReal vy, PetscReal vz)
{
    Cmpnts newVector;

    newVector.x = vx;
    newVector.y = vy;
    newVector.z = vz;

    return(newVector);
```

```
}


//*****************************************************************************************************//

inline Cmpnts nSetZero()
{
    Cmpnts newVector;

    newVector.x = 0.0;
    newVector.y = 0.0;
    newVector.z = 0.0;

    return(newVector);
}


//*****************************************************************************************************//

inline void mUnit(Cmpnts &base)
{
    PetscReal vMag
    =
    sqrt
    (
        base.x * base.x +
        base.y * base.y +
        base.z * base.z
    );

    base.x = base.x / vMag;
    base.y = base.y / vMag;
    base.z = base.z / vMag;

    return;
}


//*****************************************************************************************************//

inline void mSum(Cmpnts &base, Cmpnts add)
{
    base.x = base.x + add.x;
    base.y = base.y + add.y;
    base.z = base.z + add.z;

    return;
}


//*****************************************************************************************************//

inline void mSub(Cmpnts &base, Cmpnts sub)
{
    base.x = base.x - sub.x;
    base.y = base.y - sub.y;
    base.z = base.z - sub.z;

    return;
}


//*****************************************************************************************************//

inline void mScale(PetscReal c, Cmpnts &v)
{
    v.x = c * v.x;
    v.y = c * v.y;
    v.z = c * v.z;

    return;
}


//*****************************************************************************************************//

inline void mScaleX(PetscReal c, Cmpnts &v)
{
    v.x = c * v.x;

    return;
}


//*****************************************************************************************************//

inline void mScaleY(PetscReal c, Cmpnts &v)
```

```
{
    v.y = c * v.y;

    return;
}


//*****************************************************************************************************//

inline void mScaleZ(PetscReal c, Cmpnts &v)
{
    v.z = c * v.z;

    return;
}


//*****************************************************************************************************//

inline void mSetScale(PetscReal scale, Cmpnts &a, Cmpnts b)
{
    a.x = scale*b.x;
    a.y = scale*b.y;
    a.z = scale*b.z;
}


//*****************************************************************************************************//

inline void mRot(Cmpnts axis, Cmpnts &v, PetscReal theta)
{
    Cmpnts vRotated = {0.0,0.0,0.0};
    PetscReal costheta,sintheta;

    // normalize rotation axis vector (add a small value for division)
    PetscReal axisMag
    =
    sqrt
    (
        axis.x * axis.x +
        axis.y * axis.y +
        axis.z * axis.z
    );

    if(axisMag < 1.0e-20)
    {
        char error[512];
        sprintf(error, "provided rotation axis too small to working precision\n");
        fatalErrorInFunction("mRot",  error);
    }

    axis.x = axis.x / axisMag;
    axis.y = axis.y / axisMag;
    axis.z = axis.z / axisMag;

    // compute sine and cosine
    costheta = cos(theta);
    sintheta = sin(theta);

    // explicitly compute rotation
    vRotated.x += (costheta + (1 - costheta) * axis.x * axis.x         ) * v.x;
    vRotated.x += ((1 - costheta) * axis.x * axis.y - axis.z * sintheta) * v.y;
    vRotated.x += ((1 - costheta) * axis.x * axis.z + axis.y * sintheta) * v.z;

    vRotated.y += ((1 - costheta) * axis.x * axis.y + axis.z * sintheta) * v.x;
    vRotated.y += (costheta + (1 - costheta) * axis.y * axis.y         ) * v.y;
    vRotated.y += ((1 - costheta) * axis.y * axis.z - axis.x * sintheta) * v.z;

    vRotated.z += ((1 - costheta) * axis.x * axis.z - axis.y * sintheta) * v.x;
    vRotated.z += ((1 - costheta) * axis.y * axis.z + axis.x * sintheta) * v.y;
    vRotated.z += (costheta + (1 - costheta) * axis.z * axis.z         ) * v.z;

    v.x = vRotated.x;
    v.y = vRotated.y;
    v.z = vRotated.z;

    return;
}


//*****************************************************************************************************//

inline void mTra(Cmpnts &point, Cmpnts translation)
{
    point.x = point.x + translation.x;
    point.y = point.y + translation.y;
    point.z = point.z + translation.z;
```

```
    return;
}


//*****************************************************************************************************//

inline void mSet(Cmpnts &base, Cmpnts value)
{
    base.x = value.x;
    base.y = value.y;
    base.z = value.z;

    return;
}


//*****************************************************************************************************//

inline void mSetValue(Cmpnts &base, PetscReal value)
{
    base.x = value;
    base.y = value;
    base.z = value;

    return;
}

inline void AxByC ( PetscReal a, Cmpnts &X, PetscReal b, Cmpnts &Y, Cmpnts *C)
{
    (*C).x = a*X.x + b*Y.x;
    (*C).y = a*X.y + b*Y.y;
    (*C).z = a*X.z + b*Y.z;

    return;
}

//*****************************************************************************************************//

// domain checks
inline bool isInsideBoundingBox ( Cmpnts pt, const boundingBox &simBox)
{
    return (pt.x > simBox.xmin
       &&   pt.x < simBox.xmax
       &&   pt.y > simBox.ymin
       &&   pt.y < simBox.ymax
       &&   pt.z > simBox.zmin
       &&   pt.z < simBox.zmax ) ;

}

// FIELD AVERAGES
//
=================================================================================================
//

inline std::vector<PetscReal> jPlaneScalarMean(mesh_  *mesh, PetscReal ***s, PetscInt nLevels)
{
    DM            da = mesh->da, fda = mesh->fda;
    DMDALocalInfo info = mesh->info;
    PetscInt      xs = info.xs, xe = info.xs + info.xm;
    PetscInt      ys = info.ys, ye = info.ys + info.ym;
    PetscInt      zs = info.zs, ze = info.zs + info.zm;
    PetscInt      mx = info.mx, my = info.my, mz = info.mz;

    PetscInt      lxs, lxe, lys, lye, lzs, lze;

    lxs = xs; lxe = xe; if (xs==0) lxs = xs+1; if (xe==mx) lxe = xe-1;
    lys = ys; lye = ye; if (ys==0) lys = ys+1; if (ye==my) lye = ye-1;
    lzs = zs; lze = ze; if (zs==0) lzs = zs+1; if (ze==mz) lze = ze-1;

    PetscInt      i, j, k;
    PetscReal     ***aj;

    std::vector<PetscReal> lMean(nLevels, 0.0);
    std::vector<PetscReal> gMean(nLevels, 0.0);
    std::vector<PetscReal> lVol(nLevels, 0.0);
    std::vector<PetscReal> gVol(nLevels, 0.0);

    DMDAVecGetArray(da,  mesh->lAj, &aj);

    for (k=lzs; k<lze; k++)
    {
        for (j=lys; j<lye; j++)
        {
            for (i=lxs; i<lxe; i++)
            {
```

```
                lMean[j-1] += s[k][j][i] / aj[k][j][i];
                lVol[j-1]  += 1.0 / aj[k][j][i];
            }
        }
    }

    MPI_Allreduce(&lMean[0], &gMean[0], nLevels, MPIU_REAL, MPIU_SUM, mesh->MESH_COMM);
    MPI_Allreduce(&lVol[0], &gVol[0], nLevels, MPIU_REAL, MPIU_SUM, mesh->MESH_COMM);

    for(j=0; j<nLevels; j++)
    {
        gMean[j] = gMean[j]/gVol[j];
    }

    DMDAVecRestoreArray(da,  mesh->lAj, &aj);

    return gMean;
}

// MESH OPERATIONS
// ==========================================================================================================
//

inline void Calculate_Covariant_metrics(PetscReal g[3][3], PetscReal G[3][3])
{
    /*
        | csi.x  csi.y csi.z |-1    | x.csi   x.eta  x.zet |
        | eta.x  eta.y eta.z |    = | y.csi   y.eta  y.zet |
        | zet.x  zet.y zet.z |      | z.csi   z.eta  z.zet |

    */
    const PetscReal a11=g[0][0], a12=g[0][1], a13=g[0][2];
    const PetscReal a21=g[1][0], a22=g[1][1], a23=g[1][2];
    const PetscReal a31=g[2][0], a32=g[2][1], a33=g[2][2];

    PetscReal det= a11*(a33*a22-a32*a23)-a21*(a33*a12-a32*a13)+a31*(a23*a12-a22*a13);

    G[0][0] = (a33*a22-a32*a23)/det,    G[0][1] = - (a33*a12-a32*a13)/det,   G[0][2] = (a23*a12-a22*a13)/det;
    G[1][0] = -(a33*a21-a31*a23)/det,   G[1][1] = (a33*a11-a31*a13)/det,     G[1][2] = -(a23*a11-a21*a13)/det;
    G[2][0] = (a32*a21-a31*a22)/det,    G[2][1] = - (a32*a11-a31*a12)/det,   G[2][2] = (a22*a11-a21*a12)/det;

    return;
};

inline void calculateNormal(Cmpnts &csi, Cmpnts &eta, Cmpnts &zet, PetscReal ni[3], PetscReal nj[3], PetscReal nk[3])
{
    PetscReal g[3][3];
    PetscReal G[3][3];

    g[0][0]=csi.x, g[0][1]=csi.y, g[0][2]=csi.z;
    g[1][0]=eta.x, g[1][1]=eta.y, g[1][2]=eta.z;
    g[2][0]=zet.x, g[2][1]=zet.y, g[2][2]=zet.z;

    Calculate_Covariant_metrics(g, G);
    PetscReal xcsi=G[0][0], ycsi=G[1][0], zcsi=G[2][0];
    PetscReal xeta=G[0][1], yeta=G[1][1], zeta=G[2][1];
    PetscReal xzet=G[0][2], yzet=G[1][2], zzet=G[2][2];

    PetscReal nx_i = xcsi, ny_i = ycsi, nz_i = zcsi;
    PetscReal nx_j = xeta, ny_j = yeta, nz_j = zeta;
    PetscReal nx_k = xzet, ny_k = yzet, nz_k = zzet;

    PetscReal sum_i=sqrt(nx_i*nx_i+ny_i*ny_i+nz_i*nz_i);
    PetscReal sum_j=sqrt(nx_j*nx_j+ny_j*ny_j+nz_j*nz_j);
    PetscReal sum_k=sqrt(nx_k*nx_k+ny_k*ny_k+nz_k*nz_k);

    nx_i /= sum_i, ny_i /= sum_i, nz_i /= sum_i;
    nx_j /= sum_j, ny_j /= sum_j, nz_j /= sum_j;
    nx_k /= sum_k, ny_k /= sum_k, nz_k /= sum_k;

    ni[0] = nx_i, ni[1] = ny_i, ni[2] = nz_i;
    nj[0] = nx_j, nj[1] = ny_j, nj[2] = nz_j;
    nk[0] = nx_k, nk[1] = ny_k, nk[2] = nz_k;

    return;
}

// CELL DEFINITION TEST
// ==========================================================================================================
//
```

```
//*****************************************************************************************************//

inline bool isBoxIBMCell (int k, int j, int i, PetscReal ***nvert)
{
    return (nvert[k  ][j  ][i  ] +
            nvert[k+1][j  ][i  ] +
            nvert[k-1][j  ][i  ] +
            nvert[k  ][j+1][i  ] +
            nvert[k  ][j-1][i  ] +
            nvert[k  ][j  ][i+1] +
            nvert[k  ][j  ][i-1] > 0.1);
}


//*****************************************************************************************************//

inline bool isFluidCell (PetscInt k, PetscInt j, PetscInt i, PetscReal ***nvert)
{
    return (nvert[k][j][i] < 0.1);
}


//*****************************************************************************************************//

inline bool isIBMCell (PetscInt k, PetscInt j, PetscInt i, PetscReal ***nvert)
{
    return (nvert[k][j][i] > 0.1);
}


//*****************************************************************************************************//

inline bool isIBMFluidCell (PetscInt k, PetscInt j, PetscInt i, PetscReal ***nvert)
{
    return ((nvert[k][j][i] > 0.1) && (nvert[k][j][i] < 1.5));
}


//*****************************************************************************************************//

inline bool isIBMSolidCell (PetscInt k, PetscInt j, PetscInt i, PetscReal ***nvert)
{
    return (nvert[k][j][i] > 1.5);
}


//*****************************************************************************************************//

inline bool isFluidKFace (PetscInt kL, PetscInt j, PetscInt i, PetscInt kR, PetscReal ***nvert)
{
    return ((nvert[kL][j][i]+nvert[kR][j][i]) < 0.1);
}


//*****************************************************************************************************//

inline bool isFluidJFace (PetscInt k, PetscInt jL, PetscInt i, PetscInt jR, PetscReal ***nvert)
{
    return ((nvert[k][jL][i]+nvert[k][jR][i]) < 0.1);
}


//*****************************************************************************************************//

inline bool isFluidIFace (PetscInt k, PetscInt j, PetscInt iL, PetscInt iR, PetscReal ***nvert)
{
    return ((nvert[k][j][iL]+nvert[k][j][iR]) < 0.1);
}


//*****************************************************************************************************//

inline bool isIBMKFace (PetscInt kL, PetscInt j, PetscInt i, PetscInt kR, PetscReal ***nvert)
{
    return ((nvert[kL][j][i]+nvert[kR][j][i]) > 0.1);
}


//*****************************************************************************************************//

inline bool isIBMJFace (PetscInt k, PetscInt jL, PetscInt i, PetscInt jR, PetscReal ***nvert)
{
    return ((nvert[k][jL][i]+nvert[k][jR][i]) > 0.1);
}


```

```
//*****************************************************************************************************//

inline bool isIBMIFace (PetscInt k, PetscInt j, PetscInt iL, PetscInt iR, PetscReal ***nvert)
{
    return ((nvert[k][j][iL]+nvert[k][j][iR]) > 0.1);
}


//*****************************************************************************************************//

inline bool isIBMFluidKFace (PetscInt kL, PetscInt j, PetscInt i, PetscInt kR, PetscReal ***nvert)
{
    return ((nvert[kL][j][i] + nvert[kR][j][i] > 0.1) && (nvert[kL][j][i] + nvert[kR][j][i] < 2.5));
}


//*****************************************************************************************************//

inline bool isIBMFluidJFace (PetscInt k, PetscInt jL, PetscInt i, PetscInt jR, PetscReal ***nvert)
{
    return ((nvert[k][jL][i] + nvert[k][jR][i] > 0.1) && (nvert[k][jL][i] + nvert[k][jR][i] < 2.5));
}


//*****************************************************************************************************//

inline bool isIBMFluidIFace (PetscInt k, PetscInt j, PetscInt iL, PetscInt iR, PetscReal ***nvert)
{
    return ((nvert[k][j][iL] + nvert[k][j][iR] > 0.1) && (nvert[k][j][iL] + nvert[k][j][iR] < 2.5));
}


//*****************************************************************************************************//

// IBM solid faces - between ibm solid cells or between ibm solid and ibm fluid cell
inline bool isIBMSolidKFace (PetscInt kL, PetscInt j, PetscInt i, PetscInt kR, PetscReal ***nvert)
{
    return ((nvert[kL][j][i] + nvert[kR][j][i] > 3.5));
}


//*****************************************************************************************************//

inline bool isIBMSolidJFace (PetscInt k, PetscInt jL, PetscInt i, PetscInt jR, PetscReal ***nvert)
{
    return ((nvert[k][jL][i] + nvert[k][jR][i] > 3.5));
}


//*****************************************************************************************************//

inline bool isIBMSolidIFace (PetscInt k, PetscInt j, PetscInt iL, PetscInt iR, PetscReal ***nvert)
{
    return ((nvert[k][j][iL] + nvert[k][j][iR] > 3.5));
}

//*****************************************************************************************************//

// check if iverset cell within the neighbouring box of i,j,k
inline bool isBoxOversetCell (int k, int j, int i, PetscReal ***meshTag)
{
    return (meshTag[k  ][j  ][i  ] +
            meshTag[k+1][j  ][i  ] +
            meshTag[k-1][j  ][i  ] +
            meshTag[k  ][j+1][i  ] +
            meshTag[k  ][j-1][i  ] +
            meshTag[k  ][j  ][i+1] +
            meshTag[k  ][j  ][i-1] > 0.1);
}

//*****************************************************************************************************//

inline bool isInterpolatedCell (int k, int j, int i, PetscReal ***meshTag)
{
    return ((meshTag[k][j][i] > 0.1) && (meshTag[k][j][i] < 1.5));
}

//*****************************************************************************************************//

inline bool isZeroedCell (int k, int j, int i, PetscReal ***meshTag)
{
    return (meshTag[k][j][i] > 1.5);
}

//*****************************************************************************************************//

inline bool isOversetCell (int k, int j, int i, PetscReal ***meshTag)
```

```
{
    return (meshTag[k][j][i] > 0.1);
}

//*****************************************************************************************************//

inline bool isCalculatedCell (int k, int j, int i, PetscReal ***meshTag)
{
    return (meshTag[k][j][i] < 0.1);
}

//*****************************************************************************************************//

inline bool isInterpolatedKFace (int kL, int j, int i, int kR, PetscReal ***meshTag)
{
    return ((meshTag[kL][j][i] + meshTag[kR][j][i] > 0.1) && (meshTag[kL][j][i] + meshTag[kR][j][i] < 2.5));
}

//*****************************************************************************************************//

inline bool isInterpolatedJFace (int k, int jL, int i, int jR, PetscReal ***meshTag)
{
    return ((meshTag[k][jL][i] + meshTag[k][jR][i] > 0.1) && (meshTag[k][jL][i] + meshTag[k][jR][i] < 2.5));
}

//*****************************************************************************************************//

inline bool isInterpolatedIFace (int k, int j, int iL, int iR, PetscReal ***meshTag)
{
    return ((meshTag[k][j][iL] + meshTag[k][j][iR] > 0.1) && (meshTag[k][j][iL] + meshTag[k][j][iR] < 2.5));
}

//*****************************************************************************************************//

inline bool isZeroedKFace (int kL, int j, int i, int kR, PetscReal ***meshTag)
{
    return ((meshTag[kL][j][i]+meshTag[kR][j][i]) > 2.5);
}

//*****************************************************************************************************//

inline bool isZeroedJFace (int k, int jL, int i, int jR, PetscReal ***meshTag)
{
    return ((meshTag[k][jL][i]+meshTag[k][jR][i]) > 2.5);
}

//*****************************************************************************************************//

inline bool isZeroedIFace (int k, int j, int iL, int iR, PetscReal ***meshTag)
{
    return ((meshTag[k][j][iL]+meshTag[k][j][iR]) > 2.5);
}

//*****************************************************************************************************//

inline bool isOversetKFace (int kL, int j, int i, int kR, PetscReal ***meshTag)
{
    return ((meshTag[kL][j][i]+meshTag[kR][j][i]) > 0.1);
}

//*****************************************************************************************************//

inline bool isOversetJFace (int k, int jL, int i, int jR, PetscReal ***meshTag)
{
    return ((meshTag[k][jL][i]+meshTag[k][jR][i]) > 0.1);
}

//*****************************************************************************************************//

inline bool isOversetIFace (int k, int j, int iL, int iR, PetscReal ***meshTag)
{
    return ((meshTag[k][j][iL]+meshTag[k][j][iR]) > 0.1);
}

//*****************************************************************************************************//

inline bool isCalculatedKFace (int kL, int j, int i, int kR, PetscReal ***meshTag)
{
    return ((meshTag[kL][j][i]+meshTag[kR][j][i]) < 0.1);
}

//*****************************************************************************************************//

inline bool isCalculatedJFace (int k, int jL, int i, int jR, PetscReal ***meshTag)
```

```
{
    return ((meshTag[k][jL][i]+meshTag[k][jR][i]) < 0.1);
}

//*****************************************************************************************************//

inline bool isCalculatedIFace (int k, int j, int iL, int iR, PetscReal ***meshTag)
{
    return ((meshTag[k][j][iL]+meshTag[k][j][iR]) < 0.1);
}

//*****************************************************************************************************//

inline void resetNonResolvedCellCentersScalar(mesh_ *mesh,  Vec &V)
{
    DMDALocalInfo    info = mesh->info;
    PetscInt         xs = info.xs, xe = info.xs + info.xm;
    PetscInt         ys = info.ys, ye = info.ys + info.ym;
    PetscInt         zs = info.zs, ze = info.zs + info.zm;

    PetscInt         mx = info.mx, my = info.my, mz = info.mz;

    PetscInt         i, j, k;

    PetscReal        ***nvert, ***s, ***meshTag;

    DMDAVecGetArray(mesh->da, mesh->lNvert, &nvert);
    DMDAVecGetArray(mesh->da, mesh->lmeshTag, &meshTag);
    DMDAVecGetArray(mesh->da, V, &s);

    // Resets to zero the values of a scalar field at the non-resolved
    // faces of the mesh. Note: doesn't scatter to local.


    for (k=zs; k<ze; k++)
    {
        for (j=ys; j<ye; j++)
        {
            for (i=xs; i<xe; i++)
            {
                if
                (
                    i==0 ||
                    i==mx-1 ||
                    j==0 ||
                    j==my-1 ||
                    k==0 ||
                    k==mz-1 ||
                    isIBMCell(k, j, i, nvert) ||
                    isOversetCell(k, j, i, meshTag)
                )
                {
                    s[k][j][i] = 0.0;
                }
            }
        }
    }

    DMDAVecRestoreArray(mesh->da, V, &s);
    DMDAVecRestoreArray(mesh->da, mesh->lNvert, &nvert);
    DMDAVecRestoreArray(mesh->da, mesh->lmeshTag, &meshTag);

    return;

}


//*****************************************************************************************************//

inline void scaleNonResolvedCellCentersScalar(mesh_ *mesh,  Vec &V, PetscReal scale)
{
    DMDALocalInfo    info = mesh->info;
    PetscInt         xs = info.xs, xe = info.xs + info.xm;
    PetscInt         ys = info.ys, ye = info.ys + info.ym;
    PetscInt         zs = info.zs, ze = info.zs + info.zm;

    PetscInt         mx = info.mx, my = info.my, mz = info.mz;

    PetscInt         i, j, k;

    PetscReal        ***nvert, ***s, ***meshTag;

    DMDAVecGetArray(mesh->da, mesh->lNvert, &nvert);
    DMDAVecGetArray(mesh->da, mesh->lmeshTag, &meshTag);
    DMDAVecGetArray(mesh->da, V, &s);

    // Resets to zero the values of a scalar field at the non-resolved
```

```
    // faces of the mesh. Note: doesn't scatter to local.


    for (k=zs; k<ze; k++)
    {
        for (j=ys; j<ye; j++)
        {
            for (i=xs; i<xe; i++)
            {
                if
                (
                    i==0 ||
                    i==mx-1 ||
                    j==0 ||
                    j==my-1 ||
                    k==0 ||
                    k==mz-1 ||
                    isIBMCell(k, j, i, nvert) ||
                    isOversetCell(k, j, i, meshTag)
                )
                {
                    s[k][j][i] = scale * s[k][j][i];
                }
            }
        }
    }

    DMDAVecRestoreArray(mesh->da, V, &s);
    DMDAVecRestoreArray(mesh->da, mesh->lNvert, &nvert);
    DMDAVecRestoreArray(mesh->da, mesh->lmeshTag, &meshTag);

    return;

}


//*****************************************************************************************************************//

inline void ContravariantToCartesianPoint(Cmpnts &csi, Cmpnts &eta, Cmpnts &zet, Cmpnts &ucont, Cmpnts *ucat)
{
    // solving the linear system using Cramer method
    PetscReal det
            =
            csi.x * (eta.y * zet.z - eta.z * zet.y) -
            csi.y * (eta.x * zet.z - eta.z * zet.x) +
            csi.z * (eta.x * zet.y - eta.y * zet.x);

    PetscReal det0
            =
            ucont.x * (eta.y * zet.z - eta.z * zet.y) -
            ucont.y * (csi.y * zet.z - csi.z * zet.y) +
            ucont.z * (csi.y * eta.z - csi.z * eta.y);

    PetscReal det1
            =
           -ucont.x * (eta.x * zet.z - eta.z * zet.x) +
            ucont.y * (csi.x * zet.z - csi.z * zet.x) -
            ucont.z * (csi.x * eta.z - csi.z * eta.x);

    PetscReal det2
            =
            ucont.x * (eta.x * zet.y - eta.y * zet.x) -
            ucont.y * (csi.x * zet.y - csi.y * zet.x) +
            ucont.z * (csi.x * eta.y - csi.y * eta.x);

    (*ucat).x = det0 / det;
    (*ucat).y = det1 / det;
    (*ucat).z = det2 / det;

    return;
};


//*****************************************************************************************************************//
inline void resetNonResolvedCellFaces(mesh_ *mesh, Vec &V)
{
    DMDALocalInfo    info = mesh->info;
    PetscInt         xs = info.xs, xe = info.xs + info.xm;
    PetscInt         ys = info.ys, ye = info.ys + info.ym;
    PetscInt         zs = info.zs, ze = info.zs + info.zm;

    PetscInt         mx = info.mx, my = info.my, mz = info.mz;

    PetscInt         i, j, k;

    PetscReal        ***nvert, ***meshTag;
```

```
    Cmpnts           ***v, ***ucont_o;

    DMDAVecGetArray(mesh->fda, V, &v);
    DMDAVecGetArray(mesh->fda, mesh->access->ueqn->Ucont_o, &ucont_o);
    DMDAVecGetArray(mesh->da,  mesh->lNvert, &nvert);
    DMDAVecGetArray(mesh->da,  mesh->lmeshTag, &meshTag);

    // Resets to zero the values of a vector field at the non-resolved
    // faces of the mesh. Note: doesn't scatter to local.

    for (k=zs; k<ze; k++)
    {
        for (j=ys; j<ye; j++)
        {
            for (i=xs; i<xe; i++)
            {
                // iLeft face
                if(i==0)
                {
                // zero gradient or flow exiting
                    if(mesh->boundaryU.iLeft=="zeroGradient" || ucont_o[k][j][i].x < 0)
                    {
                        // here the value is solved
                    }
                    else
                    {
                        v[k][j][i].x = 0.0;
                    }
                }
                // unused face
                else if(i==mx-1)
                {
                    v[k][j][i].x = 0.0;
                }
                // iRight face
                else if(i==mx-2 && !(mesh->i_periodic) && !(mesh->ii_periodic))
                {
                    // zero gradient or flow exiting
                    if(mesh->boundaryU.iRight=="zeroGradient" || ucont_o[k][j][i].x > 0)
                    {
                        // here the value is solved
                    }
                    else
                    {
                        v[k][j][i].x = 0.0;
                    }
                }
                // IBM face
                else if(isIBMIFace(k, j, i, i+1, nvert))
                {
                    v[k][j][i].x = 0.0;
                }
                // OS Blanked face
                else if(isOversetIFace(k, j, i, i+1, meshTag))
                {
                    v[k][j][i].x = 0.0;
                }

                // jLeft face
                if(j==0)
                {
                    if(mesh->boundaryU.jLeft=="zeroGradient" || ucont_o[k][j][i].y < 0)
                    {
                        // here the value is solved
                    }
                    else
                    {
                        v[k][j][i].y = 0.0;
                    }
                }
                // unused face
                else if(j==my-1)
                {
                    v[k][j][i].y = 0.0;
                }
                // jRight face
                else if(j==my-2 && !(mesh->j_periodic) && !(mesh->jj_periodic))
                {
                    if(mesh->boundaryU.jRight=="zeroGradient" || ucont_o[k][j][i].y > 0)
                    {
                        // here the value is solved
                    }
                    else
                    {
                        v[k][j][i].y = 0.0;
                    }
                }
```

```
                // IBM face
                else if(isIBMJFace(k, j, i, j+1, nvert))
                {
                    v[k][j][i].y = 0.0;
                }
                // OS Blanked face
                else if(isOversetJFace(k, j, i, j+1, meshTag))
                {
                    v[k][j][i].y = 0.0;
                }

                // kLeft face
                if(k==0)
                {
                    if(mesh->boundaryU.kLeft=="zeroGradient" || ucont_o[k][j][i].z < 0)
                    {
                        // here the value is solved
                    }
                    else
                    {
                        v[k][j][i].z = 0.0;
                    }
                }
                // unused face
                else if(k==mz-1)
                {
                    v[k][j][i].z = 0.0;
                }
                // kRight face
                else if(k==mz-2 && !(mesh->k_periodic) && !(mesh->kk_periodic))
                {
                    if(mesh->boundaryU.kRight=="zeroGradient" || ucont_o[k][j][i].z > 0)
                    {
                        // here the value is solved
                    }
                    else
                    {
                        v[k][j][i].z = 0.0;
                    }
                }
                // IBM face
                else if(isIBMKFace(k, j, i, k+1, nvert))
                {
                    v[k][j][i].z = 0.0;
                }
                // OS Blanked face
                else if(isOversetKFace(k, j, i, k+1, meshTag))
                {
                    v[k][j][i].z = 0.0;
                }
            }
        }
    }

    DMDAVecRestoreArray(mesh->fda, V, &v);
    DMDAVecRestoreArray(mesh->fda, mesh->access->ueqn->Ucont_o, &ucont_o);
    DMDAVecRestoreArray(mesh->da,  mesh->lNvert, &nvert);
    DMDAVecRestoreArray(mesh->da,  mesh->lmeshTag, &meshTag);

    return;
}


//*****************************************************************************************************//

inline bool isOnNonResolvedCellCenters(PetscInt i, PetscInt j, PetscInt k, DMDALocalInfo info)
{
    PetscInt         mx = info.mx,
                     my = info.my,
                     mz = info.mz;

    bool             result;

    // i-left ghost plane
    if(i==0)
    {
        result = true;
    }
    // j-left ghost plane
    else if(j==0)
    {
        result = true;
    }
    // k-left ghost plane
    else if(k==0)
    {
        result = true;
```

```
    }
    // i-right ghost plane
    else if(i==mx-1)
    {
       result = true;
    }
    // j-right ghost plane
    else if(j==my-1)
    {
        result = true;
    }
    // k-right ghost plane
    else if(k==mz-1)
    {
        result = true;
    }
    // resolved cells
    else
    {
        result = false;
    }

    return(result);
}

//*****************************************************************************************************************//

inline bool isOnCornerCellCenters(PetscInt i, PetscInt j, PetscInt k, DMDALocalInfo info)
{
    PetscInt         mx = info.mx,
                     my = info.my,
                     mz = info.mz;

    bool             result;

    // line at i=0, k=0
    if(i==0 && k==0)
    {
        result = true;
    }
    // line at i=mx-1, k=0
    else if(i==mx-1 && k==0)
    {
        result = true;
    }
    // line at j=0, k=0
    else if(j==0 && k==0)
    {
        result = true;
    }
    // line at j=my-1, k=0
    else if(j==my-1 && k==0)
    {
       result = true;
    }
    // line at i=0, k=mz-1
    else if(i==0 && k==mz-1)
    {
        result = true;
    }
    // line at i=mx-1, k=mz-1
    else if(i==mx-1 && k==mz-1)
    {
        result = true;
    }
    // line at j=0, k=mz-1
    else if(j==0 && k==mz-1)
    {
        result = true;
    }
    // line at j=my-1, k=mz-1
    else if(j==my-1 && k==mz-1)
    {
        result = true;
    }
    // line at i=0, j=0
    else if(i==0 && j==0)
    {
        result = true;
    }
    // line at i=mx-1, j=0
    else if(i==mx-1 && j==0)
    {
        result = true;
    }
    // line at i=0, j=my-1
```

```
    else if(i==0 && j==my-1)
    {
        result = true;
    }
    // line at i=mx-1, j=my-1
    else if(i==mx-1 && j==my-1)
    {
        result = true;
    }
    // non-corner cells
    else
    {
        result = false;
    }

    return(result);
}

// STENCILS
//
==========================================================================================================
//

inline void getCell2Cell3StencilCsiNoGhost(mesh_ *mesh, PetscInt i, PetscInt mx, PetscInt *iL,
PetscInt *iR)
{
    // apply zero gradient at boundaries
    if(i == 1)
    {
        *iL = i;
        *iR = i + 1;
    }
    if(i == mx-2)
    {
        *iL = i - 1;
        *iR = i;
    }
    else
    {
        *iL = i-1, *iR = i+1;
    }

    return;
}


//*********************************************************************************************************//

inline void getCell2Cell3StencilEtaNoGhost(mesh_ *mesh, PetscInt j, PetscInt my, PetscInt *jL,
PetscInt *jR)
{
    // apply zero gradient at boundaries
    if(j == 1)
    {
        *jL = j;
        *jR = j + 1;
    }
    else if(j == my-2)
    {
        *jL = j-1;
        *jR = j;
    }
    else
    {
        *jL = j-1, *jR = j+1;
    }

    return;
}


//*********************************************************************************************************//

inline void getCell2Cell3StencilZetNoGhost(mesh_ *mesh, PetscInt k, PetscInt mz, PetscInt *kL,
PetscInt *kR)
{
    // apply zero gradient at boundaries
    if(k == 1)
    {
        *kL = k;
        *kR = k + 1;
    }
    else if(k == mz-2)
    {
        *kL = k-1;
        *kR = k;
    }
```

```
    else
    {
        *kL = k-1, *kR = k+1;
    }

    return;
}

inline void getCell2Cell3StencilCsi(mesh_ *mesh, PetscInt i, PetscInt mx, PetscInt *iL, PetscInt *iR)
{
    // if on non-periodic boundary return 2 cell-stencil extrema idxs, if periodic
    // get the right data.
    if(i==1 || i==mx-2)
    {
        if(mesh->i_periodic  && i==mx-2)
        {
            *iL = mx-3, *iR = 1;
        }
        if(mesh->i_periodic  && i==1)
        {
            *iL = mx-2, *iR = 2;
        }
        else if(mesh->ii_periodic && i==mx-2)
        {
            *iR = mx+1;
            *iL = i-1;
        }
        else if(mesh->ii_periodic && i==1)
        {
            *iL = -2;
            *iR =  2;
        }
        else
        {
            if(i == 1)
            {
                *iL = i;
                *iR = i + 1;
            }

            if(i == mx-2)
            {
                *iL = i - 1;
                *iR = i;
            }
        }
    }

    else
    {
        *iL = i-1, *iR = i+1;
    }

    return;
}


//*****************************************************************************************************************//

inline void getCell2Cell3StencilEta(mesh_ *mesh, PetscInt j, PetscInt my, PetscInt *jL, PetscInt *jR)
{
    // if on non-periodic boundary return 2 cell-stencil extrema idxs, if periodic
    // get the right data.
    if(j==1 || j==my-2)
    {
        if(mesh->j_periodic  && j==my-2)
        {
            *jL = my-3, *jR = 1;
        }
        if(mesh->j_periodic  && j==1)
        {
            *jL = my-2, *jR = 2;
        }
        else if(mesh->jj_periodic && j==my-2)
        {
            *jR = my+1;
            *jL = j-1;
        }
        else if(mesh->jj_periodic && j==1)
        {
            *jL= -2;
            *jR = 2;
        }
        else
        {
            if(j == 1)
            {
```

```
                *jL = j;
                *jR = j + 1;
            }

            if(j == my-2)
            {
                *jL = j-1;
                *jR = j;
            }
        }
    }

    else
    {
        *jL = j-1, *jR = j+1;
    }

    return;
}


//*****************************************************************************************************//

inline void getCell2Cell3StencilZet(mesh_ *mesh, PetscInt k, PetscInt mz, PetscInt *kL, PetscInt *kR)
{
    // if on non-periodic boundary return 2 cell-stencil extrema idxs, if periodic
    // get the right data.
    if(k==1 || k==mz-2)
    {
        if(mesh->k_periodic && k==mz-2)
        {
            *kL = mz-3, *kR = 1;
        }
        if(mesh->k_periodic && k==1)
        {
            *kL = mz-2, *kR = 2;
        }
        else if(mesh->kk_periodic && k==mz-2)
        {
            *kR = mz+1;
            *kL = k-1;
        }
        else if(mesh->kk_periodic && k==1)
        {
            *kL = -2;
            *kR =  2;
        }
        else
        {
            if(k == 1)
            {
                *kL = k;
                *kR = k + 1;
            }

            if(k == mz-2)
            {
                *kL = k-1;
                *kR = k;
            }
        }
    }

    else
    {
        *kL = k-1, *kR = k+1;
    }

    return;
}


//*****************************************************************************************************//

inline void getFace2Cell4StencilCsi(mesh_ *mesh, PetscInt k, PetscInt j, PetscInt i, PetscInt mx,
PetscInt *iL, PetscInt *iR, PetscReal *denom, PetscReal ***nvert, PetscReal ***meshTag)
{
    // if on non-periodic boundary return 2 cell-stencil extrema idxs, if periodic
    // get the right data.
    if(i==0 || i==mx-2)
    {
        if(mesh->i_periodic)
        {
            *iL = mx-3, *iR = 2;
            *denom = 3.;
        }
        else if(mesh->ii_periodic && i==mx-2)
```

```
        {
            *iR=mx+2;
            *iL = i-1;
            *denom = 3.;
        }
        else if(mesh->ii_periodic && i==0)
        {
            *iL=-3;
            *iR = i+2;
            *denom = 3.;
        }
        else
        {
          if(i == 0)
          {
              *iL = i;
              *iR = i + 2;
              *denom = 3.;
          }

          if(i == mx-2)
          {
              *iL = i-1;
              *iR = i + 1;
              *denom = 3.;
          }
        }
    }
    else if(isIBMCell(k, j, i, nvert) || isOversetCell(k, j, i, meshTag))
    {
        *iL = i;
        *iR = i + 2;
        *denom = 3.;
    }
    else if(isIBMCell(k, j, i+1, nvert) || isOversetCell(k, j, i+1, meshTag))
    {
        *iL = i-1;
        *iR = i + 1;
        *denom = 3.;
    }
    else
    {
        *iL = i-1, *iR = i+2;
        *denom = 3.;
    }

    return;
}


//*****************************************************************************************************//

inline void getFace2Cell4StencilEta(mesh_ *mesh, PetscInt k, PetscInt j, PetscInt i, PetscInt my,
PetscInt *jL, PetscInt *jR, PetscReal *denom, PetscReal ***nvert, PetscReal ***meshTag)
{
    // if on non-periodic boundary return 2 cell-stencil extrema idxs, if periodic
    // get the right data.
    if(j==0 || j==my-2)
    {
        if(mesh->j_periodic)
        {
            *jL = my-3, *jR = 2;
            *denom = 3.;
        }
        else if(mesh->jj_periodic && j==my-2)
        {
            *jR=my+2;
            *jL = j-1;
            *denom = 3.;
        }
        else if(mesh->jj_periodic && j==0)
        {
            *jL=-3;
            *jR = j+2;
            *denom = 3.;
        }
        else
        {
          if(j == 0)
          {
              *jL = j;
              *jR = j + 2;
              *denom = 3.;
          }

          if(j == my-2)
          {
```

```
                *jL = j-1;
                *jR = j + 1;
                *denom = 3.;
            }
        }
    }
    else if(isIBMCell(k, j, i, nvert) || isOversetCell(k, j, i, meshTag))
    {
        *jL = j;
        *jR = j + 2;
        *denom = 3.;
    }
    else if(isIBMCell(k, j+1, i, nvert) || isOversetCell(k, j+1, i, meshTag))
    {
        *jL = j-1;
        *jR = j + 1;
        *denom = 3.;
    }
    else
    {
        *jL = j-1, *jR = j+2;
        *denom = 3.;
    }

    return;
}


//*****************************************************************************************************//

inline void getFace2Cell4StencilZet(mesh_ *mesh, PetscInt k, PetscInt j, PetscInt i, PetscInt mz,
PetscInt *kL, PetscInt *kR, PetscReal *denom, PetscReal ***nvert, PetscReal ***meshTag)
{
    // if on non-periodic boundary return 2 cell-stencil extrema idxs, if periodic
    // get the right data.
    if(k==0 || k==mz-2)
    {
        if(mesh->k_periodic)
        {
            *kL = mz-3, *kR = 2;
            *denom = 3.;
        }
        else if(mesh->kk_periodic && k==mz-2)
        {
            *kR=mz+2;
            *kL = k-1;
            *denom = 3.;
        }
        else if(mesh->kk_periodic && k==0)
        {
            *kL=-3;
            *kR = k+2;
            *denom = 3.;
        }
        else
        {
          if(k == 0)
          {
              *kL = k;
              *kR = k + 2;
              *denom = 3.;
          }

          if(k == mz-2)
          {
              *kL = k-1;
              *kR = k + 1;
              *denom = 3.;
          }
        }
    }
    else if(isIBMCell(k, j, i, nvert) || isOversetCell(k, j, i, meshTag))
    {
        *kL = k;
        *kR = k + 2;
        *denom = 3.;
    }
    else if(isIBMCell(k+1, j, i, nvert) || isOversetCell(k+1, j, i, meshTag))
    {
        *kL = k-1;
        *kR = k + 1;
        *denom = 3.;
    }
    else
    {
        *kL = k-1, *kR = k+2;
        *denom = 3.;
```

```
    }

    return;
}


//*****************************************************************************************************//

inline void getFace2Face3StencilCsi(mesh_ *mesh, PetscInt i, PetscInt mx, PetscInt *iL, PetscInt *iR)
{
    // cell faces don't have ghosts, if on non-periodic boundary return the central
    // face label as the ghost face label. If periodic get neighboring face idxs.
    if(i==0 || i==mx-2)
    {
        if(mesh->i_periodic)
        {
            *iL = mx-3, *iR = 1;
        }
        else if(mesh->ii_periodic && i==mx-2)
        {
            *iR=mx+1;
            *iL = i-1;
        }
        else if(mesh->ii_periodic && i==0)
        {
            *iL=-3;
            *iR = i+1;
        }
        else if(i==0)
        {
            *iL = i;
            *iR = i + 1;
        }
        else if(i==mx-2)
        {
            *iL = i - 1;
            *iR = i;
        }
    }
    else
    {
        *iL = i-1, *iR = i+1;
    }

    return;
}


//*****************************************************************************************************//

inline void getFace2Face3StencilEta(mesh_ *mesh, PetscInt j, PetscInt my, PetscInt *jL, PetscInt *jR)
{
    // cell faces don't have ghosts, if on non-periodic boundary return the central
    // face label as the ghost face label. If periodic get neighboring face idxs.
    if(j==0 || j==my-2)
    {
        if(mesh->j_periodic)
        {
            *jL = my-3, *jR = 1;
        }
        else if(mesh->jj_periodic && j==my-2)
        {
            *jR=my+1;
            *jL = j-1;
        }
        else if(mesh->jj_periodic && j==0)
        {
            *jL=-3;
            *jR = j+1;
        }
        else if(j==0)
        {
            *jL = j;
            *jR = j + 1;
        }
        else if(j==my-2)
        {
            *jL = j - 1;
            *jR = j;
        }
    }
    else
    {
        *jL = j-1, *jR = j+1;
    }

    return;
```

```
}


//*****************************************************************************************************//

inline void getFace2Face3StencilZet(mesh_ *mesh, PetscInt k, PetscInt mz, PetscInt *kL, PetscInt *kR)
{
    // cell faces don't have ghosts, if on non-periodic boundary return the central
    // face label as the ghost face label. If periodic get neighboring face idxs.
    if(k==0 || k==mz-2)
    {
        if(mesh->k_periodic)
        {
            *kL = mz-3, *kR = 1;
        }
        else if(mesh->kk_periodic && k==mz-2)
        {
            *kR=mz+1;
            *kL = k-1;
        }
        else if(mesh->kk_periodic && k==0)
        {
            *kL=-3;
            *kR = k+1;
        }
        else if(k==0)
        {
            *kL = k;
            *kR = k + 1;
        }
        else if(k==mz-2)
        {
            *kL = k - 1;
            *kR = k;
        }
    }
    else
    {
        *kL = k-1, *kR = k+1;
    }

    return;
}


//*****************************************************************************************************//

inline void getFace2Face7StencilCsi(mesh_ *mesh, PetscInt i, PetscInt mx,
                                    PetscInt *m3, PetscInt *m2, PetscInt *m1,
                                    PetscInt *p1, PetscInt *p2, PetscInt *p3)
{
    *m3 = i-3, *m2 = i-2, *m1 = i-1, *p1 = i+1, *p2 = i+2, *p3 = i+3;
    if(mesh->i_periodic)
    {
        if(i == 0)    { *m3 = mx-5; *m2 = mx-4; *m1 = mx-3;}
        if(i == 1)    { *m3 = mx-4; *m2 = mx-3;            }
        if(i == 2)    { *m3 = mx-3;                        }
        if(i == mx-4) { *p3 = 1;                           }
        if(i == mx-3) { *p2 = 1;    *p3 = 2;               }
        if(i == mx-2) { *p1 = 1;    *p2 = 2; *p3 = 3;      }
    }
    else if(mesh->ii_periodic)
    {
        if(i == 0)    { *m3 = -3;   *m2 = -3;   *m1 = -3;  }
        if(i == 1)    { *m3 = -3;   *m2 = -3;              }
        if(i == 2)    { *m3 = -3;                          }
        if(i == mx-4) { *p3 = mx+1;                        }
        if(i == mx-3) { *p2 = mx+1; *p3 = mx+2;            }
        if(i == mx-2) { *p1 = mx+1; *p2 = mx+2; *p3 = mx+2;}
    }
    else
    {
        // Non-periodic: clamp to valid face range [0, mx-2].
        if(*m3 < 0)    *m3 = 0;
        if(*m2 < 0)    *m2 = 0;
        if(*m1 < 0)    *m1 = 0;
        if(*p1 > mx-2) *p1 = mx-2;
        if(*p2 > mx-2) *p2 = mx-2;
        if(*p3 > mx-2) *p3 = mx-2;
    }

    return;
}


//*****************************************************************************************************//

```

```
inline void getFace2Face7StencilEta(mesh_ *mesh, PetscInt j, PetscInt my,
                                    PetscInt *m3, PetscInt *m2, PetscInt *m1,
                                    PetscInt *p1, PetscInt *p2, PetscInt *p3)
{
    *m3 = j-3, *m2 = j-2, *m1 = j-1, *p1 = j+1, *p2 = j+2, *p3 = j+3;
    if(mesh->j_periodic)
    {
        if(j == 0)    { *m3 = my-5; *m2 = my-4; *m1 = my-3;}
        if(j == 1)    { *m3 = my-4; *m2 = my-3;            }
        if(j == 2)    { *m3 = my-3;                        }
        if(j == my-4) { *p3 = 1;                           }
        if(j == my-3) { *p2 = 1;    *p3 = 2;               }
        if(j == my-2) { *p1 = 1;    *p2 = 2; *p3 = 3;      }
    }
    else if(mesh->jj_periodic)
    {
        if(j == 0)    { *m3 = -3;   *m2 = -3;   *m1 = -3;  }
        if(j == 1)    { *m3 = -3;   *m2 = -3;              }
        if(j == 2)    { *m3 = -3;                          }
        if(j == my-4) { *p3 = my+1;                        }
        if(j == my-3) { *p2 = my+1; *p3 = my+2;            }
        if(j == my-2) { *p1 = my+1; *p2 = my+2; *p3 = my+2;}
    }
    else
    {
        // Non-periodic: clamp to valid face range [0, my-2].
        if(*m3 < 0)    *m3 = 0;
        if(*m2 < 0)    *m2 = 0;
        if(*m1 < 0)    *m1 = 0;
        if(*p1 > my-2) *p1 = my-2;
        if(*p2 > my-2) *p2 = my-2;
        if(*p3 > my-2) *p3 = my-2;
    }

    return;
}


//*****************************************************************************************************//

inline void getFace2Face7StencilZet(mesh_ *mesh, PetscInt k, PetscInt mz,
                                    PetscInt *m3, PetscInt *m2, PetscInt *m1,
                                    PetscInt *p1, PetscInt *p2, PetscInt *p3)
{
    *m3 = k-3, *m2 = k-2, *m1 = k-1, *p1 = k+1, *p2 = k+2, *p3 = k+3;
    if(mesh->k_periodic)
    {
        if(k == 0)    { *m3 = mz-5; *m2 = mz-4; *m1 = mz-3;}
        if(k == 1)    { *m3 = mz-4; *m2 = mz-3;            }
        if(k == 2)    { *m3 = mz-3;                        }
        if(k == mz-4) { *p3 = 1;                           }
        if(k == mz-3) { *p2 = 1;    *p3 = 2;               }
        if(k == mz-2) { *p1 = 1;    *p2 = 2; *p3 = 3;      }
    }
    else if(mesh->kk_periodic)
    {
        if(k == 0)    { *m3 = -3;   *m2 = -3;   *m1 = -3;  }
        if(k == 1)    { *m3 = -3;   *m2 = -3;              }
        if(k == 2)    { *m3 = -3;                          }
        if(k == mz-4) { *p3 = mz+1;                        }
        if(k == mz-3) { *p2 = mz+1; *p3 = mz+2;            }
        if(k == mz-2) { *p1 = mz+1; *p2 = mz+2; *p3 = mz+2;}
    }
    else
    {
        // Non-periodic: clamp to valid face range [0, my-2].
        if(*m3 < 0)    *m3 = 0;
        if(*m2 < 0)    *m2 = 0;
        if(*m1 < 0)    *m1 = 0;
        if(*p1 > mz-2) *p1 = mz-2;
        if(*p2 > mz-2) *p2 = mz-2;
        if(*p3 > mz-2) *p3 = mz-2;
    }

    return;
}


//*****************************************************************************************************//

inline void getCell2Cell7StencilCsi(mesh_ *mesh, PetscInt i, PetscInt mx,
                                    PetscInt *m3, PetscInt *m2, PetscInt *m1,
                                    PetscInt *p1, PetscInt *p2, PetscInt *p3)
{
    *m3 = i-3, *m2 = i-2, *m1 = i-1, *p1 = i+1, *p2 = i+2, *p3 = i+3;
    if(mesh->i_periodic)
    {
```

```
        if(i == 0)    { *m3 = mx-4; *m2 = mx-3; *m1 = mx-2;}
        if(i == 1)    { *m3 = mx-3; *m2 = mx-2;            }
        if(i == 2)    { *m3 = mx-2;                        }
        if(i == mx-3) { *p3 = 2;                           }
        if(i == mx-2) { *p2 = 2;    *p3 = 3;               }
        if(i == mx-1) { *p1 = 2;    *p2 = 3; *p3 = 4;      }
    }
    else if(mesh->ii_periodic)
    {
        if(i == 0)    { *m3 = -3;   *m2 = -3;   *m1 = -3;  }
        if(i == 1)    { *m3 = -3;   *m2 = -3;              }
        if(i == 2)    { *m3 = -3;                          }
        if(i == mx-3) { *p3 = mx+2;                        }
        if(i == mx-2) { *p2 = mx+2; *p3 = mx+2;            }
        if(i == mx-1) { *p1 = mx+2; *p2 = mx+2; *p3 = mx+2;}
    }
    else
    {
        // Non-periodic: clamp to valid face range [0, mx-1].
        if(*m3 < 1)    *m3 = 1;
        if(*m2 < 1)    *m2 = 1;
        if(*m1 < 1)    *m1 = 1;
        if(*p1 > mx-2) *p1 = mx-2;
        if(*p2 > mx-2) *p2 = mx-2;
        if(*p3 > mx-2) *p3 = mx-2;
    }

    return;
}


//*****************************************************************************************************//

inline void getCell2Cell7StencilEta(mesh_ *mesh, PetscInt j, PetscInt my,
                                    PetscInt *m3, PetscInt *m2, PetscInt *m1,
                                    PetscInt *p1, PetscInt *p2, PetscInt *p3)
{
    *m3 = j-3, *m2 = j-2, *m1 = j-1, *p1 = j+1, *p2 = j+2, *p3 = j+3;
    if(mesh->j_periodic)
    {
        if(j == 0)    { *m3 = my-4; *m2 = my-3; *m1 = my-2;}
        if(j == 1)    { *m3 = my-3; *m2 = my-2;            }
        if(j == 2)    { *m3 = my-2;                        }
        if(j == my-3) { *p3 = 2;                           }
        if(j == my-2) { *p2 = 2;    *p3 = 3;               }
        if(j == my-1) { *p1 = 2;    *p2 = 3; *p3 = 4;      }
    }
    else if(mesh->jj_periodic)
    {
        if(j == 0)    { *m3 = -3;   *m2 = -3;   *m1 = -3;  }
        if(j == 1)    { *m3 = -3;   *m2 = -3;              }
        if(j == 2)    { *m3 = -3;                          }
        if(j == my-3) { *p3 = my+2;                        }
        if(j == my-2) { *p2 = my+2; *p3 = my+2;            }
        if(j == my-1) { *p1 = my+2; *p2 = my+2; *p3 = my+2;}
    }
    else
    {
        // Non-periodic: clamp to valid face range [0, mx-1].
        if(*m3 < 1)    *m3 = 1;
        if(*m2 < 1)    *m2 = 1;
        if(*m1 < 1)    *m1 = 1;
        if(*p1 > my-2) *p1 = my-2;
        if(*p2 > my-2) *p2 = my-2;
        if(*p3 > my-2) *p3 = my-2;
    }

    return;
}


//*****************************************************************************************************//

inline void getCell2Cell7StencilZet(mesh_ *mesh, PetscInt k, PetscInt mz,
                                    PetscInt *m3, PetscInt *m2, PetscInt *m1,
                                    PetscInt *p1, PetscInt *p2, PetscInt *p3)
{
    *m3 = k-3, *m2 = k-2, *m1 = k-1, *p1 = k+1, *p2 = k+2, *p3 = k+3;
    if(mesh->k_periodic)
    {
        if(k == 0)    { *m3 = mz-4; *m2 = mz-3; *m1 = mz-2;}
        if(k == 1)    { *m3 = mz-3; *m2 = mz-2;            }
        if(k == 2)    { *m3 = mz-2;                        }
        if(k == mz-3) { *p3 = 2;                           }
        if(k == mz-2) { *p2 = 2;    *p3 = 3;               }
        if(k == mz-1) { *p1 = 2;    *p2 = 3; *p3 = 4;      }
    }
```

```
    else if(mesh->kk_periodic)
    {
        if(k == 0)     { *m3 = -3;    *m2 = -3;    *m1 = -3;  }
        if(k == 1)     { *m3 = -3;    *m2 = -3;               }
        if(k == 2)     { *m3 = -3;                            }
        if(k == mz-3)  { *p3 = mz+2;                          }
        if(k == mz-2)  { *p2 = mz+2;  *p3 = mz+2;             }
        if(k == mz-1)  { *p1 = mz+2;  *p2 = mz+2;  *p3 = mz+2;}
    }
    else
    {
        // Non-periodic: clamp to valid face range [0, mx-1].
        if(*m3 < 1)     *m3 = 1;
        if(*m2 < 1)     *m2 = 1;
        if(*m1 < 1)     *m1 = 1;
        if(*p1 > mz-2)  *p1 = mz-2;
        if(*p2 > mz-2)  *p2 = mz-2;
        if(*p3 > mz-2)  *p3 = mz-2;
    }

    return;
}

// GRADIENT SCHEMES
//
//

inline void Compute_du_i
(
    mesh_ *mesh,
    PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz,
    Cmpnts ***ucat, PetscReal ***nvert, PetscReal ***meshTag,
    PetscReal *dudc, PetscReal *dvdc, PetscReal *dwdc,
    PetscReal *dude, PetscReal *dvde, PetscReal *dwde,
    PetscReal *dudz, PetscReal *dvdz, PetscReal *dwdz
)
{
    *dudc = ucat[k][j][i+1].x - ucat[k][j][i].x;
    *dvdc = ucat[k][j][i+1].y - ucat[k][j][i].y;
    *dwdc = ucat[k][j][i+1].z - ucat[k][j][i].z;

    if (isIBMSolidIFace(k, j+1, i, i+1, nvert) || isZeroedIFace(k, j+1, i, i+1, meshTag) ||
            (j==my-2 && (i==0 || i==mx-2) && ((!mesh->j_periodic && !mesh->jj_periodic) ||
(!mesh->i_periodic && !mesh->ii_periodic)) ))
    {
        *dude = (ucat[k][j  ][i+1].x + ucat[k][j  ][i].x - ucat[k][j-1][i+1].x - ucat[k][j-1][i].x) *
0.5;
        *dvde = (ucat[k][j  ][i+1].y + ucat[k][j  ][i].y - ucat[k][j-1][i+1].y - ucat[k][j-1][i].y) *
0.5;
        *dwde = (ucat[k][j  ][i+1].z + ucat[k][j  ][i].z - ucat[k][j-1][i+1].z - ucat[k][j-1][i].z) *
0.5;
    }
    else if  (isIBMSolidIFace(k, j-1, i, i+1, nvert) || isZeroedIFace(k, j-1, i, i+1, meshTag) ||
            (j==1 && (i==0 || i==mx-2) && ((!mesh->j_periodic && !mesh->jj_periodic) ||
(!mesh->i_periodic && !mesh->ii_periodic)) ))
    {
        *dude = (ucat[k][j+1][i+1].x + ucat[k][j+1][i].x - ucat[k][j  ][i+1].x - ucat[k][j  ][i].x) *
0.5;
        *dvde = (ucat[k][j+1][i+1].y + ucat[k][j+1][i].y - ucat[k][j  ][i+1].y - ucat[k][j  ][i].y) *
0.5;
        *dwde = (ucat[k][j+1][i+1].z + ucat[k][j+1][i].z - ucat[k][j  ][i+1].z - ucat[k][j  ][i].z) *
0.5;
    }
    else
    {
        *dude = (ucat[k][j+1][i+1].x + ucat[k][j+1][i].x - ucat[k][j-1][i+1].x - ucat[k][j-1][i].x) *
0.25;
        *dvde = (ucat[k][j+1][i+1].y + ucat[k][j+1][i].y - ucat[k][j-1][i+1].y - ucat[k][j-1][i].y) *
0.25;
        *dwde = (ucat[k][j+1][i+1].z + ucat[k][j+1][i].z - ucat[k][j-1][i+1].z - ucat[k][j-1][i].z) *
0.25;
    }

    if (isIBMSolidIFace(k+1, j, i, i+1, nvert) || isZeroedIFace(k+1, j, i, i+1, meshTag) ||
            (k==mz-2 && (i==0 || i==mx-2) && ((!mesh->k_periodic && !mesh->kk_periodic) ||
(!mesh->i_periodic && !mesh->ii_periodic)) ))
    {
        *dudz = (ucat[k  ][j][i+1].x + ucat[k  ][j][i].x - ucat[k-1][j][i+1].x - ucat[k-1][j][i].x) *
0.5;
        *dvdz = (ucat[k  ][j][i+1].y + ucat[k  ][j][i].y - ucat[k-1][j][i+1].y - ucat[k-1][j][i].y) *
0.5;
        *dwdz = (ucat[k  ][j][i+1].z + ucat[k  ][j][i].z - ucat[k-1][j][i+1].z - ucat[k-1][j][i].z) *
0.5;
    }
    else if (isIBMSolidIFace(k-1, j, i, i+1, nvert) || isZeroedIFace(k-1, j, i, i+1, meshTag) ||
            (k==1 && (i==0 || i==mx-2) && ((!mesh->k_periodic && !mesh->kk_periodic) ||
```

```
(!mesh->i_periodic && !mesh->ii_periodic)) ))
    {
        *dudz = (ucat[k+1][j][i+1].x + ucat[k+1][j][i].x - ucat[k  ][j][i+1].x - ucat[k  ][j][i].x) * 0.5;
        *dvdz = (ucat[k+1][j][i+1].y + ucat[k+1][j][i].y - ucat[k  ][j][i+1].y - ucat[k  ][j][i].y) * 0.5;
        *dwdz = (ucat[k+1][j][i+1].z + ucat[k+1][j][i].z - ucat[k  ][j][i+1].z - ucat[k  ][j][i].z) * 0.5;
    }
    else
    {
        *dudz = (ucat[k+1][j][i+1].x + ucat[k+1][j][i].x - ucat[k-1][j][i+1].x - ucat[k-1][j][i].x) * 0.25;
        *dvdz = (ucat[k+1][j][i+1].y + ucat[k+1][j][i].y - ucat[k-1][j][i+1].y - ucat[k-1][j][i].y) * 0.25;
        *dwdz = (ucat[k+1][j][i+1].z + ucat[k+1][j][i].z - ucat[k-1][j][i+1].z - ucat[k-1][j][i].z) * 0.25;
    }

    return;
}


//*****************************************************************************************************//

inline void Compute_du_j
(
    mesh_ *mesh,
    PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz,
    Cmpnts ***ucat, PetscReal ***nvert, PetscReal ***meshTag,
    PetscReal *dudc, PetscReal *dvdc, PetscReal *dwdc,
    PetscReal *dude, PetscReal *dvde, PetscReal *dwde,
    PetscReal *dudz, PetscReal *dvdz, PetscReal *dwdz
)

{
    if (isIBMSolidJFace(k, j, i+1, j+1, nvert) || isZeroedJFace(k, j, i+1, j+1, meshTag) ||
          (i==mx-2 && (j==0 || j==my-2) && ((!mesh->i_periodic && !mesh->ii_periodic) || (!mesh->j_periodic && !mesh->jj_periodic)) ))
    {
        *dudc = (ucat[k][j+1][i  ].x + ucat[k][j][i  ].x - ucat[k][j+1][i-1].x - ucat[k][j][i-1].x) * 0.5;
        *dvdc = (ucat[k][j+1][i  ].y + ucat[k][j][i  ].y - ucat[k][j+1][i-1].y - ucat[k][j][i-1].y) * 0.5;
        *dwdc = (ucat[k][j+1][i  ].z + ucat[k][j][i  ].z - ucat[k][j+1][i-1].z - ucat[k][j][i-1].z) * 0.5;
    }
    else if (isIBMSolidJFace(k, j, i-1, j+1, nvert) || isZeroedJFace(k, j, i-1, j+1, meshTag) ||
            (i==1 && (j==0 || j==my-2) && ((!mesh->i_periodic && !mesh->ii_periodic) || (!mesh->j_periodic && !mesh->jj_periodic)) ))
    {
        *dudc = (ucat[k][j+1][i+1].x + ucat[k][j][i+1].x - ucat[k][j+1][i  ].x - ucat[k][j][i  ].x) * 0.5;
        *dvdc = (ucat[k][j+1][i+1].y + ucat[k][j][i+1].y - ucat[k][j+1][i  ].y - ucat[k][j][i  ].y) * 0.5;
        *dwdc = (ucat[k][j+1][i+1].z + ucat[k][j][i+1].z - ucat[k][j+1][i  ].z - ucat[k][j][i  ].z) * 0.5;
    }
    else
    {
        *dudc = (ucat[k][j+1][i+1].x + ucat[k][j][i+1].x - ucat[k][j+1][i-1].x - ucat[k][j][i-1].x) * 0.25;
        *dvdc = (ucat[k][j+1][i+1].y + ucat[k][j][i+1].y - ucat[k][j+1][i-1].y - ucat[k][j][i-1].y) * 0.25;
        *dwdc = (ucat[k][j+1][i+1].z + ucat[k][j][i+1].z - ucat[k][j+1][i-1].z - ucat[k][j][i-1].z) * 0.25;
    }

    *dude = ucat[k][j+1][i].x - ucat[k][j][i].x;
    *dvde = ucat[k][j+1][i].y - ucat[k][j][i].y;
    *dwde = ucat[k][j+1][i].z - ucat[k][j][i].z;

    if (isIBMSolidJFace(k+1, j, i, j+1, nvert) || isZeroedJFace(k+1, j, i, j+1, meshTag) ||
            (k==mz-2 && (j==0 || j==my-2) && ((!mesh->k_periodic && !mesh->kk_periodic) || (!mesh->j_periodic && !mesh->jj_periodic)) ))
    {
        *dudz = (ucat[k  ][j+1][i].x + ucat[k  ][j][i].x - ucat[k-1][j+1][i].x - ucat[k-1][j][i].x) * 0.5;
        *dvdz = (ucat[k  ][j+1][i].y + ucat[k  ][j][i].y - ucat[k-1][j+1][i].y - ucat[k-1][j][i].y) * 0.5;
        *dwdz = (ucat[k  ][j+1][i].z + ucat[k  ][j][i].z - ucat[k-1][j+1][i].z - ucat[k-1][j][i].z) * 0.5;
    }
    else if (isIBMSolidJFace(k-1, j, i, j+1, nvert) || isZeroedJFace(k-1, j, i, j+1, meshTag) ||
            (k==1 && (j==0 || j==my-2) && ((!mesh->k_periodic && !mesh->kk_periodic) || (!mesh->j_periodic && !mesh->jj_periodic)) ))
    {
```

```
        *dudz = (ucat[k+1][j+1][i].x + ucat[k+1][j][i].x - ucat[k  ][j+1][i].x - ucat[k  ][j][i].x) * 0.5;
        *dvdz = (ucat[k+1][j+1][i].y + ucat[k+1][j][i].y - ucat[k  ][j+1][i].y - ucat[k  ][j][i].y) * 0.5;
        *dwdz = (ucat[k+1][j+1][i].z + ucat[k+1][j][i].z - ucat[k  ][j+1][i].z - ucat[k  ][j][i].z) * 0.5;
    }
    else
    {
        *dudz = (ucat[k+1][j+1][i].x + ucat[k+1][j][i].x - ucat[k-1][j+1][i].x - ucat[k-1][j][i].x) * 0.25;
        *dvdz = (ucat[k+1][j+1][i].y + ucat[k+1][j][i].y - ucat[k-1][j+1][i].y - ucat[k-1][j][i].y) * 0.25;
        *dwdz = (ucat[k+1][j+1][i].z + ucat[k+1][j][i].z - ucat[k-1][j+1][i].z - ucat[k-1][j][i].z) * 0.25;
    }

    return;
}


//*****************************************************************************************************//

inline void Compute_du_k
(
    mesh_ *mesh,
    PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz,
    Cmpnts ***ucat, PetscReal ***nvert, PetscReal ***meshTag,
    PetscReal *dudc, PetscReal *dvdc, PetscReal *dwdc,
    PetscReal *dude, PetscReal *dvde, PetscReal *dwde,
    PetscReal *dudz, PetscReal *dvdz, PetscReal *dwdz
)

{
    if (isIBMSolidKFace(k, j, i+1, k+1, nvert) || isZeroedKFace(k, j, i+1, k+1, meshTag) ||
            (i==mx-2 && (k==0 || k==mz-2) && ((!mesh->i_periodic && !mesh->ii_periodic) || (!mesh->k_periodic && !mesh->kk_periodic)) ))
    {
        *dudc = (ucat[k+1][j][i  ].x + ucat[k][j][i  ].x - ucat[k+1][j][i-1].x - ucat[k][j][i-1].x) * 0.5;
        *dvdc = (ucat[k+1][j][i  ].y + ucat[k][j][i  ].y - ucat[k+1][j][i-1].y - ucat[k][j][i-1].y) * 0.5;
        *dwdc = (ucat[k+1][j][i  ].z + ucat[k][j][i  ].z - ucat[k+1][j][i-1].z - ucat[k][j][i-1].z) * 0.5;
    }
    else if (isIBMSolidKFace(k, j, i-1, k+1, nvert) || isZeroedKFace(k, j, i-1, k+1, meshTag) ||
            (i==1 && (k==0 || k==mz-2) && ((!mesh->i_periodic && !mesh->ii_periodic) || (!mesh->k_periodic && !mesh->kk_periodic)) ))
    {
        *dudc = (ucat[k+1][j][i+1].x + ucat[k][j][i+1].x - ucat[k+1][j][i  ].x - ucat[k][j][i  ].x) * 0.5;
        *dvdc = (ucat[k+1][j][i+1].y + ucat[k][j][i+1].y - ucat[k+1][j][i  ].y - ucat[k][j][i  ].y) * 0.5;
        *dwdc = (ucat[k+1][j][i+1].z + ucat[k][j][i+1].z - ucat[k+1][j][i  ].z - ucat[k][j][i  ].z) * 0.5;
    }
    else
    {
        *dudc = (ucat[k+1][j][i+1].x + ucat[k][j][i+1].x - ucat[k+1][j][i-1].x - ucat[k][j][i-1].x) * 0.25;
        *dvdc = (ucat[k+1][j][i+1].y + ucat[k][j][i+1].y - ucat[k+1][j][i-1].y - ucat[k][j][i-1].y) * 0.25;
        *dwdc = (ucat[k+1][j][i+1].z + ucat[k][j][i+1].z - ucat[k+1][j][i-1].z - ucat[k][j][i-1].z) * 0.25;
    }

    if (isIBMSolidKFace(k, j+1, i, k+1, nvert) || isZeroedKFace(k, j+1, i, k+1, meshTag) ||
            (j==my-2 && (k==0 || k==mz-2) && ((!mesh->j_periodic && !mesh->jj_periodic) || (!mesh->k_periodic && !mesh->kk_periodic)) ))
    {
        *dude = (ucat[k+1][j  ][i].x + ucat[k][j  ][i].x - ucat[k+1][j-1][i].x - ucat[k][j-1][i].x) * 0.5;
        *dvde = (ucat[k+1][j  ][i].y + ucat[k][j  ][i].y - ucat[k+1][j-1][i].y - ucat[k][j-1][i].y) * 0.5;
        *dwde = (ucat[k+1][j  ][i].z + ucat[k][j  ][i].z - ucat[k+1][j-1][i].z - ucat[k][j-1][i].z) * 0.5;
    }
    else if (isIBMSolidKFace(k, j-1, i, k+1, nvert) || isZeroedKFace(k, j-1, i, k+1, meshTag) ||
            (j==1 && (k==0 || k==mz-2) && ((!mesh->j_periodic && !mesh->jj_periodic) || (!mesh->k_periodic && !mesh->kk_periodic)) ))
    {
        *dude = (ucat[k+1][j+1][i].x + ucat[k][j+1][i].x - ucat[k+1][j  ][i].x - ucat[k][j  ][i].x) * 0.5;
        *dvde = (ucat[k+1][j+1][i].y + ucat[k][j+1][i].y - ucat[k+1][j  ][i].y - ucat[k][j  ][i].y) * 0.5;
        *dwde = (ucat[k+1][j+1][i].z + ucat[k][j+1][i].z - ucat[k+1][j  ][i].z - ucat[k][j  ][i].z) * 0.5;
```

```
    }
    else
    {
        *dude = (ucat[k+1][j+1][i].x + ucat[k][j+1][i].x - ucat[k+1][j-1][i].x - ucat[k][j-1][i].x) * 0.25;
        *dvde = (ucat[k+1][j+1][i].y + ucat[k][j+1][i].y - ucat[k+1][j-1][i].y - ucat[k][j-1][i].y) * 0.25;
        *dwde = (ucat[k+1][j+1][i].z + ucat[k][j+1][i].z - ucat[k+1][j-1][i].z - ucat[k][j-1][i].z) * 0.25;
    }

    *dudz = ucat[k+1][j][i].x - ucat[k][j][i].x;
    *dvdz = ucat[k+1][j][i].y - ucat[k][j][i].y;
    *dwdz = ucat[k+1][j][i].z - ucat[k][j][i].z;

    return;
}


//*****************************************************************************************************//

inline void Compute_du_i_4
(
    mesh_ *mesh,
    PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz,
    Cmpnts ***ucat, PetscReal ***nvert, PetscReal ***meshTag,
    PetscReal *dudc, PetscReal *dvdc, PetscReal *dwdc,
    PetscReal *dude, PetscReal *dvde, PetscReal *dwde,
    PetscReal *dudz, PetscReal *dvdz, PetscReal *dwdz
)
{
    if(i >= 1 && i <= mx - 3)
    {
        *dudc = (-ucat[k][j][i+2].x + 7.0 * ucat[k][j][i+1].x - 7.0 * ucat[k][j][i].x + ucat[k][j][i-1].x) / 12.0;
        *dvdc = (-ucat[k][j][i+2].y + 7.0 * ucat[k][j][i+1].y - 7.0 * ucat[k][j][i].y + ucat[k][j][i-1].y) / 12.0;
        *dwdc = (-ucat[k][j][i+2].z + 7.0 * ucat[k][j][i+1].z - 7.0 * ucat[k][j][i].z + ucat[k][j][i-1].z) / 12.0;
    }

    else
    {
        *dudc = ucat[k][j][i+1].x - ucat[k][j][i].x;
        *dvdc = ucat[k][j][i+1].y - ucat[k][j][i].y;
        *dwdc = ucat[k][j][i+1].z - ucat[k][j][i].z;
    }

    if(j >= 1 && j <= my - 3
       && !(isIBMSolidIFace(k, j+1, i, i+1, nvert) || isZeroedIFace(k, j+1, i, i+1, meshTag))
       && !(isIBMSolidIFace(k, j-1, i, i+1, nvert) || isZeroedIFace(k, j-1, i, i+1, meshTag)) )
    {
        *dude = (- (ucat[k][j+2][i+1].x + ucat[k][j+2][i].x) + 7.0 * (ucat[k][j+1][i+1].x + ucat[k][j+1][i].x) - 7.0 * (ucat[k][j][i+1].x + ucat[k][j][i].x) + (ucat[k][j-1][i+1].x + ucat[k][j-1][i].x)) / 24.0;
        *dvde = (- (ucat[k][j+2][i+1].y + ucat[k][j+2][i].y) + 7.0 * (ucat[k][j+1][i+1].y + ucat[k][j+1][i].y) - 7.0 * (ucat[k][j][i+1].y + ucat[k][j][i].y) + (ucat[k][j-1][i+1].y + ucat[k][j-1][i].y)) / 24.0;
        *dwde = (- (ucat[k][j+2][i+1].z + ucat[k][j+2][i].z) + 7.0 * (ucat[k][j+1][i+1].z + ucat[k][j+1][i].z) - 7.0 * (ucat[k][j][i+1].z + ucat[k][j][i].z) + (ucat[k][j-1][i+1].z + ucat[k][j-1][i].z)) / 24.0;
    }

    else if (isIBMSolidIFace(k, j+1, i, i+1, nvert) || isZeroedIFace(k, j+1, i, i+1, meshTag) ||
    (j==my-2 && (i==0 || i==mx-2) && ((!mesh->j_periodic && !mesh->jj_periodic) || (!mesh->i_periodic && !mesh->ii_periodic)) ))
    {
        *dude = (ucat[k][j][i+1].x + ucat[k][j][i].x - ucat[k][j-1][i+1].x - ucat[k][j-1][i].x) * 0.5;
        *dvde = (ucat[k][j][i+1].y + ucat[k][j][i].y - ucat[k][j-1][i+1].y - ucat[k][j-1][i].y) * 0.5;
        *dwde = (ucat[k][j][i+1].z + ucat[k][j][i].z - ucat[k][j-1][i+1].z - ucat[k][j-1][i].z) * 0.5;
    }

    else if  (isIBMSolidIFace(k, j-1, i, i+1, nvert) || isZeroedIFace(k, j-1, i, i+1, meshTag) ||
             (j==1 && (i==0 || i==mx-2) && ((!mesh->j_periodic && !mesh->jj_periodic) || (!mesh->i_periodic && !mesh->ii_periodic)) ))
    {
        *dude = (ucat[k][j+1][i+1].x + ucat[k][j+1][i].x - ucat[k][j][i+1].x - ucat[k][j][i].x) * 0.5;
        *dvde = (ucat[k][j+1][i+1].y + ucat[k][j+1][i].y - ucat[k][j][i+1].y - ucat[k][j][i].y) * 0.5;
        *dwde = (ucat[k][j+1][i+1].z + ucat[k][j+1][i].z - ucat[k][j][i+1].z - ucat[k][j][i].z) * 0.5;
    }

    else
    {
        *dude = (ucat[k][j+1][i+1].x + ucat[k][j+1][i].x - ucat[k][j-1][i+1].x - ucat[k][j-1][i].x) * 0.25;
        *dvde = (ucat[k][j+1][i+1].y + ucat[k][j+1][i].y - ucat[k][j-1][i+1].y - ucat[k][j-1][i].y) * 0.25;
```

```
        *dwde = (ucat[k][j+1][i+1].z + ucat[k][j+1][i].z - ucat[k][j-1][i+1].z - ucat[k][j-1][i].z) * 0.25;
    }

    if(k >= 1 && k <= mz - 3
        && !(isIBMSolidIFace(k+1, j, i, i+1, nvert) || isZeroedIFace(k+1, j, i, i+1, meshTag))
        && !(isIBMSolidIFace(k-1, j, i, i+1, nvert) || isZeroedIFace(k-1, j, i, i+1, meshTag)) )
    {
        *dudz = (- (ucat[k+2][j][i+1].x + ucat[k+2][j][i].x) + 7.0 * (ucat[k+1][j][i+1].x + ucat[k+1][j][i].x) - 7.0 * (ucat[k][j][i+1].x + ucat[k][j][i].x) + (ucat[k-1][j][i+1].x + ucat[k-1][j][i].x)) / 24.0;
        *dvdz = (- (ucat[k+2][j][i+1].y + ucat[k+2][j][i].y) + 7.0 * (ucat[k+1][j][i+1].y + ucat[k+1][j][i].y) - 7.0 * (ucat[k][j][i+1].y + ucat[k][j][i].y) + (ucat[k-1][j][i+1].y + ucat[k-1][j][i].y)) / 24.0;
        *dwdz = (- (ucat[k+2][j][i+1].z + ucat[k+2][j][i].z) + 7.0 * (ucat[k+1][j][i+1].z + ucat[k+1][j][i].z) - 7.0 * (ucat[k][j][i+1].z + ucat[k][j][i].z) + (ucat[k-1][j][i+1].z + ucat[k-1][j][i].z)) / 24.0;
    }

    else if(isIBMSolidIFace(k+1, j, i, i+1, nvert) || isZeroedIFace(k+1, j, i, i+1, meshTag) ||
    (k==mz-2 && (i==0 || i==mx-2) && ((!mesh->k_periodic && !mesh->kk_periodic) || (!mesh->i_periodic && !mesh->ii_periodic)) ))
    {
        *dudz = (ucat[k][j][i+1].x + ucat[k][j][i].x - ucat[k-1][j][i+1].x - ucat[k-1][j][i].x) * 0.5;
        *dvdz = (ucat[k][j][i+1].y + ucat[k][j][i].y - ucat[k-1][j][i+1].y - ucat[k-1][j][i].y) * 0.5;
        *dwdz = (ucat[k][j][i+1].z + ucat[k][j][i].z - ucat[k-1][j][i+1].z - ucat[k-1][j][i].z) * 0.5;
    }

    else if (isIBMSolidIFace(k-1, j, i, i+1, nvert) || isZeroedIFace(k-1, j, i, i+1, meshTag) ||
            (k==1 && (i==0 || i==mx-2) && ((!mesh->k_periodic && !mesh->kk_periodic) || (!mesh->i_periodic && !mesh->ii_periodic)) ))
    {
        *dudz = (ucat[k+1][j][i+1].x + ucat[k+1][j][i].x - ucat[k][j][i+1].x - ucat[k][j][i].x) * 0.5;
        *dvdz = (ucat[k+1][j][i+1].y + ucat[k+1][j][i].y - ucat[k][j][i+1].y - ucat[k][j][i].y) * 0.5;
        *dwdz = (ucat[k+1][j][i+1].z + ucat[k+1][j][i].z - ucat[k][j][i+1].z - ucat[k][j][i].z) * 0.5;
    }

    else
    {
        *dudz = (ucat[k+1][j][i+1].x + ucat[k+1][j][i].x - ucat[k-1][j][i+1].x - ucat[k-1][j][i].x) * 0.25;
        *dvdz = (ucat[k+1][j][i+1].y + ucat[k+1][j][i].y - ucat[k-1][j][i+1].y - ucat[k-1][j][i].y) * 0.25;
        *dwdz = (ucat[k+1][j][i+1].z + ucat[k+1][j][i].z - ucat[k-1][j][i+1].z - ucat[k-1][j][i].z) * 0.25;
    }

}

//*****************************************************************************************************//
inline void Compute_du_j_4
(
    mesh_ *mesh,
    PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz,
    Cmpnts ***ucat, PetscReal ***nvert, PetscReal ***meshTag,
    PetscReal *dudc, PetscReal *dvdc, PetscReal *dwdc,
    PetscReal *dude, PetscReal *dvde, PetscReal *dwde,
    PetscReal *dudz, PetscReal *dvdz, PetscReal *dwdz
)

{
    if(i >= 1 && i <= mx - 3
        && !(isIBMSolidJFace(k, j, i+1, j+1, nvert) || isZeroedJFace(k, j, i+1, j+1, meshTag))
        && !(isIBMSolidJFace(k, j, i-1, j+1, nvert) || isZeroedJFace(k, j, i-1, j+1, meshTag)) )
    {
        *dudc = (- (ucat[k][j][i+2].x + ucat[k][j+1][i+2].x) + 7.0*(ucat[k][j][i+1].x + ucat[k][j+1][i+1].x) - 7.0*(ucat[k][j][i].x + ucat[k][j+1][i].x) + (ucat[k][j][i-1].x + ucat[k][j+1][i-1].x)) / 24.0;
        *dvdc = (- (ucat[k][j][i+2].y + ucat[k][j+1][i+2].y) + 7.0*(ucat[k][j][i+1].y + ucat[k][j+1][i+1].y) - 7.0*(ucat[k][j][i].y + ucat[k][j+1][i].y) + (ucat[k][j][i-1].y + ucat[k][j+1][i-1].y)) / 24.0;
        *dwdc = (- (ucat[k][j][i+2].z + ucat[k][j+1][i+2].z) + 7.0*(ucat[k][j][i+1].z + ucat[k][j+1][i+1].z) - 7.0*(ucat[k][j][i].z + ucat[k][j+1][i].z) + (ucat[k][j][i-1].z + ucat[k][j+1][i-1].z)) / 24.0;
    }

    else if (isIBMSolidJFace(k, j, i+1, j+1, nvert) || isZeroedJFace(k, j, i+1, j+1, meshTag) ||
    (i==mx-2 && (j==0 || j==my-2) && ((!mesh->i_periodic && !mesh->ii_periodic) || (!mesh->j_periodic && !mesh->jj_periodic)) ))
    {
        *dudc = (ucat[k][j+1][i].x + ucat[k][j][i].x - ucat[k][j+1][i-1].x - ucat[k][j][i-1].x) * 0.5;
        *dvdc = (ucat[k][j+1][i].y + ucat[k][j][i].y - ucat[k][j+1][i-1].y - ucat[k][j][i-1].y) * 0.5;
        *dwdc = (ucat[k][j+1][i].z + ucat[k][j][i].z - ucat[k][j+1][i-1].z - ucat[k][j][i-1].z) * 0.5;
    }

    else if (isIBMSolidJFace(k, j, i-1, j+1, nvert) || isZeroedJFace(k, j, i-1, j+1, meshTag) ||
            (i==1 && (j==0 || j==my-2) && ((!mesh->i_periodic && !mesh->ii_periodic) ||
```

```
(!mesh->j_periodic && !mesh->jj_periodic)) ))
    {
        *dudc = (ucat[k][j+1][i+1].x + ucat[k][j][i+1].x - ucat[k][j+1][i].x - ucat[k][j][i].x) * 0.5;
        *dvdc = (ucat[k][j+1][i+1].y + ucat[k][j][i+1].y - ucat[k][j+1][i].y - ucat[k][j][i].y) * 0.5;
        *dwdc = (ucat[k][j+1][i+1].z + ucat[k][j][i+1].z - ucat[k][j+1][i].z - ucat[k][j][i].z) * 0.5;
    }

    else
    {
        *dudc = (ucat[k][j+1][i+1].x + ucat[k][j][i+1].x - ucat[k][j+1][i-1].x - ucat[k][j][i-1].x) * 0.25;
        *dvdc = (ucat[k][j+1][i+1].y + ucat[k][j][i+1].y - ucat[k][j+1][i-1].y - ucat[k][j][i-1].y) * 0.25;
        *dwdc = (ucat[k][j+1][i+1].z + ucat[k][j][i+1].z - ucat[k][j+1][i-1].z - ucat[k][j][i-1].z) * 0.25;
    }

    if(j >= 1 && j <= my - 3)
    {
        *dude = (-ucat[k][j+2][i].x + 7.0*ucat[k][j+1][i].x - 7.0*ucat[k][j][i].x + ucat[k][j-1][i].x)/12.0;
        *dvde = (-ucat[k][j+2][i].y + 7.0*ucat[k][j+1][i].y - 7.0*ucat[k][j][i].y + ucat[k][j-1][i].y)/12.0;
        *dwde = (-ucat[k][j+2][i].z + 7.0*ucat[k][j+1][i].z - 7.0*ucat[k][j][i].z + ucat[k][j-1][i].z)/12.0;
    }
    else
    {
        *dude = ucat[k][j+1][i].x - ucat[k][j][i].x;
        *dvde = ucat[k][j+1][i].y - ucat[k][j][i].y;
        *dwde = ucat[k][j+1][i].z - ucat[k][j][i].z;
    }

    if(k >= 1 && k <= mz - 3
        && !(isIBMSolidJFace(k+1, j, i, j+1, nvert) || isZeroedJFace(k+1, j, i, j+1, meshTag))
        && !(isIBMSolidJFace(k-1, j, i, j+1, nvert) || isZeroedJFace(k-1, j, i, j+1, meshTag)) )
    {
        *dudz = (- (ucat[k+2][j][i].x + ucat[k+2][j+1][i].x) + 7.0*(ucat[k+1][j][i].x + ucat[k+1][j+1][i].x) - 7.0*(ucat[k][j][i].x + ucat[k][j+1][i].x) + (ucat[k-1][j][i].x + ucat[k-1][j+1][i].x)) / 24.0;
        *dvdz = (- (ucat[k+2][j][i].y + ucat[k+2][j+1][i].y) + 7.0*(ucat[k+1][j][i].y + ucat[k+1][j+1][i].y) - 7.0*(ucat[k][j][i].y + ucat[k][j+1][i].y) + (ucat[k-1][j][i].y + ucat[k-1][j+1][i].y)) / 24.0;
        *dwdz = (- (ucat[k+2][j][i].z + ucat[k+2][j+1][i].z) + 7.0*(ucat[k+1][j][i].z + ucat[k+1][j+1][i].z) - 7.0*(ucat[k][j][i].z + ucat[k][j+1][i].z) + (ucat[k-1][j][i].z + ucat[k-1][j+1][i].z)) / 24.0;
    }

    else if (isIBMSolidJFace(k+1, j, i, j+1, nvert) || isZeroedJFace(k+1, j, i, j+1, meshTag) ||
    (k==mz-2 && (j==0 || j==my-2) && ((!mesh->k_periodic && !mesh->kk_periodic) || (!mesh->j_periodic && !mesh->jj_periodic)) ))
    {
        *dudz = (ucat[k][j+1][i].x + ucat[k][j][i].x - ucat[k-1][j+1][i].x - ucat[k-1][j][i].x) * 0.5;
        *dvdz = (ucat[k][j+1][i].y + ucat[k][j][i].y - ucat[k-1][j+1][i].y - ucat[k-1][j][i].y) * 0.5;
        *dwdz = (ucat[k][j+1][i].z + ucat[k][j][i].z - ucat[k-1][j+1][i].z - ucat[k-1][j][i].z) * 0.5;
    }

    else if (isIBMSolidJFace(k-1, j, i, j+1, nvert) || isZeroedJFace(k-1, j, i, j+1, meshTag) ||
    (k==1 && (j==0 || j==my-2) && ((!mesh->k_periodic && !mesh->kk_periodic) || (!mesh->j_periodic && !mesh->jj_periodic)) ))
    {
        *dudz = (ucat[k+1][j+1][i].x + ucat[k+1][j][i].x - ucat[k][j+1][i].x - ucat[k][j][i].x) * 0.5;
        *dvdz = (ucat[k+1][j+1][i].y + ucat[k+1][j][i].y - ucat[k][j+1][i].y - ucat[k][j][i].y) * 0.5;
        *dwdz = (ucat[k+1][j+1][i].z + ucat[k+1][j][i].z - ucat[k][j+1][i].z - ucat[k][j][i].z) * 0.5;
    }

    else
    {
        *dudz = (ucat[k+1][j+1][i].x + ucat[k+1][j][i].x - ucat[k-1][j+1][i].x - ucat[k-1][j][i].x) * 0.25;
        *dvdz = (ucat[k+1][j+1][i].y + ucat[k+1][j][i].y - ucat[k-1][j+1][i].y - ucat[k-1][j][i].y) * 0.25;
        *dwdz = (ucat[k+1][j+1][i].z + ucat[k+1][j][i].z - ucat[k-1][j+1][i].z - ucat[k-1][j][i].z) * 0.25;
    }

    return;

}

//*****************************************************************************************************//
inline void Compute_du_k_4
(
    mesh_ *mesh,
    PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz,
    Cmpnts ***ucat, PetscReal ***nvert, PetscReal ***meshTag,
    PetscReal *dudc, PetscReal *dvdc, PetscReal *dwdc,
```

```
    PetscReal *dude, PetscReal *dvde, PetscReal *dwde,
    PetscReal *dudz, PetscReal *dvdz, PetscReal *dwdz
)

{
    if(i >= 1 && i <= mx - 3
        && !(isIBMSolidKFace(k, j, i+1, k+1, nvert) || isZeroedKFace(k, j, i+1, k+1, meshTag))
        && !(isIBMSolidKFace(k, j, i-1, k+1, nvert) || isZeroedKFace(k, j, i-1, k+1, meshTag)) )
    {
        *dudc = (- (ucat[k+1][j][i+2].x + ucat[k][j][i+2].x) + 7.0*(ucat[k+1][j][i+1].x + ucat[k][j][i+1].x) - 7.0*(ucat[k+1][j][i].x + ucat[k][j][i].x) + (ucat[k+1][j][i-1].x + ucat[k][j][i-1].x)) / 24.0;
        *dvdc = (- (ucat[k+1][j][i+2].y + ucat[k][j][i+2].y) + 7.0*(ucat[k+1][j][i+1].y + ucat[k][j][i+1].y) - 7.0*(ucat[k+1][j][i].y + ucat[k][j][i].y) + (ucat[k+1][j][i-1].y + ucat[k][j][i-1].y)) / 24.0;
        *dwdc = (- (ucat[k+1][j][i+2].z + ucat[k][j][i+2].z) + 7.0*(ucat[k+1][j][i+1].z + ucat[k][j][i+1].z) - 7.0*(ucat[k+1][j][i].z + ucat[k][j][i].z) + (ucat[k+1][j][i-1].z + ucat[k][j][i-1].z)) / 24.0;
    }

    else if (isIBMSolidKFace(k, j, i+1, k+1, nvert) || isZeroedKFace(k, j, i+1, k+1, meshTag) ||
    (i==mx-2 && (k==0 || k==mz-2) && ((!mesh->i_periodic && !mesh->ii_periodic) || (!mesh->k_periodic && !mesh->kk_periodic)) ))
    {
        *dudc = (ucat[k+1][j][i].x + ucat[k][j][i].x - ucat[k+1][j][i-1].x - ucat[k][j][i-1].x)*0.5;
        *dvdc = (ucat[k+1][j][i].y + ucat[k][j][i].y - ucat[k+1][j][i-1].y - ucat[k][j][i-1].y)*0.5;
        *dwdc = (ucat[k+1][j][i].z + ucat[k][j][i].z - ucat[k+1][j][i-1].z - ucat[k][j][i-1].z)*0.5;
    }

    else if (isIBMSolidKFace(k, j, i-1, k+1, nvert) || isZeroedKFace(k, j, i-1, k+1, meshTag) ||
    (i==1 && (k==0 || k==mz-2) && ((!mesh->i_periodic && !mesh->ii_periodic) || (!mesh->k_periodic && !mesh->kk_periodic)) ))
    {
        *dudc = (ucat[k+1][j][i+1].x + ucat[k][j][i+1].x - ucat[k+1][j][i].x - ucat[k][j][i].x)*0.5;
        *dvdc = (ucat[k+1][j][i+1].y + ucat[k][j][i+1].y - ucat[k+1][j][i].y - ucat[k][j][i].y)*0.5;
        *dwdc = (ucat[k+1][j][i+1].z + ucat[k][j][i+1].z - ucat[k+1][j][i].z - ucat[k][j][i].z)*0.5;
    }

    else
    {
        *dudc = (ucat[k+1][j][i+1].x + ucat[k][j][i+1].x - ucat[k+1][j][i-1].x - ucat[k][j][i-1].x)*0.25;
        *dvdc = (ucat[k+1][j][i+1].y + ucat[k][j][i+1].y - ucat[k+1][j][i-1].y - ucat[k][j][i-1].y)*0.25;
        *dwdc = (ucat[k+1][j][i+1].z + ucat[k][j][i+1].z - ucat[k+1][j][i-1].z - ucat[k][j][i-1].z)*0.25;
    }

    if(j >= 1 && j <= my - 3
        && !(isIBMSolidKFace(k, j+1, i, k+1, nvert) || isZeroedKFace(k, j+1, i, k+1, meshTag))
        && !(isIBMSolidKFace(k, j-1, i, k+1, nvert) || isZeroedKFace(k, j-1, i, k+1, meshTag)) )
    {
        *dude = (- (ucat[k+1][j+2][i].x + ucat[k][j+2][i].x) + 7.0*(ucat[k+1][j+1][i].x + ucat[k][j+1][i].x) - 7.0*(ucat[k+1][j][i].x + ucat[k][j][i].x) + (ucat[k+1][j-1][i].x + ucat[k][j-1][i].x)) / 24.0;
        *dvde = (- (ucat[k+1][j+2][i].y + ucat[k][j+2][i].y) + 7.0*(ucat[k+1][j+1][i].y + ucat[k][j+1][i].y) - 7.0*(ucat[k+1][j][i].y + ucat[k][j][i].y) + (ucat[k+1][j-1][i].y + ucat[k][j-1][i].y)) / 24.0;
        *dwde = (- (ucat[k+1][j+2][i].z + ucat[k][j+2][i].z) + 7.0*(ucat[k+1][j+1][i].z + ucat[k][j+1][i].z) - 7.0*(ucat[k+1][j][i].z + ucat[k][j][i].z) + (ucat[k+1][j-1][i].z + ucat[k][j-1][i].z)) / 24.0;
    }

    else if (isIBMSolidKFace(k, j+1, i, k+1, nvert) || isZeroedKFace(k, j+1, i, k+1, meshTag) ||
    (j==my-2 && (k==0 || k==mz-2) && ((!mesh->j_periodic && !mesh->jj_periodic) || (!mesh->k_periodic && !mesh->kk_periodic)) ))
    {
        *dude = (ucat[k+1][j][i].x + ucat[k][j][i].x - ucat[k+1][j-1][i].x - ucat[k][j-1][i].x)*0.5;
        *dvde = (ucat[k+1][j][i].y + ucat[k][j][i].y - ucat[k+1][j-1][i].y - ucat[k][j-1][i].y)*0.5;
        *dwde = (ucat[k+1][j][i].z + ucat[k][j][i].z - ucat[k+1][j-1][i].z - ucat[k][j-1][i].z)*0.5;
    }

    else if (isIBMSolidKFace(k, j-1, i, k+1, nvert) || isZeroedKFace(k, j-1, i, k+1, meshTag) ||
            (j==1 && (k==0 || k==mz-2) && ((!mesh->j_periodic && !mesh->jj_periodic) || (!mesh->k_periodic && !mesh->kk_periodic)) ))
    {
        *dude = (ucat[k+1][j+1][i].x + ucat[k][j+1][i].x - ucat[k+1][j][i].x - ucat[k][j][i].x)*0.5;
        *dvde = (ucat[k+1][j+1][i].y + ucat[k][j+1][i].y - ucat[k+1][j][i].y - ucat[k][j][i].y)*0.5;
        *dwde = (ucat[k+1][j+1][i].z + ucat[k][j+1][i].z - ucat[k+1][j][i].z - ucat[k][j][i].z)*0.5;
    }

    else
    {
        *dude = (ucat[k+1][j+1][i].x + ucat[k][j+1][i].x - ucat[k+1][j-1][i].x - ucat[k][j-1][i].x)*0.25;
        *dvde = (ucat[k+1][j+1][i].y + ucat[k][j+1][i].y - ucat[k+1][j-1][i].y - ucat[k][j-1][i].y)*0.25;
```

```
        *dwde = (ucat[k+1][j+1][i].z + ucat[k][j+1][i].z - ucat[k+1][j-1][i].z - ucat[k][j-1][i].z)*0.25;
    }

    if(k >= 1 && k <= mz-3)
    {
        *dudz = (-ucat[k+2][j][i].x + 7.0*ucat[k+1][j][i].x - 7.0*ucat[k][j][i].x + ucat[k-1][j][i].x) / 12.0;
        *dvdz = (-ucat[k+2][j][i].y + 7.0*ucat[k+1][j][i].y - 7.0*ucat[k][j][i].y + ucat[k-1][j][i].y) / 12.0;
        *dwdz = (-ucat[k+2][j][i].z + 7.0*ucat[k+1][j][i].z - 7.0*ucat[k][j][i].z + ucat[k-1][j][i].z) / 12.0;
    }

    else
    {
        *dudz = ucat[k+1][j][i].x - ucat[k][j][i].x;
        *dvdz = ucat[k+1][j][i].y - ucat[k][j][i].y;
        *dwdz = ucat[k+1][j][i].z - ucat[k][j][i].z;
    }

    return;

}


//*****************************************************************************************************************//

inline void Compute_dscalar_i
(
    mesh_ *mesh,
    PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz,
    PetscReal ***K, PetscReal ***nvert, PetscReal ***meshTag,
    PetscReal *dkdc, PetscReal *dkde, PetscReal *dkdz
)
{
    *dkdc = K[k][j][i+1] - K[k][j][i];

    if (isIBMSolidIFace(k, j+1, i, i+1, nvert) || isZeroedIFace(k, j+1, i, i+1, meshTag) ||
            (j==my-2 && (i==0 || i==mx-2) && ((!mesh->j_periodic && !mesh->jj_periodic) || (!mesh->i_periodic && !mesh->ii_periodic)) ))
    {
        *dkde = (K[k][j  ][i+1] + K[k][j  ][i] - K[k][j-1][i+1] - K[k][j-1][i]) * 0.5;
    }
    else if  (isIBMSolidIFace(k, j-1, i, i+1, nvert) || isZeroedIFace(k, j-1, i, i+1, meshTag) ||
            (j==1 && (i==0 || i==mx-2) && ((!mesh->j_periodic && !mesh->jj_periodic) || (!mesh->i_periodic && !mesh->ii_periodic)) ))
    {
        *dkde = (K[k][j+1][i+1] + K[k][j+1][i] - K[k][j  ][i+1] - K[k][j  ][i]) * 0.5;
    }
    else
    {
        *dkde = (K[k][j+1][i+1] + K[k][j+1][i] - K[k][j-1][i+1] - K[k][j-1][i]) * 0.25;
    }
    if (isIBMSolidIFace(k+1, j, i, i+1, nvert) || isZeroedIFace(k+1, j, i, i+1, meshTag) ||
            (k==mz-2 && (i==0 || i==mx-2) && ((!mesh->k_periodic && !mesh->kk_periodic) || (!mesh->i_periodic && !mesh->ii_periodic)) ))
    {
        *dkdz = (K[k  ][j][i+1] + K[k  ][j][i] - K[k-1][j][i+1] - K[k-1][j][i]) * 0.5;
    }
    else if (isIBMSolidIFace(k-1, j, i, i+1, nvert) || isZeroedIFace(k-1, j, i, i+1, meshTag) ||
            (k==1 && (i==0 || i==mx-2) && ((!mesh->k_periodic && !mesh->kk_periodic) || (!mesh->i_periodic && !mesh->ii_periodic)) ))
    {
        *dkdz = (K[k+1][j][i+1] + K[k+1][j][i] - K[k  ][j][i+1] - K[k  ][j][i]) * 0.5;
    }
    else
    {
        *dkdz = (K[k+1][j][i+1] + K[k+1][j][i] - K[k-1][j][i+1] - K[k-1][j][i]) * 0.25;
    }

    return;
}


//*****************************************************************************************************************//

inline void Compute_dscalar_j
(
    mesh_ *mesh,
    PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz,
    PetscReal ***K, PetscReal ***nvert, PetscReal ***meshTag,
    PetscReal *dkdc, PetscReal *dkde, PetscReal *dkdz
)
{
    if (isIBMSolidJFace(k, j, i+1, j+1, nvert) || isZeroedJFace(k, j, i+1, j+1, meshTag) ||
          (i==mx-2 && (j==0 || j==my-2) && ((!mesh->i_periodic && !mesh->ii_periodic) ||
```

```
(!mesh->j_periodic && !mesh->jj_periodic)) ))
    {
        *dkdc = (K[k][j+1][i  ] + K[k][j][i  ] - K[k][j+1][i-1] - K[k][j][i-1]) * 0.5;
    }
    else if (isIBMSolidJFace(k, j, i-1, j+1, nvert) || isZeroedJFace(k, j, i-1, j+1, meshTag) ||
            (i==1 && (j==0 || j==my-2) && ((!mesh->i_periodic && !mesh->ii_periodic) ||
(!mesh->j_periodic && !mesh->jj_periodic)) ))
    {
        *dkdc = (K[k][j+1][i+1] + K[k][j][i+1] - K[k][j+1][i  ] - K[k][j][i  ]) * 0.5;
    }
    else
    {
        *dkdc = (K[k][j+1][i+1] + K[k][j][i+1] - K[k][j+1][i-1] - K[k][j][i-1]) * 0.25;
    }

    *dkde = K[k][j+1][i] - K[k][j][i];

    if (isIBMSolidJFace(k+1, j, i, j+1, nvert) || isZeroedJFace(k+1, j, i, j+1, meshTag) ||
            (k==mz-2 && (j==0 || j==my-2) && ((!mesh->k_periodic && !mesh->kk_periodic) ||
(!mesh->j_periodic && !mesh->jj_periodic)) ))
    {
        *dkdz = (K[k  ][j+1][i] + K[k  ][j][i] - K[k-1][j+1][i] - K[k-1][j][i]) * 0.5;
    }
    else if (isIBMSolidJFace(k-1, j, i, j+1, nvert) || isZeroedJFace(k-1, j, i, j+1, meshTag) ||
            (k==1 && (j==0 || j==my-2) && ((!mesh->k_periodic && !mesh->kk_periodic) ||
(!mesh->j_periodic && !mesh->jj_periodic)) ))
    {
        *dkdz = (K[k+1][j+1][i] + K[k+1][j][i] - K[k  ][j+1][i] - K[k  ][j][i]) * 0.5;
    }
    else
    {
        *dkdz = (K[k+1][j+1][i] + K[k+1][j][i] - K[k-1][j+1][i] - K[k-1][j][i]) * 0.25;
    }

    return;
}


//*****************************************************************************************************//

inline void Compute_dscalar_k
(
    mesh_ *mesh,
    PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz,
    PetscReal ***K, PetscReal ***nvert, PetscReal ***meshTag,
    PetscReal *dkdc, PetscReal *dkde, PetscReal *dkdz
)
{
    if (isIBMSolidKFace(k, j, i+1, k+1, nvert) || isZeroedKFace(k, j, i+1, k+1, meshTag) ||
           (i==mx-2 && (k==0 || k==mz-2) && ((!mesh->i_periodic && !mesh->ii_periodic) ||
(!mesh->k_periodic && !mesh->kk_periodic)) ))
    {
        *dkdc = (K[k+1][j][i  ] + K[k][j][i  ] - K[k+1][j][i-1] - K[k][j][i-1]) * 0.5;
    }
    else if (isIBMSolidKFace(k, j, i-1, k+1, nvert) || isZeroedKFace(k, j, i-1, k+1, meshTag) ||
            (i==1 && (k==0 || k==mz-2) && ((!mesh->i_periodic && !mesh->ii_periodic) ||
(!mesh->k_periodic && !mesh->kk_periodic)) ))
    {
        *dkdc = (K[k+1][j][i+1] + K[k][j][i+1] - K[k+1][j][i  ] - K[k][j][i  ]) * 0.5;
    }
    else
    {
        *dkdc = (K[k+1][j][i+1] + K[k][j][i+1] - K[k+1][j][i-1] - K[k][j][i-1]) * 0.25;
    }

    if (isIBMSolidKFace(k, j+1, i, k+1, nvert) || isZeroedKFace(k, j+1, i, k+1, meshTag) ||
            (j==my-2 && (k==0 || k==mz-2) && ((!mesh->j_periodic && !mesh->jj_periodic) ||
(!mesh->k_periodic && !mesh->kk_periodic)) ))
    {
        *dkde = (K[k+1][j  ][i] + K[k][j  ][i] - K[k+1][j-1][i] - K[k][j-1][i]) * 0.5;
    }
    else if (isIBMSolidKFace(k, j-1, i, k+1, nvert) || isZeroedKFace(k, j-1, i, k+1, meshTag) ||
            (j==1 && (k==0 || k==mz-2) && ((!mesh->j_periodic && !mesh->jj_periodic) ||
(!mesh->k_periodic && !mesh->kk_periodic)) ))
    {
        *dkde = (K[k+1][j+1][i] + K[k][j+1][i] - K[k+1][j  ][i] - K[k][j  ][i]) * 0.5;
    }
    else
    {
        *dkde = (K[k+1][j+1][i] + K[k][j+1][i] - K[k+1][j-1][i] - K[k][j-1][i]) * 0.25;
    }

    *dkdz = K[k+1][j][i] - K[k][j][i];

    return;
}
```

```
//*****************************************************************************************************************//

inline void Compute_du_center
(
    mesh_ *mesh,
    PetscInt i, PetscInt j, PetscInt k,  PetscInt mx, PetscInt my, PetscInt mz,
    Cmpnts ***ucat, PetscReal ***nvert, PetscReal ***meshTag,
    PetscReal *dudc, PetscReal *dvdc, PetscReal *dwdc,
    PetscReal *dude, PetscReal *dvde, PetscReal *dwde,
    PetscReal *dudz, PetscReal *dvdz, PetscReal *dwdz
)
{
    if (isIBMSolidCell(k, j, i+1, nvert) || isZeroedCell(k, j, i+1, meshTag) || (!mesh->i_periodic && !mesh->ii_periodic && i==mx-2))
    {
        *dudc = ( ucat[k][j][i].x - ucat[k][j][i-1].x );
        *dvdc = ( ucat[k][j][i].y - ucat[k][j][i-1].y );
        *dwdc = ( ucat[k][j][i].z - ucat[k][j][i-1].z );
    }
    else if (isIBMSolidCell(k, j, i-1, nvert) || isZeroedCell(k, j, i-1, meshTag) || (!mesh->i_periodic &&  !mesh->ii_periodic && i==1))
    {
        *dudc = ( ucat[k][j][i+1].x - ucat[k][j][i].x );
        *dvdc = ( ucat[k][j][i+1].y - ucat[k][j][i].y );
        *dwdc = ( ucat[k][j][i+1].z - ucat[k][j][i].z );
    }
    else
    {
        *dudc = ( ucat[k][j][i+1].x - ucat[k][j][i-1].x ) * 0.5;
        *dvdc = ( ucat[k][j][i+1].y - ucat[k][j][i-1].y ) * 0.5;
        *dwdc = ( ucat[k][j][i+1].z - ucat[k][j][i-1].z ) * 0.5;
    }

    if (isIBMSolidCell(k, j+1, i, nvert) || isZeroedCell(k, j+1, i, meshTag) || (!mesh->j_periodic && !mesh->jj_periodic && j==my-2))
    {
        *dude = ( ucat[k][j][i].x - ucat[k][j-1][i].x );
        *dvde = ( ucat[k][j][i].y - ucat[k][j-1][i].y );
        *dwde = ( ucat[k][j][i].z - ucat[k][j-1][i].z );
    }
    else if (isIBMSolidCell(k, j-1, i, nvert) || isZeroedCell(k, j-1, i, meshTag) || (!mesh->j_periodic &&  !mesh->jj_periodic && j==1))
    {
        *dude = ( ucat[k][j+1][i].x - ucat[k][j][i].x );
        *dvde = ( ucat[k][j+1][i].y - ucat[k][j][i].y );
        *dwde = ( ucat[k][j+1][i].z - ucat[k][j][i].z );

        // *dude = ( 4.0 * ucat[k][j+1][i].x - 3.0 * ucat[k][j][i].x - ucat[k][j+2][i].x) * 0.5;
        // *dvde = ( 4.0 * ucat[k][j+1][i].y - 3.0 * ucat[k][j][i].y - ucat[k][j+2][i].y) * 0.5;
        // *dwde = ( 4.0 * ucat[k][j+1][i].z - 3.0 * ucat[k][j][i].z - ucat[k][j+2][i].z) * 0.5;
    }
    else
    {
        *dude = ( ucat[k][j+1][i].x - ucat[k][j-1][i].x ) * 0.5;
        *dvde = ( ucat[k][j+1][i].y - ucat[k][j-1][i].y ) * 0.5;
        *dwde = ( ucat[k][j+1][i].z - ucat[k][j-1][i].z ) * 0.5;
    }

    if (isIBMSolidCell(k+1, j, i, nvert) || isZeroedCell(k+1, j, i, meshTag) || ( !mesh->k_periodic && !mesh->kk_periodic && k==mz-2))
    {
        *dudz = ( ucat[k][j][i].x - ucat[k-1][j][i].x );
        *dvdz = ( ucat[k][j][i].y - ucat[k-1][j][i].y );
        *dwdz = ( ucat[k][j][i].z - ucat[k-1][j][i].z );
    }
    else if (isIBMSolidCell(k-1, j, i, nvert) || isZeroedCell(k-1, j, i, meshTag) || (!mesh->k_periodic &&  !mesh->kk_periodic && k==1))
    {
        *dudz = ( ucat[k+1][j][i].x - ucat[k][j][i].x );
        *dvdz = ( ucat[k+1][j][i].y - ucat[k][j][i].y );
        *dwdz = ( ucat[k+1][j][i].z - ucat[k][j][i].z );
    }
    else
    {
        *dudz = ( ucat[k+1][j][i].x - ucat[k-1][j][i].x ) * 0.5;
        *dvdz = ( ucat[k+1][j][i].y - ucat[k-1][j][i].y ) * 0.5;
        *dwdz = ( ucat[k+1][j][i].z - ucat[k-1][j][i].z ) * 0.5;
    }

    return;
}


//*****************************************************************************************************************//

inline void Compute_du_center4th
```

```
(
    mesh_ *mesh,
    PetscInt i, PetscInt j, PetscInt k,  PetscInt mx, PetscInt my, PetscInt mz,
    Cmpnts ***ucat, PetscReal ***nvert, PetscReal ***meshTag,
    PetscReal *dudc, PetscReal *dvdc, PetscReal *dwdc,
    PetscReal *dude, PetscReal *dvde, PetscReal *dwde,
    PetscReal *dudz, PetscReal *dvdz, PetscReal *dwdz
)
{
    //csi direction
    if (isIBMSolidCell(k, j, i+1, nvert) || isZeroedCell(k, j, i+1, meshTag) || (!mesh->i_periodic && !mesh->ii_periodic && i==mx-2))
    {
        *dudc = ( ucat[k][j][i].x - ucat[k][j][i-1].x );
        *dvdc = ( ucat[k][j][i].y - ucat[k][j][i-1].y );
        *dwdc = ( ucat[k][j][i].z - ucat[k][j][i-1].z );
    }
    else if (isIBMSolidCell(k, j, i-1, nvert) || isZeroedCell(k, j, i-1, meshTag) || (!mesh->i_periodic &&  !mesh->ii_periodic && i==1))
    {
        *dudc = ( ucat[k][j][i+1].x - ucat[k][j][i].x );
        *dvdc = ( ucat[k][j][i+1].y - ucat[k][j][i].y );
        *dwdc = ( ucat[k][j][i+1].z - ucat[k][j][i].z );
    }
    else if((i==2 && !mesh->i_periodic &&  !mesh->ii_periodic) || (i==1 && (mesh->i_periodic || mesh->ii_periodic)) || (isIBMFluidIFace(k, j, i, i+1, nvert) || isInterpolatedIFace(k, j, i, i+1, meshTag)))
    {
        *dudc = ( ucat[k][j][i+1].x - ucat[k][j][i-1].x ) * 0.5;
        *dvdc = ( ucat[k][j][i+1].y - ucat[k][j][i-1].y ) * 0.5;
        *dwdc = ( ucat[k][j][i+1].z - ucat[k][j][i-1].z ) * 0.5;
    }
    else if((i==mx-3 && !mesh->i_periodic &&  !mesh->ii_periodic) || (i==mx-2 && (mesh->i_periodic || mesh->ii_periodic)) || (isIBMFluidIFace(k, j, i, i-1, nvert) || isInterpolatedIFace(k, j, i, i-1, meshTag)))
    {
        *dudc = ( ucat[k][j][i+1].x - ucat[k][j][i-1].x ) * 0.5;
        *dvdc = ( ucat[k][j][i+1].y - ucat[k][j][i-1].y ) * 0.5;
        *dwdc = ( ucat[k][j][i+1].z - ucat[k][j][i-1].z ) * 0.5;
    }
    else
    {
        *dudc = (-ucat[k][j][i+2].x + 8*ucat[k][j][i+1].x - 8*ucat[k][j][i-1].x + ucat[k][j][i-2].x) / 12.0;
        *dvdc = (-ucat[k][j][i+2].y + 8*ucat[k][j][i+1].y - 8*ucat[k][j][i-1].y + ucat[k][j][i-2].y) / 12.0;
        *dwdc = (-ucat[k][j][i+2].z + 8*ucat[k][j][i+1].z - 8*ucat[k][j][i-1].z + ucat[k][j][i-2].z) / 12.0;
    }

    //eta direction
    if (isIBMSolidCell(k, j+1, i, nvert) || isZeroedCell(k, j+1, i, meshTag) || (!mesh->j_periodic && !mesh->jj_periodic && j==my-2))
    {
        *dude = ( ucat[k][j][i].x - ucat[k][j-1][i].x );
        *dvde = ( ucat[k][j][i].y - ucat[k][j-1][i].y );
        *dwde = ( ucat[k][j][i].z - ucat[k][j-1][i].z );
    }
    else if (isIBMSolidCell(k, j-1, i, nvert) || isZeroedCell(k, j-1, i, meshTag) || (!mesh->j_periodic && !mesh->jj_periodic && j==1))
    {
        *dude = ( ucat[k][j+1][i].x - ucat[k][j][i].x );
        *dvde = ( ucat[k][j+1][i].y - ucat[k][j][i].y );
        *dwde = ( ucat[k][j+1][i].z - ucat[k][j][i].z );
    }
    else if((j==2 && !mesh->j_periodic && !mesh->jj_periodic) || (j==1 && (mesh->j_periodic || mesh->jj_periodic)) || (isIBMFluidJFace(k, j, i, j+1, nvert) || isInterpolatedJFace(k, j, i, j+1, meshTag)))
    {
        *dude = ( ucat[k][j+1][i].x - ucat[k][j-1][i].x ) * 0.5;
        *dvde = ( ucat[k][j+1][i].y - ucat[k][j-1][i].y ) * 0.5;
        *dwde = ( ucat[k][j+1][i].z - ucat[k][j-1][i].z ) * 0.5;
    }
    else if((j==my-3 && !mesh->j_periodic && !mesh->jj_periodic) || (j==my-2 && (mesh->j_periodic || mesh->jj_periodic)) || (isIBMFluidJFace(k, j, i, j-1, nvert) || isInterpolatedJFace(k, j, i, j-1, meshTag)))
    {
        *dude = ( ucat[k][j+1][i].x - ucat[k][j-1][i].x ) * 0.5;
        *dvde = ( ucat[k][j+1][i].y - ucat[k][j-1][i].y ) * 0.5;
        *dwde = ( ucat[k][j+1][i].z - ucat[k][j-1][i].z ) * 0.5;
    }
    else
    {
        *dude = (-ucat[k][j+2][i].x + 8*ucat[k][j+1][i].x - 8*ucat[k][j-1][i].x + ucat[k][j-2][i].x) / 12.0;
        *dvde = (-ucat[k][j+2][i].y + 8*ucat[k][j+1][i].y - 8*ucat[k][j-1][i].y + ucat[k][j-2][i].y) / 12.0;
```

```
        *dwde = (-ucat[k][j+2][i].z + 8*ucat[k][j+1][i].z - 8*ucat[k][j-1][i].z + ucat[k][j-2][i].z) / 12.0;
    }

    //zeta direction
    if (isIBMSolidCell(k+1, j, i, nvert) || isZeroedCell(k+1, j, i, meshTag) || (!mesh->k_periodic && !mesh->kk_periodic && k==mz-2))
    {
        *dudz = ( ucat[k][j][i].x - ucat[k-1][j][i].x );
        *dvdz = ( ucat[k][j][i].y - ucat[k-1][j][i].y );
        *dwdz = ( ucat[k][j][i].z - ucat[k-1][j][i].z );
    }
    else if (isIBMSolidCell(k-1, j, i, nvert) || isZeroedCell(k-1, j, i, meshTag) || (!mesh->k_periodic && !mesh->kk_periodic && k==1))
    {
        *dudz = ( ucat[k+1][j][i].x - ucat[k][j][i].x );
        *dvdz = ( ucat[k+1][j][i].y - ucat[k][j][i].y );
        *dwdz = ( ucat[k+1][j][i].z - ucat[k][j][i].z );
    }
    else if((k==2 && !mesh->k_periodic && !mesh->kk_periodic) || (k==1 && (mesh->k_periodic || mesh->kk_periodic)) || (isIBMFluidKFace(k, j, i, k+1, nvert) || isInterpolatedKFace(k, j, i, k+1, meshTag)))
    {
        *dudz = ( ucat[k+1][j][i].x - ucat[k-1][j][i].x ) * 0.5;
        *dvdz = ( ucat[k+1][j][i].y - ucat[k-1][j][i].y ) * 0.5;
        *dwdz = ( ucat[k+1][j][i].z - ucat[k-1][j][i].z ) * 0.5;
    }
    else if((k==mz-3 && !mesh->k_periodic && !mesh->kk_periodic) || (k==mz-2 && (mesh->k_periodic || mesh->kk_periodic)) || (isIBMFluidKFace(k, j, i, k-1, nvert) || isInterpolatedKFace(k, j, i, k-1, meshTag)))
    {
        *dudz = ( ucat[k+1][j][i].x - ucat[k-1][j][i].x ) * 0.5;
        *dvdz = ( ucat[k+1][j][i].y - ucat[k-1][j][i].y ) * 0.5;
        *dwdz = ( ucat[k+1][j][i].z - ucat[k-1][j][i].z ) * 0.5;
    }
    else
    {
        *dudz = (-ucat[k+2][j][i].x + 8*ucat[k+1][j][i].x - 8*ucat[k-1][j][i].x + ucat[k-2][j][i].x) / 12.0;
        *dvdz = (-ucat[k+2][j][i].y + 8*ucat[k+1][j][i].y - 8*ucat[k-1][j][i].y + ucat[k-2][j][i].y) / 12.0;
        *dwdz = (-ucat[k+2][j][i].z + 8*ucat[k+1][j][i].z - 8*ucat[k-1][j][i].z + ucat[k-2][j][i].z) / 12.0;
    }

    return;
}


//*****************************************************************************************************************//

inline void Compute_dscalar_center
(
    mesh_ *mesh,
    PetscInt i, PetscInt j, PetscInt k,  PetscInt mx, PetscInt my, PetscInt mz,
    PetscReal ***K, PetscReal ***nvert, PetscReal ***meshTag,
    PetscReal *dkdc, PetscReal *dkde, PetscReal *dkdz
)
{

    if      (i==mx-1) *dkdc = ( K[k][j][i] - K[k][j][i-1] );
    else if (i==0)    *dkdc = ( K[k][j][i+1] - K[k][j][i] );
    else if (isIBMSolidCell(k, j, i+1, nvert) || isZeroedCell(k, j, i+1, meshTag))
    {
        *dkdc = ( K[k][j][i] - K[k][j][i-1] );
    }
    else if (isIBMSolidCell(k, j, i-1, nvert) || isZeroedCell(k, j, i-1, meshTag))
    {
        *dkdc = ( K[k][j][i+1] - K[k][j][i] );
    }
    else
    {
        *dkdc = ( K[k][j][i+1] - K[k][j][i-1] ) * 0.5;
    }

    if      (j==my-1) *dkde = ( K[k][j][i] - K[k][j-1][i] );
    else if (j==0)    *dkde = ( K[k][j+1][i] - K[k][j][i] );
    else if (isIBMSolidCell(k, j+1, i, nvert) || isZeroedCell(k, j+1, i, meshTag))
    {
        *dkde = ( K[k][j][i] - K[k][j-1][i] );
    }
    else if (isIBMSolidCell(k, j-1, i, nvert) || isZeroedCell(k, j-1, i, meshTag))
    {
        *dkde = ( K[k][j+1][i] - K[k][j][i] );
    }
    else
    {
```

```
        *dkde = ( K[k][j+1][i] - K[k][j-1][i] ) * 0.5;
    }

    if      (k==mz-1) *dkdz = ( K[k][j][i] - K[k-1][j][i] );
    else if (k==0)    *dkdz = ( K[k+1][j][i] - K[k][j][i] );
    else if (isIBMSolidCell(k+1, j, i, nvert) || isZeroedCell(k+1, j, i, meshTag) )
    {
        *dkdz = ( K[k][j][i] - K[k-1][j][i] );
    }
    else if (isIBMSolidCell(k-1, j, i, nvert) || isZeroedCell(k-1, j, i, meshTag))
    {
        *dkdz = ( K[k+1][j][i] - K[k][j][i] );
    }
    else
    {
        *dkdz = ( K[k+1][j][i] - K[k-1][j][i] ) * 0.5;
    }

    return;
}


//*****************************************************************************************************************//

// Simpson integration algorithm

inline PetscReal integrateTestfilterSimpson(PetscReal val[3][3][3], PetscReal w[3][3][3])
{

    PetscReal wsum=0, valsum=0;

    for(PetscInt i=0; i<3; i++)
    for(PetscInt j=0; j<3; j++)
    for(PetscInt k=0; k<3; k++)
    {
        PetscReal simpson_w = 1.0;
        if(i==1) simpson_w *= 4.;
        if(j==1) simpson_w *= 4.;
        if(k==1) simpson_w *= 4.;

        wsum   += simpson_w * w[i][j][k];
        valsum += simpson_w * w[i][j][k] * val[i][j][k];
    }

    return valsum / wsum;
};

inline PetscReal integrateTestfilterSimpson5x5(PetscReal val[5][5][5], PetscReal w[5][5][5])
{
    PetscReal wsum=0, valsum=0;

    for(PetscInt i=0; i<5; i++)
    for(PetscInt j=0; j<5; j++)
    for(PetscInt k=0; k<5; k++)
    {
        PetscReal simpson_w = 1.0;
        simpson_w *= (i == 0 || i == 4) ? 1 : (i % 2 == 0 ? 2 : 4);
        simpson_w *= (j == 0 || j == 4) ? 1 : (j % 2 == 0 ? 2 : 4);
        simpson_w *= (k == 0 || k == 4) ? 1 : (k % 2 == 0 ? 2 : 4);

        wsum   += simpson_w * w[i][j][k];
        valsum += simpson_w * w[i][j][k] * val[i][j][k];
    }

    return valsum / wsum;
};


//*****************************************************************************************************************//

inline void Compute_du_dxyz
(
    mesh_ *mesh,
    PetscReal csi0, PetscReal csi1, PetscReal csi2,
    PetscReal eta0, PetscReal eta1, PetscReal eta2,
    PetscReal zet0, PetscReal zet1, PetscReal zet2,
    PetscReal ajc,
    PetscReal dudc, PetscReal dvdc, PetscReal dwdc,
    PetscReal dude, PetscReal dvde, PetscReal dwde,
    PetscReal dudz, PetscReal dvdz, PetscReal dwdz,
    PetscReal *du_dx, PetscReal *dv_dx, PetscReal *dw_dx,
    PetscReal *du_dy, PetscReal *dv_dy, PetscReal *dw_dy,
    PetscReal *du_dz, PetscReal *dv_dz, PetscReal *dw_dz
)
{
    *du_dx = (dudc * csi0 + dude * eta0 + dudz * zet0) * ajc;
    *du_dy = (dudc * csi1 + dude * eta1 + dudz * zet1) * ajc;
```

```
    *du_dz = (dudc * csi2 + dude * eta2 + dudz * zet2) * ajc;
    *dv_dx = (dvdc * csi0 + dvde * eta0 + dvdz * zet0) * ajc;
    *dv_dy = (dvdc * csi1 + dvde * eta1 + dvdz * zet1) * ajc;
    *dv_dz = (dvdc * csi2 + dvde * eta2 + dvdz * zet2) * ajc;
    *dw_dx = (dwdc * csi0 + dwde * eta0 + dwdz * zet0) * ajc;
    *dw_dy = (dwdc * csi1 + dwde * eta1 + dwdz * zet1) * ajc;
    *dw_dz = (dwdc * csi2 + dwde * eta2 + dwdz * zet2) * ajc;

    return;
}


//*****************************************************************************************************//

inline void Compute_dscalar_dxyz
(
    mesh_ *mesh,
    PetscReal csi0, PetscReal csi1, PetscReal csi2,
    PetscReal eta0, PetscReal eta1, PetscReal eta2,
    PetscReal zet0, PetscReal zet1, PetscReal zet2,
    PetscReal ajc,
    PetscReal dkdc, PetscReal dkde, PetscReal dkdz,
    PetscReal *dk_dx, PetscReal *dk_dy, PetscReal *dk_dz
)
{
    *dk_dx = (dkdc * csi0 + dkde * eta0 + dkdz * zet0) * ajc;
    *dk_dy = (dkdc * csi1 + dkde * eta1 + dkdz * zet1) * ajc;
    *dk_dz = (dkdc * csi2 + dkde * eta2 + dkdz * zet2) * ajc;

    return;
}


//*****************************************************************************************************//

inline void Compute_du_wmLocal
(
    mesh_ *mesh,
    Cmpnts eN, Cmpnts eT1, Cmpnts eT2,
    PetscReal du_dx, PetscReal dv_dx, PetscReal dw_dx,
    PetscReal du_dy, PetscReal dv_dy, PetscReal dw_dy,
    PetscReal du_dz, PetscReal dv_dz, PetscReal dw_dz,
    PetscReal *dut1dn, PetscReal *dut2dn, PetscReal *dundn,
    PetscReal *dut1dt1, PetscReal *dut2dt1, PetscReal *dundt1,
    PetscReal *dut1dt2, PetscReal *dut2dt2, PetscReal *dundt2
)
{
    PetscReal dudn = du_dx * eN.x + du_dy * eN.y + du_dz * eN.z;
    PetscReal dvdn = dv_dx * eN.x + dv_dy * eN.y + dv_dz * eN.z;
    PetscReal dwdn = dw_dx * eN.x + dw_dy * eN.y + dw_dz * eN.z;

    PetscReal dudt1 = du_dx * eT1.x + du_dy * eT1.y + du_dz * eT1.z;
    PetscReal dvdt1 = dv_dx * eT1.x + dv_dy * eT1.y + dv_dz * eT1.z;
    PetscReal dwdt1 = dw_dx * eT1.x + dw_dy * eT1.y + dw_dz * eT1.z;

    PetscReal dudt2 = du_dx * eT2.x + du_dy * eT2.y + du_dz * eT2.z;
    PetscReal dvdt2 = dv_dx * eT2.x + dv_dy * eT2.y + dv_dz * eT2.z;
    PetscReal dwdt2 = dw_dx * eT2.x + dw_dy * eT2.y + dw_dz * eT2.z;

    *dut1dn = dudn * eT1.x + dvdn * eT1.y + dwdn * eT1.z;
    *dut2dn = dudn * eT2.x + dvdn * eT2.y + dwdn * eT2.z;
    *dundn = dudn * eN.x + dvdn * eN.y + dwdn * eN.z;

    *dut1dt1 = dudt1 * eT1.x + dvdt1 * eT1.y + dwdt1 * eT1.z;
    *dut2dt1 = dudt1 * eT2.x + dvdt1 * eT2.y + dwdt1 * eT2.z;
    *dundt1 = dudt1 * eN.x + dvdt1 * eN.y + dwdt1 * eN.z;

    *dut1dt2 = dudt2 * eT1.x + dvdt2 * eT1.y + dwdt2 * eT1.z;
    *dut2dt2 = dudt2 * eT2.x + dvdt2 * eT2.y + dwdt2 * eT2.z;
    *dundt2 = dudt2 * eN.x + dvdt2 * eN.y + dwdt2 * eN.z;

    return;
}


//*****************************************************************************************************//

inline void Comput_JacobTensor_i
(
    PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz,
    Cmpnts ***coor, PetscReal *dxdc, PetscReal *dxde, PetscReal *dxdz,
    PetscReal *dydc, PetscReal *dyde, PetscReal *dydz, PetscReal *dzdc, PetscReal *dzde, PetscReal
*dzdz
)
{

    PetscReal centx, centy, centz;
```

```
    PetscReal centx_ip1, centy_ip1, centz_ip1;
    PetscReal centx_im1, centy_im1, centz_im1;
    PetscReal centx_jp1, centy_jp1, centz_jp1;
    PetscReal centx_jm1, centy_jm1, centz_jm1;
    PetscReal centx_kp1, centy_kp1, centz_kp1;
    PetscReal centx_km1, centy_km1, centz_km1;


    PetscInt i1=i,j1=j,k1=k;

    centx = (coor[k1  ][j1  ][i1].x + coor[k1-1][j1  ][i1].x +
                 coor[k1  ][j1-1][i1].x + coor[k1-1][j1-1][i1].x) * 0.25;
    centy = (coor[k1  ][j1  ][i1].y + coor[k1-1][j1  ][i1].y +
                 coor[k1  ][j1-1][i1].y + coor[k1-1][j1-1][i1].y) * 0.25;
    centz = (coor[k1  ][j1  ][i1].z + coor[k1-1][j1  ][i1].z +
                 coor[k1  ][j1-1][i1].z + coor[k1-1][j1-1][i1].z) * 0.25;

    if (i!=mx-2)
    {
        i1=i+1,j1=j,k1=k;

        centx_ip1 = (coor[k1  ][j1  ][i1].x + coor[k1-1][j1  ][i1].x +
                     coor[k1  ][j1-1][i1].x + coor[k1-1][j1-1][i1].x) * 0.25;
        centy_ip1 = (coor[k1  ][j1  ][i1].y + coor[k1-1][j1  ][i1].y +
                     coor[k1  ][j1-1][i1].y + coor[k1-1][j1-1][i1].y) * 0.25;
        centz_ip1 = (coor[k1  ][j1  ][i1].z + coor[k1-1][j1  ][i1].z +
                     coor[k1  ][j1-1][i1].z + coor[k1-1][j1-1][i1].z) * 0.25;
    }

    if (i!=0)
    {
        i1=i-1,j1=j,k1=k;

        centx_im1 = (coor[k1  ][j1  ][i1].x + coor[k1-1][j1  ][i1].x +
                     coor[k1  ][j1-1][i1].x + coor[k1-1][j1-1][i1].x) * 0.25;
        centy_im1 = (coor[k1  ][j1  ][i1].y + coor[k1-1][j1  ][i1].y +
                     coor[k1  ][j1-1][i1].y + coor[k1-1][j1-1][i1].y) * 0.25;
        centz_im1 = (coor[k1  ][j1  ][i1].z + coor[k1-1][j1  ][i1].z +
                     coor[k1  ][j1-1][i1].z + coor[k1-1][j1-1][i1].z) * 0.25;
    }

    if (j!=my-2)
    {
        i1=i,j1=j+1,k1=k;

        centx_jp1 = (coor[k1  ][j1  ][i1].x + coor[k1-1][j1  ][i1].x +
                     coor[k1  ][j1-1][i1].x + coor[k1-1][j1-1][i1].x) * 0.25;
        centy_jp1 = (coor[k1  ][j1  ][i1].y + coor[k1-1][j1  ][i1].y +
                     coor[k1  ][j1-1][i1].y + coor[k1-1][j1-1][i1].y) * 0.25;
        centz_jp1 = (coor[k1  ][j1  ][i1].z + coor[k1-1][j1  ][i1].z +
                     coor[k1  ][j1-1][i1].z + coor[k1-1][j1-1][i1].z) * 0.25;
    }

    if (j!=1)
    {
        i1=i,j1=j-1,k1=k;

        centx_jm1 = (coor[k1  ][j1  ][i1].x + coor[k1-1][j1  ][i1].x +
                     coor[k1  ][j1-1][i1].x + coor[k1-1][j1-1][i1].x) * 0.25;
        centy_jm1 = (coor[k1  ][j1  ][i1].y + coor[k1-1][j1  ][i1].y +
                     coor[k1  ][j1-1][i1].y + coor[k1-1][j1-1][i1].y) * 0.25;
        centz_jm1 = (coor[k1  ][j1  ][i1].z + coor[k1-1][j1  ][i1].z +
                     coor[k1  ][j1-1][i1].z + coor[k1-1][j1-1][i1].z) * 0.25;
    }

    if (k!=mz-2)
    {
        i1=i,j1=j,k1=k+1;

        centx_kp1 = (coor[k1  ][j1  ][i1].x + coor[k1-1][j1  ][i1].x +
                     coor[k1  ][j1-1][i1].x + coor[k1-1][j1-1][i1].x) * 0.25;
        centy_kp1 = (coor[k1  ][j1  ][i1].y + coor[k1-1][j1  ][i1].y +
                     coor[k1  ][j1-1][i1].y + coor[k1-1][j1-1][i1].y) * 0.25;
        centz_kp1 = (coor[k1  ][j1  ][i1].z + coor[k1-1][j1  ][i1].z +
                     coor[k1  ][j1-1][i1].z + coor[k1-1][j1-1][i1].z) * 0.25;
    }

    if (k!=1)
    {
        i1=i,j1=j,k1=k-1;

        centx_km1 = (coor[k1  ][j1  ][i1].x + coor[k1-1][j1  ][i1].x +
                     coor[k1  ][j1-1][i1].x + coor[k1-1][j1-1][i1].x) * 0.25;
        centy_km1 = (coor[k1  ][j1  ][i1].y + coor[k1-1][j1  ][i1].y +
                     coor[k1  ][j1-1][i1].y + coor[k1-1][j1-1][i1].y) * 0.25;
        centz_km1 = (coor[k1  ][j1  ][i1].z + coor[k1-1][j1  ][i1].z +
                     coor[k1  ][j1-1][i1].z + coor[k1-1][j1-1][i1].z) * 0.25;
```

```
    }

    if (i==0)
    {
      *dxdc = centx_ip1 - centx;
      *dydc = centy_ip1 - centy;
      *dzdc = centz_ip1 - centz;
    }
    else if (i==mx-2)
    {
      *dxdc = centx - centx_im1;
      *dydc = centy - centy_im1;
      *dzdc = centz - centz_im1;
    }
    else
    {
      *dxdc = (centx_ip1 - centx_im1) * 0.5;
      *dydc = (centy_ip1 - centy_im1) * 0.5;
      *dzdc = (centz_ip1 - centz_im1) * 0.5;
    }


    if (j==1)
    {
      *dxde = centx_jp1 - centx;
      *dyde = centy_jp1 - centy;
      *dzde = centz_jp1 - centz;
    }
    else if (j==my-2)
    {
      *dxde = centx - centx_jm1;
      *dyde = centy - centy_jm1;
      *dzde = centz - centz_jm1;
    }
    else
    {
      *dxde = (centx_jp1 - centx_jm1) * 0.5;
      *dyde = (centy_jp1 - centy_jm1) * 0.5;
      *dzde = (centz_jp1 - centz_jm1) * 0.5;
    }

    if (k==1)
    {
      *dxdz = (centx_kp1 - centx);
      *dydz = (centy_kp1 - centy);
      *dzdz = (centz_kp1 - centz);
    }
    else if (k==mz-2)
    {
      *dxdz = (centx - centx_km1);
      *dydz = (centy - centy_km1);
      *dzdz = (centz - centz_km1);
    }
    else
    {
      *dxdz = (centx_kp1 - centx_km1) * 0.5;
      *dydz = (centy_kp1 - centy_km1) * 0.5;
      *dzdz = (centz_kp1 - centz_km1) * 0.5;
    }

    return;
}


//*****************************************************************************************************//

inline void Comput_JacobTensor_j
(
    PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz,
    Cmpnts ***coor, PetscReal *dxdc, PetscReal *dxde, PetscReal *dxdz,
    PetscReal *dydc, PetscReal *dyde, PetscReal *dydz, PetscReal *dzdc, PetscReal *dzde, PetscReal *dzdz
)
{
    PetscReal centx, centy, centz;
    PetscReal centx_ip1, centy_ip1, centz_ip1;
    PetscReal centx_im1, centy_im1, centz_im1;
    PetscReal centx_jp1, centy_jp1, centz_jp1;
    PetscReal centx_jm1, centy_jm1, centz_jm1;
    PetscReal centx_kp1, centy_kp1, centz_kp1;
    PetscReal centx_km1, centy_km1, centz_km1;

    PetscInt i1=i,j1=j,k1=k;
    centx = (coor[k1  ][j1][i1  ].x + coor[k1-1][j1][i1  ].x +
             coor[k1  ][j1][i1-1].x + coor[k1-1][j1][i1-1].x) * 0.25;
    centy = (coor[k1  ][j1][i1  ].y + coor[k1-1][j1][i1  ].y +
             coor[k1  ][j1][i1-1].y + coor[k1-1][j1][i1-1].y) * 0.25;
```

```
    centz = (coor[k1  ][j1][i1  ].z + coor[k1-1][j1][i1  ].z +
             coor[k1  ][j1][i1-1].z + coor[k1-1][j1][i1-1].z) * 0.25;

    if (i!=mx-2)
    {
            i1=i+1,j1=j,k1=k;

            centx_ip1 = (coor[k1  ][j1][i1  ].x + coor[k1-1][j1][i1  ].x +
                         coor[k1  ][j1][i1-1].x + coor[k1-1][j1][i1-1].x) * 0.25;
            centy_ip1 = (coor[k1  ][j1][i1  ].y + coor[k1-1][j1][i1  ].y +
                         coor[k1  ][j1][i1-1].y + coor[k1-1][j1][i1-1].y) * 0.25;
            centz_ip1 = (coor[k1  ][j1][i1  ].z + coor[k1-1][j1][i1  ].z +
                         coor[k1  ][j1][i1-1].z + coor[k1-1][j1][i1-1].z) * 0.25;

    }

    if (i!=1)
    {
        i1=i-1,j1=j,k1=k;

        centx_im1 = (coor[k1  ][j1][i1  ].x + coor[k1-1][j1][i1  ].x +
                coor[k1  ][j1][i1-1].x + coor[k1-1][j1][i1-1].x) * 0.25;
        centy_im1 = (coor[k1  ][j1][i1  ].y + coor[k1-1][j1][i1  ].y +
                coor[k1  ][j1][i1-1].y + coor[k1-1][j1][i1-1].y) * 0.25;
        centz_im1 = (coor[k1  ][j1][i1  ].z + coor[k1-1][j1][i1  ].z +
                coor[k1  ][j1][i1-1].z + coor[k1-1][j1][i1-1].z) * 0.25;

    }

    if (j!=my-2)
    {
        i1=i,j1=j+1,k1=k;

        centx_jp1 = (coor[k1  ][j1][i1  ].x + coor[k1-1][j1][i1  ].x +
                coor[k1  ][j1][i1-1].x + coor[k1-1][j1][i1-1].x) * 0.25;
        centy_jp1 = (coor[k1  ][j1][i1  ].y + coor[k1-1][j1][i1  ].y +
                coor[k1  ][j1][i1-1].y + coor[k1-1][j1][i1-1].y) * 0.25;
        centz_jp1 = (coor[k1  ][j1][i1  ].z + coor[k1-1][j1][i1  ].z +
                coor[k1  ][j1][i1-1].z + coor[k1-1][j1][i1-1].z) * 0.25;

    }

    if (j!=0)
    {
        i1=i,j1=j-1,k1=k;

        centx_jm1 = (coor[k1  ][j1][i1  ].x + coor[k1-1][j1][i1  ].x +
                     coor[k1  ][j1][i1-1].x + coor[k1-1][j1][i1-1].x) * 0.25;
        centy_jm1 = (coor[k1  ][j1][i1  ].y + coor[k1-1][j1][i1  ].y +
                coor[k1  ][j1][i1-1].y + coor[k1-1][j1][i1-1].y) * 0.25;
        centz_jm1 = (coor[k1  ][j1][i1  ].z + coor[k1-1][j1][i1  ].z +
                coor[k1  ][j1][i1-1].z + coor[k1-1][j1][i1-1].z) * 0.25;

    }

    if (k!=mz-2)
    {
        i1=i,j1=j,k1=k+1;

        centx_kp1 = (coor[k1  ][j1][i1  ].x + coor[k1-1][j1][i1  ].x +
                coor[k1  ][j1][i1-1].x + coor[k1-1][j1][i1-1].x) * 0.25;
        centy_kp1 = (coor[k1  ][j1][i1  ].y + coor[k1-1][j1][i1  ].y +
                coor[k1  ][j1][i1-1].y + coor[k1-1][j1][i1-1].y) * 0.25;
        centz_kp1 = (coor[k1  ][j1][i1  ].z + coor[k1-1][j1][i1  ].z +
                coor[k1  ][j1][i1-1].z + coor[k1-1][j1][i1-1].z) * 0.25;

    }

    if (k!=1)
    {
        i1=i,j1=j,k1=k-1;

        centx_km1 = (coor[k1  ][j1][i1  ].x + coor[k1-1][j1][i1  ].x +
                coor[k1  ][j1][i1-1].x + coor[k1-1][j1][i1-1].x) * 0.25;
        centy_km1 = (coor[k1  ][j1][i1  ].y + coor[k1-1][j1][i1  ].y +
                coor[k1  ][j1][i1-1].y + coor[k1-1][j1][i1-1].y) * 0.25;
        centz_km1 = (coor[k1  ][j1][i1  ].z + coor[k1-1][j1][i1  ].z +
                coor[k1  ][j1][i1-1].z + coor[k1-1][j1][i1-1].z) * 0.25;

    }

    if (i==1)
    {
      *dxdc = centx_ip1 - centx;
      *dydc = centy_ip1 - centy;
      *dzdc = centz_ip1 - centz;
    }
```

```
    else if (i==mx-2)
    {
      *dxdc = centx - centx_im1;
      *dydc = centy - centy_im1;
      *dzdc = centz - centz_im1;
    }
    else {
      *dxdc = (centx_ip1 - centx_im1) * 0.5;
      *dydc = (centy_ip1 - centy_im1) * 0.5;
      *dzdc = (centz_ip1 - centz_im1) * 0.5;
    }


    if (j==0)
    {
      *dxde = centx_jp1 - centx;
      *dyde = centy_jp1 - centy;
      *dzde = centz_jp1 - centz;
    }
    else if (j==my-2)
    {
      *dxde = centx - centx_jm1;
      *dyde = centy - centy_jm1;
      *dzde = centz - centz_jm1;
    }
    else
    {
      *dxde = (centx_jp1 - centx_jm1) * 0.5;
      *dyde = (centy_jp1 - centy_jm1) * 0.5;
      *dzde = (centz_jp1 - centz_jm1) * 0.5;
    }

    if (k==1)
    {
      *dxdz = (centx_kp1 - centx);
      *dydz = (centy_kp1 - centy);
      *dzdz = (centz_kp1 - centz);
    }
    else if (k==mz-2)
    {
      *dxdz = (centx - centx_km1);
      *dydz = (centy - centy_km1);
      *dzdz = (centz - centz_km1);
    }
    else
    {
      *dxdz = (centx_kp1 - centx_km1) * 0.5;
      *dydz = (centy_kp1 - centy_km1) * 0.5;
      *dzdz = (centz_kp1 - centz_km1) * 0.5;
    }

}


//*****************************************************************************************************//

inline void Comput_JacobTensor_k
(
    PetscInt i, PetscInt j, PetscInt k, PetscInt mx, PetscInt my, PetscInt mz,
    Cmpnts ***coor, PetscReal *dxdc, PetscReal *dxde, PetscReal *dxdz,
    PetscReal *dydc, PetscReal *dyde, PetscReal *dydz, PetscReal *dzdc, PetscReal *dzde, PetscReal *dzdz
)
{
    PetscReal centx, centy, centz;
    PetscReal centx_ip1, centy_ip1, centz_ip1;
    PetscReal centx_im1, centy_im1, centz_im1;
    PetscReal centx_jp1, centy_jp1, centz_jp1;
    PetscReal centx_jm1, centy_jm1, centz_jm1;
    PetscReal centx_kp1, centy_kp1, centz_kp1;
    PetscReal centx_km1, centy_km1, centz_km1;

    PetscInt i1=i,j1=j,k1=k;

    centx = (coor[k1  ][j1][i1  ].x + coor[k1][j1-1][i1  ].x +
             coor[k1  ][j1][i1-1].x + coor[k1][j1-1][i1-1].x) * 0.25;
    centy = (coor[k1  ][j1][i1  ].y + coor[k1][j1-1][i1  ].y +
             coor[k1  ][j1][i1-1].y + coor[k1][j1-1][i1-1].y) * 0.25;
    centz = (coor[k1  ][j1][i1  ].z + coor[k1][j1-1][i1  ].z +
             coor[k1  ][j1][i1-1].z + coor[k1][j1-1][i1-1].z) * 0.25;

    if (i!=mx-2)
    {
        i1=i+1,j1=j,k1=k;

        centx_ip1 = (coor[k1  ][j1][i1  ].x + coor[k1][j1-1][i1  ].x +
                     coor[k1  ][j1][i1-1].x + coor[k1][j1-1][i1-1].x) * 0.25;
```

```
        centy_ip1 = (coor[k1  ][j1][i1  ].y + coor[k1][j1-1][i1  ].y +
                     coor[k1  ][j1][i1-1].y + coor[k1][j1-1][i1-1].y) * 0.25;
        centz_ip1 = (coor[k1  ][j1][i1  ].z + coor[k1][j1-1][i1  ].z +
                     coor[k1  ][j1][i1-1].z + coor[k1][j1-1][i1-1].z) * 0.25;

    }

    if (i!=1)
    {
        i1=i-1,j1=j,k1=k;

        centx_im1 = (coor[k1  ][j1][i1  ].x + coor[k1][j1-1][i1  ].x +
                     coor[k1  ][j1][i1-1].x + coor[k1][j1-1][i1-1].x) * 0.25;
        centy_im1 = (coor[k1  ][j1][i1  ].y + coor[k1][j1-1][i1  ].y +
                     coor[k1  ][j1][i1-1].y + coor[k1][j1-1][i1-1].y) * 0.25;
        centz_im1 = (coor[k1  ][j1][i1  ].z + coor[k1][j1-1][i1  ].z +
                     coor[k1  ][j1][i1-1].z + coor[k1][j1-1][i1-1].z) * 0.25;

    }

    if (j!=my-2)
    {
        i1=i,j1=j+1,k1=k;

        centx_jp1 = (coor[k1  ][j1][i1  ].x + coor[k1][j1-1][i1  ].x +
                     coor[k1  ][j1][i1-1].x + coor[k1][j1-1][i1-1].x) * 0.25;
        centy_jp1 = (coor[k1  ][j1][i1  ].y + coor[k1][j1-1][i1  ].y +
                     coor[k1  ][j1][i1-1].y + coor[k1][j1-1][i1-1].y) * 0.25;
        centz_jp1 = (coor[k1  ][j1][i1  ].z + coor[k1][j1-1][i1  ].z +
                     coor[k1  ][j1][i1-1].z + coor[k1][j1-1][i1-1].z) * 0.25;

    }

    if (j!=1)
    {
        i1=i,j1=j-1,k1=k;

        centx_jm1 = (coor[k1  ][j1][i1  ].x + coor[k1][j1-1][i1  ].x +
                     coor[k1  ][j1][i1-1].x + coor[k1][j1-1][i1-1].x) * 0.25;
        centy_jm1 = (coor[k1  ][j1][i1  ].y + coor[k1][j1-1][i1  ].y +
                     coor[k1  ][j1][i1-1].y + coor[k1][j1-1][i1-1].y) * 0.25;
        centz_jm1 = (coor[k1  ][j1][i1  ].z + coor[k1][j1-1][i1  ].z +
                     coor[k1  ][j1][i1-1].z + coor[k1][j1-1][i1-1].z) * 0.25;

    }

    if (k!=mz-2)
    {
        i1=i,j1=j,k1=k+1;

        centx_kp1 = (coor[k1  ][j1][i1  ].x + coor[k1][j1-1][i1  ].x +
                     coor[k1  ][j1][i1-1].x + coor[k1][j1-1][i1-1].x) * 0.25;
        centy_kp1 = (coor[k1  ][j1][i1  ].y + coor[k1][j1-1][i1  ].y +
                     coor[k1  ][j1][i1-1].y + coor[k1][j1-1][i1-1].y) * 0.25;
        centz_kp1 = (coor[k1  ][j1][i1  ].z + coor[k1][j1-1][i1  ].z +
                     coor[k1  ][j1][i1-1].z + coor[k1][j1-1][i1-1].z) * 0.25;

    }

    if (k!=0)
    {
        i1=i,j1=j,k1=k-1;

        centx_km1 = (coor[k1  ][j1][i1  ].x + coor[k1][j1-1][i1  ].x +
                     coor[k1  ][j1][i1-1].x + coor[k1][j1-1][i1-1].x) * 0.25;
        centy_km1 = (coor[k1  ][j1][i1  ].y + coor[k1][j1-1][i1  ].y +
                     coor[k1  ][j1][i1-1].y + coor[k1][j1-1][i1-1].y) * 0.25;
        centz_km1 = (coor[k1  ][j1][i1  ].z + coor[k1][j1-1][i1  ].z +
                     coor[k1  ][j1][i1-1].z + coor[k1][j1-1][i1-1].z) * 0.25;

    }

    if (i==1)
    {
      *dxdc = centx_ip1 - centx;
      *dydc = centy_ip1 - centy;
      *dzdc = centz_ip1 - centz;
    }
    else if (i==mx-2)
    {
      *dxdc = centx - centx_im1;
      *dydc = centy - centy_im1;
      *dzdc = centz - centz_im1;
    }
    else
    {
      *dxdc = (centx_ip1 - centx_im1) * 0.5;
```

```
      *dydc = (centy_ip1 - centy_im1) * 0.5;
      *dzdc = (centz_ip1 - centz_im1) * 0.5;
    }


    if (j==1)
    {
      *dxde = centx_jp1 - centx;
      *dyde = centy_jp1 - centy;
      *dzde = centz_jp1 - centz;
    }
    else if (j==my-2)
    {
      *dxde = centx - centx_jm1;
      *dyde = centy - centy_jm1;
      *dzde = centz - centz_jm1;
    }
    else
    {
      *dxde = (centx_jp1 - centx_jm1) * 0.5;
      *dyde = (centy_jp1 - centy_jm1) * 0.5;
      *dzde = (centz_jp1 - centz_jm1) * 0.5;
    }

    if (k==0)
    {
      *dxdz = (centx_kp1 - centx);
      *dydz = (centy_kp1 - centy);
      *dzdz = (centz_kp1 - centz);
    }
    else if (k==mz-2)
    {
      *dxdz = (centx - centx_km1);
      *dydz = (centy - centy_km1);
      *dzdz = (centz - centz_km1);
    }
    else
    {
      *dxdz = (centx_kp1 - centx_km1) * 0.5;
      *dydz = (centy_kp1 - centy_km1) * 0.5;
      *dzdz = (centz_kp1 - centz_km1) * 0.5;
    }

}


//*****************************************************************************************************//

// From local wall model grid to computational grid
inline void Compute_du_Compgrid
(
    PetscReal dxdc, PetscReal dxde, PetscReal dxdz, PetscReal dydc, PetscReal dyde, PetscReal dydz, PetscReal dzdc, PetscReal dzde, PetscReal dzdz,
    PetscReal nx, PetscReal ny, PetscReal nz, PetscReal t1x, PetscReal t1y, PetscReal t1z, PetscReal t2x, PetscReal t2y, PetscReal t2z,
    PetscReal dut1dn, PetscReal dut2dn, PetscReal dundn, PetscReal dut1dt1, PetscReal dut2dt1, PetscReal dundt1, PetscReal dut1dt2, PetscReal dut2dt2, PetscReal dundt2,
    PetscReal *dudc, PetscReal *dvdc, PetscReal *dwdc, PetscReal *dude, PetscReal *dvde, PetscReal *dwde, PetscReal *dudz, PetscReal *dvdz, PetscReal *dwdz
)
{

    PetscReal dxdn=nx, dydn=ny, dzdn=nz;
    PetscReal dxdt1=t1x, dydt1=t1y, dzdt1=t1z;
    PetscReal dxdt2=t2x, dydt2=t2y, dzdt2=t2z;

    PetscReal dndx = dydt1*dzdt2-dydt2*dzdt1;
    PetscReal dt1dx = dydt2*dzdn-dydn*dzdt2;
    PetscReal dt2dx = dydn*dzdt1-dydt1*dzdn;

    PetscReal dndy = dzdt1*dxdt2-dzdt2*dxdt1;
    PetscReal dt1dy = dzdt2*dxdn-dzdn*dxdt2;
    PetscReal dt2dy = dzdn*dxdt1-dzdt1*dxdn;

    PetscReal dndz = dxdt1*dydt2-dxdt2*dydt1;
    PetscReal dt1dz = dxdt2*dydn-dxdn*dydt2;
    PetscReal dt2dz = dxdn*dydt1-dxdt1*dydn;


    PetscReal dundx = dundn*dndx+dundt1*dt1dx+dundt2*dt2dx;
    PetscReal dundy = dundn*dndy+dundt1*dt1dy+dundt2*dt2dy;
    PetscReal dundz = dundn*dndz+dundt1*dt1dz+dundt2*dt2dz;

    PetscReal dut1dx = dut1dn*dndx+dut1dt1*dt1dx+dut1dt2*dt2dx;
    PetscReal dut1dy = dut1dn*dndy+dut1dt1*dt1dy+dut1dt2*dt2dy;
    PetscReal dut1dz = dut1dn*dndz+dut1dt1*dt1dz+dut1dt2*dt2dz;

```

```
    PetscReal dut2dx = dut2dn*dndx+dut2dt1*dt1dx+dut2dt2*dt2dx;
    PetscReal dut2dy = dut2dn*dndy+dut2dt1*dt1dy+dut2dt2*dt2dy;
    PetscReal dut2dz = dut2dn*dndz+dut2dt1*dt1dz+dut2dt2*dt2dz;


    PetscReal du_dx = dundx*nx+dut1dx*t1x+dut2dx*t2x;
    PetscReal du_dy = dundy*nx+dut1dy*t1x+dut2dy*t2x;
    PetscReal du_dz = dundz*nx+dut1dz*t1x+dut2dz*t2x;

    PetscReal dv_dx = dundx*ny+dut1dx*t1y+dut2dx*t2y;
    PetscReal dv_dy = dundy*ny+dut1dy*t1y+dut2dy*t2y;
    PetscReal dv_dz = dundz*ny+dut1dz*t1y+dut2dz*t2y;

    PetscReal dw_dx = dundx*nz+dut1dx*t1z+dut2dx*t2z;
    PetscReal dw_dy = dundy*nz+dut1dy*t1z+dut2dy*t2z;
    PetscReal dw_dz = dundz*nz+dut1dz*t1z+dut2dz*t2z;

    *dudc = du_dx*dxdc+du_dy*dydc+du_dz*dzdc;
    *dude = du_dx*dxde+du_dy*dyde+du_dz*dzde;
    *dudz = du_dx*dxdz+du_dy*dydz+du_dz*dzdz;

    *dvdc = dv_dx*dxdc+dv_dy*dydc+dv_dz*dzdc;
    *dvde = dv_dx*dxde+dv_dy*dyde+dv_dz*dzde;
    *dvdz = dv_dx*dxdz+dv_dy*dydz+dv_dz*dzdz;

    *dwdc = dw_dx*dxdc+dw_dy*dydc+dw_dz*dzdc;
    *dwde = dw_dx*dxde+dw_dy*dyde+dw_dz*dzde;
    *dwdz = dw_dx*dxdz+dw_dy*dydz+dw_dz*dzdz;
}

// DIVERGENCE SCHEMES
//
=================================================================================================================
//

inline PetscReal minMod(PetscReal m1, PetscReal m2)
{
    return 0.5 * ( sign(m1)+sign(m2) ) * PetscMin ( fabs(m1), fabs(m2) );
}


//*****************************************************************************************************************//

inline PetscReal vanLeer(PetscReal f0, PetscReal f1, PetscReal f2)
{
    PetscReal r, n, d;

    if(f1>0)
    {
        n = f1 - f0; d = PetscMax(f2 - f1, 1e-5); r = n/d;
    }
    else
    {
        n = f1 - f2; d = PetscMax(f0 - f1, 1e-5); r = n/d;
    }

    // Van Leer limiter function
    return 0.5 * (r + fabs(r)) / (1.0 + fabs(r));
}


//*****************************************************************************************************************//

inline PetscReal central(PetscReal f0, PetscReal f1)
{
    return 0.5 * (f0 + f1);
}


//*****************************************************************************************************************//

inline Cmpnts centralVec(Cmpnts f0, Cmpnts f1)
{
    Cmpnts result;
    result.x = 0.5 * (f0.x + f1.x);
    result.y = 0.5 * (f0.y + f1.y);
    result.z = 0.5 * (f0.z + f1.z);

    return result;
}


//*****************************************************************************************************************//

inline symmTensor centralSymmT(symmTensor f0, symmTensor f1)
{
    symmTensor result;
```

```
    result.xx = 0.5 * (f0.xx + f1.xx);
    result.yy = 0.5 * (f0.yy + f1.yy);
    result.zz = 0.5 * (f0.zz + f1.zz);
    result.xy = 0.5 * (f0.xy + f1.xy);
    result.xz = 0.5 * (f0.xz + f1.xz);
    result.yz = 0.5 * (f0.yz + f1.yz);

    return result;
}


//*****************************************************************************************************//

inline PetscReal wCentral(PetscReal f0, PetscReal f1, PetscReal d0, PetscReal d1)
{
    // distance btw upwind and downwind cell centers
    PetscReal d  = 0.5 * (d0 + d1);

    // distance btw upwind cell center and face center
    PetscReal dF = 0.5 * d0;

    return((f1 - f0)/d * dF + f0);
}


//*****************************************************************************************************//

inline Cmpnts wCentralVec(Cmpnts f0, Cmpnts f1, PetscReal d0, PetscReal d1)
{
    // distance btw upwind and downwind cell centers
    PetscReal d  = 0.5 * (d0 + d1);

    // distance btw upwind cell center and face center
    PetscReal dF = 0.5 * d0;

    return
    (
        nSum
        (
            f0,
            nScale(dF/d, nSub(f1, f0))
        )
    );
}


//*****************************************************************************************************//

inline Cmpnts centralVec4thCsi(mesh_ *mesh, PetscInt k, PetscInt j, PetscInt i, PetscInt mx,
        PetscReal ***nvert, PetscReal ***meshTag, Cmpnts ***ucat, PetscReal wavespeed, PetscReal hyperVisc)
{
    // if on non-periodic boundary return 2 cell-stencil extrema idxs, if periodic
    // get the right data.
    PetscInt im1, i0, ip1, ip2;
    Cmpnts result;
    if(i==0 || i==mx-2)
    {
        if(mesh->i_periodic)
        {
            im1 = mx-3, i0  = i, ip1  = i+1, ip2 = 2;

            result.x = (-ucat[k][j][im1].x + 7.0 * ucat[k][j][i0].x + 7.0 * ucat[k][j][ip1].x - ucat[k][j][ip2].x) / 12.0;
            result.y = (-ucat[k][j][im1].y + 7.0 * ucat[k][j][i0].y + 7.0 * ucat[k][j][ip1].y - ucat[k][j][ip2].y) / 12.0;
            result.z = (-ucat[k][j][im1].z + 7.0 * ucat[k][j][i0].z + 7.0 * ucat[k][j][ip1].z - ucat[k][j][ip2].z) / 12.0;

            result.x += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[k][j][ip2].x - ucat[k][j][im1].x) - 3.0 * (ucat[k][j][ip1].x - ucat[k][j][i0].x));
            result.y += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[k][j][ip2].y - ucat[k][j][im1].y) - 3.0 * (ucat[k][j][ip1].y - ucat[k][j][i0].y));
            result.z += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[k][j][ip2].z - ucat[k][j][im1].z) - 3.0 * (ucat[k][j][ip1].z - ucat[k][j][i0].z));
        }
        else if(mesh->ii_periodic && i==mx-2)
        {
            im1 = i-1, i0  = i, ip1  = i+1, ip2 = mx+2;

            result.x = (-ucat[k][j][im1].x + 7.0 * ucat[k][j][i0].x + 7.0 * ucat[k][j][ip1].x - ucat[k][j][ip2].x) / 12.0;
            result.y = (-ucat[k][j][im1].y + 7.0 * ucat[k][j][i0].y + 7.0 * ucat[k][j][ip1].y - ucat[k][j][ip2].y) / 12.0;
            result.z = (-ucat[k][j][im1].z + 7.0 * ucat[k][j][i0].z + 7.0 * ucat[k][j][ip1].z - ucat[k][j][ip2].z) / 12.0;

```

```
            result.x += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[k][j][ip2].x - ucat[k][j][im1].x) - 3.0 * (ucat[k][j][ip1].x - ucat[k][j][i0].x));
            result.y += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[k][j][ip2].y - ucat[k][j][im1].y) - 3.0 * (ucat[k][j][ip1].y - ucat[k][j][i0].y));
            result.z += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[k][j][ip2].z - ucat[k][j][im1].z) - 3.0 * (ucat[k][j][ip1].z - ucat[k][j][i0].z));
        }
        else if(mesh->ii_periodic && i==0)
        {
            im1 = -3, i0  = i, ip1  = i+1, ip2 = i+2;

            result.x = (-ucat[k][j][im1].x + 7.0 * ucat[k][j][i0].x + 7.0 * ucat[k][j][ip1].x - ucat[k][j][ip2].x) / 12.0;
            result.y = (-ucat[k][j][im1].y + 7.0 * ucat[k][j][i0].y + 7.0 * ucat[k][j][ip1].y - ucat[k][j][ip2].y) / 12.0;
            result.z = (-ucat[k][j][im1].z + 7.0 * ucat[k][j][i0].z + 7.0 * ucat[k][j][ip1].z - ucat[k][j][ip2].z) / 12.0;

            result.x += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[k][j][ip2].x - ucat[k][j][im1].x) - 3.0 * (ucat[k][j][ip1].x - ucat[k][j][i0].x));
            result.y += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[k][j][ip2].y - ucat[k][j][im1].y) - 3.0 * (ucat[k][j][ip1].y - ucat[k][j][i0].y));
            result.z += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[k][j][ip2].z - ucat[k][j][im1].z) - 3.0 * (ucat[k][j][ip1].z - ucat[k][j][i0].z));
        }
        else
        {
            i0 = i, ip1 = i+1;

            result.x = 0.5 * (ucat[k][j][i0].x + ucat[k][j][ip1].x);
            result.y = 0.5 * (ucat[k][j][i0].y + ucat[k][j][ip1].y);
            result.z = 0.5 * (ucat[k][j][i0].z + ucat[k][j][ip1].z);
        }
    }
    else if ( (i == 1 || i == mx-3) && (!isIBMIFace(k, j, i, i+1, nvert)) && (!isOversetIFace(k, j, i, i+1, nvert)) && (!(mesh->i_periodic || mesh->ii_periodic))) // Quick/Second-order central scheme for j=1 and j=my-3 depending on direction of flow
    {
        im1 = i-1, i0 = i, ip1 = i+1, ip2 = i+2;

        if (i == 1)
        {
            if(wavespeed>0)
            {
                result.x = 0.5 * (ucat[k][j][i0].x + ucat[k][j][ip1].x);
                result.y = 0.5 * (ucat[k][j][i0].y + ucat[k][j][ip1].y);
                result.z = 0.5 * (ucat[k][j][i0].z + ucat[k][j][ip1].z);
            }
            else
            {
                result.x = 0.5 * (ucat[k][j][ip1].x + ucat[k][j][i0].x) + (2.0*ucat[k][j][ip1].x - ucat[k][j][i0].x - ucat[k][j][ip2].x) / 8.0;
                result.y = 0.5 * (ucat[k][j][ip1].y + ucat[k][j][i0].y) + (2.0*ucat[k][j][ip1].y - ucat[k][j][i0].y - ucat[k][j][ip2].y) / 8.0;
                result.z = 0.5 * (ucat[k][j][ip1].z + ucat[k][j][i0].z) + (2.0*ucat[k][j][ip1].z - ucat[k][j][i0].z - ucat[k][j][ip2].z) / 8.0;
            }
        }

        if (i == mx-3)
        {
            if(wavespeed>0)
            {
                result.x = 0.5 * (ucat[k][j][ip1].x + ucat[k][j][i0].x) + (2.0*ucat[k][j][i0].x - ucat[k][j][ip1].x - ucat[k][j][im1].x) / 8.0;
                result.y = 0.5 * (ucat[k][j][ip1].y + ucat[k][j][i0].y) + (2.0*ucat[k][j][i0].y - ucat[k][j][ip1].y - ucat[k][j][im1].y) / 8.0;
                result.z = 0.5 * (ucat[k][j][ip1].z + ucat[k][j][i0].z) + (2.0*ucat[k][j][i0].z - ucat[k][j][ip1].z - ucat[k][j][im1].z) / 8.0;
            }
            else
            {
                result.x = 0.5 * (ucat[k][j][i0].x + ucat[k][j][ip1].x);
                result.y = 0.5 * (ucat[k][j][i0].y + ucat[k][j][ip1].y);
                result.z = 0.5 * (ucat[k][j][i0].z + ucat[k][j][ip1].z);
            }
        }
    }
    else if(isIBMCell(k, j, i, nvert) || isOversetCell(k, j, i, meshTag))
    {
        im1 = i-1, i0 = i, ip1 = i+1, ip2 = i+2;

        if(wavespeed>0)
        {
            result.x = 0.5 * (ucat[k][j][i0].x + ucat[k][j][ip1].x);
            result.y = 0.5 * (ucat[k][j][i0].y + ucat[k][j][ip1].y);
            result.z = 0.5 * (ucat[k][j][i0].z + ucat[k][j][ip1].z);
```

```
        }
        else
        {
            result.x = 0.5 * (ucat[k][j][ip1].x + ucat[k][j][i0].x) + (2.0*ucat[k][j][ip1].x - ucat[k][j][i0].x - ucat[k][j][ip2].x) / 8.0;
            result.y = 0.5 * (ucat[k][j][ip1].y + ucat[k][j][i0].y) + (2.0*ucat[k][j][ip1].y - ucat[k][j][i0].y - ucat[k][j][ip2].y) / 8.0;
            result.z = 0.5 * (ucat[k][j][ip1].z + ucat[k][j][i0].z) + (2.0*ucat[k][j][ip1].z - ucat[k][j][i0].z - ucat[k][j][ip2].z) / 8.0;
        }
    }
    else if(isIBMCell(k, j, i+1, nvert) || isOversetCell(k, j, i+1, meshTag))
    {
        im1 = i-1, i0 = i, ip1 = i+1, ip2 = i+2;

        if(wavespeed>0)
        {
            result.x = 0.5 * (ucat[k][j][ip1].x + ucat[k][j][i0].x) + (2.0*ucat[k][j][i0].x - ucat[k][j][ip1].x - ucat[k][j][im1].x) / 8.0;
            result.y = 0.5 * (ucat[k][j][ip1].y + ucat[k][j][i0].y) + (2.0*ucat[k][j][i0].y - ucat[k][j][ip1].y - ucat[k][j][im1].y) / 8.0;
            result.z = 0.5 * (ucat[k][j][ip1].z + ucat[k][j][i0].z) + (2.0*ucat[k][j][i0].z - ucat[k][j][ip1].z - ucat[k][j][im1].z) / 8.0;
        }
        else
        {
            result.x = 0.5 * (ucat[k][j][i0].x + ucat[k][j][ip1].x);
            result.y = 0.5 * (ucat[k][j][i0].y + ucat[k][j][ip1].y);
            result.z = 0.5 * (ucat[k][j][i0].z + ucat[k][j][ip1].z);
        }
    }
    else
    {
        im1 = i-1, i0  = i, ip1  = i+1, ip2 = i+2;

        result.x = (-ucat[k][j][im1].x + 7.0 * ucat[k][j][i0].x + 7.0 * ucat[k][j][ip1].x - ucat[k][j][ip2].x) / 12.0;
        result.y = (-ucat[k][j][im1].y + 7.0 * ucat[k][j][i0].y + 7.0 * ucat[k][j][ip1].y - ucat[k][j][ip2].y) / 12.0;
        result.z = (-ucat[k][j][im1].z + 7.0 * ucat[k][j][i0].z + 7.0 * ucat[k][j][ip1].z - ucat[k][j][ip2].z) / 12.0;

        result.x += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[k][j][ip2].x - ucat[k][j][im1].x) - 3.0 * (ucat[k][j][ip1].x - ucat[k][j][i0].x));
        result.y += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[k][j][ip2].y - ucat[k][j][im1].y) - 3.0 * (ucat[k][j][ip1].y - ucat[k][j][i0].y));
        result.z += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[k][j][ip2].z - ucat[k][j][im1].z) - 3.0 * (ucat[k][j][ip1].z - ucat[k][j][i0].z));
    }

    return result;
}


//*****************************************************************************************************************//

inline Cmpnts centralVec4thEta(mesh_ *mesh, PetscInt k, PetscInt j, PetscInt i, PetscInt my,
                               PetscReal ***nvert, PetscReal ***meshTag, Cmpnts ***ucat, PetscReal wavespeed, PetscReal hyperVisc)
{
    PetscInt jm1, j0, jp1, jp2;
    Cmpnts result;

    if (j == 0 || j == my-2)
    {
        if (mesh->j_periodic) // Assuming j_periodic for y-direction periodicity
        {
            jm1 = my-3, j0 = j, jp1 = j+1, jp2 = 2;

            result.x = (-ucat[k][jm1][i].x + 7.0 * ucat[k][j0][i].x + 7.0 * ucat[k][jp1][i].x - ucat[k][jp2][i].x) / 12.0;
            result.y = (-ucat[k][jm1][i].y + 7.0 * ucat[k][j0][i].y + 7.0 * ucat[k][jp1][i].y - ucat[k][jp2][i].y) / 12.0;
            result.z = (-ucat[k][jm1][i].z + 7.0 * ucat[k][j0][i].z + 7.0 * ucat[k][jp1][i].z - ucat[k][jp2][i].z) / 12.0;

            result.x += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[k][jp2][i].x - ucat[k][jm1][i].x) - 3.0 * (ucat[k][jp1][i].x - ucat[k][j0][i].x));
            result.y += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[k][jp2][i].y - ucat[k][jm1][i].y) - 3.0 * (ucat[k][jp1][i].y - ucat[k][j0][i].y));
            result.z += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[k][jp2][i].z - ucat[k][jm1][i].z) - 3.0 * (ucat[k][jp1][i].z - ucat[k][j0][i].z));
        }
        else if (mesh->jj_periodic && j == my-2) // Assuming jj_periodic for y-direction
        {
            jm1 = j-1, j0 = j, jp1 = j+1, jp2 = my+2;

```

```
            result.x = (-ucat[k][jm1][i].x + 7.0 * ucat[k][j0][i].x + 7.0 * ucat[k][jp1][i].x - ucat[k][jp2][i].x) / 12.0;
            result.y = (-ucat[k][jm1][i].y + 7.0 * ucat[k][j0][i].y + 7.0 * ucat[k][jp1][i].y - ucat[k][jp2][i].y) / 12.0;
            result.z = (-ucat[k][jm1][i].z + 7.0 * ucat[k][j0][i].z + 7.0 * ucat[k][jp1][i].z - ucat[k][jp2][i].z) / 12.0;

            result.x += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[k][jp2][i].x - ucat[k][jm1][i].x) - 3.0 * (ucat[k][jp1][i].x - ucat[k][j0][i].x));
            result.y += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[k][jp2][i].y - ucat[k][jm1][i].y) - 3.0 * (ucat[k][jp1][i].y - ucat[k][j0][i].y));
            result.z += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[k][jp2][i].z - ucat[k][jm1][i].z) - 3.0 * (ucat[k][jp1][i].z - ucat[k][j0][i].z));
        }
        else if (mesh->jj_periodic && j == 0)
        {
            jm1 = -3, j0 = j, jp1 = j+1, jp2 = j+2;

            result.x = (-ucat[k][jm1][i].x + 7.0 * ucat[k][j0][i].x + 7.0 * ucat[k][jp1][i].x - ucat[k][jp2][i].x) / 12.0;
            result.y = (-ucat[k][jm1][i].y + 7.0 * ucat[k][j0][i].y + 7.0 * ucat[k][jp1][i].y - ucat[k][jp2][i].y) / 12.0;
            result.z = (-ucat[k][jm1][i].z + 7.0 * ucat[k][j0][i].z + 7.0 * ucat[k][jp1][i].z - ucat[k][jp2][i].z) / 12.0;

            result.x += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[k][jp2][i].x - ucat[k][jm1][i].x) - 3.0 * (ucat[k][jp1][i].x - ucat[k][j0][i].x));
            result.y += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[k][jp2][i].y - ucat[k][jm1][i].y) - 3.0 * (ucat[k][jp1][i].y - ucat[k][j0][i].y));
            result.z += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[k][jp2][i].z - ucat[k][jm1][i].z) - 3.0 * (ucat[k][jp1][i].z - ucat[k][j0][i].z));
        }
        else
        {
            // use second order central at boundary
            j0 = j, jp1 = j+1;

            result.x = 0.5 * (ucat[k][jp1][i].x + ucat[k][j0][i].x);
            result.y = 0.5 * (ucat[k][jp1][i].y + ucat[k][j0][i].y);
            result.z = 0.5 * (ucat[k][jp1][i].z + ucat[k][j0][i].z);
        }
    }
    else if ( (j == 1 || j == my-3) && (!isIBMJFace(k, j, i, j+1, nvert)) && (!isOversetJFace(k, j, i, j+1, nvert)) && (!(mesh->j_periodic || mesh->jj_periodic))) // Quick/Second-order central scheme for j=1 and j=my-3 depending on direction of flow
    {
        jm1 = j-1, j0 = j, jp1 = j+1, jp2 = j+2;

        if (j == 1)
        {
            if(wavespeed>0)
            {
                result.x = 0.5 * (ucat[k][j0][i].x + ucat[k][jp1][i].x);
                result.y = 0.5 * (ucat[k][j0][i].y + ucat[k][jp1][i].y);
                result.z = 0.5 * (ucat[k][j0][i].z + ucat[k][jp1][i].z);
            }
            else
            {
                result.x = 0.5 * (ucat[k][jp1][i].x + ucat[k][j0][i].x) + (2.0*ucat[k][jp1][i].x - ucat[k][j0][i].x - ucat[k][jp2][i].x) / 8.0;
                result.y = 0.5 * (ucat[k][jp1][i].y + ucat[k][j0][i].y) + (2.0*ucat[k][jp1][i].y - ucat[k][j0][i].y - ucat[k][jp2][i].y) / 8.0;
                result.z = 0.5 * (ucat[k][jp1][i].z + ucat[k][j0][i].z) + (2.0*ucat[k][jp1][i].z - ucat[k][j0][i].z - ucat[k][jp2][i].z) / 8.0;
            }
        }

        if (j == my-3)
        {
            if(wavespeed>0)
            {
                result.x = 0.5 * (ucat[k][jp1][i].x + ucat[k][j0][i].x) + (2.0*ucat[k][j0][i].x - ucat[k][jp1][i].x - ucat[k][jm1][i].x) / 8.0;
                result.y = 0.5 * (ucat[k][jp1][i].y + ucat[k][j0][i].y) + (2.0*ucat[k][j0][i].y - ucat[k][jp1][i].y - ucat[k][jm1][i].y) / 8.0;
                result.z = 0.5 * (ucat[k][jp1][i].z + ucat[k][j0][i].z) + (2.0*ucat[k][j0][i].z - ucat[k][jp1][i].z - ucat[k][jm1][i].z) / 8.0;
            }
            else
            {
                result.x = 0.5 * (ucat[k][j0][i].x + ucat[k][jp1][i].x);
                result.y = 0.5 * (ucat[k][j0][i].y + ucat[k][jp1][i].y);
                result.z = 0.5 * (ucat[k][j0][i].z + ucat[k][jp1][i].z);
            }
        }
    }
    else if (isIBMCell(k, j, i, nvert) || isOversetCell(k, j, i, meshTag))
```

```
    {
        jm1 = j-1, j0 = j, jp1 = j+1, jp2 = j+2;

        if(wavespeed>0)
        {
            result.x = 0.5 * (ucat[k][j0][i].x + ucat[k][jp1][i].x);
            result.y = 0.5 * (ucat[k][j0][i].y + ucat[k][jp1][i].y);
            result.z = 0.5 * (ucat[k][j0][i].z + ucat[k][jp1][i].z);
        }
        else
        {
            result.x = 0.5 * (ucat[k][jp1][i].x + ucat[k][j0][i].x) + (2.0*ucat[k][jp1][i].x - ucat[k][j0][i].x - ucat[k][jp2][i].x) / 8.0;
            result.y = 0.5 * (ucat[k][jp1][i].y + ucat[k][j0][i].y) + (2.0*ucat[k][jp1][i].y - ucat[k][j0][i].y - ucat[k][jp2][i].y) / 8.0;
            result.z = 0.5 * (ucat[k][jp1][i].z + ucat[k][j0][i].z) + (2.0*ucat[k][jp1][i].z - ucat[k][j0][i].z - ucat[k][jp2][i].z) / 8.0;
        }
    }
    else if (isIBMCell(k, j+1, i, nvert) || isOversetCell(k, j+1, i, meshTag))
    {
        jm1 = j-1, j0 = j, jp1 = j+1, jp2 = j+2;

        if(wavespeed>0)
        {
            result.x = 0.5 * (ucat[k][jp1][i].x + ucat[k][j0][i].x) + (2.0*ucat[k][j0][i].x - ucat[k][jp1][i].x - ucat[k][jm1][i].x) / 8.0;
            result.y = 0.5 * (ucat[k][jp1][i].y + ucat[k][j0][i].y) + (2.0*ucat[k][j0][i].y - ucat[k][jp1][i].y - ucat[k][jm1][i].y) / 8.0;
            result.z = 0.5 * (ucat[k][jp1][i].z + ucat[k][j0][i].z) + (2.0*ucat[k][j0][i].z - ucat[k][jp1][i].z - ucat[k][jm1][i].z) / 8.0;
        }
        else
        {
            result.x = 0.5 * (ucat[k][j0][i].x + ucat[k][jp1][i].x);
            result.y = 0.5 * (ucat[k][j0][i].y + ucat[k][jp1][i].y);
            result.z = 0.5 * (ucat[k][j0][i].z + ucat[k][jp1][i].z);
        }
    }
    else
    {
        jm1 = j-1, j0 = j, jp1 = j+1, jp2 = j+2;

        result.x = (-ucat[k][jm1][i].x + 7.0 * ucat[k][j0][i].x + 7.0 * ucat[k][jp1][i].x - ucat[k][jp2][i].x) / 12.0;
        result.y = (-ucat[k][jm1][i].y + 7.0 * ucat[k][j0][i].y + 7.0 * ucat[k][jp1][i].y - ucat[k][jp2][i].y) / 12.0;
        result.z = (-ucat[k][jm1][i].z + 7.0 * ucat[k][j0][i].z + 7.0 * ucat[k][jp1][i].z - ucat[k][jp2][i].z) / 12.0;

        result.x += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[k][jp2][i].x - ucat[k][jm1][i].x) - 3.0 * (ucat[k][jp1][i].x - ucat[k][j0][i].x));
        result.y += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[k][jp2][i].y - ucat[k][jm1][i].y) - 3.0 * (ucat[k][jp1][i].y - ucat[k][j0][i].y));
        result.z += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[k][jp2][i].z - ucat[k][jm1][i].z) - 3.0 * (ucat[k][jp1][i].z - ucat[k][j0][i].z));
    }

    return result;
}


//*****************************************************************************************************//

inline Cmpnts centralVec4thZet(mesh_ *mesh, PetscInt k, PetscInt j, PetscInt i, PetscInt mz,
                               PetscReal ***nvert, PetscReal ***meshTag, Cmpnts ***ucat, PetscReal wavespeed, PetscReal hyperVisc)
{
    PetscInt km1, k0, kp1, kp2;
    Cmpnts result;

    if (k == 0 || k == mz-2)
    {
        if (mesh->k_periodic) // Assuming k_periodic for z-direction periodicity
        {
            km1 = mz-3, k0 = k, kp1 = k+1, kp2 = 2;

            result.x = (-ucat[km1][j][i].x + 7.0 * ucat[k0][j][i].x + 7.0 * ucat[kp1][j][i].x - ucat[kp2][j][i].x) / 12.0;
            result.y = (-ucat[km1][j][i].y + 7.0 * ucat[k0][j][i].y + 7.0 * ucat[kp1][j][i].y - ucat[kp2][j][i].y) / 12.0;
            result.z = (-ucat[km1][j][i].z + 7.0 * ucat[k0][j][i].z + 7.0 * ucat[kp1][j][i].z - ucat[kp2][j][i].z) / 12.0;

            result.x += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[kp2][j][i].x - ucat[km1][j][i].x) - 3.0 * (ucat[kp1][j][i].x - ucat[k0][j][i].x));
            result.y += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[kp2][j][i].y -
```

```
ucat[km1][j][i].y) - 3.0 * (ucat[kp1][j][i].y - ucat[k0][j][i].y));
            result.z += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[kp2][j][i].z - ucat[km1][j][i].z) - 3.0 * (ucat[kp1][j][i].z - ucat[k0][j][i].z));
        }
        else if (mesh->kk_periodic && k == mz-2) // Assuming kk_periodic for z-direction
        {
            km1 = k-1, k0 = k, kp1 = k+1, kp2 = mz+2;

            result.x = (-ucat[km1][j][i].x + 7.0 * ucat[k0][j][i].x + 7.0 * ucat[kp1][j][i].x - ucat[kp2][j][i].x) / 12.0;
            result.y = (-ucat[km1][j][i].y + 7.0 * ucat[k0][j][i].y + 7.0 * ucat[kp1][j][i].y - ucat[kp2][j][i].y) / 12.0;
            result.z = (-ucat[km1][j][i].z + 7.0 * ucat[k0][j][i].z + 7.0 * ucat[kp1][j][i].z - ucat[kp2][j][i].z) / 12.0;

            result.x += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[kp2][j][i].x - ucat[km1][j][i].x) - 3.0 * (ucat[kp1][j][i].x - ucat[k0][j][i].x));
            result.y += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[kp2][j][i].y - ucat[km1][j][i].y) - 3.0 * (ucat[kp1][j][i].y - ucat[k0][j][i].y));
            result.z += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[kp2][j][i].z - ucat[km1][j][i].z) - 3.0 * (ucat[kp1][j][i].z - ucat[k0][j][i].z));
        }
        else if (mesh->kk_periodic && k == 0)
        {
            km1 = -3, k0 = k, kp1 = k+1, kp2 = k+2;

            result.x = (-ucat[km1][j][i].x + 7.0 * ucat[k0][j][i].x + 7.0 * ucat[kp1][j][i].x - ucat[kp2][j][i].x) / 12.0;
            result.y = (-ucat[km1][j][i].y + 7.0 * ucat[k0][j][i].y + 7.0 * ucat[kp1][j][i].y - ucat[kp2][j][i].y) / 12.0;
            result.z = (-ucat[km1][j][i].z + 7.0 * ucat[k0][j][i].z + 7.0 * ucat[kp1][j][i].z - ucat[kp2][j][i].z) / 12.0;

            result.x += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[kp2][j][i].x - ucat[km1][j][i].x) - 3.0 * (ucat[kp1][j][i].x - ucat[k0][j][i].x));
            result.y += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[kp2][j][i].y - ucat[km1][j][i].y) - 3.0 * (ucat[kp1][j][i].y - ucat[k0][j][i].y));
            result.z += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[kp2][j][i].z - ucat[km1][j][i].z) - 3.0 * (ucat[kp1][j][i].z - ucat[k0][j][i].z));
        }
        else
        {
            k0 = k, kp1 = k+1;

            result.x = 0.5 * (ucat[k0][j][i].x + ucat[kp1][j][i].x);
            result.y = 0.5 * (ucat[k0][j][i].y + ucat[kp1][j][i].y);
            result.z = 0.5 * (ucat[k0][j][i].z + ucat[kp1][j][i].z);
        }
    }
    else if ( (k == 1 || k == mz-3) && (!isIBMKFace(k, j, i, k+1, nvert)) && (!isOversetKFace(k, j, i, k+1, nvert)) && (!(mesh->k_periodic || mesh->kk_periodic))) // Quick/Second-order central scheme for k=1 and k=mz-3 depending on direction of flow
    {
        km1 = k-1, k0 = k, kp1 = k+1, kp2 = k+2;

        if (k == 1)
        {
            if(wavespeed>0)
            {
                result.x = 0.5 * (ucat[k0][j][i].x + ucat[kp1][j][i].x);
                result.y = 0.5 * (ucat[k0][j][i].y + ucat[kp1][j][i].y);
                result.z = 0.5 * (ucat[k0][j][i].z + ucat[kp1][j][i].z);
            }
            else
            {
                result.x = 0.5 * (ucat[kp1][j][i].x + ucat[k0][j][i].x) + (2.0*ucat[kp1][j][i].x - ucat[k0][j][i].x - ucat[kp2][j][i].x) / 8.0;
                result.y = 0.5 * (ucat[kp1][j][i].y + ucat[k0][j][i].y) + (2.0*ucat[kp1][j][i].y - ucat[k0][j][i].y - ucat[kp2][j][i].y) / 8.0;
                result.z = 0.5 * (ucat[kp1][j][i].z + ucat[k0][j][i].z) + (2.0*ucat[kp1][j][i].z - ucat[k0][j][i].z - ucat[kp2][j][i].z) / 8.0;
            }
        }

        if (k == mz-3)
        {
            if(wavespeed>0)
            {
                result.x = 0.5 * (ucat[kp1][j][i].x + ucat[k0][j][i].x) + (2.0*ucat[k0][j][i].x - ucat[kp1][j][i].x - ucat[km1][j][i].x) / 8.0;
                result.y = 0.5 * (ucat[kp1][j][i].y + ucat[k0][j][i].y) + (2.0*ucat[k0][j][i].y - ucat[kp1][j][i].y - ucat[km1][j][i].y) / 8.0;
                result.z = 0.5 * (ucat[kp1][j][i].z + ucat[k0][j][i].z) + (2.0*ucat[k0][j][i].z - ucat[kp1][j][i].z - ucat[km1][j][i].z) / 8.0;
            }
            else
            {
```

```
                result.x = 0.5 * (ucat[k0][j][i].x + ucat[kp1][j][i].x);
                result.y = 0.5 * (ucat[k0][j][i].y + ucat[kp1][j][i].y);
                result.z = 0.5 * (ucat[k0][j][i].z + ucat[kp1][j][i].z);
            }
        }
    }
    else if (isIBMCell(k, j, i, nvert) || isOversetCell(k, j, i, meshTag))
    {
        km1 = k-1, k0 = k, kp1 = k+1, kp2 = k+2;

        if(wavespeed>0)
        {
            result.x = 0.5 * (ucat[k0][j][i].x + ucat[kp1][j][i].x);
            result.y = 0.5 * (ucat[k0][j][i].y + ucat[kp1][j][i].y);
            result.z = 0.5 * (ucat[k0][j][i].z + ucat[kp1][j][i].z);
        }
        else
        {
            result.x = 0.5 * (ucat[kp1][j][i].x + ucat[k0][j][i].x) + (2.0*ucat[kp1][j][i].x - ucat[k0][j][i].x - ucat[kp2][j][i].x) / 8.0;
            result.y = 0.5 * (ucat[kp1][j][i].y + ucat[k0][j][i].y) + (2.0*ucat[kp1][j][i].y - ucat[k0][j][i].y - ucat[kp2][j][i].y) / 8.0;
            result.z = 0.5 * (ucat[kp1][j][i].z + ucat[k0][j][i].z) + (2.0*ucat[kp1][j][i].z - ucat[k0][j][i].z - ucat[kp2][j][i].z) / 8.0;
        }
    }
    else if (isIBMCell(k+1, j, i, nvert) || isOversetCell(k+1, j, i, meshTag))
    {
        km1 = k-1, k0 = k, kp1 = k+1, kp2 = k+2;

        if(wavespeed>0)
        {
            result.x = 0.5 * (ucat[kp1][j][i].x + ucat[k0][j][i].x) + (2.0*ucat[k0][j][i].x - ucat[kp1][j][i].x - ucat[km1][j][i].x) / 8.0;
            result.y = 0.5 * (ucat[kp1][j][i].y + ucat[k0][j][i].y) + (2.0*ucat[k0][j][i].y - ucat[kp1][j][i].y - ucat[km1][j][i].y) / 8.0;
            result.z = 0.5 * (ucat[kp1][j][i].z + ucat[k0][j][i].z) + (2.0*ucat[k0][j][i].z - ucat[kp1][j][i].z - ucat[km1][j][i].z) / 8.0;
        }
        else
        {
            result.x = 0.5 * (ucat[k0][j][i].x + ucat[kp1][j][i].x);
            result.y = 0.5 * (ucat[k0][j][i].y + ucat[kp1][j][i].y);
            result.z = 0.5 * (ucat[k0][j][i].z + ucat[kp1][j][i].z);
        }
    }
    else
    {
        km1 = k-1, k0 = k, kp1 = k+1, kp2 = k+2;

        result.x = (-ucat[km1][j][i].x + 7.0 * ucat[k0][j][i].x + 7.0 * ucat[kp1][j][i].x - ucat[kp2][j][i].x) / 12.0;
        result.y = (-ucat[km1][j][i].y + 7.0 * ucat[k0][j][i].y + 7.0 * ucat[kp1][j][i].y - ucat[kp2][j][i].y) / 12.0;
        result.z = (-ucat[km1][j][i].z + 7.0 * ucat[k0][j][i].z + 7.0 * ucat[kp1][j][i].z - ucat[kp2][j][i].z) / 12.0;

        result.x += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[kp2][j][i].x - ucat[km1][j][i].x) - 3.0 * (ucat[kp1][j][i].x - ucat[k0][j][i].x));
        result.y += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[kp2][j][i].y - ucat[km1][j][i].y) - 3.0 * (ucat[kp1][j][i].y - ucat[k0][j][i].y));
        result.z += PetscSign(wavespeed) * ((1.0-hyperVisc)/12.0)*((ucat[kp2][j][i].z - ucat[km1][j][i].z) - 3.0 * (ucat[kp1][j][i].z - ucat[k0][j][i].z));
    }

    return result;
}


//*****************************************************************************************************//
inline PetscReal central4(PetscReal f0, PetscReal f1, PetscReal f2, PetscReal f3)
{
    return 0.0625 * (- f0 + 9. * f1 + 9.* f2 - f3);
}


//*****************************************************************************************************//

inline PetscReal centralUpwind(PetscReal f0, PetscReal f1, PetscReal f2, PetscReal f3, PetscReal wavespeed)
{
    PetscReal r, n, d;
    PetscReal fUU, fU, fD;

    if(wavespeed>0)
    {
        n = f1 - f0; d = PetscMax(f2 - f1, 1e-5); r = n/d;
```

```
        fUU = f0; fU  = f1; fD  = f2;
    }
    else
    {
        n = f2 - f3; d = PetscMax(f1 - f2, 1e-5); r = n/d;
        fUU = f3; fU  = f2; fD  = f1;
    }

    // Van Leer limiter function
    PetscReal lim = 0.5 * (r + fabs(r)) / (1.0 + fabs(r));

    // central scheme
    PetscReal C = 0.5 * (fU + fD);

    // quickDiv correction to the central scheme
    PetscReal corr = (2.0*fU - fD - fUU) / 8.0;

    return
    (
       C + (1.0-lim)*corr
    );
}


//*****************************************************************************************************//

inline Cmpnts centralUpwindVec(Cmpnts f0, Cmpnts f1, Cmpnts f2, Cmpnts f3, PetscReal wavespeed)
{
    Cmpnts r, n, d;
    Cmpnts fUU, fU, fD;

    if(wavespeed>0)
    {
        n = nSub(f1,f0); d = nMax(nSub(f2,f1), 1e-5); r = nDivElWise(n,d);
        fUU = nSet(f0); fU  = nSet(f1); fD  = nSet(f2);
    }
    else
    {
        n = nSub(f2,f3); d = nMax(nSub(f1,f2), 1e-5); r = nDivElWise(n,d);
        fUU = nSet(f3); fU  = nSet(f2); fD  = nSet(f1);
    }

    // Van Leer limiter function
    Cmpnts lim = nScale(0.5, nDivElWise(nSum(r, nAbs(r)), nSum(nSetFromComponents(1.0, 1.0, 1.0), nAbs(r))));

    // central scheme
    Cmpnts C = nScale(0.5, nSum(fU, fD));

    // quickDiv correction to the central scheme
    Cmpnts corr = nScale(1./8., nSub(nScale(2.0, fU), nSum(fD, fUU)));

    return
    (
       nSum(C, nMultElWise(nSub(nSetFromComponents(1.0, 1.0, 1.0),lim),corr))
    );
}



//*****************************************************************************************************//

inline PetscReal centralUpwind
(
    PetscReal f0, PetscReal f1, PetscReal f2, PetscReal f3,
    PetscReal wavespeed, PetscReal limiter
)
{
    PetscReal fUU, fU, fD;

    if(wavespeed>0)
    {
        fUU = f0; fU  = f1; fD  = f2;
    }
    else
    {
        fUU = f3; fU  = f2; fD  = f1;
    }

    // central scheme
    PetscReal C = 0.5 * (fU + fD);

    // quickDiv correction to the central scheme
    PetscReal corr = (2.0*fU - fD - fUU) / 8.0;

    return
    (
```

```
        C + (1.0-limiter)*corr
    );
}


//*****************************************************************************************************//

inline PetscReal upwind
(
    PetscReal f1, PetscReal f2,
    PetscReal wavespeed
)
{
    PetscReal fU, fD;

    if(wavespeed>0)
    {
        fU  = f1;
    }
    else
    {
        fU  = f2;
    }

    return
    (
       fU
    );
}


//*****************************************************************************************************//

inline Cmpnts centralUpwindVec
(
    Cmpnts f0, Cmpnts f1, Cmpnts f2, Cmpnts f3,
    PetscReal wavespeed, PetscReal limiter
)
{
    Cmpnts fUU, fU, fD;

    if(wavespeed>0)
    {
        fUU = nSet(f0); fU  = nSet(f1); fD  = nSet(f2);
    }
    else
    {
        fUU = nSet(f3); fU  = nSet(f2); fD  = nSet(f1);
    }

    // central scheme
    Cmpnts C = nScale(0.5, nSum(fU,fD));

    // quickDiv correction to the central scheme
    Cmpnts corr = nScale(1./8., nSub(nScale(2.0,fU), nSum(fD,fUU)));

    return
    (
       nSum(C, nScale(1.0-limiter,corr))
    );
}


//*****************************************************************************************************//

inline PetscReal wCentralUpwind
(
    PetscReal f0, PetscReal f1, PetscReal f2, PetscReal f3,
    PetscReal d0, PetscReal d1, PetscReal d2, PetscReal d3,
    PetscReal wavespeed, PetscReal limiter
)
{
    PetscReal fUU, fU, fD;
    PetscReal dUU, dU, dD;

    if(d0 != d0 || d1 != d1 || d2 != d2 || d3 != d3)
    {
        char error[512];
        sprintf(error, "nan-distances provided\n");
        fatalErrorInFunction("wCentralUpwind",  error);
    }

    if(wavespeed > 0)
    {
        fUU = f0; fU  = f1; fD  = f2;
        dUU = d0; dU  = d1; dD  = d2;
    }
```

```
    else
    {
        fUU = f3; fU  = f2; fD  = f1;
        dUU = d3; dU  = d2; dD  = d1;
    }

    // compute cell to cell and cell to face distances
    PetscReal da  = 0.5 * (dUU + dU);
    PetscReal db  = 0.5 * (dU  + dD);
    PetscReal dc  = 0.5 * (dUU + 2.0*dU + dD);
    PetscReal dfQ = 0.5 * (2.0*dU + dUU);
    PetscReal dfC = 0.5 * dU;

    // central scheme
    PetscReal C = (fD - fU) / db * dfC + fU;

    // quickDiv correction a, b, c terms
    PetscReal a = (da*fD - dc*fU + db*fUU);
    PetscReal b = ((dc*dc+dc*da)*fU - (da*da+dc*da)*fD - (dc*dc-da*da)*fUU);
    PetscReal c = ((fD - fU) / db * da + fUU - fU ) * (da*db*dc);

    // quadratic quickDiv correction to the central scheme
    PetscReal corr = (a*dfQ*dfQ + b*dfQ + c ) / (da*db*dc);

    return
    (
       C + (1.0 - limiter) * corr
    );
}


//*****************************************************************************************************//

inline Cmpnts wCentralUpwindVec
(
    Cmpnts f0, Cmpnts f1, Cmpnts f2, Cmpnts f3,
    PetscReal d0, PetscReal d1, PetscReal d2, PetscReal d3,
    PetscReal wavespeed, PetscReal limiter
)
{
    Cmpnts    fUU, fU, fD;
    PetscReal dUU, dU, dD;

    if(d0 != d0 || d1 != d1 || d2 != d2 || d3 != d3)
    {
        char error[512];
        sprintf(error, "nan-distances provided\n");
        fatalErrorInFunction("wCentralUpwind",  error);
    }

    if(wavespeed > 0)
    {
        fUU = nSet(f0); fU  = nSet(f1); fD  = nSet(f2);
        dUU = d0; dU  = d1; dD  = d2;
    }
    else
    {
        fUU = nSet(f3); fU  = nSet(f2); fD  = nSet(f1);
        dUU = d3; dU  = d2; dD  = d1;
    }

    // compute cell to cell and cell to face distances
    PetscReal da  = 0.5 * (dUU + dU);
    PetscReal db  = 0.5 * (dU  + dD);
    PetscReal dc  = 0.5 * (dUU + 2.0*dU + dD);
    PetscReal dfQ = 0.5 * (2.0*dU + dUU);
    PetscReal dfC = 0.5 * dU;

    // central scheme
    Cmpnts C = nSum(nScale(dfC/ db, nSub(fD, fU)), fU);

    // quickDiv correction a, b, c terms
    Cmpnts a = nSum(nSub(nScale(da,fD), nScale(dc,fU)), nScale(db,fUU));
    Cmpnts b = nSub(nScale(dc*dc+dc*da,fU), nSum(nScale(da*da+dc*da,fD), nScale(dc*dc-da*da,fUU)));
    Cmpnts c = nScale(da*db*dc, nSub(nSum(nScale(da/db, nSub(fD,fU)), fUU), fU));

    // quadratic quickDiv correction to the central scheme
    Cmpnts corr = nScale(1.0/(da*db*dc), nSum(nSum(nScale(dfQ*dfQ, a), nScale(dfQ, b)), c));

    return
    (
       nSum(C, nScale(1.0 - limiter, corr))
    );
}


//*****************************************************************************************************//
```

```
inline PetscReal quadraticUpwind
(
    PetscReal f0, PetscReal f1, PetscReal f2, PetscReal f3,
    PetscReal wavespeed
)
{
    PetscReal fUU, fU, fD;

    if(wavespeed>0)
    {
        fUU = f0; fU  = f1; fD  = f2;
    }
    else
    {
        fUU = f3; fU  = f2; fD  = f1;
    }

    return 0.5 * (fU + fD) + (2.0*fU - fD - fUU) / 8.0;
}


//*****************************************************************************************************************//

inline Cmpnts quadraticUpwindVec
(
    Cmpnts f0, Cmpnts f1, Cmpnts f2, Cmpnts f3,
    PetscReal wavespeed
)
{
    Cmpnts fUU, fU, fD;

    if(wavespeed>0)
    {
        fUU = nSet(f0); fU  = nSet(f1); fD  = nSet(f2);
    }
    else
    {
        fUU = nSet(f3); fU  = nSet(f2); fD  = nSet(f1);
    }

    return nSum(nScale(0.5, nSum(fU, fD)), nScale(1./8., nSub(nScale(2.0,fU), nSum(fD, fUU))));
}


//*****************************************************************************************************************//

inline PetscReal wQuadraticUpwind
(
    PetscReal f0, PetscReal f1, PetscReal f2, PetscReal f3,
    PetscReal d0, PetscReal d1, PetscReal d2, PetscReal d3,
    PetscReal wavespeed
)
{
    PetscReal fUU, fU, fD;
    PetscReal dUU, dU, dD;

    if(wavespeed > 0)
    {
        fUU = f0; fU  = f1; fD  = f2;
        dUU = d0; dU  = d1; dD  = d2;
    }
    else
    {
        fUU = f3; fU  = f2; fD  = f1;
        dUU = d3; dU  = d2; dD  = d1;
    }

    // compute cell to cell and cell to face distances
    PetscReal da  = 0.5 * (dUU + dU);
    PetscReal db  = 0.5 * (dU  + dD);
    PetscReal dc  = 0.5 * (dUU + 2.0*dU + dD);
    PetscReal dfQ = 0.5 * (2.0*dU + dUU);

    // quickDiv scheme a, b terms (c = fUU)
    PetscReal a = (da*fD - dc*fU + db*fUU) / (da*db*dc);
    PetscReal b = (dc*dc*fU - da*da*fD - (dc*dc-da*da)*fUU) / (da*db*dc);

    return(a*dfQ*dfQ + b*dfQ + fUU);
}


//*****************************************************************************************************************//

inline PetscReal weno3(PetscReal f0, PetscReal f1, PetscReal f2, PetscReal f3, PetscReal wavespeed)
{
    PetscReal fL, fC, fR;
```

```
    if(wavespeed>0)
    {
        fL = f0; fC = f1; fR = f2;
    }
    else
    {
        fL = f3; fC = f2; fR = f1;
    }

    PetscReal d0=2./3., d1=1./3.;

    const PetscReal eps=1.e-6;

    PetscReal beta0 = pow( fC - fR,  2. );
    PetscReal beta1 = pow( fL - fC,  2. );

    PetscReal alpha0 = d0 / pow( eps + beta0, 2. );
    PetscReal alpha1 = d1 / pow( eps + beta1, 2. );

    PetscReal sumalpha = alpha0 + alpha1;

    PetscReal w0 = alpha0 / sumalpha;
    PetscReal w1 = alpha1 / sumalpha;

    PetscReal u0 = ( fC*0.5  + fR*0.5 );
    PetscReal u1 = ( -fL*0.5 + fC*1.5 );

    return w0*u0 + w1*u1;
}


//*****************************************************************************************************//

inline Cmpnts weno3Vec(Cmpnts f0, Cmpnts f1, Cmpnts f2, Cmpnts f3, PetscReal wavespeed)
{
    Cmpnts fL, fC, fR;

    if(wavespeed>0)
    {
        fL = nSet(f0); fC = nSet(f1); fR = nSet(f2);
    }
    else
    {
        fL = nSet(f3); fC = nSet(f2); fR = nSet(f1);
    }

    PetscReal d0=2./3., d1=1./3.;

    const Cmpnts eps = nSetFromComponents(1e-6, 1e-6, 1e-6);

    Cmpnts beta0 = nPow(nSub(fC,fR),  2.);
    Cmpnts beta1 = nPow(nSub(fL,fC),  2.);

    Cmpnts alpha0 = nScale(d0, nInv(nPow( nSum(eps,beta0), 2. )));
    Cmpnts alpha1 = nScale(d1, nInv(nPow( nSum(eps,beta1), 2. )));

    Cmpnts sumalpha = nSum(alpha0, alpha1);

    Cmpnts w0 = nMultElWise(alpha0, nInv(sumalpha));
    Cmpnts w1 = nMultElWise(alpha1, nInv(sumalpha));

    Cmpnts u0 = nSum(nScale(0.5,fC),  nScale(0.5,fR));
    Cmpnts u1 = nSum(nScale(-0.5,fL), nScale(1.5,fC));

    return nSum(nMultElWise(w0,u0), nMultElWise(w1,u1));
}


//*****************************************************************************************************//

inline PetscReal weno5(PetscReal f0, PetscReal f1, PetscReal f2, PetscReal f3, PetscReal f4, PetscReal
f5, PetscReal wavespeed)
{
    PetscReal A, B, C, D, E;

    if(wavespeed>0)
    {
        A = f0; B = f1; C = f2; D = f3; E = f4;
    }
    else
    {
        A = f5; B = f4; C = f3; D = f2; E = f1;
    }

    PetscReal eps = 1.e-6;
    PetscReal d0, d1, d2;
```

```
    d0=0.3, d1=0.6, d2=0.1;

    PetscReal beta0 = 13./12. * pow( A - 2. * B + C, 2. ) + 1./4. * pow ( A - 4. * B  + 3. * C, 2. );
    PetscReal beta1 = 13./12. * pow( B - 2. * C + D, 2. ) + 1./4. * pow ( B - D, 2. );
    PetscReal beta2 = 13./12. * pow( C - 2. * D + E, 2. ) + 1./4. * pow ( 3. * C - 4. * D  + E, 2. );

    PetscReal alpha0 = d0 / pow( eps + beta0, 2.);
    PetscReal alpha1 = d1 / pow( eps + beta1, 2.);
    PetscReal alpha2 = d2 / pow( eps + beta2, 2.);

    PetscReal sumalpha = alpha0 + alpha1 + alpha2;

    PetscReal w0 = alpha0 / sumalpha;
    PetscReal w1 = alpha1 / sumalpha;
    PetscReal w2 = alpha2 / sumalpha;

    PetscReal u0 = 2./6. * A - 7./6. * B + 11./6. * C;
    PetscReal u1 = -1./6. * B + 5./6. * C + 2./6. * D;
    PetscReal u2 = 2./6. * C + 5./6. * E - 1./6. * E;

    return w0*u0 + w1*u1 + w2*u2;

}

// FIELD OPERATIONS
//
=================================================================================================================
//

inline void resetNoPenetrationFluxes(ueqn_ *ueqn)
{
    mesh_             *mesh = ueqn->access->mesh;
    DM               da = mesh->da, fda = mesh->fda;
    DMDALocalInfo    info = mesh->info;
    PetscInt         xs = info.xs, xe = info.xs + info.xm;
    PetscInt         ys = info.ys, ye = info.ys + info.ym;
    PetscInt         zs = info.zs, ze = info.zs + info.zm;
    PetscInt         mx = info.mx, my = info.my, mz = info.mz;

    PetscInt         lxs, lxe, lys, lye, lzs, lze;
    PetscInt         i, j, k;

    Cmpnts           ***ucont;

    lxs = xs; lxe = xe; if (xs==0) lxs = xs+1; if (xe==mx) lxe = xe-1;
    lys = ys; lye = ye; if (ys==0) lys = ys+1; if (ye==my) lye = ye-1;
    lzs = zs; lze = ze; if (zs==0) lzs = zs+1; if (ze==mz) lze = ze-1;

    // We go through all the boundary conditions implying no penetration and set
    // the wall normal flux to zero in order to exactly enforce mass conservation.
    // This is done because, especially the non-linear solver, could give fluxes not
    // exactly zero at the wall.

    DMDAVecGetArray(fda, ueqn->Ucont, &ucont);

    for (k=zs; k<lze; k++)
    {
        for (j=ys; j<lye; j++)
        {
            for (i=xs; i<lxe; i++)
            {
                // noslip BC
                if(i==0 && mesh->boundaryU.iLeft=="noSlip")      ucont[k][j][i].x = 0.0;
                if(i==mx-2 && mesh->boundaryU.iRight=="noSlip") ucont[k][j][i].x = 0.0;
                if(j==0 && mesh->boundaryU.jLeft=="noSlip")      ucont[k][j][i].y = 0.0;
                if(j==my-2 && mesh->boundaryU.jRight=="noSlip") ucont[k][j][i].y = 0.0;
                if(k==0 && mesh->boundaryU.kLeft=="noSlip")      ucont[k][j][i].z = 0.0;
                if(k==mz-2 && mesh->boundaryU.kRight=="noSlip") ucont[k][j][i].z = 0.0;

                // wall models
                if(i==0 && mesh->boundaryU.iLeft=="velocityWallFunction")      ucont[k][j][i].x = 0.0;
                if(i==mx-2 && mesh->boundaryU.iRight=="velocityWallFunction") ucont[k][j][i].x = 0.0;
                if(j==0 && mesh->boundaryU.jLeft=="velocityWallFunction")      ucont[k][j][i].y = 0.0;
                if(j==my-2 && mesh->boundaryU.jRight=="velocityWallFunction") ucont[k][j][i].y = 0.0;
                if(k==0 && mesh->boundaryU.kLeft=="velocityWallFunction")      ucont[k][j][i].z = 0.0;
                if(k==mz-2 && mesh->boundaryU.kRight=="velocityWallFunction") ucont[k][j][i].z = 0.0;

                // slip BC
                if (mesh->boundaryU.iLeft=="slip" && i==0)      ucont[k][j][i].x = 0.0;
                if (mesh->boundaryU.iRight=="slip" && i==mx-2) ucont[k][j][i].x = 0.0;
                if (mesh->boundaryU.jLeft=="slip" && j==0)      ucont[k][j][i].y = 0.0;
                if (mesh->boundaryU.jRight=="slip" && j==my-2) ucont[k][j][i].y = 0.0;
                if (mesh->boundaryU.kLeft=="slip" && k==0)      ucont[k][j][i].z = 0.0;
                if (mesh->boundaryU.kRight=="slip" && k==mz-2) ucont[k][j][i].z = 0.0;

                // zero gradient BC (if reverse flow)
```

```
                if (mesh->boundaryU.iLeft=="zeroGradient" && i==0)
                {
                    if(ucont[k][j][i].x > 0.0) ucont[k][j][i].x = 0.0;
                }
                if (mesh->boundaryU.iRight=="zeroGradient" && i==mx-2)
                {
                    if(ucont[k][j][i].x < 0.0) ucont[k][j][i].x = 0.0;
                }
                if (mesh->boundaryU.jLeft=="zeroGradient" && j==0)
                {
                    if(ucont[k][j][i].y > 0.0) ucont[k][j][i].y = 0.0;
                }
                if (mesh->boundaryU.jRight=="zeroGradient" && j==my-2)
                {
                    if(ucont[k][j][i].y < 0.0) ucont[k][j][i].y = 0.0;
                }
                if (mesh->boundaryU.kLeft=="zeroGradient" && k==0)
                {
                    if(ucont[k][j][i].z > 0.0) ucont[k][j][i].z = 0.0;
                }
                if (mesh->boundaryU.kRight=="zeroGradient" && k==mz-2)
                {
                    if(ucont[k][j][i].z < 0.0) ucont[k][j][i].z = 0.0;
                }
            }
        }
    }

    DMDAVecRestoreArray (fda, ueqn->Ucont, &ucont);

    DMGlobalToLocalBegin(fda, ueqn->Ucont, INSERT_VALUES, ueqn->lUcont);
    DMGlobalToLocalEnd  (fda, ueqn->Ucont, INSERT_VALUES, ueqn->lUcont);

    return;
}


//*****************************************************************************************************//

inline void resetFacePeriodicFluxesVector(mesh_ *mesh, Vec V, Vec lV, const char *scatterType)
{
    DMDALocalInfo    info = mesh->info;
    PetscInt         xs = info.xs, xe = info.xs + info.xm;
    PetscInt         ys = info.ys, ye = info.ys + info.ym;
    PetscInt         zs = info.zs, ze = info.zs + info.zm;
    PetscInt         mx = info.mx, my = info.my, mz = info.mz;

    PetscInt         i, j, k;

    Cmpnts           ***v, ***lv;

    // If the flow is periodic in l-direction, the variable is solved also
    // at the right boundary faces. Its value must be copied at the left boundary
    // faces to ensure consistency in the momentum computation.

    if(scatterType=="globalToLocal")
    {
        DMDAVecGetArray(mesh->fda, V, &v);
        DMDAVecGetArray(mesh->fda, lV, &lv);
    }
    else if(scatterType=="localToLocal")
    {
        DMDAVecGetArray(mesh->fda, V, &v);
    }

    if(mesh->access->info->periodic)
    {
        for (k=zs; k<ze; k++)
        {
            for (j=ys; j<ye; j++)
            {
                for (i=xs; i<xe; i++)
                {
                    PetscInt flag=0, a=i, b=j, c=k;

                    if(mesh->i_periodic && i==0) a=mx-2, flag=1;
                    //else if(i_periodic && i==mx-1) a=1, flag=1;

                    if(mesh->j_periodic && j==0) b=my-2, flag=1;
                    //else if(j_periodic && j==my-1) b=1, flag=1;

                    if(mesh->k_periodic && k==0) c=mz-2, flag=1;
                    //else if(k_periodic && k==mz-1) c=1, flag=1;

                    if(mesh->ii_periodic && i==0) a=-2, flag=1;
                    //else if(ii_periodic && i==mx-1) a=mx+1, flag=1;

```

```
                    if(mesh->jj_periodic && j==0) b=-2, flag=1;
                    //else if(jj_periodic && j==my-1) b=my+1, flag=1;

                    if(mesh->kk_periodic && k==0) c=-2, flag=1;
                    //else if(kk_periodic && k==mz-1) c=mz+1, flag=1;

                    if(flag)
                    {
                        if(scatterType=="globalToLocal")
                        {
                            v[k][j][i] = lv[c][b][a];
                        }
                        else if(scatterType=="localToLocal")
                        {
                            v[k][j][i] = v[c][b][a];
                        }
                    }
                }
            }
        }
    }

    if(scatterType=="globalToLocal")
    {
        DMDAVecRestoreArray (mesh->fda, V, &v);
        DMDAVecRestoreArray (mesh->fda, lV, &lv);
        DMGlobalToLocalBegin(mesh->fda, V, INSERT_VALUES, lV);
        DMGlobalToLocalEnd  (mesh->fda, V, INSERT_VALUES, lV);
    }
    else if(scatterType=="localToLocal")
    {
        DMDAVecRestoreArray (mesh->fda, V, &v);
        DMLocalToLocalBegin (mesh->fda, V, INSERT_VALUES, V);
        DMLocalToLocalEnd   (mesh->fda, V, INSERT_VALUES, V);
    }
    else
    {
        char error[512];
        sprintf(error, "wrong scatter type selected, available types are\n    - globalToLocal\n    -
localToLocal\n");
        fatalErrorInFunction("resetCellPeriodicFluxes",  error);
    }

    return;
}


//*****************************************************************************************************//

inline void resetCellPeriodicFluxes(mesh_ *mesh, Vec &V, Vec &lV, const char *type, const char
*scatterType)
{
    DM            da    = mesh->da, fda = mesh->fda;
    DMDALocalInfo  info = mesh->info;
    PetscInt         xs = info.xs, xe = info.xs + info.xm;
    PetscInt         ys = info.ys, ye = info.ys + info.ym;
    PetscInt         zs = info.zs, ze = info.zs + info.zm;
    PetscInt         mx = info.mx, my = info.my, mz = info.mz;

    PetscInt         i, j, k;

    Cmpnts           ***v, ***lv;
    PetscScalar      ***s, ***ls;

    // If the flow is periodic in l-th direction, put the first internal value
    // at the ghost cell of the opposite patch.

    if(type=="scalar")
    {
        if(scatterType=="globalToLocal")
        {
            DMDAVecGetArray(da, V, &s);
            DMDAVecGetArray(da, lV, &ls);
        }
        else if(scatterType=="localToLocal")
        {
            DMDAVecGetArray(da, V, &s);
        }
    }
    else if(type=="vector")
    {
        if(scatterType=="globalToLocal")
        {
            DMDAVecGetArray(fda, V, &v);
            DMDAVecGetArray(fda, lV, &lv);
        }
        else if(scatterType=="localToLocal")
```

```
        {
            DMDAVecGetArray(fda, V, &v);
        }
    }
    else
    {
        char error[512];
        sprintf(error, "wrong type selected, available types are\n    - scalar\n    - vector\n");
        fatalErrorInFunction("resetCellPeriodicFluxes",  error);
    }

    if(mesh->access->info->periodic)
    {
        for (k=zs; k<ze; k++)
        {
            for (j=ys; j<ye; j++)
            {
                for (i=xs; i<xe; i++)
                {
                    PetscInt flag=0, a=i, b=j, c=k;

                    if(mesh->i_periodic && i==0) a=mx-2, flag=1;
                    else if(mesh->i_periodic && i==mx-1) a=1, flag=1;

                    if(mesh->j_periodic && j==0) b=my-2, flag=1;
                    else if(mesh->j_periodic && j==my-1) b=1, flag=1;

                    if(mesh->k_periodic && k==0) c=mz-2, flag=1;
                    else if(mesh->k_periodic && k==mz-1) c=1, flag=1;

                    if(mesh->ii_periodic && i==0) a=-2, flag=1;
                    else if(mesh->ii_periodic && i==mx-1) a=mx+1, flag=1;

                    if(mesh->jj_periodic && j==0) b=-2, flag=1;
                    else if(mesh->jj_periodic && j==my-1) b=my+1, flag=1;

                    if(mesh->kk_periodic && k==0) c=-2, flag=1;
                    else if(mesh->kk_periodic && k==mz-1) c=mz+1, flag=1;

                    if(flag)
                    {
                        if(type=="scalar")
                        {
                            if(scatterType=="globalToLocal")
                            {
                                s[k][j][i] = ls[c][b][a];
                            }
                            else if(scatterType=="localToLocal")
                            {
                                s[k][j][i] = s[c][b][a];
                            }
                        }
                        else if(type=="vector")
                        {
                            if(scatterType=="globalToLocal")
                            {
                                v[k][j][i] = lv[c][b][a];
                            }
                            else if(scatterType=="localToLocal")
                            {
                                v[k][j][i] = v[c][b][a];
                            }
                        }
                    }
                }
            }
        }
    }

    if(type=="scalar")
    {
        if(scatterType=="globalToLocal")
        {
            DMDAVecRestoreArray(da, V, &s);
            DMDAVecRestoreArray(da, lV, &ls);
            DMGlobalToLocalBegin(da, V, INSERT_VALUES, lV);
            DMGlobalToLocalEnd(da, V, INSERT_VALUES, lV);
        }
        else if(scatterType=="localToLocal")
        {
            DMDAVecRestoreArray(da, V, &s);
            DMLocalToLocalBegin(da, V, INSERT_VALUES, V);
            DMLocalToLocalEnd(da, V, INSERT_VALUES, V);
        }
        else
        {
            char error[512];
```

```
            sprintf(error, "wrong scatter type selected, available types are\n    - globalToLocal\n - localToLocal\n");
            fatalErrorInFunction("resetCellPeriodicFluxes",  error);
        }
    }
    else if(type=="vector")
    {
        if(scatterType=="globalToLocal")
        {
            DMDAVecRestoreArray(fda, V, &v);
            DMDAVecRestoreArray(fda, lV, &lv);
            DMGlobalToLocalBegin(fda, V, INSERT_VALUES, lV);
            DMGlobalToLocalEnd(fda, V, INSERT_VALUES, lV);
        }
        else if(scatterType=="localToLocal")
        {
            DMDAVecRestoreArray(fda, V, &v);
            DMLocalToLocalBegin(fda, V, INSERT_VALUES, V);
            DMLocalToLocalEnd(fda, V, INSERT_VALUES, V);
        }
        else
        {
            char error[512];
            sprintf(error, "wrong scatter type selected, available types are\n    - globalToLocal\n - localToLocal\n");
            fatalErrorInFunction("resetCellPeriodicFluxes",  error);
        }
    }

    return;
}

// DAMPING VISCOSITY & FRINGE REGION RELATED
//==========================================================================================================//

inline PetscReal viscRayleigh(PetscReal &alpha, PetscReal &hS, PetscReal &hE, PetscReal &h)
{
    PetscReal h_hat = (h-hS)/(hE-hS);
    PetscReal viscosity;

    viscosity
    =
    alpha *
    (
        1 - std::cos
        (
            M_PI / 2.0 * PetscMax(h_hat, 0.0)
        )
    );

    return(viscosity);
}


//**********************************************************************************************************//

inline PetscReal viscCosAscending(PetscReal &alpha, PetscReal &hS, PetscReal &hE, PetscReal &h)
{
    PetscReal h_hat = M_PI*(h-hS)/(hE-hS);
    PetscReal viscosity = 0.0;

    if(h > hS && h < hE)
    {
        viscosity
        =
        alpha * 0.5 *
        (
            1 - std::cos(h_hat)
        );
    }

    return(viscosity);
}


//**********************************************************************************************************//

inline PetscReal viscCosDescending(PetscReal &alpha, PetscReal &hS, PetscReal &hE, PetscReal &h)
{
    PetscReal h_hat = M_PI*(h-hS)/(hE-hS);
    PetscReal viscosity = 0;

    if(h > hS && h < hE)
    {
        viscosity
```

```
        =
        alpha * 0.5 *
        (
            1 + std::cos(h_hat)
        );
    }

    return(viscosity);
}


//*****************************************************************************************************//
inline PetscReal viscNordstrom(PetscReal &alpha, PetscReal &hS, PetscReal &hE, PetscReal &delta, PetscReal &h)
{
    // make sure delta is positive and strictly greater than zero
    delta = PetscMax(fabs(delta), 1e-5);

    PetscReal h1_hat = (h-hS)/delta;
    PetscReal h2_hat = (h-hE)/delta + 1.0;

    PetscReal viscosity;

    PetscReal s1, s2;

    // compute smoothing function rise component
    if(h1_hat <= 0.0)
    {
        s1 = 0.0;
    }
    else if(h1_hat >= 1.0)
    {
        s1 = 1.0;
    }
    else
    {
        s1 = 1.0 / (1.0 + std::exp(1.0/(h1_hat - 1.0) + 1.0 / h1_hat));
    }


    // compute smoothing function falling component
    if(h2_hat <= 0.0)
    {
        s2 = 0.0;
    }
    else if(h2_hat >= 1.0)
    {
        s2 = 1.0;
    }
    else
    {
        s2 = 1.0 / (1.0 + std::exp(1.0/(h2_hat - 1.0) + 1.0 / h2_hat));
    }

    viscosity = alpha * (s1 - s2);

    return(viscosity);
}


//*****************************************************************************************************//

inline double viscNordstromNoVertFilter(double hS, double hE, double &delta, double &h)
{
    // make sure delta is positive and strictly greater than zero
    delta = std::max(fabs(delta), 1e-5);

    double h1_hat = (h-hS)/delta;
    double h2_hat = (h-hE)/delta + 1.0;

    double viscosity;

    double s1, s2;

    // compute smoothing function rise component
    if(h1_hat <= 0.0)
    {
        s1 = 0.0;
    }
    else if(h1_hat >= 1.0)
    {
        s1 = 1.0;
    }
    else
    {
        s1 = 1.0 / (1.0 + std::exp(1.0/(h1_hat - 1.0) + 1.0 / h1_hat));
    }
```

```

    // compute smoothing function falling component
    if(h2_hat <= 0.0)
    {
        s2 = 0.0;
    }
    else if(h2_hat >= 1.0)
    {
        s2 = 1.0;
    }
    else
    {
        s2 = 1.0 / (1.0 + std::exp(1.0/(h2_hat - 1.0) + 1.0 / h2_hat));
    }

    viscosity = 1.0 - (s1 - s2);

    return(viscosity);
}


//*****************************************************************************************************//

inline double viscStipa(double &hS, double &hE, double &delta, double &h, double &z, double &H)
{
    // make sure delta is positive and strictly greater than zero
    delta = std::max(fabs(delta), 1e-5);

    double h1_hat = (h-hS)/delta;
    double h2_hat = (h-hE)/delta + 1.0;

    double viscosity;

    double s1, s2;

    // compute smoothing function rise component
    if(h1_hat <= 0.0)
    {
        s1 = 0.0;
    }
    else if(h1_hat >= 1.0)
    {
        s1 = 1.0;
    }
    else
    {
        s1 = 1.0 / (1.0 + std::exp(1.0/(h1_hat - 1.0) + 1.0 / h1_hat));
    }

    // compute smoothing function falling component
    if(h2_hat <= 0.0)
    {
        s2 = 0.0;
    }
    else if(h2_hat >= 1.0)
    {
        s2 = 1.0;
    }
    else
    {
        s2 = 1.0 / (1.0 + std::exp(1.0/(h2_hat - 1.0) + 1.0 / h2_hat));
    }

    if(z >= H)
    {
        viscosity = 1.0 - (s1 - s2);
    }
    else
    {
        viscosity = 1.0;
    }

    return(viscosity);
}


//*****************************************************************************************************//

inline double viscStipaDelta(double &hS, double &hE, double &deltaS, double &deltaE, double &h, double
&z, double &H)
{
    // make sure delta is positive and strictly greater than zero
    deltaS = std::max(fabs(deltaS), 1e-5);
    deltaE = std::max(fabs(deltaE), 1e-5);

    double h1_hat = (h-hS)/deltaS;
    double h2_hat = (h-hE)/deltaE + 1.0;
```

```

    double viscosity;

    double s1, s2;

    // compute smoothing function rise component
    if(h1_hat <= 0.0)
    {
        s1 = 0.0;
    }
    else if(h1_hat >= 1.0)
    {
        s1 = 1.0;
    }
    else
    {
        s1 = 1.0 / (1.0 + std::exp(1.0/(h1_hat - 1.0) + 1.0 / h1_hat));
    }

    // compute smoothing function falling component
    if(h2_hat <= 0.0)
    {
        s2 = 0.0;
    }
    else if(h2_hat >= 1.0)
    {
        s2 = 1.0;
    }
    else
    {
        s2 = 1.0 / (1.0 + std::exp(1.0/(h2_hat - 1.0) + 1.0 / h2_hat));
    }

    if(z >= H)
    {
        viscosity = 1.0 - (s1 - s2);
    }
    else
    {
        viscosity = 1.0;
    }

    return(viscosity);
}


//*****************************************************************************************************//

inline void findInterpolationWeights(PetscReal *weights, PetscInt *labels, PetscReal *pvec, PetscInt npts, PetscReal pval)
{
    // equality tolerance: if equal data points up to re-tol = 1e-5 are found,
    // that interpolation point is skipped because it does not provide information
    // and would result in a division by zero when computing the weights.
    PetscReal tol1, tol2, eqTol = 1e-5;

    // check that extrapolation is not necessary
    // (up to a minimum tol of (pEnd-pStart) / npts that it is small enough not to have extrapolation errors)
    PetscReal eps = (pvec[npts-1] - pvec[0]) / npts;
    // if(pval > pvec[npts-1]+eps || pval < pvec[0]-eps)
    // {
    //    char error[512];
    //     sprintf(error, "query point %lf outside of bounds [%lf - %lf]... was told not to extrapolate\n", pval, pvec[0]-eps, pvec[npts-1]+eps);
    //     fatalErrorInFunction("findInterpolationWeights",  error);
    // }

    PetscReal diff[npts];
    PetscInt    idx_1 = 0,
            idx_2 = 1;

    // find the difference
    for(PetscInt i=0; i<npts; i++)
    {
        diff[i] = fabs(pvec[i]-pval);
    }

    // find the two closest labels
    for(PetscInt i=0; i<npts; i++)
    {
        tol1 = fabs(diff[i] - diff[idx_2]) / diff[i];

        if(diff[i] < diff[idx_1] && tol1 > eqTol)
        {
            idx_2 = idx_1;
            idx_1 = i;
```

```
        }

        tol2 = fabs(diff[i] - diff[idx_1]) / diff[i];

        if(diff[i] < diff[idx_2] && i != idx_1 && tol2 > eqTol)
        {
            idx_2 = i;
        }
    }

    // always put the lower value at idx_1 and higher at idx_2
    if(pvec[idx_2] < pvec[idx_1])
    {
        PetscInt idx_tmp = idx_2;
        idx_2 = idx_1;
        idx_1 = idx_tmp;
    }

    // store the labels
    labels[0] = idx_1;
    labels[1] = idx_2;

    // should never divide by zero (increase equality tolerance if it happens)
    PetscReal idx = (idx_2 - idx_1) / (pvec[idx_2] - pvec[idx_1]) * (pval - pvec[idx_1]) + idx_1;

    // store the weights
    weights[0] = (idx_2 - idx) / (idx_2 - idx_1);
    weights[1] = (idx - idx_1) / (idx_2 - idx_1);

    return;
}


//*****************************************************************************************************//

inline void findInterpolationWeightsWithExtrap(PetscReal *weights, PetscInt *labels, PetscReal *pvec, PetscInt npts, PetscReal pval)
{
    if(pval >= pvec[npts-1])
    {
        weights[0] = 0.5;
        weights[1] = 0.5;

        labels[0]  = npts-1;
        labels[1]  = npts-1;
    }
    else if (pval <= pvec[0])
    {
        weights[0] = 0.5;
        weights[1] = 0.5;

        labels[0]  = 0;
        labels[1]  = 0;
    }
    else
    {
        findInterpolationWeights(weights, labels, pvec, npts, pval);
    }

    return;
}

//==================================================================================================//

inline PetscReal scaleHyperTangBot(PetscReal h, PetscReal H, PetscReal delta)
{
    // the input parameter H is the height at which the function is almost zero.
    // cb is the center of the smoothing region, i.e. H - 0.5*delta
    PetscReal cb = H - 0.5*delta;
    return
    (
        0.5 *
        (
            1.0 - std::tanh(7.0 * (h-cb)/delta)
        )
    );
}

//==================================================================================================//

inline PetscReal scaleHyperTangTop(PetscReal h, PetscReal H, PetscReal delta)
{
    // the input parameter H is the height at which the function is almost one.
```

```
    // cb is the center of the smoothing region, i.e. H - 0.5*delta
    PetscReal cb = H - 0.5*delta;
    return
    (
        0.5 *
        (
            1.0 + std::tanh(7.0 * (h-cb)/delta)
        )
    );
}


// FLUID QUANTITIES
//
=================================================================================================================
//

inline PetscReal computeEm(Cmpnts &avgU, symmTensor &avgUprimeUprime, PetscReal avgP)
{
    return
    (
        0.5 *
        (
            avgU.x*avgU.x + avgU.y*avgU.y + avgU.z*avgU.z +
            avgUprimeUprime.xx + avgUprimeUprime.yy + avgUprimeUprime.zz
        ) + avgP
    );
}

inline PetscReal computeEmTilde(Cmpnts &avgU, symmTensor &avgUprimeUprime)
{
    return
    (
        0.5 *
        (
            avgU.x*avgU.x + avgU.y*avgU.y + avgU.z*avgU.z +
            avgUprimeUprime.xx + avgUprimeUprime.yy + avgUprimeUprime.zz
        )
    );
}

inline PetscReal computeMKE(Cmpnts &avgU)
{
    return
    (
        0.5 *
        (
            avgU.x*avgU.x + avgU.y*avgU.y + avgU.z*avgU.z
        )
    );
}

inline PetscReal computeTKE(symmTensor &avgUprimeUprime)
{
    return
    (
        0.5 *
        (
            avgUprimeUprime.xx + avgUprimeUprime.yy + avgUprimeUprime.zz
        )
    );
}

// INTERPOLATION
//
=================================================================================================================
//

inline void scalarPointLocalVolumeInterpolation
(
    mesh_ *mesh,
    PetscReal px, PetscReal py, PetscReal pz,
    PetscInt ic, PetscInt jc, PetscInt kc,
    Cmpnts ***cent,
    PetscReal ***v,
    PetscReal &result
)
{
    // Given generic point coordinates px, py, pz and the indices of the closest
    // cell center, this functions finds the closest 8 cells to the given point which form
    // its surrounding box and interpolates the point value using the values at the 8
    // surrounding cell centers (tri-linear interpolation).
    // Note: cell indices passed to this function must be internal cells, not ghosts.
    // Throws an error otherwise.
    // Interpolation weights are bounded from 0 to 1 to handle query points lying outside
    // of the boundaries: this means that those points are projected on the boundaries and
    // the value there is returned (Prevents extrapolation ouside of boundaries).
```

```

    DMDALocalInfo    info = mesh->info;
    PetscInt         xs   = info.xs, xe = info.xs + info.xm;
    PetscInt         ys   = info.ys, ye = info.ys + info.ym;
    PetscInt         zs   = info.zs, ze = info.zs + info.zm;
    PetscInt         mx   = info.mx, my = info.my, mz = info.mz;
    PetscInt         gxs  = info.gxs, gxe = info.gxs + info.gxm;
    PetscInt         gys  = info.gys, gye = info.gys + info.gym;
    PetscInt         gzs  = info.gzs, gze = info.gzs + info.gzm;

    PetscInt         i, j, k;

    PetscInt iL, iR;
    // do the search if surrounded by cells
    if(ic > 1 && ic < mx-2)
    {
        if (ic-1 < gxs)
        {
            iL = ic; iR = ic + 1;
        }
        else if(ic+1 >= gxe)
        {
            iL = ic - 1; iR = ic;
        }
        else
        {
            PetscReal dcsiRightSq
            =
            std::pow(cent[kc][jc][ic+1].x - px,2) +
            std::pow(cent[kc][jc][ic+1].y - py,2) +
            std::pow(cent[kc][jc][ic+1].z - pz,2);

            PetscReal dcsiLeftSq
            =
            std::pow(px - cent[kc][jc][ic-1].x,2) +
            std::pow(py - cent[kc][jc][ic-1].y,2) +
            std::pow(pz - cent[kc][jc][ic-1].z,2);

            if(dcsiRightSq <= dcsiLeftSq)
            {
                iR = ic + 1; iL = ic;
            }
            else
            {
                iR = ic; iL = ic-1;
            }
        }

    }
    // on boundary, already know where to pick data
    else if(ic == 1)
    {
        iR = ic + 1; iL = ic;
    }
    // on boundary, already know where to pick data
    else if(ic == mx-2)
    {
        iR = ic; iL = ic-1;
    }
    // outside the boundary: the Vec V might not contain useful data.
    else
    {
        char error[512];
        sprintf(error, "i-index out of bounds: ended up in the ghosts when looking for interpolation data\n");
        fatalErrorInFunction("pointLocalVolumeInterpolation",  error);
    }

    PetscInt jL, jR;
    // do the search if surrounded by cells
    if(jc > 1 && jc < my-2)
    {
        // check processor bounds
        if (jc-1 < gys)
        {
            jL = jc; jR = jc + 1;
        }
        else if(jc+1 >= gye)
        {
            jL = jc - 1; jR = jc;
        }
        else
        {
            PetscReal detaRightSq
            =
            std::pow(cent[kc][jc+1][ic].x - px,2) +
            std::pow(cent[kc][jc+1][ic].y - py,2) +
```

```
            std::pow(cent[kc][jc+1][ic].z - pz,2);

            PetscReal detaLeftSq
            =
            std::pow(px - cent[kc][jc-1][ic].x,2) +
            std::pow(py - cent[kc][jc-1][ic].y,2) +
            std::pow(pz - cent[kc][jc-1][ic].z,2);

            if(detaRightSq <= detaLeftSq)
            {
                jR = jc + 1; jL = jc;
            }
            else
            {
                jR = jc; jL = jc-1;
            }
        }

    }
    // on boundary, already know where to pick data
    else if(jc == 1)
    {
        jR = jc + 1; jL = jc;
    }
    // on boundary, already know where to pick data
    else if(jc == my-2)
    {
        jR = jc; jL = jc-1;
    }
    // outside the boundary: the Vec V might not contain useful data.
    else
    {
        char error[512];
        sprintf(error, "j-index out of bounds: ended up in the ghosts when looking for interpolation data\n");
        fatalErrorInFunction("pointLocalVolumeInterpolation",  error);
    }

    PetscInt kL, kR;
    // do the search if surrounded by cells
    if(kc > 1 && kc < mz-2)
    {
        // check processor bounds
        if (kc-1 < gzs)
        {
            kL = kc; kR = kc + 1;
        }
        else if(kc+1 >= gze)
        {
            kL = kc - 1; kR = kc;
        }
        else
        {
            PetscReal dzetRightSq
            =
            std::pow(cent[kc+1][jc][ic].x - px,2) +
            std::pow(cent[kc+1][jc][ic].y - py,2) +
            std::pow(cent[kc+1][jc][ic].z - pz,2);

            PetscReal dzetLeftSq
            =
            std::pow(px - cent[kc-1][jc][ic].x,2) +
            std::pow(py - cent[kc-1][jc][ic].y,2) +
            std::pow(pz - cent[kc-1][jc][ic].z,2);

            if(dzetRightSq <= dzetLeftSq)
            {
                kR = kc + 1; kL = kc;
            }
            else
            {
                kR = kc; kL = kc-1;
            }
        }

    }
    // on boundary, already know where to pick data
    else if(kc == 1)
    {
        kR = kc + 1; kL = kc;
    }
    // on boundary, already know where to pick data
    else if(kc == mz-2)
    {
        kR = kc; kL = kc-1;
    }
    // outside the boundary: the Vec V might not contain useful data.
```

```
    else
    {
        char error[512];
        sprintf(error, "k-index out of bounds: ended up in the ghosts when looking for interpolation data\n");
        fatalErrorInFunction("pointLocalVolumeInterpolation",  error);
    }

    /*
    // Debugging information (only i and j)
    PetscPrintf(PETSC_COMM_WORLD, "Neighboring cells\n");
    PetscPrintf(PETSC_COMM_WORLD, "(iL,jL) idxs: %d %d, C = %.2f %.2f\n", iL, jL, cent[kL][jL][iL].y, cent[kL][jL][iL].z);
    PetscPrintf(PETSC_COMM_WORLD, "(iL,jR) idxs: %d %d, C = %.2f %.2f\n", iL, jR, cent[kL][jR][iL].y, cent[kL][jR][iL].z);
    PetscPrintf(PETSC_COMM_WORLD, "(iR,jL) idxs: %d %d, C = %.2f %.2f\n", iR, jL, cent[kL][jL][iR].y, cent[kL][jL][iR].z);
    PetscPrintf(PETSC_COMM_WORLD, "(iR,jR) idxs: %d %d, C = %.2f %.2f\n", iR, jR, cent[kL][jR][iR].y, cent[kL][jR][iR].z);
    PetscPrintf(PETSC_COMM_WORLD, "(ic,jc) idxs: %d %d\n", ic, jc);
    PetscPrintf(PETSC_COMM_WORLD, "P = %.2f %.2f\n\n", py, pz);
    */

    // find directional gradients
    Cmpnts csiHat, etaHat, zetHat;
    PetscReal csiNor, etaNor, zetNor;

    csiHat.x = cent[kL][jL][iR].x - cent[kL][jL][iL].x;
    csiHat.y = cent[kL][jL][iR].y - cent[kL][jL][iL].y;
    csiHat.z = cent[kL][jL][iR].z - cent[kL][jL][iL].z;

    etaHat.x = cent[kL][jR][iL].x - cent[kL][jL][iL].x;
    etaHat.y = cent[kL][jR][iL].y - cent[kL][jL][iL].y;
    etaHat.z = cent[kL][jR][iL].z - cent[kL][jL][iL].z;

    zetHat.x = cent[kR][jL][iL].x - cent[kL][jL][iL].x;
    zetHat.y = cent[kR][jL][iL].y - cent[kL][jL][iL].y;
    zetHat.z = cent[kR][jL][iL].z - cent[kL][jL][iL].z;

    csiNor = std::sqrt(csiHat.x*csiHat.x + csiHat.y*csiHat.y + csiHat.z*csiHat.z);
    etaNor = std::sqrt(etaHat.x*etaHat.x + etaHat.y*etaHat.y + etaHat.z*etaHat.z);
    zetNor = std::sqrt(zetHat.x*zetHat.x + zetHat.y*zetHat.y + zetHat.z*zetHat.z);

    csiHat.x = csiHat.x / csiNor; csiHat.y = csiHat.y / csiNor; csiHat.z = csiHat.z / csiNor;
    etaHat.x = etaHat.x / etaNor; etaHat.y = etaHat.y / etaNor; etaHat.z = etaHat.z / etaNor;
    zetHat.x = zetHat.x / zetNor; zetHat.y = zetHat.y / zetNor; zetHat.z = zetHat.z / zetNor;

    // bound deltas from 0 to 1: this means that if the query point is outside of boundaries (we have
    // no data there), we project that point on the boundary and give the interpolated value at that
    // point as a result.

    PetscReal deltaCsi
    =
    PetscMax
    (
        PetscMin
        (
            (
                csiHat.x * (px - cent[kL][jL][iL].x) +
                csiHat.y * (py - cent[kL][jL][iL].y) +
                csiHat.z * (pz - cent[kL][jL][iL].z)

            ) / csiNor,
            1.0
        ),
        0.0
    );

    PetscReal deltaEta
    =
    PetscMax
    (
        PetscMin
        (
            (
                etaHat.x * (px - cent[kL][jL][iL].x) +
                etaHat.y * (py - cent[kL][jL][iL].y) +
                etaHat.z * (pz - cent[kL][jL][iL].z)

            ) / etaNor,
            1.0
        ),
        0.0
    );

    PetscReal deltaZet
    =
```

```
    PetscMax
    (
        PetscMin
        (
            (
                zetHat.x * (px - cent[kL][jL][iL].x) +
                zetHat.y * (py - cent[kL][jL][iL].y) +
                zetHat.z * (pz - cent[kL][jL][iL].z)

            ) / zetNor,
            1.0
        ),
        0.0
    );

    // find interpolation weights
    PetscReal v000 = v[kL][jL][iL],
           v100 = v[kL][jL][iR],
           v110 = v[kL][jR][iR],
           v010 = v[kL][jR][iL],
           v001 = v[kR][jL][iL],
           v101 = v[kR][jL][iR],
           v111 = v[kR][jR][iR],
           v011 = v[kR][jR][iL];

    PetscReal c0 = v000,
           c1 = v100 - v000,
           c2 = v010 - v000,
           c3 = v001 - v000,
           c4 = v110 - v010 - v100 + v000,
           c5 = v011 - v001 - v010 + v000,
           c6 = v101 - v001 - v100 + v000,
           c7 = v111 - v011 - v101 - v110 + v100 + v001 + v010 - v000;

    // interpolate
    result
    =
    c0 +
    c1 * deltaCsi +
    c2 * deltaEta +
    c3 * deltaZet +
    c4 * deltaCsi * deltaEta +
    c5 * deltaEta * deltaZet +
    c6 * deltaZet * deltaCsi +
    c7 * deltaCsi * deltaEta * deltaZet;

    return;
}


//*****************************************************************************************************************//

inline void PointInterpolationWeights
(
    mesh_ *mesh,
    PetscReal px, PetscReal py, PetscReal pz,
    PetscInt ic, PetscInt jc, PetscInt kc,
    Cmpnts ***cent,
    PetscReal *intWts,
    PetscInt *intId
)
{
    // Given generic point coordinates px, py, pz and the indices of the closest
    // cell center, this functions finds the closest 8 cells to the given point which form
    // its surrounding box and interpolates the point value using the values at the 8
    // surrounding cell centers (tri-linear interpolation).
    // Note: cell indices passed to this function must be internal cells, not ghosts.
    // Throws an error otherwise.
    // Interpolation weights are bounded from 0 to 1 to handle query points lying outside
    // of the boundaries: this means that those points are projected on the boundaries and
    // the value there is returned (Prevents extrapolation ouside of boundaries).

    DMDALocalInfo    info = mesh->info;
    PetscInt         xs   = info.xs, xe = info.xs + info.xm;
    PetscInt         ys   = info.ys, ye = info.ys + info.ym;
    PetscInt         zs   = info.zs, ze = info.zs + info.zm;
    PetscInt         mx   = info.mx, my = info.my, mz = info.mz;
    PetscInt         gxs  = info.gxs, gxe = info.gxs + info.gxm;
    PetscInt         gys  = info.gys, gye = info.gys + info.gym;
    PetscInt         gzs  = info.gzs, gze = info.gzs + info.gzm;

    PetscInt         i, j, k;

    PetscInt iL, iR;

    // do the search if surrounded by cells
    if(ic > 1 && ic < mx-2)
```

```
    {
        if (ic-1 < gxs)
        {
            iL = ic; iR = ic + 1;
        }
        else if(ic+1 >= gxe)
        {
            iL = ic - 1; iR = ic;
        }
        else
        {
            PetscReal dcsiRightSq
            =
            std::pow(cent[kc][jc][ic+1].x - px,2) +
            std::pow(cent[kc][jc][ic+1].y - py,2) +
            std::pow(cent[kc][jc][ic+1].z - pz,2);

            PetscReal dcsiLeftSq
            =
            std::pow(px - cent[kc][jc][ic-1].x,2) +
            std::pow(py - cent[kc][jc][ic-1].y,2) +
            std::pow(pz - cent[kc][jc][ic-1].z,2);

            if(dcsiRightSq <= dcsiLeftSq)
            {
                iR = ic + 1; iL = ic;
            }
            else
            {
                iR = ic; iL = ic-1;
            }
        }

    }
    // on boundary, already know where to pick data
    else if(ic == 1)
    {
        iR = ic + 1; iL = ic;
    }
    // on boundary, already know where to pick data
    else if(ic == mx-2)
    {
        iR = ic; iL = ic-1;
    }
    // outside the boundary: the Vec V might not contain useful data.
    else
    {
        char error[512];
        sprintf(error, "i-index out of bounds: ended up in the ghosts when looking for interpolation (i = %ld)\n", ic);
        fatalErrorInFunction("scalarPointInterpolationWeights",  error);
    }

    PetscInt jL, jR;
    // do the search if surrounded by cells
    if(jc > 1 && jc < my-2)
    {
        // check processor bounds
        if (jc-1 < gys)
        {
            jL = jc; jR = jc + 1;
        }
        else if(jc+1 >= gye)
        {
            jL = jc - 1; jR = jc;
        }
        else
        {
            PetscReal detaRightSq
            =
            std::pow(cent[kc][jc+1][ic].x - px,2) +
            std::pow(cent[kc][jc+1][ic].y - py,2) +
            std::pow(cent[kc][jc+1][ic].z - pz,2);

            PetscReal detaLeftSq
            =
            std::pow(px - cent[kc][jc-1][ic].x,2) +
            std::pow(py - cent[kc][jc-1][ic].y,2) +
            std::pow(pz - cent[kc][jc-1][ic].z,2);

            if(detaRightSq <= detaLeftSq)
            {
                jR = jc + 1; jL = jc;
            }
            else
            {
                jR = jc; jL = jc-1;
```

```
            }
        }

    }
    // on boundary, already know where to pick data
    else if(jc == 1)
    {
        jR = jc + 1; jL = jc;
    }
    // on boundary, already know where to pick data
    else if(jc == my-2)
    {
        jR = jc; jL = jc-1;
    }
    // outside the boundary: the Vec V might not contain useful data.
    else
    {
        char error[512];
        sprintf(error, "j-index out of bounds: ended up in the ghosts when looking for interpolation (j = %ld)\n", jc);
        fatalErrorInFunction("scalarPointInterpolationWeights",  error);
    }

    PetscInt kL, kR;
    // do the search if surrounded by cells
    if(kc > 1 && kc < mz-2)
    {
        // check processor bounds
        if (kc-1 < gzs)
        {
            kL = kc; kR = kc + 1;
        }
        else if(kc+1 >= gze)
        {
            kL = kc - 1; kR = kc;
        }
        else
        {
            PetscReal dzetRightSq
            =
            std::pow(cent[kc+1][jc][ic].x - px,2) +
            std::pow(cent[kc+1][jc][ic].y - py,2) +
            std::pow(cent[kc+1][jc][ic].z - pz,2);

            PetscReal dzetLeftSq
            =
            std::pow(px - cent[kc-1][jc][ic].x,2) +
            std::pow(py - cent[kc-1][jc][ic].y,2) +
            std::pow(pz - cent[kc-1][jc][ic].z,2);

            if(dzetRightSq <= dzetLeftSq)
            {
                kR = kc + 1; kL = kc;
            }
            else
            {
                kR = kc; kL = kc-1;
            }
        }

    }
    // on boundary, already know where to pick data
    else if(kc == 1)
    {
        kR = kc + 1; kL = kc;
    }
    // on boundary, already know where to pick data
    else if(kc == mz-2)
    {
        kR = kc; kL = kc-1;
    }
    // outside the boundary: the Vec V might not contain useful data.
    else
    {
        char error[512];
        sprintf(error, "k-index out of bounds: ended up in the ghosts when looking for interpolation (k = %ld)\n", kc);
        fatalErrorInFunction("scalarPointInterpolationWeights",  error);
    }

    intId[0] = kL;
    intId[1] = kR;
    intId[2] = jL;
    intId[3] = jR;
    intId[4] = iL;
    intId[5] = iR;

```

```
    //check that the point is inside the interpolation box, send warning if not
    // if
    // (
    //     ( ( (px < cent[kL][jL][iL].x) && (px < cent[kR][jR][iR].x) ) || ( (px > cent[kR][jR][iR].x) && (px > cent[kL][jL][iL].x) ) ) ||
    //     ( ( (py < cent[kL][jL][iL].y) && (py < cent[kR][jR][iR].y) ) || ( (py > cent[kR][jR][iR].y) && (py > cent[kL][jL][iL].y) ) )||
    //     ( ( (pz < cent[kL][jL][iL].z) && (pz < cent[kR][jR][iR].z) ) || ( (pz > cent[kR][jR][iR].z) && (pz > cent[kL][jL][iL].z) ) )
    //
    // )
    // {
    //   char warning[512];
    //   sprintf(warning, "\n\n\npoint %lf %lf %lf is outside the trilinear interpolation box of %ld %ld %ld, %ld %ld %ld - min bound = %lf %lf %lf, max bound %lf %lf %lf, results may be incorrect!!\n\n\n", px, py, pz, kL, jL, iL, kR, jR, iR, cent[kL][jL][iL].x, cent[kL][jL][iL].y, cent[kL][jL][iL].z, cent[kR][jR][iR].x, cent[kR][jR][iR].y, cent[kR][jR][iR].z);
    //   warningInFunction("PointInterpolationWeights",  warning);
    // }

    // find directional gradients
    Cmpnts csiHat, etaHat, zetHat;
    PetscReal csiNor, etaNor, zetNor;

    csiHat.x = cent[kL][jL][iR].x - cent[kL][jL][iL].x;
    csiHat.y = cent[kL][jL][iR].y - cent[kL][jL][iL].y;
    csiHat.z = cent[kL][jL][iR].z - cent[kL][jL][iL].z;

    etaHat.x = cent[kL][jR][iL].x - cent[kL][jL][iL].x;
    etaHat.y = cent[kL][jR][iL].y - cent[kL][jL][iL].y;
    etaHat.z = cent[kL][jR][iL].z - cent[kL][jL][iL].z;

    zetHat.x = cent[kR][jL][iL].x - cent[kL][jL][iL].x;
    zetHat.y = cent[kR][jL][iL].y - cent[kL][jL][iL].y;
    zetHat.z = cent[kR][jL][iL].z - cent[kL][jL][iL].z;

    csiNor = std::sqrt(csiHat.x*csiHat.x + csiHat.y*csiHat.y + csiHat.z*csiHat.z);
    etaNor = std::sqrt(etaHat.x*etaHat.x + etaHat.y*etaHat.y + etaHat.z*etaHat.z);
    zetNor = std::sqrt(zetHat.x*zetHat.x + zetHat.y*zetHat.y + zetHat.z*zetHat.z);

    csiHat.x = csiHat.x / csiNor; csiHat.y = csiHat.y / csiNor; csiHat.z = csiHat.z / csiNor;
    etaHat.x = etaHat.x / etaNor; etaHat.y = etaHat.y / etaNor; etaHat.z = etaHat.z / etaNor;
    zetHat.x = zetHat.x / zetNor; zetHat.y = zetHat.y / zetNor; zetHat.z = zetHat.z / zetNor;

    // bound deltas from 0 to 1: this means that if the query point is outside of boundaries (we have
    // no data there), we project that point on the boundary and give the interpolated value at that
    // point as a result.

    PetscReal deltaCsi
    =
    PetscMax
    (
        PetscMin
        (
            (
                csiHat.x * (px - cent[kL][jL][iL].x) +
                csiHat.y * (py - cent[kL][jL][iL].y) +
                csiHat.z * (pz - cent[kL][jL][iL].z)

            ) / csiNor,
            1.0
        ),
        0.0
    );

    PetscReal deltaEta
    =
    PetscMax
    (
        PetscMin
        (
            (
                etaHat.x * (px - cent[kL][jL][iL].x) +
                etaHat.y * (py - cent[kL][jL][iL].y) +
                etaHat.z * (pz - cent[kL][jL][iL].z)

            ) / etaNor,
            1.0
        ),
        0.0
    );

    PetscReal deltaZet
    =
    PetscMax
    (
        PetscMin
```

```
        (
            (
                zetHat.x * (px - cent[kL][jL][iL].x) +
                zetHat.y * (py - cent[kL][jL][iL].y) +
                zetHat.z * (pz - cent[kL][jL][iL].z)

            ) / zetNor,
            1.0
        ),
        0.0
    );

    // find interpolation weights
    intWts[0] = 1 - deltaCsi - deltaEta - deltaZet + (deltaCsi * deltaEta) + (deltaEta * deltaZet) + (deltaZet * deltaCsi) - (deltaCsi * deltaEta * deltaZet),
    intWts[1] = deltaCsi - (deltaCsi * deltaEta) - (deltaZet * deltaCsi) + (deltaCsi * deltaEta * deltaZet),
    intWts[2] = deltaEta - (deltaCsi * deltaEta) - (deltaEta * deltaZet) + (deltaCsi * deltaEta * deltaZet),
    intWts[3] = (deltaCsi * deltaEta) - (deltaCsi * deltaEta * deltaZet),
    intWts[4] = deltaZet - (deltaEta * deltaZet) - (deltaZet * deltaCsi) + (deltaCsi * deltaEta * deltaZet),
    intWts[5] = (deltaZet * deltaCsi) - (deltaCsi * deltaEta * deltaZet),
    intWts[6] = (deltaEta * deltaZet) - (deltaCsi * deltaEta * deltaZet),
    intWts[7] = (deltaCsi * deltaEta * deltaZet);

    return;
}

//*****************************************************************************************************//
inline void PointInterpolationCells
(
    mesh_ *mesh,
    PetscReal px, PetscReal py, PetscReal pz,
    PetscInt ic, PetscInt jc, PetscInt kc,
    Cmpnts ***cent,
    PetscInt *intId
)
{
    // Given generic point coordinates px, py, pz and the indices of the closest
    // cell center, this functions finds the closest 8 cells to the given point which form
    // its surrounding box and interpolates the point value using the values at the 8
    // surrounding cell centers (tri-linear interpolation).
    // Note: cell indices passed to this function must be internal cells, not ghosts.
    // Throws an error otherwise.
    // Interpolation weights are bounded from 0 to 1 to handle query points lying outside
    // of the boundaries: this means that those points are projected on the boundaries and
    // the value there is returned (Prevents extrapolation ouside of boundaries).

    DMDALocalInfo    info = mesh->info;
    PetscInt         xs   = info.xs, xe = info.xs + info.xm;
    PetscInt         ys   = info.ys, ye = info.ys + info.ym;
    PetscInt         zs   = info.zs, ze = info.zs + info.zm;
    PetscInt         mx   = info.mx, my = info.my, mz = info.mz;
    PetscInt         gxs  = info.gxs, gxe = info.gxs + info.gxm;
    PetscInt         gys  = info.gys, gye = info.gys + info.gym;
    PetscInt         gzs  = info.gzs, gze = info.gzs + info.gzm;

    PetscInt         i, j, k;

    PetscInt iL, iR;

    // do the search if surrounded by cells
    if(ic > 1 && ic < mx-2)
    {
        if (ic-1 < gxs)
        {
            iL = ic; iR = ic + 1;
        }
        else if(ic+1 >= gxe)
        {
            iL = ic - 1; iR = ic;
        }
        else
        {
            PetscReal dcsiRightSq
            =
            std::pow(cent[kc][jc][ic+1].x - px,2) +
            std::pow(cent[kc][jc][ic+1].y - py,2) +
            std::pow(cent[kc][jc][ic+1].z - pz,2);

            PetscReal dcsiLeftSq
            =
            std::pow(px - cent[kc][jc][ic-1].x,2) +
            std::pow(py - cent[kc][jc][ic-1].y,2) +
            std::pow(pz - cent[kc][jc][ic-1].z,2);

```

```
            if(dcsiRightSq <= dcsiLeftSq)
            {
                iR = ic + 1; iL = ic;
            }
            else
            {
                iR = ic; iL = ic-1;
            }
        }

    }
    // on boundary, already know where to pick data
    else if(ic == 1)
    {
        iR = ic + 1; iL = ic;
    }
    // on boundary, already know where to pick data
    else if(ic == mx-2)
    {
        iR = ic; iL = ic-1;
    }
    // outside the boundary: the Vec V might not contain useful data.
    else
    {
        char error[512];
        sprintf(error, "i-index out of bounds: ended up in the ghosts when looking for interpolation (i = %ld)\n", ic);
        fatalErrorInFunction("scalarPointInterpolationCells",  error);
    }

    PetscInt jL, jR;
    // do the search if surrounded by cells
    if(jc > 1 && jc < my-2)
    {
        // check processor bounds
        if (jc-1 < gys)
        {
            jL = jc; jR = jc + 1;
        }
        else if(jc+1 >= gye)
        {
            jL = jc - 1; jR = jc;
        }
        else
        {
            PetscReal detaRightSq
            =
            std::pow(cent[kc][jc+1][ic].x - px,2) +
            std::pow(cent[kc][jc+1][ic].y - py,2) +
            std::pow(cent[kc][jc+1][ic].z - pz,2);

            PetscReal detaLeftSq
            =
            std::pow(px - cent[kc][jc-1][ic].x,2) +
            std::pow(py - cent[kc][jc-1][ic].y,2) +
            std::pow(pz - cent[kc][jc-1][ic].z,2);

            if(detaRightSq <= detaLeftSq)
            {
                jR = jc + 1; jL = jc;
            }
            else
            {
                jR = jc; jL = jc-1;
            }
        }

    }
    // on boundary, already know where to pick data
    else if(jc == 1)
    {
        jR = jc + 1; jL = jc;
    }
    // on boundary, already know where to pick data
    else if(jc == my-2)
    {
        jR = jc; jL = jc-1;
    }
    // outside the boundary: the Vec V might not contain useful data.
    else
    {
        char error[512];
        sprintf(error, "j-index out of bounds: ended up in the ghosts when looking for interpolation (j = %ld)\n", jc);
        fatalErrorInFunction("scalarPointInterpolationCells",  error);
    }

```

```
    PetscInt kL, kR;
    // do the search if surrounded by cells
    if(kc > 1 && kc < mz-2)
    {
        // check processor bounds
        if (kc-1 < gzs)
        {
            kL = kc; kR = kc + 1;
        }
        else if(kc+1 >= gze)
        {
            kL = kc - 1; kR = kc;
        }
        else
        {
            PetscReal dzetRightSq
            =
            std::pow(cent[kc+1][jc][ic].x - px,2) +
            std::pow(cent[kc+1][jc][ic].y - py,2) +
            std::pow(cent[kc+1][jc][ic].z - pz,2);

            PetscReal dzetLeftSq
            =
            std::pow(px - cent[kc-1][jc][ic].x,2) +
            std::pow(py - cent[kc-1][jc][ic].y,2) +
            std::pow(pz - cent[kc-1][jc][ic].z,2);

            if(dzetRightSq <= dzetLeftSq)
            {
                kR = kc + 1; kL = kc;
            }
            else
            {
                kR = kc; kL = kc-1;
            }
        }

    }
    // on boundary, already know where to pick data
    else if(kc == 1)
    {
        kR = kc + 1; kL = kc;
    }
    // on boundary, already know where to pick data
    else if(kc == mz-2)
    {
        kR = kc; kL = kc-1;
    }
    // outside the boundary: the Vec V might not contain useful data.
    else
    {
        char error[512];
        sprintf(error, "k-index out of bounds: ended up in the ghosts when looking for interpolation (k = %ld)\n", kc);
        fatalErrorInFunction("scalarPointInterpolationCells",  error);
    }

    intId[0] = kL;
    intId[1] = kR;
    intId[2] = jL;
    intId[3] = jR;
    intId[4] = iL;
    intId[5] = iR;

    return;
}

//*****************************************************************************************************************//
inline void vectorPointLocalVolumeInterpolation
(
    mesh_ *mesh,
    PetscReal px, PetscReal py, PetscReal pz,
    PetscInt ic, PetscInt jc, PetscInt kc,
    Cmpnts ***cent,
    Cmpnts ***v,
    Cmpnts &result
)
{
    // Given generic point coordinates px, py, pz and the indices of the closest
    // cell center, this functions finds the closest 8 cells to the given point which form
    // its surrounding box and interpolates the point value using the values at the 8
    // surrounding cell centers (tri-linear interpolation).
    // Note: cell indices passed to this function must be internal cells, not ghosts.
    // Throws an error otherwise.
    // Interpolation weights are bounded from 0 to 1 to handle query points lying outside
    // of the boundaries: this means that those points are projected on the boundaries and
    // the value there is returned (Prevents extrapolation ouside of boundaries).
```

```
    DMDALocalInfo    info = mesh->info;
    PetscInt         xs   = info.xs, xe = info.xs + info.xm;
    PetscInt         ys   = info.ys, ye = info.ys + info.ym;
    PetscInt         zs   = info.zs, ze = info.zs + info.zm;
    PetscInt         mx   = info.mx, my = info.my, mz = info.mz;
    PetscInt         gxs  = info.gxs, gxe = info.gxs + info.gxm;
    PetscInt         gys  = info.gys, gye = info.gys + info.gym;
    PetscInt         gzs  = info.gzs, gze = info.gzs + info.gzm;

    PetscInt         i, j, k;

    PetscInt iL, iR;
    // do the search if surrounded by cells
    if(ic > 1 && ic < mx-2)
    {
        if (ic-1 < gxs)
        {
            iL = ic; iR = ic + 1;
        }
        else if(ic+1 >= gxe)
        {
            iL = ic - 1; iR = ic;
        }
        else
        {
            PetscReal dcsiRightSq
            =
            std::pow(cent[kc][jc][ic+1].x - px,2) +
            std::pow(cent[kc][jc][ic+1].y - py,2) +
            std::pow(cent[kc][jc][ic+1].z - pz,2);

            PetscReal dcsiLeftSq
            =
            std::pow(px - cent[kc][jc][ic-1].x,2) +
            std::pow(py - cent[kc][jc][ic-1].y,2) +
            std::pow(pz - cent[kc][jc][ic-1].z,2);

            if(dcsiRightSq <= dcsiLeftSq)
            {
                iR = ic + 1; iL = ic;
            }
            else
            {
                iR = ic; iL = ic-1;
            }
        }

    }
    // on boundary, already know where to pick data
    else if(ic == 1)
    {
        iR = ic + 1; iL = ic;
    }
    // on boundary, already know where to pick data
    else if(ic == mx-2)
    {
        iR = ic; iL = ic-1;
    }
    // outside the boundary: the Vec V might not contain useful data.
    else
    {
        char error[512];
        sprintf(error, "i-index out of bounds: ended up in the ghosts when looking for interpolation data (i = %ld)\n", ic);
        fatalErrorInFunction("vectorPointLocalVolumeInterpolation",  error);
    }

    PetscInt jL, jR;
    // do the search if surrounded by cells
    if(jc > 1 && jc < my-2)
    {
        // check processor bounds
        if (jc-1 < gys)
        {
            jL = jc; jR = jc + 1;
        }
        else if(jc+1 >= gye)
        {
            jL = jc - 1; jR = jc;
        }
        else
        {
            PetscReal detaRightSq
            =
            std::pow(cent[kc][jc+1][ic].x - px,2) +
            std::pow(cent[kc][jc+1][ic].y - py,2) +
```

```
            std::pow(cent[kc][jc+1][ic].z - pz,2);

            PetscReal detaLeftSq
            =
            std::pow(px - cent[kc][jc-1][ic].x,2) +
            std::pow(py - cent[kc][jc-1][ic].y,2) +
            std::pow(pz - cent[kc][jc-1][ic].z,2);

            if(detaRightSq <= detaLeftSq)
            {
                jR = jc + 1; jL = jc;
            }
            else
            {
                jR = jc; jL = jc-1;
            }
        }

    }
    // on boundary, already know where to pick data
    else if(jc == 1)
    {
        jR = jc + 1; jL = jc;
    }
    // on boundary, already know where to pick data
    else if(jc == my-2)
    {
        jR = jc; jL = jc-1;
    }
    // outside the boundary: the Vec V might not contain useful data.
    else
    {
        char error[512];
        sprintf(error, "j-index out of bounds: ended up in the ghosts when looking for interpolation data (j = %ld)\n", jc);
        fatalErrorInFunction("vectorPointLocalVolumeInterpolation",  error);
    }

    PetscInt kL, kR;
    // do the search if surrounded by cells
    if(kc > 1 && kc < mz-2)
    {
        // check processor bounds
        if (kc-1 < gzs)
        {
            kL = kc; kR = kc + 1;
        }
        else if(kc+1 >= gze)
        {
            kL = kc - 1; kR = kc;
        }
        else
        {
            PetscReal dzetRightSq
            =
            std::pow(cent[kc+1][jc][ic].x - px,2) +
            std::pow(cent[kc+1][jc][ic].y - py,2) +
            std::pow(cent[kc+1][jc][ic].z - pz,2);

            PetscReal dzetLeftSq
            =
            std::pow(px - cent[kc-1][jc][ic].x,2) +
            std::pow(py - cent[kc-1][jc][ic].y,2) +
            std::pow(pz - cent[kc-1][jc][ic].z,2);

            if(dzetRightSq <= dzetLeftSq)
            {
                kR = kc + 1; kL = kc;
            }
            else
            {
                kR = kc; kL = kc-1;
            }
        }

    }
    // on boundary, already know where to pick data
    else if(kc == 1)
    {
        kR = kc + 1; kL = kc;
    }
    // on boundary, already know where to pick data
    else if(kc == mz-2)
    {
        kR = kc; kL = kc-1;
    }
    // outside the boundary: the Vec V might not contain useful data.
```

```
    else
    {
        char error[512];
        sprintf(error, "k-index out of bounds: ended up in the ghosts when looking for interpolation data (k = %ld)\n", kc);
        fatalErrorInFunction("vectorPointLocalVolumeInterpolation",  error);
    }

    /*
    // Debugging information (only i and j)
    PetscPrintf(PETSC_COMM_WORLD, "Neighboring cells\n");
    PetscPrintf(PETSC_COMM_WORLD, "(iL,jL) idxs: %d %d, C = %.2f %.2f\n", iL, jL, cent[kL][jL][iL].y, cent[kL][jL][iL].z);
    PetscPrintf(PETSC_COMM_WORLD, "(iL,jR) idxs: %d %d, C = %.2f %.2f\n", iL, jR, cent[kL][jR][iL].y, cent[kL][jR][iL].z);
    PetscPrintf(PETSC_COMM_WORLD, "(iR,jL) idxs: %d %d, C = %.2f %.2f\n", iR, jL, cent[kL][jL][iR].y, cent[kL][jL][iR].z);
    PetscPrintf(PETSC_COMM_WORLD, "(iR,jR) idxs: %d %d, C = %.2f %.2f\n", iR, jR, cent[kL][jR][iR].y, cent[kL][jR][iR].z);
    PetscPrintf(PETSC_COMM_WORLD, "(ic,jc) idxs: %d %d\n", ic, jc);
    PetscPrintf(PETSC_COMM_WORLD, "P = %.2f %.2f\n\n", py, pz);
    */

    // find directional gradients
    Cmpnts csiHat, etaHat, zetHat;
    PetscReal csiNor, etaNor, zetNor;

    csiHat.x = cent[kL][jL][iR].x - cent[kL][jL][iL].x;
    csiHat.y = cent[kL][jL][iR].y - cent[kL][jL][iL].y;
    csiHat.z = cent[kL][jL][iR].z - cent[kL][jL][iL].z;

    etaHat.x = cent[kL][jR][iL].x - cent[kL][jL][iL].x;
    etaHat.y = cent[kL][jR][iL].y - cent[kL][jL][iL].y;
    etaHat.z = cent[kL][jR][iL].z - cent[kL][jL][iL].z;

    zetHat.x = cent[kR][jL][iL].x - cent[kL][jL][iL].x;
    zetHat.y = cent[kR][jL][iL].y - cent[kL][jL][iL].y;
    zetHat.z = cent[kR][jL][iL].z - cent[kL][jL][iL].z;

    csiNor = std::sqrt(csiHat.x*csiHat.x + csiHat.y*csiHat.y + csiHat.z*csiHat.z);
    etaNor = std::sqrt(etaHat.x*etaHat.x + etaHat.y*etaHat.y + etaHat.z*etaHat.z);
    zetNor = std::sqrt(zetHat.x*zetHat.x + zetHat.y*zetHat.y + zetHat.z*zetHat.z);

    csiHat.x = csiHat.x / csiNor; csiHat.y = csiHat.y / csiNor; csiHat.z = csiHat.z / csiNor;
    etaHat.x = etaHat.x / etaNor; etaHat.y = etaHat.y / etaNor; etaHat.z = etaHat.z / etaNor;
    zetHat.x = zetHat.x / zetNor; zetHat.y = zetHat.y / zetNor; zetHat.z = zetHat.z / zetNor;

    // bound deltas from 0 to 1: this means that if the query point is outside of boundaries (we have
    // no data there), we project that point on the boundary and give the interpolated value at that
    // point as a result.

    PetscReal deltaCsi
    =
    PetscMax
    (
        PetscMin
        (
            (
                csiHat.x * (px - cent[kL][jL][iL].x) +
                csiHat.y * (py - cent[kL][jL][iL].y) +
                csiHat.z * (pz - cent[kL][jL][iL].z)

            ) / csiNor,
            1.0
        ),
        0.0
    );

    PetscReal deltaEta
    =
    PetscMax
    (
        PetscMin
        (
            (
                etaHat.x * (px - cent[kL][jL][iL].x) +
                etaHat.y * (py - cent[kL][jL][iL].y) +
                etaHat.z * (pz - cent[kL][jL][iL].z)

            ) / etaNor,
            1.0
        ),
        0.0
    );

    PetscReal deltaZet
    =
```

```
    PetscMax
    (
        PetscMin
        (
            (
                zetHat.x * (px - cent[kL][jL][iL].x) +
                zetHat.y * (py - cent[kL][jL][iL].y) +
                zetHat.z * (pz - cent[kL][jL][iL].z)

            ) / zetNor,
            1.0
        ),
        0.0
    );

    // find interpolation weights
    Cmpnts v000 = nSet(v[kL][jL][iL]),
           v100 = nSet(v[kL][jL][iR]),
           v110 = nSet(v[kL][jR][iR]),
           v010 = nSet(v[kL][jR][iL]),
           v001 = nSet(v[kR][jL][iL]),
           v101 = nSet(v[kR][jL][iR]),
           v111 = nSet(v[kR][jR][iR]),
           v011 = nSet(v[kR][jR][iL]);

    Cmpnts c0, c1, c2, c3, c4, c5, c6, c7;

    c0.x = v000.x,
    c1.x = v100.x - v000.x,
    c2.x = v010.x - v000.x,
    c3.x = v001.x - v000.x,
    c4.x = v110.x - v010.x - v100.x + v000.x,
    c5.x = v011.x - v001.x - v010.x + v000.x,
    c6.x = v101.x - v001.x - v100.x + v000.x,
    c7.x = v111.x - v011.x - v101.x - v110.x + v100.x + v001.x + v010.x - v000.x;

    c0.y = v000.y,
    c1.y = v100.y - v000.y,
    c2.y = v010.y - v000.y,
    c3.y = v001.y - v000.y,
    c4.y = v110.y - v010.y - v100.y + v000.y,
    c5.y = v011.y - v001.y - v010.y + v000.y,
    c6.y = v101.y - v001.y - v100.y + v000.y,
    c7.y = v111.y - v011.y - v101.y - v110.y + v100.y + v001.y + v010.y - v000.y;

    c0.z = v000.z,
    c1.z = v100.z - v000.z,
    c2.z = v010.z - v000.z,
    c3.z = v001.z - v000.z,
    c4.z = v110.z - v010.z - v100.z + v000.z,
    c5.z = v011.z - v001.z - v010.z + v000.z,
    c6.z = v101.z - v001.z - v100.z + v000.z,
    c7.z = v111.z - v011.z - v101.z - v110.z + v100.z + v001.z + v010.z - v000.z;


    // interpolate
    result.x
    =
    c0.x +
    c1.x * deltaCsi +
    c2.x * deltaEta +
    c3.x * deltaZet +
    c4.x * deltaCsi * deltaEta +
    c5.x * deltaEta * deltaZet +
    c6.x * deltaZet * deltaCsi +
    c7.x * deltaCsi * deltaEta * deltaZet;

    result.y
    =
    c0.y +
    c1.y * deltaCsi +
    c2.y * deltaEta +
    c3.y * deltaZet +
    c4.y * deltaCsi * deltaEta +
    c5.y * deltaEta * deltaZet +
    c6.y * deltaZet * deltaCsi +
    c7.y * deltaCsi * deltaEta * deltaZet;

    result.z
    =
    c0.z +
    c1.z * deltaCsi +
    c2.z * deltaEta +
    c3.z * deltaZet +
    c4.z * deltaCsi * deltaEta +
    c5.z * deltaEta * deltaZet +
    c6.z * deltaZet * deltaCsi +
```

```
    c7.z * deltaCsi * deltaEta * deltaZet;

    return;
}


//*****************************************************************************************************//

inline void scalarPointB2SplineVolumeInterpolation(
    mesh_ *mesh,
    PetscReal px, PetscReal py, PetscReal pz, // Query point coordinates
    PetscInt ic, PetscInt jc, PetscInt kc,    // Closest cell center indices
    Cmpnts ***cent,                           // Cell center coordinates
    PetscReal ***v,                           // Scalar field
    PetscReal &result                         // Interpolated result
)
{
    // Performs quadratic B-spline interpolation using a 3x3x3 stencil centered at
    // (ic, jc, kc). Assumes ic, jc, kc are internal cells (not ghosts). Normalizes
    // weights near boundaries to handle incomplete stencils.

    DMDALocalInfo info = mesh->info;
    PetscInt xs = info.xs, xe = info.xs + info.xm;
    PetscInt ys = info.ys, ye = info.ys + info.ym;
    PetscInt zs = info.zs, ze = info.zs + info.zm;
    PetscInt mx = info.mx, my = info.my, mz = info.mz;
    PetscInt gxs = info.gxs, gxe = info.gxs + info.gxm;
    PetscInt gys = info.gys, gye = info.gys + info.gym;
    PetscInt gzs = info.gzs, gze = info.gzs + info.gzm;

    // Check that the center point is not in ghost cells
    if (ic < 1 || ic >= mx - 1 || jc < 1 || jc >= my - 1 || kc < 1 || kc >= mz - 1) {
        char error[512];
        sprintf(error, "Center indices (%ld, %ld, %ld) are in ghost cells or out of bounds\n", ic, jc, kc);
        fatalErrorInFunction("scalarPointLocalVolumeInterpolationQuad", error);
    }

    if(ic == 1 || ic == mx-2 || jc == 1 || jc == my-2 || kc == 1 || kc == mz-2)
    {
        scalarPointLocalVolumeInterpolation
        (
                mesh,
                px, py, pz,
                ic, jc, kc,
                cent,
                v,
                result
        );
    }
    else
    {
        // Define the 3x3x3 stencil bounds
        PetscInt iL = ic - 1, iR = ic + 1;
        PetscInt jL = jc - 1, jR = jc + 1;
        PetscInt kL = kc - 1, kR = kc + 1;

        // Compute cell sizes for normalization (assuming graded mesh)
        PetscReal dx = (cent[kR][jc][ic].x - cent[kL][jc][ic].x) / 2.0;
        PetscReal dy = (cent[kc][jc][iR].y - cent[kc][jc][iL].y) / 2.0;
        PetscReal dz = (cent[kc][jR][ic].z - cent[kc][jL][ic].z) / 2.0;

        // Normalized distances from the center cell (ic, jc, kc)
        PetscReal xd = (px - cent[kc][jc][ic].x) / dx; // [-1.5, 1.5] maps to splineB2 support
        PetscReal yd = (py - cent[kc][jc][ic].y) / dy;
        PetscReal zd = (pz - cent[kc][jc][ic].z) / dz;

        // Interpolation using splineB2 over 3x3x3 stencil
        result = 0.0;
        PetscReal weightSum = 0.0; // For normalization near boundaries

        for (PetscInt k = kL; k <= kR; k++)
        {
            for (PetscInt j = jL; j <= jR; j++)
            {
                for (PetscInt i = iL; i <= iR; i++)
                {
                    // Compute weights based on distance from center
                    PetscReal wx = splineB2(xd - (k - kc));
                    PetscReal wy = splineB2(yd - (i - ic));
                    PetscReal wz = splineB2(zd - (j - jc));
                    PetscReal weight = wx * wy * wz;

                    // Clamp indices to valid range (no extrapolation)
                    PetscInt ii = PetscMax(0, PetscMin(i, mx - 1));
                    PetscInt jj = PetscMax(0, PetscMin(j, my - 1));
                    PetscInt kk = PetscMax(0, PetscMin(k, mz - 1));
```

```
                    // Only contribute if within local domain (avoid ghosts)
                    if (ii >= gxs && ii < gxe && jj >= gys && jj < gye && kk >= gzs && kk < gze)
                    {
                        result += v[kk][jj][ii] * weight;
                        weightSum += weight;
                    }
                }
            }
        }

        // Normalize result to account for incomplete stencils near boundaries
        if (weightSum > 1e-10) {
            result /= weightSum;
        } else {
            result = v[kc][jc][ic]; // Fallback to nearest value if no valid weights
        }
    }

    return;
}


inline void vectorPointB2SplineVolumeInterpolation(
    mesh_ *mesh,
    PetscReal px, PetscReal py, PetscReal pz, // Query point coordinates
    PetscInt ic, PetscInt jc, PetscInt kc,    // Closest cell center indices
    Cmpnts ***cent,                           // Cell center coordinates
    Cmpnts ***v,                              // Vector field
    Cmpnts &result                            // Interpolated result (vector)
) {
    // Performs quadratic B-spline interpolation using a 3x3x3 stencil centered at
    // (ic, jc, kc) for a vector field. Assumes ic, jc, kc are internal cells (not ghosts).
    // Falls back to trilinear interpolation near boundaries. Normalizes weights to handle
    // incomplete stencils.

    DMDALocalInfo info = mesh->info;
    PetscInt xs = info.xs, xe = info.xs + info.xm;
    PetscInt ys = info.ys, ye = info.ys + info.ym;
    PetscInt zs = info.zs, ze = info.zs + info.zm;
    PetscInt mx = info.mx, my = info.my, mz = info.mz;
    PetscInt gxs = info.gxs, gxe = info.gxs + info.gxm;
    PetscInt gys = info.gys, gye = info.gys + info.gym;
    PetscInt gzs = info.gzs, gze = info.gzs + info.gzm;

    // Check that the center point is not in ghost cells
    if (ic < 1 || ic >= mx - 1 || jc < 1 || jc >= my - 1 || kc < 1 || kc >= mz - 1) {
        char error[512];
        sprintf(error, "Center indices (i=%ld, j=%ld, k=%ld) are in ghost cells or out of bounds\n", ic, jc, kc);
        fatalErrorInFunction("vectorPointB2SplineVolumeInterpolation", error);
    }

    // Near boundaries, fall back to trilinear interpolation
    if (ic == 1 || ic == mx - 2 || jc == 1 || jc == my - 2 || kc == 1 || kc == mz - 2)
    {
        vectorPointLocalVolumeInterpolation
        (
            mesh,
            px, py, pz,
            ic, jc, kc,
            cent,
            v,
            result
        );
    }
    else
    {
        // Define the 3x3x3 stencil bounds
        PetscInt iL = ic - 1, iR = ic + 1;
        PetscInt jL = jc - 1, jR = jc + 1;
        PetscInt kL = kc - 1, kR = kc + 1;

        // Compute cell sizes for normalization
        PetscReal dx = (cent[kR][jc][ic].x - cent[kL][jc][ic].x) / 2.0;
        PetscReal dy = (cent[kc][jc][iR].y - cent[kc][jc][iL].y) / 2.0;
        PetscReal dz = (cent[kc][jR][ic].z - cent[kc][jL][ic].z) / 2.0;

        // Normalized distances from the center cell (ic, jc, kc)
        PetscReal xd = (px - cent[kc][jc][ic].x) / dx; // [-1.5, 1.5] maps to splineB2 support
        PetscReal yd = (py - cent[kc][jc][ic].y) / dy;
        PetscReal zd = (pz - cent[kc][jc][ic].z) / dz;

        // Interpolation using splineB2 over 3x3x3 stencil
        result.x = 0.0; result.y = 0.0; result.z = 0.0;
        PetscReal weightSum = 0.0; // For normalization near boundaries

```

```
        for (PetscInt k = kL; k <= kR; k++)
        {
            for (PetscInt j = jL; j <= jR; j++)
            {
                for (PetscInt i = iL; i <= iR; i++)
                {
                    // Compute weights based on distance from center
                    PetscReal wx = splineB2(xd - (k - kc));
                    PetscReal wy = splineB2(yd - (i - ic));
                    PetscReal wz = splineB2(zd - (j - jc));
                    PetscReal weight = wx * wy * wz;

                    // Clamp indices to valid range (no extrapolation)
                    PetscInt ii = PetscMax(0, PetscMin(i, mx - 1));
                    PetscInt jj = PetscMax(0, PetscMin(j, my - 1));
                    PetscInt kk = PetscMax(0, PetscMin(k, mz - 1));

                    // Only contribute if within local domain (avoid ghosts)
                    if (ii >= gxs && ii < gxe && jj >= gys && jj < gye && kk >= gzs && kk < gze)
                    {
                        result.x += v[kk][jj][ii].x * weight;
                        result.y += v[kk][jj][ii].y * weight;
                        result.z += v[kk][jj][ii].z * weight;
                        weightSum += weight;
                    }
                }
            }
        }

        // PetscPrintf(PETSC_COMM_SELF, "Bspline for point %ld %ld %ld %lf %lf %lf weight sum = %lf\n", kc, jc, ic, xd, yd, zd, weightSum);

        // Normalize result to account for incomplete stencils near boundaries
        if (weightSum > 1e-10) {
            result.x /= weightSum;
            result.y /= weightSum;
            result.z /= weightSum;
        } else {
            // Fallback to nearest value if no valid weights
            result.x = v[kc][jc][ic].x;
            result.y = v[kc][jc][ic].y;
            result.z = v[kc][jc][ic].z;
        }
    }

    return;
}


//*****************************************************************************************************************//

// Dynamic array allocation for increasing array size: linked list
inline void initlist(list *ilist)
{

  ilist->head = PETSC_NULL;

  return;
}


//*****************************************************************************************************************//

// initialize a cell list to null pointer
inline void initCellList(cellList *ilist)
{

  ilist->head = PETSC_NULL;
  return;
}


//*****************************************************************************************************************//

// insert a node into a list and check if previously added
inline bool insertnode(list *ilist, PetscInt Node)
{

    node *_new;
    node *current;

    current = ilist->head;

    PetscBool Exist = PETSC_FALSE;

    while(current)
    {
```

```
        if (Node == current->Node)
        {
            Exist = PETSC_TRUE;
        }

        if (Exist) break;
        current = current->next;
    }

    if (!Exist)
    {
        PetscMalloc(sizeof(node), &_new);
        _new->next = ilist->head;
        _new->Node = Node;
        ilist->head = _new;
    }

    if(Exist)
        return false;
    else
        return true;
}


//*****************************************************************************************************************//

// insert a node into a list
inline void insertnode1(list *ilist, PetscInt Node)
{
    node *_new;

    PetscMalloc(sizeof(node), &_new);

    _new->next = ilist->head;

    _new->Node = Node;

    ilist->head = _new;

    return;
}


//*****************************************************************************************************************//

// insert a node into a list and check for previous existing node
inline bool insertCellNode(cellList *ilist, cellIds Node)
{

  cellNode *_new;
  cellNode *current;
  current = ilist->head;

  PetscBool Exist = PETSC_FALSE;
  while(current) {
    if (Node.i == current->Node.i
      && Node.j == current->Node.j
      && Node.k == current->Node.k)
       {
         Exist = PETSC_TRUE;
       }

    if (Exist) break;
    current = current->next;
  }

  if (!Exist) {
    PetscMalloc(sizeof(cellNode), &_new);
    _new->next = ilist->head;

    _new->Node.i = Node.i;
    _new->Node.j = Node.j;
    _new->Node.k = Node.k;

    ilist->head = _new;
  }

  if(Exist)
    return false;
  else
    return true;
}


//*****************************************************************************************************************//

// insert a node into a list and dont check if it previously exists
```

```
inline void insertCellNode1(cellList *ilist, cellIds Node)
{
  cellNode *_new;

  PetscMalloc(sizeof(cellNode), &_new);

  _new->next = ilist->head;

  _new->Node.i = Node.i;
  _new->Node.j = Node.j;
  _new->Node.k = Node.k;

  ilist->head = _new;

  return;
}

// detroy a list
inline void destroy(list *ilist)
{

  node *current;
  while (ilist->head) {
    current = ilist->head->next;
    PetscFree(ilist->head);
    ilist->head = current;
  }

  return;
}


//*****************************************************************************************************************//

// detroy a cell list
inline void destroyCellList(cellList *ilist)
{

  cellNode *current;
  while (ilist->head) {
    current = ilist->head->next;
    PetscFree(ilist->head);
    ilist->head = current;
  }

  return;
}


//*****************************************************************************************************************//
// inverse matrix functions


//*****************************************************************************************************************//

inline void reorderMatrix(PetscReal **A, PetscReal **B, PetscInt N, PetscInt pivot)
{
    PetscInt  maxrow=pivot;
    PetscInt  row,col;
    PetscReal temp;

    for (row=pivot+1;row<N;row++)
    {
        if ( fabs(A[row][pivot]) > fabs(A[maxrow][pivot]) ) maxrow = row;
    }


   if (fabs(A[maxrow][pivot])<1.e-11)
   {
        char error[512];
        sprintf(error, "matrix inverse reorder algorithm failed\n");
        fatalErrorInFunction("reorderMatrix",  error);
   }

    if (maxrow != pivot)
    {
        for (col=0; col<N; col++)
        {
            temp           = A[maxrow][col];
            A[maxrow][col] = A[pivot][col];
            A[pivot][col]  = temp;

            temp           = B[maxrow][col];
            B[maxrow][col] = B[pivot][col];
            B[pivot][col]  = temp;
        }
    }
```

```
    return;
}

//*****************************************************************************************************//

inline void elimJordanAlg(PetscReal **A, PetscReal **B, PetscInt N, PetscInt pivot)
{

    PetscInt row,col;

    for (col=pivot+1; col<N; col++)
    {
        A[pivot][col] = A[pivot][col]/A[pivot][pivot];
    }

    for (col=0; col<N; col++)
    {
        B[pivot][col] = B[pivot][col]/A[pivot][pivot];
    }

    A[pivot][pivot]=1.;

    for (row=0; row<N; row++)
    {
        if (row == pivot) continue;

        for (col=pivot+1; col<N; col++) A[row][col] -= A[row][pivot]*A[pivot][col];
        for (col=0; col<N; col++)       B[row][col] -= A[row][pivot]*B[pivot][col];
        A[row][pivot]=0.;
    }

    return;
}


//*****************************************************************************************************//
// inverse of generic matrix of any order
inline void inverseMatrix(PetscReal **A, PetscReal **inv_A, PetscInt size)
{
    PetscInt i,j,k;

    for (i=0; i<size; i++)
    {
        for (j=0; j<size; j++)
        {
            inv_A[i][j]=0.;
        }
    }

    for (i=0; i<size; i++)
    {
        inv_A[i][i]=1.0;
    }

    PetscInt pivot=0;

    while (pivot < size)
    {
        reorderMatrix(A, inv_A, size, pivot);
        elimJordanAlg(A, inv_A, size, pivot);

        pivot++;
    }

    return;
}


//*****************************************************************************************************//
//invert a 4 by 4 matrix  -- faster
inline void inv_4by4(PetscReal **A, PetscReal **inv_A, PetscInt size)
{

    if( size!= 4 )
    {
        char error[512];
        sprintf(error, "expected array of size 4\n");
        fatalErrorInFunction("inv_4by4",  error);
    }

    PetscReal A2323 = A[2][2] * A[3][3] - A[2][3] * A[3][2] ;
    PetscReal A1323 = A[2][1] * A[3][3] - A[2][3] * A[3][1] ;
    PetscReal A1223 = A[2][1] * A[3][2] - A[2][2] * A[3][1] ;
    PetscReal A0323 = A[2][0] * A[3][3] - A[2][3] * A[3][0] ;
    PetscReal A0223 = A[2][0] * A[3][2] - A[2][2] * A[3][0] ;
```

```
    PetscReal A0123 = A[2][0] * A[3][1] - A[2][1] * A[3][0] ;
    PetscReal A2313 = A[1][2] * A[3][3] - A[1][3] * A[3][2] ;
    PetscReal A1313 = A[1][1] * A[3][3] - A[1][3] * A[3][1] ;
    PetscReal A1213 = A[1][1] * A[3][2] - A[1][2] * A[3][1] ;
    PetscReal A2312 = A[1][2] * A[2][3] - A[1][3] * A[2][2] ;
    PetscReal A1312 = A[1][1] * A[2][3] - A[1][3] * A[2][1] ;
    PetscReal A1212 = A[1][1] * A[2][2] - A[1][2] * A[2][1] ;
    PetscReal A0313 = A[1][0] * A[3][3] - A[1][3] * A[3][0] ;
    PetscReal A0213 = A[1][0] * A[3][2] - A[1][2] * A[3][0] ;
    PetscReal A0312 = A[1][0] * A[2][3] - A[1][3] * A[2][0] ;
    PetscReal A0212 = A[1][0] * A[2][2] - A[1][2] * A[2][0] ;
    PetscReal A0113 = A[1][0] * A[3][1] - A[1][1] * A[3][0] ;
    PetscReal A0112 = A[1][0] * A[2][1] - A[1][1] * A[2][0] ;

    PetscReal det = A[0][0] * ( A[1][1] * A2323 - A[1][2] * A1323 + A[1][3] * A1223 )
        - A[0][1] * ( A[1][0] * A2323 - A[1][2] * A0323 + A[1][3] * A0223 )
        + A[0][2] * ( A[1][0] * A1323 - A[1][1] * A0323 + A[1][3] * A0123 )
        - A[0][3] * ( A[1][0] * A1223 - A[1][1] * A0223 + A[1][2] * A0123 ) ;

    det = 1 / det;

    inv_A[0][0] = det *   ( A[1][1] * A2323 - A[1][2] * A1323 + A[1][3] * A1223 );
    inv_A[0][1] = det * - ( A[0][1] * A2323 - A[0][2] * A1323 + A[0][3] * A1223 );
    inv_A[0][2] = det *   ( A[0][1] * A2313 - A[0][2] * A1313 + A[0][3] * A1213 );
    inv_A[0][3] = det * - ( A[0][1] * A2312 - A[0][2] * A1312 + A[0][3] * A1212 );
    inv_A[1][0] = det * - ( A[1][0] * A2323 - A[1][2] * A0323 + A[1][3] * A0223 );
    inv_A[1][1] = det *   ( A[0][0] * A2323 - A[0][2] * A0323 + A[0][3] * A0223 );
    inv_A[1][2] = det * - ( A[0][0] * A2313 - A[0][2] * A0313 + A[0][3] * A0213 );
    inv_A[1][3] = det *   ( A[0][0] * A2312 - A[0][2] * A0312 + A[0][3] * A0212 );
    inv_A[2][0] = det *   ( A[1][0] * A1323 - A[1][1] * A0323 + A[1][3] * A0123 );
    inv_A[2][1] = det * - ( A[0][0] * A1323 - A[0][1] * A0323 + A[0][3] * A0123 );
    inv_A[2][2] = det *   ( A[0][0] * A1313 - A[0][1] * A0313 + A[0][3] * A0113 );
    inv_A[2][3] = det * - ( A[0][0] * A1312 - A[0][1] * A0312 + A[0][3] * A0112 );
    inv_A[3][0] = det * - ( A[1][0] * A1223 - A[1][1] * A0223 + A[1][2] * A0123 );
    inv_A[3][1] = det *   ( A[0][0] * A1223 - A[0][1] * A0223 + A[0][2] * A0123 );
    inv_A[3][2] = det * - ( A[0][0] * A1213 - A[0][1] * A0213 + A[0][2] * A0113 );
    inv_A[3][3] = det *   ( A[0][0] * A1212 - A[0][1] * A0212 + A[0][2] * A0112 );

    return;
}


//*****************************************************************************************************//
//invert a 3 by 3 matrix
inline void inv_3by3(PetscReal **A, PetscReal **inv_A, PetscInt size)
{
    if( size!= 3 )
    {
        char error[512];
        sprintf(error, "expected array of size 3\n");
        fatalErrorInFunction("inv_3by3",  error);
    }

    PetscReal det = A[0][0] * (A[1][1] * A[2][2] - A[2][1] * A[1][2]) -
                    A[0][1] * (A[1][0] * A[2][2] - A[1][2] * A[2][0]) +
                    A[0][2] * (A[1][0] * A[2][1] - A[1][1] * A[2][0]);

    det = 1 / det;

    // Compute elements of the inverse matrix
    inv_A[0][0] = (A[1][1] * A[2][2] - A[2][1] * A[1][2]) * det;
    inv_A[0][1] = (A[0][2] * A[2][1] - A[0][1] * A[2][2]) * det;
    inv_A[0][2] = (A[0][1] * A[1][2] - A[0][2] * A[1][1]) * det;
    inv_A[1][0] = (A[1][2] * A[2][0] - A[1][0] * A[2][2]) * det;
    inv_A[1][1] = (A[0][0] * A[2][2] - A[0][2] * A[2][0]) * det;
    inv_A[1][2] = (A[1][0] * A[0][2] - A[0][0] * A[1][2]) * det;
    inv_A[2][0] = (A[1][0] * A[2][1] - A[2][0] * A[1][1]) * det;
    inv_A[2][1] = (A[2][0] * A[0][1] - A[0][0] * A[2][1]) * det;
    inv_A[2][2] = (A[0][0] * A[1][1] - A[1][0] * A[0][1]) * det;

    return;
}


//*****************************************************************************************************//

inline void reorder_20(PetscReal A[][20], PetscReal B[][20],PetscInt N,PetscInt pivot)
{
    PetscInt maxrow=pivot;
    PetscInt row,col;
    PetscReal temp;
    for (row=pivot+1;row<N;row++)
        if (fabs(A[row][pivot])>fabs(A[maxrow][pivot])) maxrow=row;

//    if (fabs(A[maxrow][pivot])<1.e-11)
//        exit(0);
    if (maxrow!=pivot)
```

```
        for (col=0;col<N;col++){
            temp= A[maxrow][col];
            A[maxrow][col]=A[pivot][col];
            A[pivot][col]=temp;
            temp= B[maxrow][col];
            B[maxrow][col]=B[pivot][col];
            B[pivot][col]=temp;
        }
    return;
}


//*****************************************************************************************************//

inline void elim_jordan_20(PetscReal A[][20], PetscReal B[][20],PetscInt N,PetscInt pivot){
    PetscInt row,col;
    for (col=pivot+1;col<N;col++) A[pivot][col]=A[pivot][col]/A[pivot][pivot];
    for (col=0;col<N;col++)
        B[pivot][col]=B[pivot][col]/A[pivot][pivot];
    A[pivot][pivot]=1.;
    for (row=0;row<N;row++){
        if (row==pivot) continue;
        for (col=pivot+1;col<N;col++) A[row][col] -= A[row][pivot]*A[pivot][col];
        for (col=0;col<N;col++)       B[row][col] -= A[row][pivot]*B[pivot][col];
        A[row][pivot]=0.;
    }

    return;
}


//*****************************************************************************************************//

inline void inv_20(PetscReal A[20][20], PetscReal inv_A[][20],PetscInt N){
    PetscInt i,j,k;
    for (i=0;i<N;i++)
        for (j=0;j<N;j++)
            inv_A[i][j]=0.;
    for (i=0;i<N;i++) inv_A[i][i]=1.;
    PetscInt pivot=0;
    while (pivot<N){
        reorder_20(A, inv_A, N, pivot);
        elim_jordan_20(A,inv_A,N,pivot);
        pivot++;
    }

    return;
}


//*****************************************************************************************************//

inline void reorder_10(PetscReal A[][10], PetscReal B[][10],PetscInt N,PetscInt pivot){
    PetscInt maxrow=pivot;
    PetscInt row,col;
    PetscReal temp;
    for (row=pivot+1;row<N;row++)
        if (fabs(A[row][pivot])>fabs(A[maxrow][pivot])) maxrow=row;

//    if (fabs(A[maxrow][pivot])<1.e-11)
//        exit(0);
    if (maxrow!=pivot)
        for (col=0;col<N;col++){
            temp= A[maxrow][col];
            A[maxrow][col]=A[pivot][col];
            A[pivot][col]=temp;
            temp= B[maxrow][col];
            B[maxrow][col]=B[pivot][col];
            B[pivot][col]=temp;
        }
    return;
}


//*****************************************************************************************************//

inline void elim_jordan_10(PetscReal A[][10], PetscReal B[][10],PetscInt N,PetscInt pivot){
    PetscInt row,col;
    for (col=pivot+1;col<N;col++) A[pivot][col]=A[pivot][col]/A[pivot][pivot];
    for (col=0;col<N;col++)
        B[pivot][col]=B[pivot][col]/A[pivot][pivot];
    A[pivot][pivot]=1.;
    for (row=0;row<N;row++){
        if (row==pivot) continue;
        for (col=pivot+1;col<N;col++) A[row][col] -= A[row][pivot]*A[pivot][col];
        for (col=0;col<N;col++)       B[row][col] -= A[row][pivot]*B[pivot][col];
        A[row][pivot]=0.;
```

```
    }

    return;
}


//*****************************************************************************************************//

inline void inv_10(PetscReal A[10][10], PetscReal inv_A[][10],PetscInt N){
    PetscInt i,j,k;
    for (i=0;i<N;i++)
        for (j=0;j<N;j++)
            inv_A[i][j]=0.;
    for (i=0;i<N;i++) inv_A[i][i]=1.;
    PetscInt pivot=0;
    while (pivot<N){
        reorder_10(A, inv_A, N, pivot);
        elim_jordan_10(A,inv_A,N,pivot);
        pivot++;
    }

    return;
}


//*****************************************************************************************************//

inline void mult_mats3_lin(PetscReal **inv_A,PetscReal **B,PetscInt nsupport,PetscReal *PHI){

  for (PetscInt i=0;i<nsupport;i++) {
    PHI[i]=0.;
    PetscReal temp=0.;
    for (PetscInt k=0;k<4;k++) temp += inv_A[0][k]*B[k][i];
    PHI[i] = temp;
  }

  return;
}


#endif
```

5.45 /zfs60/nobackup/stipa/software/TOSCA/src/include/io.h File Reference

Contains i/o operations function headers.

Classes

- struct io_

  Struct defining io settings (simulation checkpointing)

Functions

- void **fatalErrorInFunction** (const char ∗functionName, const char ∗errorMsg)

  Calls fatal error and exits.
- void **warningInFunction** (const char ∗functionName, const char ∗wrngMsg)

  Calls warning.
- word **thisCaseName** ()

  Retrieves this case name.
- PetscErrorCode **getTimeList** (const char ∗dataLoc, std::vector< PetscReal > &timeSeries, PetscInt &ntimes)

  Returns the time list (from folder names) and size contained in a folder.
- PetscErrorCode **getFileList** (const char ∗dataLoc, std::vector< word > &fileSeries, PetscInt &nfiles)

Returns the file list (from folder names) and number of files contained in a folder.

- PetscInt **foundInString** (const char ∗str, word keyword)

  Finds a word inside a string.
- PetscInt **file_exist** (const char ∗str)

  Checks if file exists.
- PetscInt **dir_exist** (const char ∗str)

  Checks if directory exists.
- PetscInt **count_files** (const char ∗path)

  Count number of files.
- void **createDir** (MPI_Comm comm, const char ∗path)

  Creat directory (remove stuff it exists)
- void **createDirNoRemove** (MPI_Comm comm, const char ∗path)

  Creat directory (leave stuff it exists)
- void **remove_dir** (MPI_Comm comm, const char ∗path2dir)

  Removes directory.
- void **remove_subdirs** (MPI_Comm comm, const char ∗path2dir)

  Removes all subdirectory of a given directory.
- void **remove_subdirs_except** (MPI_Comm comm, const char ∗path2dir, const word name)

  Removes all subdirectory of a given directory except the name provided.
- void **remove_subdirs_except_keep_n** (MPI_Comm comm, const char ∗path2dir, const word name, PetscInt nKeep)

  Removes all subdirectory except the latest n times inside a given directory and the name provided.
- void **remove_subdirs_except2** (MPI_Comm comm, const char ∗path2dir, const word name1, const word name2)

  Removes all subdirectory of a given directory except the 2 names provided.
- void **remove_subdirs_except3** (MPI_Comm comm, const char ∗path2dir, const word name1, const word name2, const word name3)

  Removes all subdirectory of a given directory except the 3 names provided.
- void **remove_subdirs_except4** (MPI_Comm comm, const char ∗path2dir, const word name1, const word name2, const word name3, const word name4)

  Removes all subdirectory of a given directory except the 4 names provided.
- void **remove_subdirs_except4_keep_n** (MPI_Comm comm, const char ∗path2dir, const word name1, const word name2, const word name3, const word name4, PetscInt nKeep)

  Removes all subdirectory except the latest n times inside a given directory and the 4 names provided.
- void **SetWriteDir** (mesh_ ∗mesh, const word str)

  Set write directory for a given mesh.
- void **writeBinaryField** (mesh_ ∗mesh, Vec &V, char ∗file)

  Write binary vector file.
- PetscErrorCode **InitializeIO** (io_ ∗io)

  Initializes io data structure.
- PetscErrorCode **ReadIO** (domain_ ∗domain)

  Re-read IO write parameters.
- PetscErrorCode **UpdateInput** (io_ ∗io, word &modified)

  Called by ReadIO, triggers updates in each domain.
- PetscErrorCode **readFields** (domain_ ∗domain, PetscReal timeValue)

  read and save the different fields stored in the fields/meshName/time folder
- word **getTimeName** (clock_ ∗clock)

  Get time name in string format with required digits.
- word **getStartTimeName** (clock_ ∗clock)

  Get start time name in string format with required digits.
- word **getArbitraryTimeName** (clock_ ∗clock, double timeValue)

Get time value in string format with required digits.

- PetscErrorCode **setRunTimeWrite** (domain_ *domain)

  Sets the runTimeWrite flag and creates initial output directory.
- PetscErrorCode **initializeTimeDir** (io_ *io)

  Creates the checkpoint directory with a trailing _tmp (indicates that it is not finalized)
- PetscErrorCode **finalizeTimeDir** (io_ *io)

  Renames the temporary time directory to the final name (without _tmp) and deletes prev chekpoints if purge is active.
- PetscErrorCode **writeFields** (io_ *io)

  Write output fields.
- PetscErrorCode **readDictDouble** (const char *dictName, const char *keyword, PetscReal *value)

  Read mandatory PetscReal from dictionary.
- PetscErrorCode **readDictVector** (const char *dictName, const char *keyword, Cmpnts *value)

  Read mandatory vector from dictionary.
- PetscErrorCode **readDictVector2D** (const char *dictName, const char *keyword, Cpt2D *value)

  Read mandatory 2D vector from dictionary.
- PetscErrorCode **readDictInt** (const char *dictName, const char *keyword, PetscInt *value)

  Read mandatory PetscInt from dictionary.
- PetscErrorCode **readDictWord** (const char *dictName, const char *keyword, word *value)

  Read mandatory word from dictionary.
- PetscErrorCode **readDictWordAndDouble** (const char *dictName, const char *keyword, word *value1, PetscReal *value2)

  Read mandatory word and PetscReal from dictionary.
- PetscErrorCode **readDictWordAndVector** (const char *dictName, const char *keyword, word *value1, Cmpnts *value2)

  Read mandatory word and vector from dictionary.
- PetscErrorCode **readSubDictDouble** (const char *dictName, const char *subdict, const char *keyword, PetscReal *value)

  Read mandatory PetscReal from sub-dictionary.
- PetscErrorCode **readSubDictInt** (const char *dictName, const char *subdict, const char *keyword, PetscInt *value)

  Read mandatory PetscInt from sub-dictionary.
- PetscErrorCode **readSubDictVector** (const char *dictName, const char *subdict, const char *keyword, Cmpnts *value)

  Read mandatory vector from sub-dictionary.
- PetscErrorCode **readSubDictWord** (const char *dictName, const char *subdict, const char *keyword, word *value)

  Read mandatory word from sub-dictionary.
- PetscErrorCode **readSubDictIntArray** (const char *dictName, const char *subdict, const char *keyword, labelList &value)

  Read mandatory array of PetscInt from sub-dictionary.
- std::string * **readSubDictWordArray** (const char *dictName, const char *subdict, const char *keyword, PetscInt numW)
- word **trim** (const word &str)

  Trims a string removing pre and trail spaces.
- bool **isNumber** (const word &str)

  Check if is a number.

5.45.1 Detailed Description

Contains i/o operations function headers.

5.46 io.h

Go to the documentation of this file.

```
#ifndef _IO_H_
#define _IO_H_

struct io_
{
    word      intervalType;
    PetscReal timeInterval;

    PetscInt  runTimeWrite;

    PetscInt  purgeWrite;

    PetscInt  averaging;
    PetscReal avgPrd;
    PetscReal avgStartTime;
    PetscInt  phaseAveraging;
    PetscReal phAvgPrd;
    PetscReal phAvgStartTime;
    PetscInt  keBudgets;
    PetscInt  writePForce;

    PetscInt  avgWeight;
    PetscInt  pAvgWeight;
    PetscInt  keAvgWeight;

    PetscInt  qCrit;
    PetscInt  l2Crit;
    PetscInt  vgtQg;
    PetscInt  vgtRg;
    PetscInt  vgtQs;
    PetscInt  vgtRs;
    PetscInt  vgtQr;
    PetscInt  windFarmForce;
    PetscInt  sources;
    PetscInt  buoyancy;
    PetscInt  continuity;

    // runtime modifiable
    word      lastModification;

    // catalyst interface
    PetscReal     timeIntervalCatalyst;
    PetscReal     startTimeCatalyst;
    word          outputTypeCatalyst;
    word          ioTypeCatalyst;
    word          scriptNameCatalyst;

    // access database
    access_   *access;
};


#endif

void fatalErrorInFunction(const char* functionName, const char* errorMsg);

void warningInFunction(const char* functionName, const char* wrngMsg);

word thisCaseName();

PetscErrorCode getTimeList(const char* dataLoc, std::vector<PetscReal> &timeSeries, PetscInt &ntimes);

PetscErrorCode getFileList(const char* dataLoc, std::vector<word> &fileSeries, PetscInt &nfiles);

PetscInt foundInString(const char *str, word keyword);

PetscInt file_exist(const char *str);

PetscInt dir_exist(const char *str);

PetscInt count_files(const char* path);

void createDir(MPI_Comm comm, const char* path);

void createDirNoRemove(MPI_Comm comm, const char* path);

void remove_dir(MPI_Comm comm, const char *path2dir);

void remove_subdirs(MPI_Comm comm, const char *path2dir);

```

```
void remove_subdirs_except(MPI_Comm comm, const char *path2dir, const word name);

void remove_subdirs_except_keep_n(MPI_Comm comm, const char *path2dir, const word name, PetscInt nKeep);

void remove_subdirs_except2(MPI_Comm comm, const char *path2dir, const word name1, const word name2);

void remove_subdirs_except3(MPI_Comm comm, const char *path2dir, const word name1, const word name2, const word name3);

void remove_subdirs_except4(MPI_Comm comm, const char *path2dir, const word name1, const word name2, const word name3, const word name4);

void remove_subdirs_except4_keep_n(MPI_Comm comm, const char *path2dir, const word name1, const word name2, const word name3, const word name4, PetscInt nKeep);

void SetWriteDir(mesh_ *mesh, const word str);

void writeBinaryField(mesh_ *mesh, Vec &V, char *file);

PetscErrorCode InitializeIO(io_ *io);

PetscErrorCode ReadIO(domain_ *domain);

PetscErrorCode UpdateInput(io_ *io, word &modified);

PetscErrorCode readFields(domain_ *domain, PetscReal timeValue);

word getTimeName(clock_ *clock);

word getStartTimeName(clock_ *clock);

word getArbitraryTimeName(clock_ *clock, double timeValue);

PetscErrorCode setRunTimeWrite(domain_ *domain);

PetscErrorCode initializeTimeDir(io_ *io);

PetscErrorCode finalizeTimeDir(io_ *io);

PetscErrorCode writeFields(io_ *io);

PetscErrorCode readDictDouble(const char *dictName, const char *keyword, PetscReal *value);

PetscErrorCode readDictVector(const char *dictName, const char *keyword, Cmpnts *value);

PetscErrorCode readDictVector2D(const char *dictName, const char *keyword, Cpt2D *value);

PetscErrorCode readDictInt(const char *dictName, const char *keyword, PetscInt *value);

PetscErrorCode readDictWord(const char *dictName, const char *keyword, word *value);

PetscErrorCode readDictWordAndDouble(const char *dictName, const char *keyword, word *value1, PetscReal *value2);

PetscErrorCode readDictWordAndVector(const char *dictName, const char *keyword, word *value1, Cmpnts *value2);

PetscErrorCode readSubDictDouble(const char *dictName, const char *subdict, const char *keyword, PetscReal *value);

PetscErrorCode readSubDictInt(const char *dictName, const char *subdict, const char *keyword, PetscInt *value);

PetscErrorCode readSubDictVector(const char *dictName, const char *subdict, const char *keyword, Cmpnts *value);

PetscErrorCode readSubDictWord(const char *dictName, const char *subdict, const char *keyword, word *value);

PetscErrorCode readSubDictIntArray(const char *dictName, const char *subdict, const char *keyword, labelList &value);

std::string* readSubDictWordArray(const char *dictName, const char *subdict, const char *keyword, PetscInt numW);

word trim(const word& str);

bool isNumber(const word& str);
```

5.47 les.h

```
```

```
#ifndef _LES_H_
#define _LES_H_

typedef enum {
    SMAGORINSKY,
    STABILITY_BASED,
    DSM,
    DLASI,
    DLASD,
    DPASD,
    AMD,
    VREMAN,
    BV,
    BAMD
} LesModel;

struct les_
{
    LesModel      model;
    PetscReal     maxCs;
    PetscReal     amdCs;
    PetscInt      turbStat;

    // Dynamic Smagorisnky
    Vec           lSx, lSy, lSz, lS;
    Vec           lLM, lMM;
    Vec           lQN, lNN;
    Vec           lLM_old, lMM_old;
    Vec           lQN_old, lNN_old;
    Vec           lCs;

    // Scale dependent model
    Vec           lN;
    Vec           lCh;
    Vec           L;

    // kEquation

    // general
    Vec           lNu_t;
    Vec           lKsgs;

    //eddy diffusivity parameters
    Vec           lk_t;
    Vec           lCsk;
    Vec           ldTheta;
    Vec           lKX, lXX;
    Vec           lPY, lYY;
    Vec           lKX_old, lXX_old;
    Vec           lPY_old, lYY_old;

    Vec           lTau;
    Vec           lQ;

    // initial nut field
    word          initFieldType;

    // access
    access_       *access;

};

#endif

PetscErrorCode InitializeLES(les_ *les);

PetscErrorCode UpdateCs(les_ *les);

PetscErrorCode UpdateNut(les_ *les);

PetscErrorCode ComputeTurbulenceStatistics(les_ *les);

PetscErrorCode updateLESStructuralModelCartesianForm(les_ *les);

PetscErrorCode updateLESStructuralModelContravariantForm(les_ *les);
```

5.48 /zfs60/nobackup/stipa/software/TOSCA/src/include/lesScalar.h File Reference

LES model header file for scalar quantities.

Functions

- PetscErrorCode **InitializeLESScalar** (les_ *les)
- PetscErrorCode UpdateCsk (les_ *les)
- PetscErrorCode **UpdatekT** (les_ *les)
- PetscErrorCode **updateLESScalarStructuralModel** (les_ *les)

5.48.1 Detailed Description

LES model header file for scalar quantities.

5.48.2 Function Documentation

5.48.2.1 UpdateCsk()

```
PetscErrorCode UpdateCsk (
            les_ * les)
```

isOnCornerCellCenters(I, J, K, mesh->info) <- use also ghost? Not for now

isOnCornerCellCenters(I, J, K, mesh->info) <- use also ghost? Not for now

5.49 lesScalar.h

Go to the documentation of this file.

```
00001
00003
00004 PetscErrorCode InitializeLESScalar(les_ *les);
00005
00006 PetscErrorCode UpdateCsk (les_ *les);
00007
00008 PetscErrorCode UpdatekT(les_ *les);
00009
00010 PetscErrorCode updateLESScalarStructuralModel(les_ *les);
```

5.50 /zfs60/nobackup/stipa/software/TOSCA/src/include/mesh.h File Reference

Mesh header file.

```
#include "wallmodel.h"
#include "boundary.h"
```

Classes

- struct boundingBox

  Structure defining the domain bounding box (easy access to xmin, xmax, ymin, ymax, zmin, zmax)
- struct mesh_

  mesh structure storing mesh and curvilinear coordinates

Functions

- PetscErrorCode **InitializeMesh** (mesh_ *mesh)

  Initialize mesh.
- PetscErrorCode **SetDistributedArrays** (mesh_ *mesh)

  Set distributed arrays.
- PetscErrorCode **SetMeshMetrics** (mesh_ *mesh)

  Set curvilinear coordinates metrics.
- PetscErrorCode **SetBoundingBox** (mesh_ *mesh)

  Set bounding box.
- PetscErrorCode **ghostnodesCellcenter** (mesh_ *mesh)

  Find the cell center for the ghost nodes.
- PetscErrorCode **DeformMeshBasedOnBLDisp** (mesh_ *mesh)

  Deform the mesh according to prescribed BL Disp.

5.50.1 Detailed Description

Mesh header file.

5.51 mesh.h

Go to the documentation of this file.

```
00001
00003
00004 #ifndef _MESH_H_
00005 #define _MESH_H_
00006
00007 #include "wallmodel.h"
00008 #include "boundary.h"
00009
00011 typedef struct
00012 {
00013     PetscReal      xmin,                        // domain bounding coordinates in meters
00014                    xmax,
00015                    ymin,
00016                    ymax,
00017                    zmin,
00018                    zmax;
00019     PetscReal      Lx,                          // domain length in meters in x,y,z directions
00020                    Ly,
00021                    Lz;
00022 } boundingBox;
00023
00025 struct mesh_
00026 {
00027     word           meshName;
00028     word           meshFileType;
00029
00030     boundingBox    bounds;
00031     PetscReal      grndLevel;
00032
00033     PetscInt       IM, JM, KM;
00034
00035     // distributed arrays
00036     DM             da;
00037     DM             fda;
00038     DM             sda;
00039     DM             tda;
00040     DMDALocalInfo  info;
00041
00042     // metrics
00043     Vec            Cent, lCent;
00044     Vec            lCsi, lEta, lZet;
00045     Vec            lAj;
00046     Vec            lICsi, lIEta, lIZet;
00047     Vec            lIAj;
00048     Vec            lJCsi, lJEta, lJZet;
```

```
    Vec            lJAj;
    Vec            lKCsi, lKEta, lKZet;
    Vec            lKAj;

    // IBM markup
    Vec            Nvert, Nvert_o;
    Vec            lNvert, lNvert_o;
    Vec            meshTag, lmeshTag;
    // periodic connectivity
    PetscInt       i_periodic, ii_periodic;
    PetscInt       j_periodic, jj_periodic;
    PetscInt       k_periodic, kk_periodic;

    // boundary conditions
    scalarBC       boundaryNut;
    scalarBC       boundaryT;
    vectorBC       boundaryU;

    // special inflow boundary conditions
    inletFunctions inletF;

    // limiters and numerical fields
    Vec            fluxLimiter;

    // this mesh communicator
    MPI_Comm       MESH_COMM;

    // access database
    access_        *access;

};

#endif

// ==========================================================================
// FUNCTIONS
// ==========================================================================

PetscErrorCode InitializeMesh(mesh_ *mesh);

PetscErrorCode SetDistributedArrays(mesh_ *mesh);

PetscErrorCode SetMeshMetrics(mesh_ *mesh);

PetscErrorCode SetBoundingBox(mesh_ *mesh);

PetscErrorCode ghostnodesCellcenter(mesh_ *mesh);

PetscErrorCode DeformMeshBasedOnBLDisp(mesh_ *mesh);
```

5.52 /zfs60/nobackup/stipa/software/TOSCA/src/include/objects.h File Reference

Contains forward object declarations.

5.52.1 Detailed Description

Contains forward object declarations.

5.53 objects.h

Go to the documentation of this file.

```


struct access_;

struct flags_;

```

```
struct domain_;

struct mesh_;

struct io_;

struct ueqn_;

struct peqn_;

struct teqn_;

struct les_;

struct farm_;

struct abl_;

struct acquisition_;

struct ibm_;

struct overset_;

struct precursor_;
```

5.54 /zfs60/nobackup/stipa/software/TOSCA/src/include/overset.h File Reference

Definitions for overset objects and functions for two–way coupling.

```
#include <unordered_map>
#include <map>
```

Classes

- struct HoleObject
- struct Acell

  Overset acceptor cell (used for both overset and base mesh acceptor cells)
- struct Dcell

  Overset donor cell (contains donor cell indices, owner, and distance)
- struct oversetMotion

  Overset motion structure (for dynamic overset motion)
- struct overset_

  Main overset structure.
  This structure holds the parameters, connectivity, and interpolation data for both the overset (child) mesh and the base (background) mesh update.
- struct BinIndex
- struct std::hash< BinIndex >

Functions

- PetscErrorCode **InitializeOverset** (domain_ *dm)

  Initialize overset coupling.
- PetscErrorCode **SetInitialFieldsOverset** (PetscInt d, domain_ *domain)
- PetscErrorCode **UpdateOversetInterpolation** (domain_ *domain)
- PetscErrorCode **UpdateDomainInterpolation** (PetscInt d, domain_ *domain, PetscInt level)

  Update overset interpolation for all domains.

- PetscErrorCode **findClosestDomainDonors** (PetscInt d, domain_ *domain, PetscInt level, const std::vector< HoleObject > &holeObjects)
- PetscErrorCode **findAcceptorCells** (PetscInt d, domain_ *domain, PetscInt level, const std::vector< HoleObject > &holeObjects)
- PetscErrorCode **readOversetProperties** (overset_ *os)
- PetscErrorCode **readBlankingIBMObject** (overset_ *os, domain_ *domain, char *holeObjectName, const std::vector< HoleObject > &holeObjects)
- PetscErrorCode **readHoleObjects** (std::vector< HoleObject > &holeObjects, PetscInt numHoleObjects)
- PetscErrorCode **FindHoleObject** (const std::vector< HoleObject > &holeObjects, PetscInt parentId, PetscInt childId, char **holeObjectName)
- PetscErrorCode **updateAcceptorCoordinates** (overset_ *os)
- PetscErrorCode **createAcceptorCellOverset** (overset_ *os)
- PetscErrorCode **createAcceptorCellBackground** (overset_ *os, PetscInt donorMeshId)

  Create the list of background acceptor cells.
- PetscErrorCode **findClosestDonorC2P** (mesh_ *meshDonor, mesh_ *meshAcceptor, PetscInt donorId)
- PetscErrorCode **findClosestDonorP2C** (mesh_ *meshDonor, mesh_ *meshAcceptor)
- PetscErrorCode **interpolateACellTrilinearP2C** (mesh_ *donorMesh, mesh_ *acceptorMesh)
- PetscErrorCode **interpolateACellTrilinearC2P** (mesh_ *meshD, mesh_ *meshA, PetscInt donorId)
- PetscErrorCode **oversetContravariantBC** (mesh_ *mesh, PetscInt i, PetscInt j, PetscInt k, Cmpnts ucart, PetscInt face)
- PetscErrorCode **setOversetBC** (mesh_ *meshA)
- PetscErrorCode **setBackgroundBC** (mesh_ *meshA)
- PetscErrorCode **acell1Dcell0Connectivity** (mesh_ *meshP, mesh_ *mesh)
- PetscErrorCode **getLSWeights** (mesh_ *meshP, mesh_ *mesh)
- PetscErrorCode **getLSWeights_2nd** (mesh_ *meshP, mesh_ *mesh)
- PetscErrorCode **getLSWeights_3rd** (mesh_ *meshP, mesh_ *mesh)
- PetscErrorCode **interpolateACellInvD** (mesh_ *meshP, mesh_ *mesh)
- PetscErrorCode **interpolateACellLS** (mesh_ *meshP, mesh_ *mesh)
- PetscErrorCode **updateAcellCoordinates** (domain_ *domain)
- PetscErrorCode **updateIntersectingProcessors** (domain_ *domain)
- PetscErrorCode **updateDonorCells** (domain_ *domain)
- PetscErrorCode **oversetMeshTranslation** (overset_ *os)
- void **sum_struct_Acell** (void *in, void *inout, int *len, MPI_Datatype *type)
- void **defineStruct_Acell** (MPI_Datatype *tstype)
- PetscErrorCode **computeOversetIBMElementNormal** (ibm_ *ibm)
- PetscErrorCode **oversetIbmSearch** (ibm_ *ibm)

5.54.1 Detailed Description

Definitions for overset objects and functions for two–way coupling.

5.55 overset.h

Go to the documentation of this file.

```
00001
00003
00004 #ifndef _OVERSET_H_
00005 #define _OVERSET_H_
00006
00007 #include <unordered_map>
00008 #include <map>
00009 // Structure to hold hole object data
00010 typedef struct {
00011     word bodyName;
00012     PetscInt ownerMesh;
```

```
    PetscInt donorMesh;
    word fileType;
    Cmpnts baseLocation;
    PetscInt searchCellRatio;
} HoleObject;

typedef struct {
    PetscInt      indi, indj, indk;
    PetscReal     coorx, coory, coorz;
    PetscMPIInt   rank;
    PetscReal     cell_size;
    PetscInt      face;
    PetscInt      donorId;
    PetscInt      parentCellId;
} Acell;

typedef struct
{
    PetscInt      indi, indj, indk;
    PetscMPIInt   rank;
    PetscReal     dist2p;
} Dcell;

typedef struct
{
    Cmpnts        prescribedVel;
    word          motionType;
    PetscInt      setMotion;
    PetscInt      ibmAttached;
} oversetMotion;

struct overset_
{
    // Mesh connectivity information
    labelList    parentMeshId;
    labelList    childMeshId;

    word         interpolationType;

    // Dynamic overset parameters
    PetscInt                    dynamicOverset;
    PetscInt                    procChange;
    oversetMotion               *oMotion;

    // In general there are 2 sets of acceptor cells, acceptor cell at domain boundary(aCellDb) and acceptor cell at holeCut boundary(aCellHc)
    // All variables need to be created for both sets of acceptors

    std::vector<std::vector<PetscInt»      AcellProcMatDb;
    std::vector<std::vector<PetscInt»      AcellProcMatHc;

    std::vector<PetscInt>                   NumAcellPerProcDb;
    std::vector<PetscInt>                   NumAcellPerProcHc;

    std::vector<Acell>                      aCellDb;
    std::vector<Acell>                      aCellHc;

    std::vector<Dcell>                      closestDonorDb;
    std::vector<Dcell>                      closestDonorHc;

    //for other interpolation methods
    std::vector<std::vector<PetscReal»     DWeights;
    std::vector<std::vector<Dcell»         dCell;

    // Communication-related arrays for processor connectivity (for donor/acceptor mapping)
    std::vector<MPI_Comm>       oset_comm;

    // MLS (Least-Squares) interpolation parameters
    PetscReal                   cellAvg;
    PetscReal                   cellFactor;

    ibm_    *oibm;
    access_ *access;
};

//structs for 3D binning
struct BinIndex {
    int ix, iy, iz;
    bool operator==(const BinIndex& other) const {
        return ix == other.ix && iy == other.iy && iz == other.iz;
    }
};

namespace std {
    template <>
    struct hash<BinIndex> {
        std::size_t operator()(const BinIndex& b) const {
```

```
            return ((std::hash<int>()(b.ix) ^ (std::hash<int>()(b.iy) « 1)) » 1) ^ (std::hash<int>()(b.iz) « 1);
        }
    };
}

#endif


// Initialization and update
PetscErrorCode InitializeOverset(domain_ *dm);

PetscErrorCode SetInitialFieldsOverset(PetscInt d, domain_ *domain);

PetscErrorCode UpdateOversetInterpolation(domain_ *domain);

PetscErrorCode UpdateDomainInterpolation(PetscInt d, domain_ *domain, PetscInt level);

PetscErrorCode findClosestDomainDonors(PetscInt d, domain_ *domain, PetscInt level, const std::vector<HoleObject> &holeObjects);

PetscErrorCode findAcceptorCells(PetscInt d, domain_ *domain, PetscInt level,
    const std::vector<HoleObject> &holeObjects);

PetscErrorCode readOversetProperties(overset_ *os);

PetscErrorCode readBlankingIBMObject(overset_ *os, domain_ *domain, char *holeObjectName, const std::vector<HoleObject> &holeObjects);

PetscErrorCode readHoleObjects(std::vector<HoleObject> &holeObjects, PetscInt numHoleObjects);

PetscErrorCode FindHoleObject(const std::vector<HoleObject> &holeObjects,
    PetscInt parentId, PetscInt childId,
    char **holeObjectName);

PetscErrorCode updateAcceptorCoordinates(overset_ *os);

PetscErrorCode createAcceptorCellOverset(overset_ *os);

PetscErrorCode createAcceptorCellBackground(overset_ *os, PetscInt donorMeshId);

PetscErrorCode findClosestDonorC2P(mesh_ *meshDonor, mesh_ *meshAcceptor, PetscInt donorId);

PetscErrorCode findClosestDonorP2C(mesh_ *meshDonor, mesh_ *meshAcceptor);

// Interpolation routines for overset (child) mesh
PetscErrorCode interpolateACellTrilinearP2C(mesh_ *donorMesh, mesh_ *acceptorMesh);

PetscErrorCode interpolateACellTrilinearC2P(mesh_ *meshD, mesh_ *meshA, PetscInt donorId);

PetscErrorCode oversetContravariantBC(mesh_ *mesh, PetscInt i, PetscInt j, PetscInt k, Cmpnts ucart, PetscInt face);

PetscErrorCode setOversetBC(mesh_ *meshA);

PetscErrorCode setBackgroundBC(mesh_ *meshA);

PetscErrorCode acell1Dcell0Connectivity(mesh_ *meshP, mesh_ *mesh);

// Least-Squares weights (for overset interpolation from base donor cells)
PetscErrorCode getLSWeights(mesh_ *meshP, mesh_ *mesh);

PetscErrorCode getLSWeights_2nd(mesh_ *meshP, mesh_ *mesh);

PetscErrorCode getLSWeights_3rd(mesh_ *meshP, mesh_ *mesh);

PetscErrorCode interpolateACellInvD(mesh_ *meshP, mesh_ *mesh);

PetscErrorCode interpolateACellLS(mesh_ *meshP, mesh_ *mesh);

PetscErrorCode updateAcellCoordinates(domain_ *domain);

PetscErrorCode updateIntersectingProcessors(domain_ *domain);

PetscErrorCode updateDonorCells(domain_ *domain);

// Dynamic overset motion
PetscErrorCode oversetMeshTranslation(overset_ *os);

void sum_struct_Acell(void *in, void *inout, int *len, MPI_Datatype *type);
void defineStruct_Acell(MPI_Datatype *tstype);

PetscErrorCode computeOversetIBMElementNormal(ibm_ *ibm);

PetscErrorCode oversetIbmSearch(ibm_ *ibm);
```

5.56 /zfs60/nobackup/stipa/software/TOSCA/src/include/peqn.h File Reference

P equation solution header file.

Classes

- struct peqn_

  struct storing pressure equation

Functions

- PetscErrorCode **InitializePEqn** (peqn_ *peqn)

  Initializes Peqn environment.
- PetscErrorCode **CreateHypreSolver** (peqn_ *peqn)

  Initializes HYPRE solver.
- PetscErrorCode **CreatePETScSolver** (peqn_ *peqn)

  Initializes PETSc solver.
- PetscErrorCode **CreateHypreMatrix** (peqn_ *peqn)

  Initializes HYPRE matrix.
- PetscErrorCode **CreatePETScMatrix** (peqn_ *peqn)

  Initializes PETSc matrix.
- PetscErrorCode **CreateHypreVector** (peqn_ *peqn)

  Initializes HYPRE vector.
- PetscErrorCode **CreatePETScVector** (peqn_ *peqn)

  Initializes PETSc vector.
- PetscErrorCode **DestroyHypreSolver** (peqn_ *peqn)

  Destroys HYPRE solver.
- PetscErrorCode **DestroyPETScSolver** (peqn_ *peqn)

  Destroys PETSc solver.
- PetscErrorCode **DestroyHypreMatrix** (peqn_ *peqn)

  Destroys HYPRE matrix.
- PetscErrorCode **DestroyHypreVector** (peqn_ *peqn)

  Destroys HYPRE vector.
- PetscErrorCode **DestroyPETScVector** (peqn_ *peqn)

  Destroys PETSc vector.
- PetscErrorCode **Petsc2HypreVector** (Vec &A, HYPRE_IJVector &B, HYPRE_Int startID)

  Transfer vector values from Petsc 2 Hypre.
- PetscErrorCode **Hypre2PetscVector** (HYPRE_IJVector &B, Vec &A, HYPRE_Int startID)

  Transfer vector values from Hypre 2 Petsc.
- PetscErrorCode **phiToPhi** (peqn_ *peqn)

  Convert phi to Phi (1D to 3D vector)
- PetscReal **L2NormHypre** (peqn_ *peqn, HYPRE_IJMatrix &A, HYPRE_IJVector &X, HYPRE_IJVector &B)

  Compute L2 norm of system residual.
- PetscReal **L2NormPETSc** (peqn_ *peqn, Mat &A, Vec &X, Vec &B)

  Compute L2 norm of system residual.
- PetscErrorCode **GradP** (peqn_ *peqn)

  Compute pressure gradient term.
- PetscErrorCode **GradP4thOrder** (peqn_ *peqn)

Compute 4th order pressure gradient term.

- PetscErrorCode **SetPoissonConnectivity** (peqn_ *peqn)

  Compute cell-matrix connectivity.
- PetscErrorCode **SetCoeffMatrix** (peqn_ *peqn)

  Compute coefficient matrix.
- PetscErrorCode **ZeroCoeffMatrix** (peqn_ *peqn)

  Zeroes coefficient matrix.
- PetscErrorCode **SetRHS** (peqn_ *peqn)

  Set RHS of the pressure equation.
- PetscErrorCode **SubtractAverageHypre** (peqn_ *peqn, HYPRE_IJVector &B)

  Subtract average from the solution.
- PetscErrorCode **SubtractAveragePETSc** (peqn_ *peqn, Vec &B)

  Subtract average from the solution.
- PetscErrorCode **AdjustIBMFlux** (peqn_ *peqn)

  Correct IBM volume flux.
- PetscErrorCode **UpdatePressure** (peqn_ *peqn)

  Update pressure and subtract average.
- PetscErrorCode **updateIBMPhi** (ibm_ *ibm)
- PetscErrorCode **ProjectVelocity** (peqn_ *peqn)

  Project Ucont into an incompressible space.
- PetscErrorCode **SolvePEqn** (peqn_ *peqn)

  Compute pressure gradient term.
- PetscErrorCode **SetPressureReference** (peqn_ *peqn)

  Set pressure = 0 at k = 0, j = 0, i = 0.
- PetscErrorCode **ContinuityErrors** (peqn_ *peqn)

  Compute continuity errors (also calculates which cell and processor has the max)
- PetscErrorCode **ContinuityErrorsOptimized** (peqn_ *peqn)

  Compute continuity errors (only prints the max)
- PetscErrorCode **InitGravityWaveInducedPressure** (peqn_ *peqn)

  Initialize pressure with given gravity wave large-scale pressure field from 3LM.
- cellIds **GetIdFromStencil** (int stencil, int k, int j, int i)

  get the cell id from stencil position

5.56.1 Detailed Description

P equation solution header file.

5.57 peqn.h

Go to the documentation of this file.

```
00001
00003
00004 #ifndef _PEQN_H_
00005 #define _PEQN_H_
00006
00008 struct peqn_
00009 {
00010     Vec           phi;
00011     Vec           Phi, lPhi;
00012     Vec           P, lP;
00013     Vec           lLid, lGid;
00014
00015     HYPRE_Int     thisRankSize,
```

```
                    thisRankStart,
                    thisRankEnd;
    HYPRE_Int       totalSize;
    PetscReal       initialPoissonRes,
                    finalPoissonRes;
    HYPRE_Int       poissonIterations;

    PetscInt        hypreSolverType;
    PetscInt        poissonIt;
    PetscReal       poissonTol;

    // solver parameters
    PetscInt        amgAgg;
    PetscInt        amgCoarsenType;
    PetscReal       amgThresh;

    word            solverType;

    // hypre solver
    HYPRE_Solver        hypreSlvr;
    HYPRE_Solver        hyprePC;
    HYPRE_IJMatrix      hypreA;
    HYPRE_ParCSRMatrix hypreParA;
    HYPRE_IJVector      hypreP, hypreRhs;
    HYPRE_ParVector     hypreParP, hypreParRhs;

    // PETSC solver
    Mat                 petscA;
    KSP                 ksp;
    PC                  petscPC;
    Vec                 petscRhs;
    MatNullSpace        petscNs;

    // access
    access_             *access;
    word                initialFieldType;

};

#endif

PetscErrorCode InitializePEqn(peqn_ *peqn);

PetscErrorCode CreateHypreSolver(peqn_ *peqn);

PetscErrorCode CreatePETScSolver(peqn_ *peqn);

PetscErrorCode CreateHypreMatrix(peqn_ *peqn);

PetscErrorCode CreatePETScMatrix(peqn_ *peqn);

PetscErrorCode CreateHypreVector(peqn_ *peqn);

PetscErrorCode CreatePETScVector(peqn_ *peqn);

PetscErrorCode DestroyHypreSolver(peqn_ *peqn);

PetscErrorCode DestroyPETScSolver(peqn_ *peqn);

PetscErrorCode DestroyHypreMatrix(peqn_ *peqn);

PetscErrorCode DestroyHypreVector(peqn_ *peqn);

PetscErrorCode DestroyPETScVector(peqn_ *peqn);

PetscErrorCode Petsc2HypreVector(Vec &A, HYPRE_IJVector &B, HYPRE_Int startID);

PetscErrorCode Hypre2PetscVector(HYPRE_IJVector &B, Vec &A, HYPRE_Int startID);

PetscErrorCode phiToPhi(peqn_ *peqn);

PetscReal      L2NormHypre(peqn_ *peqn, HYPRE_IJMatrix &A, HYPRE_IJVector &X, HYPRE_IJVector &B);

PetscReal      L2NormPETSc(peqn_ *peqn, Mat &A, Vec &X, Vec &B);

PetscErrorCode GradP(peqn_ *peqn);

PetscErrorCode GradP4thOrder(peqn_ *peqn);

PetscErrorCode SetPoissonConnectivity(peqn_ *peqn);

PetscErrorCode SetCoeffMatrix(peqn_ *peqn);

PetscErrorCode ZeroCoeffMatrix(peqn_ *peqn);

PetscErrorCode SetRHS(peqn_ *peqn);
```

```
PetscErrorCode SubtractAverageHypre(peqn_ *peqn, HYPRE_IJVector &B);

PetscErrorCode SubtractAveragePETSc(peqn_ *peqn, Vec &B);

PetscErrorCode AdjustIBMFlux(peqn_ *peqn);

PetscErrorCode UpdatePressure(peqn_ *peqn);

PetscErrorCode updateIBMPhi(ibm_ *ibm);

PetscErrorCode ProjectVelocity(peqn_ *peqn);

PetscErrorCode SolvePEqn(peqn_ *peqn);

PetscErrorCode SetPressureReference(peqn_ *peqn);

PetscErrorCode ContinuityErrors(peqn_ *peqn);

PetscErrorCode ContinuityErrorsOptimized(peqn_ *peqn);

PetscErrorCode InitGravityWaveInducedPressure(peqn_ *peqn);

cellIds GetIdFromStencil(int stencil, int k, int j, int i);
```

5.58 /zfs60/nobackup/stipa/software/TOSCA/src/include/precursor.h File Reference

Concurrent precursor header file.

Classes

- struct mapInfo

  concurrent precursor mapping info
- struct precursor_

  concurrent precursor database

Functions

- PetscErrorCode **concurrentPrecursorInitialize** (abl_ ∗abl)

  initialize concurrent precursor
- PetscErrorCode **SetSolutionFlagsPrecursor** (domain_ ∗domain)

  Precursor solution flags definition.
- PetscErrorCode **SetStartTimePrecursor** (domain_ ∗domain, abl_ ∗abl)

  Checks that start time is available to read if spinUp == 0.
- PetscErrorCode **InitializeMeshPrecursor** (abl_ ∗abl)

  Precursor mesh initialization.
- PetscErrorCode **SetBoundaryConditionsPrecursor** (mesh_ ∗mesh)

  Precursor boundary conditions initialization function.
- PetscErrorCode **SetInflowFunctionsPrecursor** (mesh_ ∗mesh)

  Sets the inflow function on the precursor.
- PetscErrorCode **MapInitialConditionPrecursor** (abl_ ∗abl)

  Map fields from successor to precursor.
- PetscErrorCode **successorPrecursorMapVectorField** (abl_ ∗abl, Vec &Source, Vec &Target, Vec &ITarget)

  Map vector field from successor to precursor.
- PetscErrorCode **successorPrecursorMapScalarField** (abl_ ∗abl, Vec &Source, Vec &Target, Vec &ITarget)

  Map scalar tor field from successor to precursor.
- PetscErrorCode **ABLInitializePrecursor** (domain_ ∗domain)

  Initialize abl for precursor (x damping layer is not set)
- PetscErrorCode **concurrentPrecursorSolve** (abl_ ∗abl)

  Solve concurrent precursor.
- PetscErrorCode **saveSourcePrecursor2Successor** (abl_ ∗precursorABL, abl_ ∗successorABL)
- PetscErrorCode **InitializePrecursorIBM** (ibm_ ∗ibm, ibm_ ∗succIbm)

5.58.1 Detailed Description

Concurrent precursor header file.

5.59 precursor.h

Go to the documentation of this file.

```
00001
00003
00004 #ifndef _PRECURSOR_H_
00005 #define _PRECURSOR_H_
00006
00008 typedef struct
00009 {
00010     PetscInt kFringeStart, kFringeEnd;
00011     PetscInt kStart, kEnd;
00012 } mapInfo;
00013
00015 struct precursor_
00016 {
00017     PetscInt      thisProcessorInFringe;
00018     mapInfo       map;
00019
00020     domain_       *domain;
00021 };
00022
00023 #endif
00024
00026 PetscErrorCode concurrentPrecursorInitialize(abl_ *abl);
00027
00029 PetscErrorCode SetSolutionFlagsPrecursor(domain_ *domain);
00030
00032 PetscErrorCode SetStartTimePrecursor(domain_ *domain, abl_ *abl);
00033
00035 PetscErrorCode InitializeMeshPrecursor(abl_ *abl);
00036
00038 PetscErrorCode SetBoundaryConditionsPrecursor(mesh_ *mesh);
00039
00041 PetscErrorCode SetInflowFunctionsPrecursor(mesh_ *mesh);
00042
00044 PetscErrorCode MapInitialConditionPrecursor(abl_ *abl);
00045
00047 PetscErrorCode successorPrecursorMapVectorField(abl_ *abl, Vec &Source, Vec &Target, Vec &lTarget);
00048
00050 PetscErrorCode successorPrecursorMapScalarField(abl_ *abl, Vec &Source, Vec &Target, Vec &lTarget);
00051
00053 PetscErrorCode ABLInitializePrecursor(domain_ *domain);
00054
00056 PetscErrorCode concurrentPrecursorSolve(abl_ *abl);
00057
00058 PetscErrorCode saveSourcePrecursor2Successor(abl_ *precursorABL, abl_ *successorABL);
00059
00060 PetscErrorCode InitializePrecursorIBM(ibm_ *ibm, ibm_ *succIbm);
```

5.60 /zfs60/nobackup/stipa/software/TOSCA/src/include/teqn.h File Reference

T equation solution header file.

Classes

- struct teqn_

  struct storing temperature equation

Functions

- PetscErrorCode **InitializeTEqn** (teqn_ *teqn)

  Initializes Teqn environment.
- PetscErrorCode **SolveTEqn** (teqn_ *teqn)

  Solve T equation.
- PetscErrorCode **ghGradRhoK** (teqn_ *teqn)

  *Computes g*h times gradient of rho_k / rho_0.*
- PetscErrorCode **TmprtPredictor** (teqn_ *teqn)

  Forward-Euler conv-only T predictor: T = T^n + dt*N(T^n) -> lTmprt (for buoyancy in SolveUEqn)*
- PetscErrorCode **TmprtRestoreFromOld** (teqn_ *teqn)

  Restore Tmprt/lTmprt to T^n from Tmprt_o (undo TmprtPredictor before SolveTEqn)

5.60.1 Detailed Description

T equation solution header file.

5.61 teqn.h

Go to the documentation of this file.

```
00001
00003
00004 #ifndef _TEQN_H_
00005 #define _TEQN_H_
00006
00008 struct teqn_
00009 {
00010     // solver name
00011     word          solverType;
00012
00013     // temperature variables
00014     SNES          snesT;
00015     Mat           JT;
00016     Mat           AT, CT;
00017     KSP           ksp;
00018     PC            pc;
00019     KSP           kspIMEX;
00020     Mat           JvIMEX;
00021     word          kspType;
00022     PetscInt      gmresRestart;
00023     Vec           Rhs;
00024     Vec           bT;
00025     Vec           RhsConv;
00026     Vec           RhsConv_o;
00027
00028     Vec           TmprtTmp;
00029     Vec           Tmprt, lTmprt,
00030                   Tmprt_o, lTmprt_o;
00031     Vec           Tmprt_oo;
00032
00033     Vec           lDivT, lViscT, lViscIBMT;
00034     Vec           sourceT;
00035
00036     Vec           lRhoK;
00037     Vec           ghGradRhok;
00038     Vec           ghGradRhok_o;
00039
00040     PetscReal     absExitTol;
00041     PetscReal     relExitTol;
00042
00043     word          ddtScheme;
00044
00045     PetscReal     hyperVisc4i;
00046     PetscReal     hyperVisc4j;
00047     PetscReal     hyperVisc4k;
00048
00049     // wall model patch
00050     wallModel     *iLWM;
```

```
    wallModel     *iRWM;
    wallModel     *jLWM;
    wallModel     *jRWM;

    // t equation flags
    PetscInt      pTildeFormulation;

    // initial field
    word          initFieldType;

    // access
    access_       *access;

};

#endif

PetscErrorCode InitializeTEqn(teqn_ *teqn);

PetscErrorCode SolveTEqn(teqn_ *teqn);

PetscErrorCode ghGradRhoK(teqn_ *teqn);

PetscErrorCode TmprtPredictor(teqn_ *teqn);

PetscErrorCode TmprtRestoreFromOld(teqn_ *teqn);
```

5.62 /zfs60/nobackup/stipa/software/TOSCA/src/include/tosca2PV.h File Reference

post processing header file.

Classes

- struct postProcess

  Structure defining the variables for postProcessing.

Functions

- PetscErrorCode **binary3DToXMF** (domain_ ∗domain, postProcess ∗pp)

  Reads binary fields and writes paraview data into XMF folder.
- PetscErrorCode **binaryISectionsToXMF** (domain_ ∗domain)

  Reads binary i-section data and writes paraview data into XMF folder.
- PetscErrorCode **fieldISectionsToXMF** (domain_ ∗domain)

  Reads i-section data from average fields and writes paraview data into XMF folder.
- PetscErrorCode **binaryISectionsPerturbToXMF** (domain_ ∗domain)

  Reads binary i-section pertirbation data and writes paraview data into XMF folder.
- PetscErrorCode **binaryJSectionsToXMF** (domain_ ∗domain, postProcess ∗pp)

  Reads binary j-section data and writes paraview data into XMF folder.
- PetscErrorCode **fieldJSectionsToXMF** (domain_ ∗domain)

  Reads j-section data from average fields and writes paraview data into XMF folder.
- PetscErrorCode **binaryJSectionsPerturbToXMF** (domain_ ∗domain, postProcess ∗pp)

  Reads binary j-section perturbation data and writes paraview data into XMF folder.
- PetscErrorCode **binaryKSectionsToXMF** (domain_ ∗domain)

  Reads binary k-section data and writes paraview data into XMF folder.
- PetscErrorCode **binaryUserSectionsToXMF** (domain_ ∗domain)
- PetscErrorCode **fieldKSectionsToXMF** (domain_ ∗domain)

  Reads k-section data from average fields and writes paraview data into XMF folder.

- PetscErrorCode **fieldUserDefinedPlaneToXMF** (domain_ *domain)

  Reads a user defined surface file and writes paraview average field data at the surface points into the XMF folder.
- PetscErrorCode **binaryKSectionsPerturbToXMF** (domain_ *domain)

  Reads binary k-section perturbation data and writes paraview data into XMF folder.
- PetscErrorCode **postProcessInitialize** (domain_ **domainAddr, clock_ *clock, simInfo_ *info, flags_ *flags)

  Initialize post processing parameters.
- PetscErrorCode **postProcessInitializePrecursor** (postProcess *pp, clock_ *clock)

  Initialize concurrent precursor post processing parameters.
- PetscErrorCode **writeFieldsToXMF** (domain_ *domain, const char *filexmf, PetscReal time)
- PetscErrorCode **postProcessWriteProbes** (domain_ *domain)

  \broer On the fly probes cration of average and phase average fields only
- PetscErrorCode getTimeList (const char *dataLoc, std::vector< PetscReal > &timeSeries, PetscInt &ntimes)

  gets list of time folders contained in a directory
- PetscErrorCode **sectionsReadAndAllocate** (domain_ *domain)

  Reads i,j,k - sections info and allocates moemry. Some info are not necessary thus not read.
- PetscErrorCode **kSectionLoadSymmTensorFromField** (Vec &V, mesh_ *mesh, sections *sec, PetscInt kplane, const word &fieldName, PetscReal time)

  Generate the section on-the-fly in the post processing phase (for average fields that only have to be done at the end)
- PetscErrorCode **kSectionLoadVectorFromField** (Vec &V, mesh_ *mesh, sections *sec, PetscInt kplane, const word &fieldName, PetscReal time)
- PetscErrorCode **kSectionLoadScalarFromField** (Vec &V, mesh_ *mesh, sections *sec, PetscInt kplane, const word &fieldName, PetscReal time)
- PetscErrorCode **userSectionLoadVectorFromField** (Vec &V, mesh_ *mesh, uSections *uSection, const word &fieldName, PetscReal time)
- PetscErrorCode **userSectionLoadScalarFromField** (Vec &V, mesh_ *mesh, uSections *uSection, const word &fieldName, PetscReal time)
- PetscErrorCode **kSectionLoadVector** (mesh_ *mesh, sections *sec, PetscInt kplane, const word &field↩ Name, PetscReal time)

  \briefReads from k-slices time series and loads the velocity, temperature and nut planes. Important: assumes T and nut databases have the same times of U.
- PetscErrorCode **kSectionLoadScalar** (mesh_ *mesh, sections *sec, PetscInt kplane, const word &field↩ Name, PetscReal time)
- PetscErrorCode **userSectionLoadVector** (mesh_ *mesh, uSections *uSection, const word &fieldName, PetscReal time)
- PetscErrorCode **userSectionLoadScalar** (mesh_ *mesh, uSections *uSection, const word &fieldName, PetscReal time)
- PetscErrorCode **iSectionLoadSymmTensorFromField** (Vec &V, mesh_ *mesh, sections *sec, PetscInt kplane, const word &fieldName, PetscReal time)

  Generate the section on-the-fly in the post processing phase (for average fields that only have to be done at the end)
- PetscErrorCode **iSectionLoadVectorFromField** (Vec &V, mesh_ *mesh, sections *sec, PetscInt iplane, const word &fieldName, PetscReal time)
- PetscErrorCode **iSectionLoadScalarFromField** (Vec &V, mesh_ *mesh, sections *sec, PetscInt iplane, const word &fieldName, PetscReal time)
- PetscErrorCode **iSectionLoadVector** (mesh_ *mesh, sections *sec, PetscInt iplane, const word &fieldName, PetscReal time)

  \briefReads from i-slices time series and loads the velocity, temperature and nut planes. Important: assumes T and nut databases have the same times of U.
- PetscErrorCode **iSectionLoadScalar** (mesh_ *mesh, sections *sec, PetscInt iplane, const word &fieldName, PetscReal time)
- PetscErrorCode **jSectionLoadSymmTensorFromField** (Vec &V, mesh_ *mesh, sections *sec, PetscInt kplane, const word &fieldName, PetscReal time)

  Generate the section on-the-fly in the post processing phase (for average fields that only have to be done at the end)
- PetscErrorCode **jSectionLoadVectorFromField** (Vec &V, mesh_ *mesh, sections *sec, PetscInt jplane, const word &fieldName, PetscReal time)

- PetscErrorCode **jSectionLoadScalarFromField** (Vec &V, mesh_ *mesh, sections *sec, PetscInt jplane, const word &fieldName, PetscReal time)
- PetscErrorCode **jSectionLoadVector** (mesh_ *mesh, sections *sec, PetscInt jplane, const word &fieldName, PetscReal time)

  \briefReads from j-slices time series and loads the velocity, temperature and nut planes. Important: assumes T and nut databases have the same times of U.
- PetscErrorCode **jSectionLoadScalar** (mesh_ *mesh, sections *sec, PetscInt jplane, const word &fieldName, PetscReal time)
- void **xmfWriteFileStartTimeSection** (FILE *Xmf, const char *FileXmf, PetscInt Size_x, PetscInt Size_↩y, PetscInt Size_z, const char *Topology, PetscReal Time)

  Opens a time section in the XMF file.
- void **xmfWriteFileEndTimeSection** (FILE *Xmf, const char *FileXmf)

  Closes a time section in the XMF file.
- void **xmfWriteFileGeometry** (FILE *Xmf, const char *FileXmf, PetscInt Size_x, PetscInt Size_y, PetscInt Size_z, const char *PathSave)

  Writes geometry info in the XMF file.
- void **xmfWriteFileSymmTensor** (FILE *xmf, const char *filexmf, PetscInt size_x, PetscInt size_y, PetscInt size_z, const char *PathSave, const char *symmTensorName, const char *XX, const char *YY, const char *ZZ, const char *XY, const char *XZ, const char *YZ, const char *center="Cell")

  Writes a symmetric tensor in the XMF files.
- void **xmfWriteFileTensor** (FILE *xmf, const char *filexmf, PetscInt size_x, PetscInt size_y, PetscInt size_z, const char *PathSave, const char *TensorName, const char *XX, const char *XY, const char *XZ, const char *YX, const char *YY, const char *YZ, const char *ZX, const char *ZY, const char *ZZ, const char *center="Cell")

  Writes a full tensor in the XMF files.
- void **xmfWriteFileVector** (FILE *xmf, const char *filexmf, PetscInt size_x, PetscInt size_y, PetscInt size_z, const char *PathSave, const char *Vecname, const char *V1, const char *V2, const char *V3, const char *center="Cell")

  Writes a vector in the XMF file.
- void **xmfWriteFileScalar** (FILE *Xmf, const char *Filexmf, PetscInt Size_x, PetscInt Size_y, PetscInt Size_z, const char *PathSave, const char *ScalName, const char *Scal, const char *Center="Cell")

  Writes a scalar in the XMF file.
- void **hdfWriteDataset** (hid_t *file_id, hid_t *dataspace_id, char const *var, float *x)

  Writes a dataset to HDF format file.
- PetscErrorCode **writeScalarToXMF** (domain_ *domain, const char *filexmf, const char *hdfilen, hid_t *file↩_id, hid_t *dataspace_id, PetscReal time, const char *fieldName, Vec V)

  Writes a scalar field (appends to XMF and creates HDF)
- PetscErrorCode **writeVectorToXMF** (domain_ *domain, const char *filexmf, const char *hdfilen, hid_t *file↩_id, hid_t *dataspace_id, PetscReal time, const char *fieldName, Vec V)

  Writes a vector field (appends to XMF and creates HDF)
- PetscErrorCode **writeSymmTensorToXMF** (domain_ *domain, const char *filexmf, const char *hdfilen, hid_t *file_id, hid_t *dataspace_id, PetscReal time, const char *fieldName, Vec V)

  Writes a symmetric tensor field (appends to XMF and creates HDF)
- PetscErrorCode **writeTensorToXMF** (domain_ *domain, const char *filexmf, const char *hdfilen, hid_t *file↩_id, hid_t *dataspace_id, PetscReal time, const char *fieldName, Vec V)

  Writes a full tensor field (appends to XMF and creates HDF)
- PetscErrorCode **writeISectionScalarToXMF** (mesh_ *mesh, const char *filexmf, const char *hdfilen, hid_t *file_id, hid_t *dataspace_id, PetscReal time, const char *fieldName, PetscReal **field)

  Writes a scalar defined on an i-section (appends to XMF and creates HDF)
- PetscErrorCode **writeJSectionScalarToXMF** (mesh_ *mesh, const char *filexmf, const char *hdfilen, hid_t *file_id, hid_t *dataspace_id, PetscReal time, const char *fieldName, PetscReal **field)

  Writes a scalar defined on an j-section (appends to XMF and creates HDF)
- PetscErrorCode **writeKSectionScalarToXMF** (mesh_ *mesh, const char *filexmf, const char *hdfilen, hid_t *file_id, hid_t *dataspace_id, PetscReal time, const char *fieldName, PetscReal **field)

Writes a scalar defined on an k-section (appends to XMF and creates HDF)

- PetscErrorCode **writeUserSectionScalarToXMF** (mesh_ *mesh, const char *filexmf, const char *hdfilen, hid_t *file_id, hid_t *dataspace_id, PetscReal time, const char *fieldName, uSections *uSection)

  Writes a scalar defined on a user defined section (appends to XMF and creates HDF)

- PetscErrorCode **writeISectionVectorToXMF** (mesh_ *mesh, const char *filexmf, const char *hdfilen, hid_t *file_id, hid_t *dataspace_id, PetscReal time, const char *fieldName, Cmpnts **field)

  Writes a vector defined on an i-section (appends to XMF and creates HDF)

- PetscErrorCode **writeJSectionVectorToXMF** (mesh_ *mesh, const char *filexmf, const char *hdfilen, hid_t *file_id, hid_t *dataspace_id, PetscReal time, const char *fieldName, Cmpnts **field)

  Writes a vector defined on an j-section (appends to XMF and creates HDF)

- PetscErrorCode **writeKSectionVectorToXMF** (mesh_ *mesh, const char *filexmf, const char *hdfilen, hid↩_t *file_id, hid_t *dataspace_id, PetscReal time, const char *fieldName, Cmpnts **field)

  Writes a vector defined on an k-section (appends to XMF and creates HDF)

- PetscErrorCode **writeUserSectionVectorToXMF** (mesh_ *mesh, const char *filexmf, const char *hdfilen, hid_t *file_id, hid_t *dataspace_id, PetscReal time, const char *fieldName, uSections *uSection)

  Writes a vector defined on a user defined section (appends to XMF and creates HDF)

- PetscErrorCode **writeISectionSymmTensorToXMF** (mesh_ *mesh, const char *filexmf, const char *hdfilen, hid_t *file_id, hid_t *dataspace_id, PetscReal time, const char *fieldName, symmTensor **field)

  Writes a vector defined on an i-section (appends to XMF and creates HDF)

- PetscErrorCode **writeJSectionSymmTensorToXMF** (mesh_ *mesh, const char *filexmf, const char *hdfilen, hid_t *file_id, hid_t *dataspace_id, PetscReal time, const char *fieldName, symmTensor **field)

  Writes a vector defined on an j-section (appends to XMF and creates HDF)

- PetscErrorCode **writeKSectionSymmTensorToXMF** (mesh_ *mesh, const char *filexmf, const char *hdfilen, hid_t *file_id, hid_t *dataspace_id, PetscReal time, const char *fieldName, symmTensor **field)

  Writes a vector defined on an k-section (appends to XMF and creates HDF)

- PetscErrorCode **writePointsToXMF** (mesh_ *mesh, const char *filexmf, const char *hdfilen, hid_t *file_id, hid_t *dataspace_id, PetscReal time)

  Writes 3D mesh (appends to XMF and creates HDF)

- PetscErrorCode **writeISectionPointsToXMF** (mesh_ *mesh, const char *filexmf, const char *hdfilen, hid_t *file_id, hid_t *dataspace_id, PetscReal time, PetscInt iIndex)

  Writes 2D i-section mesh (appends to XMF and creates HDF)

- PetscErrorCode **writeJSectionPointsToXMF** (mesh_ *mesh, const char *filexmf, const char *hdfilen, hid_t *file_id, hid_t *dataspace_id, PetscReal time, PetscInt jIndex)

  Writes 2D j-section mesh (appends to XMF and creates HDF)

- PetscErrorCode **writeKSectionPointsToXMF** (mesh_ *mesh, const char *filexmf, const char *hdfilen, hid_t *file_id, hid_t *dataspace_id, PetscReal time, PetscInt kIndex)

  Writes 2D k-section mesh (appends to XMF and creates HDF)

- PetscErrorCode **writeUserSectionPointsToXMF** (mesh_ *mesh, const char *filexmf, const char *hdfilen, hid_t *file_id, hid_t *dataspace_id, PetscReal time, uSections *uSection)

  Writes 2D user-section mesh (appends to XMF and creates HDF)

- void **setToZero** (float *vec, PetscInt n)
- PetscErrorCode **writeJSectionToRaster** (mesh_ *mesh, PetscInt jIndex)

5.62.1 Detailed Description

post processing header file.

5.62.2 Function Documentation

5.62.2.1 getTimeList()

```
PetscErrorCode getTimeList (
            const char * dataLoc,
            std::vector< PetscReal > & timeSeries,
            PetscInt & ntimes)
```

gets list of time folders contained in a directory

gets list of time folders contained in a directory

5.63 tosca2PV.h

Go to the documentation of this file.

```
00001
00003
00004 #ifndef _TOSCA2PV_H_
00005 #define _TOSCA2PV_H_
00006
00008 typedef struct
00009 {
00010   PetscInt              postProcessFields;
00011   PetscInt              writeRaster;
00012   PetscInt              samplingSections;
00013
00014   PetscInt              postProcessPrecursor;
00015   precursor_            *precursor;
00016 } postProcess;
00017
00018 #endif
00019
00020 // Top level functions
00021 // -------------------------------------------------------
00022
00024 PetscErrorCode binary3DToXMF(domain_ *domain, postProcess *pp);
00025
00027 PetscErrorCode binaryISectionsToXMF(domain_ *domain);
00028
00030 PetscErrorCode fieldISectionsToXMF(domain_ *domain);
00031
00033 PetscErrorCode binaryISectionsPerturbToXMF(domain_ *domain);
00034
00036 PetscErrorCode binaryJSectionsToXMF(domain_ *domain, postProcess *pp);
00037
00039 PetscErrorCode fieldJSectionsToXMF(domain_ *domain);
00040
00042 PetscErrorCode binaryJSectionsPerturbToXMF(domain_ *domain, postProcess *pp);
00043
00045 PetscErrorCode binaryKSectionsToXMF(domain_ *domain);
00046
00047 PetscErrorCode binaryUserSectionsToXMF(domain_ *domain);
00048
00050 PetscErrorCode fieldKSectionsToXMF(domain_ *domain);
00051
00053 PetscErrorCode fieldUserDefinedPlaneToXMF(domain_ *domain);
00054
00056 PetscErrorCode binaryKSectionsPerturbToXMF(domain_ *domain);
00057
00059 PetscErrorCode postProcessInitialize(domain_ **domainAddr, clock_ *clock, simInfo_ *info, flags_
      *flags);
00060
00062 PetscErrorCode postProcessInitializePrecursor(postProcess *pp, clock_ *clock);
00063
00064 // Operation functions
00065 // -------------------------------------------------------
00066 PetscErrorCode writeFieldsToXMF(domain_ *domain, const char* filexmf, PetscReal time);
00067
00069 PetscErrorCode postProcessWriteProbes(domain_ *domain);
00070
00072 PetscErrorCode getTimeList(const char* dataLoc, std::vector<PetscReal> &timeSeries, PetscInt &ntimes);
00073
00075 PetscErrorCode sectionsReadAndAllocate(domain_ *domain);
```

```
PetscErrorCode kSectionLoadSymmTensorFromField(Vec &V, mesh_ *mesh, sections *sec, PetscInt kplane, const word &fieldName, PetscReal time);
PetscErrorCode kSectionLoadVectorFromField(Vec &V, mesh_ *mesh, sections *sec, PetscInt kplane, const word &fieldName, PetscReal time);
PetscErrorCode kSectionLoadScalarFromField(Vec &V, mesh_ *mesh, sections *sec, PetscInt kplane, const word &fieldName, PetscReal time);
PetscErrorCode userSectionLoadVectorFromField(Vec &V, mesh_ *mesh, uSections *uSection, const word &fieldName, PetscReal time);
PetscErrorCode userSectionLoadScalarFromField(Vec &V, mesh_ *mesh, uSections *uSection, const word &fieldName, PetscReal time);


PetscErrorCode kSectionLoadVector(mesh_ *mesh, sections *sec, PetscInt kplane, const word &fieldName, PetscReal time);
PetscErrorCode kSectionLoadScalar(mesh_ *mesh, sections *sec, PetscInt kplane, const word &fieldName, PetscReal time);
PetscErrorCode userSectionLoadVector(mesh_ *mesh, uSections *uSection, const word &fieldName, PetscReal time);
PetscErrorCode userSectionLoadScalar(mesh_ *mesh, uSections *uSection, const word &fieldName, PetscReal time);

PetscErrorCode iSectionLoadSymmTensorFromField(Vec &V, mesh_ *mesh, sections *sec, PetscInt kplane, const word &fieldName, PetscReal time);
PetscErrorCode iSectionLoadVectorFromField(Vec &V, mesh_ *mesh, sections *sec, PetscInt iplane, const word &fieldName, PetscReal time);
PetscErrorCode iSectionLoadScalarFromField(Vec &V, mesh_ *mesh, sections *sec, PetscInt iplane, const word &fieldName, PetscReal time);

PetscErrorCode iSectionLoadVector(mesh_ *mesh, sections *sec, PetscInt iplane, const word &fieldName, PetscReal time);
PetscErrorCode iSectionLoadScalar(mesh_ *mesh, sections *sec, PetscInt iplane, const word &fieldName, PetscReal time);

PetscErrorCode jSectionLoadSymmTensorFromField(Vec &V, mesh_ *mesh, sections *sec, PetscInt kplane, const word &fieldName, PetscReal time);
PetscErrorCode jSectionLoadVectorFromField(Vec &V, mesh_ *mesh, sections *sec, PetscInt jplane, const word &fieldName, PetscReal time);
PetscErrorCode jSectionLoadScalarFromField(Vec &V, mesh_ *mesh, sections *sec, PetscInt jplane, const word &fieldName, PetscReal time);

PetscErrorCode jSectionLoadVector(mesh_ *mesh, sections *sec, PetscInt jplane, const word &fieldName, PetscReal time);
PetscErrorCode jSectionLoadScalar(mesh_ *mesh, sections *sec, PetscInt jplane, const word &fieldName, PetscReal time);


// The following functions append text the the XMF file
// ----------------------------------------------------

void xmfWriteFileStartTimeSection
(
    FILE *Xmf, const char *FileXmf,
    PetscInt Size_x, PetscInt Size_y, PetscInt Size_z,
    const char *Topology,
    PetscReal Time
);

void xmfWriteFileEndTimeSection
(
    FILE *Xmf, const char *FileXmf
);

void xmfWriteFileGeometry
(
    FILE *Xmf, const char *FileXmf,
    PetscInt Size_x, PetscInt Size_y, PetscInt Size_z,
    const char *PathSave
);

void xmfWriteFileSymmTensor
(
    FILE *xmf, const char *filexmf,
    PetscInt size_x, PetscInt size_y, PetscInt size_z,
    const char *PathSave, const char *symmTensorName,
    const char * XX, const char * YY, const char *ZZ,
    const char * XY, const char * XZ, const char *YZ,
    const char *center = "Cell"
);

void xmfWriteFileTensor
(
    FILE *xmf, const char *filexmf,
    PetscInt size_x, PetscInt size_y, PetscInt size_z,
    const char *PathSave, const char *TensorName,
    const char * XX, const char * XY, const char *XZ,
    const char * YX, const char * YY, const char *YZ,
```

```
    const char * ZX, const char * ZY, const char *ZZ,
    const char *center = "Cell"
);

void xmfWriteFileVector
(
    FILE *xmf, const char *filexmf,
    PetscInt size_x, PetscInt size_y, PetscInt size_z,
    const char *PathSave, const char *Vecname,
    const char * V1, const char * V2, const char *V3,
    const char *center = "Cell"
);

void xmfWriteFileScalar
(
    FILE *Xmf, const char *Filexmf,
    PetscInt Size_x, PetscInt Size_y, PetscInt Size_z,
    const char *PathSave, const char *ScalName,
    const char *Scal,
    const char *Center = "Cell"
);

// The following functions deal with the 1D-data HDF5 file creation
// ----------------------------------------------------------------

void hdfWriteDataset
(
    hid_t *file_id,
    hid_t *dataspace_id,
    char const *var,
    float *x
);

// Interface functions for writing a single field (3D or 2D)
// ---------------------------------------------------------

PetscErrorCode writeScalarToXMF
(
    domain_    *domain,
    const char *filexmf,
    const char *hdfilen,
    hid_t        *file_id,
    hid_t      *dataspace_id,
    PetscReal     time,
    const char *fieldName,
    Vec        V
);

PetscErrorCode writeVectorToXMF
(
    domain_    *domain,
    const char *filexmf,
    const char *hdfilen,
    hid_t      *file_id,
    hid_t      *dataspace_id,
    PetscReal     time,
    const char *fieldName,
    Vec        V
);

PetscErrorCode writeSymmTensorToXMF
(
    domain_    *domain,
    const char *filexmf,
    const char *hdfilen,
    hid_t      *file_id,
    hid_t      *dataspace_id,
    PetscReal     time,
    const char *fieldName,
    Vec        V
);

PetscErrorCode writeTensorToXMF
(
    domain_    *domain,
    const char *filexmf,
    const char *hdfilen,
    hid_t      *file_id,
    hid_t      *dataspace_id,
    PetscReal     time,
    const char *fieldName,
    Vec        V
);

PetscErrorCode writeISectionScalarToXMF
(
    mesh_    *mesh,
```

```
    const char *filexmf,
    const char *hdfilen,
    hid_t      *file_id,
    hid_t      *dataspace_id,
    PetscReal     time,
    const char *fieldName,
    PetscReal     **field
);

PetscErrorCode writeJSectionScalarToXMF
(
    mesh_    *mesh,
    const char *filexmf,
    const char *hdfilen,
    hid_t      *file_id,
    hid_t      *dataspace_id,
    PetscReal     time,
    const char *fieldName,
    PetscReal     **field
);

PetscErrorCode writeKSectionScalarToXMF
(
    mesh_    *mesh,
    const char *filexmf,
    const char *hdfilen,
    hid_t      *file_id,
    hid_t      *dataspace_id,
    PetscReal     time,
    const char *fieldName,
    PetscReal     **field
);

PetscErrorCode writeUserSectionScalarToXMF
(
    mesh_    *mesh,        // user context
    const char *filexmf,     // name of the XMF file to append
    const char *hdfilen,     // name of the HDF file to refer
    hid_t      *file_id,     // id of the HDF file to refer
    hid_t      *dataspace_id,// id of the HDF dataspace to refer
    PetscReal     time,          // time
    const char *fieldName,   // field name as it appears in ParaView
    uSections  *uSection    // pointer to the user defined section
);

PetscErrorCode writeISectionVectorToXMF
(
    mesh_    *mesh,
    const char *filexmf,
    const char *hdfilen,
    hid_t      *file_id,
    hid_t      *dataspace_id,
    PetscReal     time,
    const char *fieldName,
    Cmpnts     **field
);

PetscErrorCode writeJSectionVectorToXMF
(
    mesh_    *mesh,
    const char *filexmf,
    const char *hdfilen,
    hid_t      *file_id,
    hid_t      *dataspace_id,
    PetscReal     time,
    const char *fieldName,
    Cmpnts     **field
);

PetscErrorCode writeKSectionVectorToXMF
(
    mesh_    *mesh,
    const char *filexmf,
    const char *hdfilen,
    hid_t      *file_id,
    hid_t      *dataspace_id,
    PetscReal     time,
    const char *fieldName,
    Cmpnts     **field
);

PetscErrorCode writeUserSectionVectorToXMF
(
    mesh_    *mesh,
    const char *filexmf,
    const char *hdfilen,
    hid_t      *file_id,
```

```
    hid_t      *dataspace_id,
    PetscReal     time,
    const char *fieldName,
    uSections  *uSection
);

PetscErrorCode writeISectionSymmTensorToXMF
(
    mesh_    *mesh,
    const char *filexmf,
    const char *hdfilen,
    hid_t      *file_id,
    hid_t      *dataspace_id,
    PetscReal     time,
    const char *fieldName,
    symmTensor     **field
);

PetscErrorCode writeJSectionSymmTensorToXMF
(
    mesh_    *mesh,
    const char *filexmf,
    const char *hdfilen,
    hid_t      *file_id,
    hid_t      *dataspace_id,
    PetscReal     time,
    const char *fieldName,
    symmTensor     **field
);

PetscErrorCode writeKSectionSymmTensorToXMF
(
    mesh_    *mesh,
    const char *filexmf,
    const char *hdfilen,
    hid_t      *file_id,
    hid_t      *dataspace_id,
    PetscReal     time,
    const char *fieldName,
    symmTensor     **field
);

// Interface functions for writing the mesh (3D or 2D)
// ---------------------------------------------------------

PetscErrorCode writePointsToXMF
(
    mesh_      *mesh,
    const char *filexmf,
    const char *hdfilen,
    hid_t        *file_id,
    hid_t      *dataspace_id,
    PetscReal     time
);

PetscErrorCode writeISectionPointsToXMF
(
    mesh_      *mesh,
    const char *filexmf,
    const char *hdfilen,
    hid_t        *file_id,
    hid_t      *dataspace_id,
    PetscReal     time,
    PetscInt   iIndex
);

PetscErrorCode writeJSectionPointsToXMF
(
    mesh_      *mesh,
    const char *filexmf,
    const char *hdfilen,
    hid_t        *file_id,
    hid_t      *dataspace_id,
    PetscReal     time,
    PetscInt   jIndex
);

PetscErrorCode writeKSectionPointsToXMF
(
    mesh_      *mesh,
    const char *filexmf,
    const char *hdfilen,
    hid_t        *file_id,
    hid_t      *dataspace_id,
    PetscReal     time,
    PetscInt   kIndex
);
```

```
PetscErrorCode writeUserSectionPointsToXMF
(
    mesh_      *mesh,        // user context
    const char *filexmf,     // name of the XMF file to append
    const char *hdfilen,     // name of the HDF file to refer
    hid_t        *file_id,     // id of the HDF file to refer
    hid_t      *dataspace_id,// id of the HDF dataspace to refer
    PetscReal     time,          // time
    uSections  *uSection    // pointer to the user defined section
);

void setToZero(float *vec, PetscInt n);

// Other file formats
// -------------------------------------------------------

PetscErrorCode writeJSectionToRaster
(
    mesh_    *mesh,        // user context
    PetscInt        jIndex        // index of the j-section
);
```

5.64 /zfs60/nobackup/stipa/software/TOSCA/src/include/turbines.h File Reference

Wind Farm Objects for the arrays of std::string and arrays of structs we use dynamic allocation through array of pointers to each std::string or struct respectively.

```
#include "base.h"
#include "domain.h"
#include "io.h"
#include "inline.h"
```

5.64.1 Detailed Description

Wind Farm Objects for the arrays of std::string and arrays of structs we use dynamic allocation through array of pointers to each std::string or struct respectively.

5.65 turbines.h

Go to the documentation of this file.

```
#include "base.h"
#include "domain.h"
#include "io.h"
#include "inline.h"

#if USE_OPENFAST
    #include "OpenFAST.H"
#endif


#ifndef FARM_H
#define FARM_H

typedef struct
{
    word                    name;
    PetscInt                size;
    PetscReal               *aoa;
    PetscReal                *cl;
    PetscReal                *cd;
}foilInfo;

```

```
typedef struct
{
    PetscInt                size;
    PetscReal             *radius;
    PetscReal              *chord;
    PetscReal              *twist;
    PetscReal              *thick;
    PetscInt             *foilIds;
}bladeAeroInfo;

typedef struct
{
    PetscInt                size;
    PetscReal              *Uref;
    PetscReal              *Ct;
    PetscReal              *Cp;
}ctTable;

typedef struct
{
    PetscInt                size;
    PetscReal              *Uref;
    PetscReal              *pitch;
}pitchTable;

typedef struct
{
    PetscInt                size;
    PetscReal              *Uref;
    PetscReal              *rpm;
}rpmTable;

typedef struct
{
    // mesh level preoperties
    PetscInt              nPoints;
    Cmpnts               *points;
    PetscReal                 *dA;

    Cmpnts                center;

    PetscInt   thisRigControlled;
    MPI_Comm             UPW_COMM;

    cellIds     *controlledCells;
    PetscInt         nControlled;

    // time-varying variables
    PetscInt   *thisPtControlled;
    cellIds        *closestCells;

    PetscReal               Uref;

} upSampling;

typedef struct
{
    Cmpnts                 point;
    PetscReal               Uref;
    PetscReal                 Ct;

    word          projectionType;
    word              sampleType;
    PetscReal     rtrUFilterFreq;

    // time-varying variables
    cellIds          closestCell;
    Cmpnts                     U;
    Cmpnts                     B;
    PetscReal             axialF;

    PetscReal          rtrThrust;
    PetscReal            aeroPwr;

    PetscInt    thisPtControlled;
    PetscInt          searchDone;

    // debug switch
    PetscInt                 dbg;

} AFM;

typedef struct
{
    // settings
    word          projectionType;
    word              sampleType;
```

```
    // mesh level preoperties
    PetscInt             nPoints;
    Cmpnts               *points;
    PetscReal                *dr;

    PetscInt             nRadial;
    PetscInt            nAzimuth;

    PetscReal               Uref;

    PetscReal             *chord;
    PetscReal             *twist;
    PetscReal             *thick;
    PetscReal          *solidity;
    PetscInt           **foilIds;
    PetscReal               **iw;

    // time-varying variables
    PetscInt   *thisPtControlled;
    cellIds        *closestCells;
    PetscReal                *Cd;
    PetscReal                *Cl;
    PetscReal             *alpha;
    Cmpnts                    *U;
    Cmpnts                *gWind;
    Cmpnts                    *B;
    PetscReal            *axialF;
    PetscReal            *tangtF;

    PetscReal         rtrAvgMagU;
    PetscReal          rtrTorque;
    PetscReal          rtrThrust;
    PetscReal            aeroPwr;

    PetscReal            azimuth;

    // debug switch
    PetscInt                 dbg;

} ALM;

typedef struct
{
    // mesh level preoperties
    PetscInt             nPoints;
    Cmpnts               *points;
    PetscReal                *dr;

    PetscInt             nRadial;
    PetscInt            nAzimuth;

    PetscReal               Uref;

    PetscReal             *chord;
    PetscReal             *twist;
    PetscReal          *solidity;
    PetscInt           **foilIds;
    PetscReal               **iw;

    // time-varying variables
    PetscInt   *thisPtControlled;
    cellIds        *closestCells;
    PetscReal                *Cd;
    PetscReal                *Cl;
    PetscReal             *alpha;
    Cmpnts                    *U;
    Cmpnts                    *B;
    PetscReal            *axialF;
    PetscReal            *tangtF;

    PetscReal         rtrAvgMagU;
    PetscReal          rtrTorque;
    PetscReal          rtrThrust;
    PetscReal            aeroPwr;

    // debug switch
    PetscInt                 dbg;

} ADM;

typedef struct
{
    // mesh level preoperties
    PetscInt             nPoints;
    Cmpnts               *points;
    PetscReal                *dA;
```

```
    PetscInt             nRadial;
    PetscInt            nAzimuth;

    word              sampleType;
    PetscReal               Uref;
    PetscReal                 Ct;
    PetscReal             axiInd;

    // time-varying variables
    PetscInt   *thisPtControlled;
    cellIds        *closestCells;
    Cmpnts                    *U;
    Cmpnts                    *B;
    PetscReal            *axialF;

    PetscReal         rtrAvgMagU;
    PetscReal          rtrThrust;
    PetscReal            aeroPwr;

    // debug switch
    PetscInt                 dbg;
} UADM;

typedef struct
{
    PetscReal              rBase;
    PetscReal               rTop;
    PetscReal                 Cd;
    PetscReal                eps;
    PetscReal          prjNSigma;

    PetscInt             nPoints;

    Cmpnts               *points;
    PetscReal                *dA;

    cellIds     *controlledCells;
    PetscInt         nControlled;

    PetscInt   *thisPtControlled;
    cellIds        *closestCells;

    Cmpnts                    *U;
    Cmpnts                    *B;
    PetscReal             *tangF;

    PetscReal          twrThrust;

    MPI_Comm            TWR_COMM;
    PetscMPIInt        nProcsTwr;

} towerModel;

typedef struct
{
    PetscReal                 Cd;
    PetscReal                eps;
    PetscReal          prjNSigma;

    Cmpnts                 point;
    PetscReal                  A;

    cellIds     *controlledCells;
    PetscInt         nControlled;

    PetscInt    thisPtControlled;
    cellIds          closestCell;

    Cmpnts                     U;
    Cmpnts                     B;
    PetscReal              tangF;

    PetscReal          nacThrust;

    MPI_Comm            NAC_COMM;
    PetscMPIInt        nProcsNac;

} nacelleModel;

typedef struct
{
    Cmpnts            samplePoint;
    cellIds           closestCell;
    PetscInt anemometerControlled;
    Cmpnts                      U;
    Cmpnts         anemFilterFreq;
```

```
    Cmpnts        acquisitionFreq;
} anemometer;

typedef struct
{
    // global parameters (all turbine models)
    PetscInt               index;
    word                      id;
    word                    type;
    PetscInt             nBlades;
    PetscReal               rTip;
    PetscReal               rHub;
    PetscReal               hTwr;
    PetscReal            ovrHang;
    PetscReal            precone;
    Cmpnts                twrDir;
    PetscReal             upTilt;
    PetscReal             genEff;

    // moving turbine flag
    PetscInt            trbMoved;

    // additional info for ADM and ALM models
    word                  rotDir;
    PetscInt              nFoils;
    word             **foilNames;
    foilInfo             **foils;
    bladeAeroInfo          blade;
    Cmpnts             omega_hat;
    PetscReal           rtrOmega;

    anemometer              WDAS;

    // time-varying parameters
    Cmpnts             rotCenter;
    Cmpnts                twrTop;
    Cmpnts                rtrDir;
    Cmpnts               rtrAxis;

    // rotor models
    ADM                       adm;
    UADM                     uadm;
    ALM                       alm;
    AFM                       afm;

    // tower model
    towerModel                twr;
    PetscInt          includeTwr;

    // nacelle model
    nacelleModel              nac;
    PetscInt      includeNacelle;

    // sampling points 2.5 RD upstream
    upSampling         *upPoints;

    // useful conversion factors
    PetscReal            deg2rad;
    PetscReal            rad2deg;
    PetscReal         rpm2RadSec;

    // communication color
    MPI_Comm            TRB_COMM;
    PetscMPIInt       writerRank;
    PetscMPIInt        nProcsTrb;

    // torque controller (ADM and ALM models)
    word       genControllerType;
    PetscReal           genOmega;
    PetscReal       rtrOmegaFilt;
    PetscReal   rtrSpdFilterFreq;
    PetscReal        cutInGenSpd;
    PetscReal         cutInGenTq;
    PetscReal  regTwoStartGenSpd;
    PetscReal    regTwoEndGenSpd;
    PetscReal         ratedGenTq;
    PetscReal            omegaKP;
    PetscReal          genTorque;
    PetscReal             genPwr;
    PetscInt       tqRateLimiter;
    PetscInt       rtrSpdLimiter;
    PetscReal          tqMaxRate;
    PetscReal      ratedRotorSpd;

    // Ct curve for UADM/AFM models
    ctTable                ctTbl;
    word                  ctType;
```

```
    PetscInt          variableCp;

    // rotor dynamics (if torque controller is active)
    PetscReal  driveTrainInertia;
    PetscReal         genInertia;
    PetscReal         hubInertia;
    PetscReal         bldInertia;
    PetscReal        gbxRatioG2R;
    PetscReal             gbxEff;
    rpmTable              rpmTbl;
    word       rpmControllerType;

    // pitch controller (ADM and ALM models)
    word     pitchControllerType;
    PetscReal             *pitch;
    PetscReal          collPitch;
    PetscReal            pitchKP;
    PetscReal            pitchKI;
    PetscReal            pitchKD;
    PetscReal           pitchS2R;
    PetscReal             errPID;
    PetscReal          intErrPID;
    pitchTable          pitchTbl;

    PetscInt    pitchRateLimiter;
    PetscInt   pitchAngleLimiter;
    PetscReal       pitchMaxRate;
    PetscReal           pitchMin;
    PetscReal           pitchMax;

    // yaw controller (ADM and ALM models)
    word       yawControllerType;
    word         yawSamplingType;
    PetscReal   yawAverageWindow;
    PetscReal    yawAllowedError;
    PetscReal             yawMin;
    PetscReal             yawMax;
    PetscReal           yawAngle;
    PetscReal          flowAngle;
    PetscReal           yawError;
    PetscReal           yawSpeed;
    cellIds         yawSampleIds;

    // wind farm overall controller
    PetscReal wfControlCollPitch;
    PetscReal        wfControlCt;
    PetscInt      wfControlNData;
    PetscReal    *wfControlTimes;
    PetscReal   *wfControlValues;
    PetscInt     currentCloseIdx;

    // dynamic individual pitch controller
    word       dipcControllerType;
    PetscReal        dipcHelixAmp;
    word             dipcHelixDir;
    PetscReal       dipcHelixFreq;

    // numerical parameters and flags
    PetscInt   turbineControlled;
    cellIds     *controlledCells;
    PetscInt         nControlled;
    PetscReal                 eps;
    PetscReal              eps_x;
    PetscReal              eps_y;
    PetscReal              eps_z;
    PetscReal               flat;
    PetscReal                r12;
    PetscReal                  I;
    PetscReal          prjNSigma;

    // debug switch
    PetscInt                 dbg;

    // OpenFAST coupling parameters
    PetscInt         useOpenFAST;   //< turbine specific flag to use OpenFAST (it exists and is always zero when USE_OPENFAST is off)

#if USE_OPENFAST

    // access and connectivity
    fast::OpenFAST         *FAST;
    fast::fastInputs         *fi;
    PetscInt       openfastIndex;
    PetscInt         initialized;

    // time-varying parameters
    Cmpnts                 base;
```

```
    // number of points for velocity sampling and force projection
    PetscInt     nBladeVelPtsOF;
    PetscInt       nTwrVelPtsOF;
    PetscInt   nBladeForcePtsOF;
    PetscInt     nTwrForcePtsOF;

    // arrays containing the coordinates and values of the velocity and force points
    Cmpnts         *velPtsBlade;
    Cmpnts        *velValsBlade;
    Cmpnts       *forcePtsBlade;
    Cmpnts      *forceValsBlade;
    Cmpnts           *velPtsTwr;
    Cmpnts          *velValsTwr;
    Cmpnts         *forcePtsTwr;
    Cmpnts        *forceValsTwr;

    // helper variables
    Cmpnts      forcePtBladeOld;

    // processors controlling the OpenFAST points
    PetscInt   *thisBladeVelPtControlled;
    cellIds        *closestBladeVelCells;
    PetscInt *thisBladeForcePtControlled;
    cellIds      *closestBladeForceCells;
    PetscInt     *thisTwrVelPtControlled;
    cellIds          *closestTwrVelCells;
    PetscInt   *thisTwrForcePtControlled;
    cellIds        *closestTwrForceCells;

#endif

} windTurbine;

struct farm_
{
    // main parametes
    word                    name;
    PetscInt                size;
    Cmpnts                 *base;
    PetscInt  *farmControlActive;

    word          **turbineTypes;
    word            **turbineIds;
    word         **turbineModels;

    windTurbine             **wt;

    // output parameters
    PetscReal          timeStart;
    word            intervalType;
    PetscReal       timeInterval;
    PetscInt         writeNumber;

    // debug switch
    PetscInt                 dbg;

    Vec           lsourceFarmCat,
                  sourceFarmCont;

    // access database
    access_              *access;

    // CFL control for ALM: blade tip cannot move more than one cell
    PetscInt       checkCFL;
    PetscReal      maxTipSpeed;

#if USE_OPENFAST

    // OpenFAST coupling parameters
    fast::OpenFAST         *FAST;
    fast::fastInputs          fi;

    PetscInt           nOpenFAST;
    PetscInt        *openfastIds;
    PetscInt       nFastSubSteps;
    PetscInt    fastTimeInterval;
    PetscInt         fastRestart;
#endif

};

// Top level functions
// ---------------------------------------------------------------------------

PetscErrorCode InitializeWindFarm(farm_ *farm);

```

```
PetscErrorCode UpdateWindTurbines(farm_ *farm);

PetscErrorCode computeMaxTipSpeed(farm_ *farm);

#endif
```

5.66 /zfs60/nobackup/stipa/software/TOSCA/src/include/ueqn.h File Reference

U equation solution header file.

Classes

- struct ueqn_

  structure storing momentum equation

Functions

- PetscErrorCode **InitializeUEqn** (ueqn_ *ueqn)

  Initializes Ueqn environment.
- PetscErrorCode **UpdateFluxLimiter** (ueqn_ *ueqn)

  Updates flux limiter.
- PetscErrorCode **contravariantToCartesian** (ueqn_ *ueqn)

  Transform velocity from contravariant to cartesian.
- PetscErrorCode **contravariantToCartesianGeneric** (mesh_ *mesh, Vec &lCont, Vec &lCat)

  Transform generic local vector from contravariant to cartesian.
- PetscErrorCode **adjustFluxesLocal** (ueqn_ *ueqn)

  Adjust fluxes to obey mass conservation (on a per-cell basis)
- PetscErrorCode **SolveUEqn** (ueqn_ *ueqn)

  Solve the momentum equation.

5.66.1 Detailed Description

U equation solution header file.

5.67 ueqn.h

Go to the documentation of this file.

```
#ifndef _UEQN_H_
#define _UEQN_H_

struct ueqn_
{
    // solver name
    word          solverType;

    // implicit time stepping
    SNES          snesU;
    Mat           JU;
    Mat           A, C;
```

```
    KSP           ksp;
    PC            pc;
    KSP           kspIMEX;
    Mat           JvIMEX;
    PetscInt      snesMaxIter;
    word          kspType;
    PetscInt      gmresRestart;

    // momentum variables
    Vec           Utmp;
    Vec           Rhs;
    Vec           Rhs_o;
    Vec           lFp;
    Vec           lDiv1, lDiv2, lDiv3;
    Vec           lVisc1, lVisc2, lVisc3;
    Vec           lViscIBM1, lViscIBM2, lViscIBM3;
    Vec           dP;
    Vec           dPAGW;
    Vec           bTheta;
    Vec           bTheta_o;
    Vec           sourceU;
    Vec           gCont;
    Vec           Ucont, lUcont;
    Vec           Ucat, lUcat;
    Vec           Ucont_o;
    Vec           bU;
    Vec           RhsVisc;
    Vec           RhsConv;
    Vec           RhsConv_o;

    Vec           lUstar;
    Cmpnts        meanGradP;
    Cmpnts        uBulk;

    // momentum settings
    word          ddtScheme;
    word          divScheme;
    word          viscScheme;                 //<! viscous gradient scheme
    PetscReal     relExitTol;
    PetscReal     absExitTol;
    PetscInt      inviscid;
    PetscInt      buoyancy;
    PetscInt      coriolis;
    PetscInt      fringe;

    // divergence schemes
    PetscInt      centralDiv;
    PetscInt      central4Div;
    PetscInt      centralUpwindDiv;
    PetscInt      centralUpwindWDiv;
    PetscInt      quickDiv;
    PetscInt      weno3Div;

    PetscReal     hyperVisc;
    PetscReal     hyperVisc4i;
    PetscReal     hyperVisc4j;
    PetscReal     hyperVisc4k;

    // wall model patch
    wallModel     *iLWM;
    wallModel     *iRWM;
    wallModel     *jLWM;
    wallModel     *jRWM;

    // initial field
    word           initFieldType;

    // access
    access_       *access;

};

#endif

PetscErrorCode InitializeUEqn(ueqn_ *ueqn);

PetscErrorCode UpdateFluxLimiter(ueqn_ *ueqn);

PetscErrorCode contravariantToCartesian(ueqn_ *ueqn);

PetscErrorCode contravariantToCartesianGeneric(mesh_ *mesh, Vec &lCont, Vec &lCat);

PetscErrorCode adjustFluxesLocal(ueqn_ *ueqn);

PetscErrorCode SolveUEqn(ueqn_ *ueqn);
```

5.68 /zfs60/nobackup/stipa/software/TOSCA/src/include/wallfunctions.h File Reference

wallfunction functions

```
#include "io.h"
```

Functions

- PetscReal **uTauCabot** (PetscReal nu, PetscReal u, PetscReal y, PetscReal guess, PetscReal dpdn)
- PetscReal **utau_wf** (PetscReal nu, PetscReal ks, PetscReal sb, PetscReal Ut_mag)
- PetscReal **uTauCabotRoughness** (PetscReal nu, PetscReal u, PetscReal y, PetscReal guess, PetscReal dpdn, PetscReal ks)
- void **wallFunctionCabot** (PetscReal nu, PetscReal sc, PetscReal sb, Cmpnts Ua, Cmpnts Uc, Cmpnts ∗Ub, PetscReal ∗ustar, Cmpnts nf)
- void **wallFunctionCabotRoughness** (PetscReal nu, PetscReal ks, PetscReal sc, PetscReal sb, Cmpnts Ua, Cmpnts Uc, Cmpnts ∗Ub, PetscReal ∗ustar, Cmpnts nf)
- void **wallFunctionPowerlaw** (PetscReal nu, PetscReal sc, PetscReal sb, Cmpnts Ua, Cmpnts Uc, Cmpnts ∗Ub, PetscReal ∗ustar, Cmpnts nf)
- void **wallFunctionPowerlawAPG** (PetscReal nu, PetscReal sc, PetscReal sb, PetscReal roughness, Petsc↩ Real kappa, Cmpnts Ua, Cmpnts Uc, Cmpnts ∗Ub, PetscReal ∗ustar, Cmpnts nf, PetscReal dpdx, PetscReal dpdy, PetscReal dpdz)
- void **wallFunctionLogLawAPG** (PetscReal nu, PetscReal sc, PetscReal sb, PetscReal roughness, Petsc↩ Real kappa, Cmpnts Ua, Cmpnts Uc, Cmpnts ∗Ub, PetscReal ∗ustar, Cmpnts nf, PetscReal dpdx, PetscReal dpdy, PetscReal dpdz)
- void **slipBC** (PetscReal sc, PetscReal sb, Cmpnts Ua, Cmpnts Uc, Cmpnts ∗Ub, Cmpnts nf)
- void **wallFunctionSchumann** (PetscReal nu, PetscReal sc, PetscReal sb, PetscReal roughness, PetscReal kappa, Cmpnts Ua, Cmpnts Uc, Cmpnts ∗Ub, PetscReal ∗ustar, Cmpnts nf)
- void **wallShearVelocityBC** (PetscReal nu, PetscReal sc, PetscReal sb, PetscReal roughness, PetscReal kappa, Cmpnts Ua, Cmpnts Uc, Cmpnts ∗Ub, PetscReal ∗ustar, Cmpnts nf)
- void **wallShearVelocityBCQuadratic** (PetscReal nu, PetscReal sd, PetscReal sc, PetscReal sb, PetscReal roughness, PetscReal kappa, Cmpnts Ua, Cmpnts Ud, Cmpnts Uc, Cmpnts ∗Ub, PetscReal ∗ustar, Cmpnts nf)
- void **wallShearGhostVelocityBC** (PetscReal sd, PetscReal sc, PetscReal sb, PetscReal roughness, Petsc↩ Real kappa, Cmpnts Ua, Cmpnts Ud, Cmpnts Uc, Cmpnts ∗Ub, PetscReal ∗ustar, Cmpnts nf)
- void **ghostTempVelocityBCShumann** (Shumann ∗wm, PetscReal sd, PetscReal sc, PetscReal sb, Cmpnts Ua, Cmpnts Ud, Cmpnts ∗Ub, PetscReal Td, PetscReal surfaceTemp, Cmpnts nf, PetscInt k, PetscInt j, PetscInt i)
- void **uStarShumann** (PetscReal &UParallelMeanMag, PetscReal &wallDist, PetscReal &z0, PetscReal &gammaM, PetscReal &kappa, PetscReal &qwall, PetscReal &thetaRef, PetscReal &uStar, PetscReal &phiM, PetscReal &L, PetscInt k, PetscInt j, PetscInt i)
- void **qWallShumann** (PetscReal &UParallelMeanMag, PetscReal &wallDist, PetscReal &z0, PetscReal &gammaM, PetscReal &gammaH, PetscReal &alphaH, PetscReal &thetaRef, PetscReal &deltaTheta, PetscReal &kappa, PetscReal &qWall, PetscReal &uStar, PetscReal &phiM, PetscReal &phiH, PetscReal &L, PetscInt k, PetscInt j, PetscInt i)

5.68.1 Detailed Description

wallfunction functions

5.69 wallfunctions.h

Go to the documentation of this file.

```
#include "io.h"

// WALL FUNCTIONS
//
==========================================================================================================
//

PetscReal uTauCabot(PetscReal nu, PetscReal u, PetscReal y, PetscReal guess, PetscReal dpdn);

PetscReal utau_wf(PetscReal nu, PetscReal ks, PetscReal sb, PetscReal Ut_mag);

PetscReal uTauCabotRoughness(PetscReal nu, PetscReal u, PetscReal y, PetscReal guess, PetscReal dpdn, PetscReal ks);

PetscReal uTauCabot(PetscReal nu, PetscReal u, PetscReal y, PetscReal guess, PetscReal dpdn);

void      wallFunctionCabot(PetscReal nu, PetscReal sc, PetscReal sb, Cmpnts Ua, Cmpnts Uc, Cmpnts *Ub, PetscReal *ustar, Cmpnts nf);

void      wallFunctionCabotRoughness(PetscReal nu, PetscReal ks, PetscReal sc, PetscReal sb,
            Cmpnts Ua, Cmpnts Uc, Cmpnts *Ub, PetscReal *ustar, Cmpnts nf);

void      wallFunctionPowerlaw(PetscReal nu, PetscReal sc, PetscReal sb, Cmpnts Ua, Cmpnts Uc, Cmpnts *Ub, PetscReal *ustar, Cmpnts nf);

void      wallFunctionPowerlawAPG(PetscReal nu, PetscReal sc, PetscReal sb, PetscReal roughness, PetscReal kappa, Cmpnts Ua,
                                  Cmpnts Uc, Cmpnts *Ub, PetscReal *ustar, Cmpnts nf, PetscReal dpdx, PetscReal dpdy, PetscReal dpdz);

void      wallFunctionLogLawAPG(PetscReal nu, PetscReal sc, PetscReal sb, PetscReal roughness, PetscReal kappa, Cmpnts Ua,
                                  Cmpnts Uc, Cmpnts *Ub, PetscReal *ustar, Cmpnts nf, PetscReal dpdx, PetscReal dpdy, PetscReal dpdz);

void      slipBC(PetscReal sc, PetscReal sb, Cmpnts Ua, Cmpnts Uc, Cmpnts *Ub, Cmpnts nf);

void      wallFunctionSchumann(PetscReal nu, PetscReal sc, PetscReal sb, PetscReal roughness,
                                    PetscReal kappa, Cmpnts Ua, Cmpnts Uc, Cmpnts *Ub, PetscReal *ustar, Cmpnts nf);

void      wallShearVelocityBC(PetscReal nu, PetscReal sc, PetscReal sb, PetscReal roughness,
                                    PetscReal kappa, Cmpnts Ua, Cmpnts Uc, Cmpnts *Ub, PetscReal *ustar, Cmpnts nf);

void      wallShearVelocityBCQuadratic(PetscReal nu,  PetscReal sd, PetscReal sc, PetscReal sb, PetscReal roughness,
                                    PetscReal kappa, Cmpnts Ua, Cmpnts Ud, Cmpnts Uc, Cmpnts *Ub, PetscReal *ustar, Cmpnts nf);

void      wallShearGhostVelocityBC(PetscReal sd, PetscReal sc, PetscReal sb, PetscReal roughness,
                                    PetscReal kappa, Cmpnts Ua, Cmpnts Ud, Cmpnts Uc, Cmpnts *Ub, PetscReal *ustar, Cmpnts nf);

void      ghostTempVelocityBCShumann(Shumann *wm, PetscReal sd, PetscReal sc, PetscReal sb, Cmpnts Ua, Cmpnts Ud, Cmpnts *Ub,
                                    PetscReal Td, PetscReal surfaceTemp, Cmpnts nf, PetscInt k, PetscInt j, PetscInt i);

void uStarShumann
(
    PetscReal &UParallelMeanMag, PetscReal &wallDist, PetscReal &z0,
    PetscReal &gammaM, PetscReal &kappa, PetscReal &qwall, PetscReal &thetaRef,
    PetscReal &uStar, PetscReal &phiM, PetscReal &L,
    PetscInt k, PetscInt j, PetscInt i
);

void qWallShumann
(
    PetscReal &UParallelMeanMag, PetscReal &wallDist, PetscReal &z0,
    PetscReal &gammaM, PetscReal &gammaH, PetscReal &alphaH,
    PetscReal &thetaRef, PetscReal &deltaTheta, PetscReal &kappa,
    PetscReal &qWall, PetscReal &uStar, PetscReal &phiM, PetscReal &phiH, PetscReal &L,
    PetscInt k, PetscInt j, PetscInt i
);
```

5.70 /zfs60/nobackup/stipa/software/TOSCA/src/include/wallmodel.h File Reference

Wall models header file.

Classes

- struct Shumann

  structure storing the Shumann wall models information for U and T
- struct Cabot

  structure storing the Shumann wall models information
- struct PowerLawAPG

  structure storing the Shumann wall models information
- struct LogLawAPG
- struct wallModel

  wall models container

5.70.1 Detailed Description

Wall models header file.

5.71 wallmodel.h

Go to the documentation of this file.

```
00001
00003
00005 struct Shumann
00006 {
00007     word            wfEvalType;
00008     PetscReal       kappa;
00009     PetscReal       thetaRef;
00010     PetscReal       roughness;
00011     PetscReal       gammaM;
00012     PetscReal       betaM;
00013     PetscReal       gammaH;
00014     PetscReal       betaH;
00015     PetscReal       alphaH;
00016     PetscReal       tLast;
00017     PetscReal       heatingRate;
00018     PetscReal       **surfaceTheta;
00019     PetscInt        surfaceThetaSet;
00020     PetscReal       qWall;
00021     PetscReal       *surfTemp;
00022     PetscReal       *surfL;
00023     PetscReal       *timeVec;
00024     PetscInt        numT;
00025 };
00026
00028 struct Cabot
00029 {
00030     PetscReal       roughness;
00031     PetscReal       kappa;
00032 };
00033
00035 struct PowerLawAPG
00036 {
00037     PetscReal       roughness;
00038     PetscReal       kappa;
00039 };
00040
00041 struct LogLawAPG
00042 {
00043     PetscReal       roughness;
```

```
    PetscReal        kappa;
};

struct wallModel
{
    Shumann          *wmShumann;
    Cabot            *wmCabot;
    PowerLawAPG      *wmPowerLawAPG;
    LogLawAPG        *wmLogLawAPG;

    patchVectorField tauWall;
    patchVectorField qWall;
    patchVectorField uFilt;

    PetscReal        **uMeanMag;
    PetscInt         uFiltSet;
};
```

5.72 /zfs60/nobackup/stipa/software/TOSCA/src/initialField.c File Reference

Contains initial field function definitions.

```
#include "include/base.h"
#include "include/domain.h"
#include "include/io.h"
#include "include/inline.h"
#include "include/initialField.h"
```

Functions

- PetscErrorCode SetInitialField (domain_ *domain)

  <

- PetscErrorCode **SetInitialFieldPrecursor** (abl_ *abl)

  set the initial internal contravariant and cartesian velocity field

- PetscErrorCode **SetInitialFieldU** (ueqn_ *ueqn)

  set the initial temperature field

- PetscErrorCode **SetInitialFieldT** (teqn_ *teqn)

  set the initial pressure field

- PetscErrorCode **SetInitialFieldP** (peqn_ *peqn)

  set the initial variables for LES

- PetscErrorCode **SetInitialFieldLES** (les_ *les)

  set uniform value for the cartesian velocity (add perturbations if applicable)

- PetscErrorCode **SetUniformFieldU** (ueqn_ *ueqn, Cmpnts &uRef, PetscInt &addPerturbations)

  set initial ABL flow U

- PetscErrorCode **SetABLInitialFlowU** (ueqn_ *ueqn)
- PetscErrorCode **SetABLInitialFlowUZilitinkevich** (ueqn_ *ueqn)

  set initial field Taylor green vortex problem

- PetscErrorCode **SetTaylorGreenFieldU** (ueqn_ *ueqn, PetscReal &u0, PetscReal &freq)

  set the internal field as the spreaded inlet flow condition

- PetscErrorCode **SpreadInletFlowU** (ueqn_ *ueqn)

  set uniform value for the temperature

- PetscErrorCode **SetUniformFieldT** (teqn_ *teqn, PetscReal &tRef)

  set linear profile for the temperature

- PetscErrorCode **SetLinearFieldT** (teqn_ *teqn, PetscReal &tRef, PetscReal &tLapse)

set const tRef + linear profile for the temperature above it like CNBL

- PetscErrorCode **SetLinearCNBLFieldT** (teqn_ *teqn, PetscReal &tRef, PetscReal &tLapse, PetscReal &h↩Ref)

  set initial ABL flow T

- PetscErrorCode **SetABLInitialFlowT** (teqn_ *teqn)

  set the internal field as the spreaded inlet flow condition

- PetscErrorCode **SpreadInletFlowT** (teqn_ *teqn)

  set the internal pressure field as the TGV flow

- PetscErrorCode **SetTaylorGreenFieldP** (peqn_ *peqn)

5.72.1 Detailed Description

Contains initial field function definitions.

5.72.2 Function Documentation

5.72.2.1 SetInitialField()

```
PetscErrorCode SetInitialField (
            domain_ * domain)
```

<

set the initial internal contravariant and cartesian velocity field

set the initial internal fields

set the initial internal fields for the concurrent precursor simulation

5.73 /zfs60/nobackup/stipa/software/TOSCA/src/initialization.c File Reference

Contains inflow boundary condition function definitions.

```
#include "include/base.h"
#include "include/domain.h"
#include "include/io.h"
#include "include/inline.h"
#include "include/boundary.h"
#include "include/inflow.h"
#include "include/abl.h"
#include "include/turbines.h"
#include "include/initialization.h"
```

Functions

- PetscErrorCode **PrintOkWindLogo** ()

  Print OkWind logo.
- PetscErrorCode **PrintNumberOfProcs** ()

  Print number of processors.
- PetscErrorCode simulationInitialize (domain_ ∗∗domainAddr, clock_ ∗clock, simInfo_ ∗info, flags_ ∗flags)

  Initialize the simulation parameters.
- PetscErrorCode **SetSimulationFlags** (flags_ ∗flags)

  Set simulation flags.
- PetscErrorCode **SetSimulationInfo** (simInfo_ ∗info)

  Set simulation global info.
- PetscErrorCode **SetDomainsAndAllocate** (domain_ ∗∗domainAddr, flags_ ∗flags, simInfo_ ∗info)

  Set number of domains (reads from OversetInput.dat file if overset is active)
- PetscErrorCode **ReadTimeControls** (clock_ ∗clock)

  Read time controls.
- PetscErrorCode **SetStartTime** (clock_ ∗clock, domain_ ∗domain, simInfo_ ∗info)

  Get startFrom parameter and set initial time (check multiple domains cosistency)
- PetscErrorCode **ReadPhysicalConstants** (domain_ ∗domain)

  Read physical constants.
- PetscErrorCode **SetDomainMemory** (domain_ ∗domain)

  Allocate memory for domain pointers.
- PetscErrorCode **SetAccessPointers** (domain_ ∗domain)

  Set access database pointers.

5.73.1 Detailed Description

Contains inflow boundary condition function definitions.

Contains simulation initialization function definitions.

5.73.2 Function Documentation

5.73.2.1 simulationInitialize()

```
PetscErrorCode simulationInitialize (
            domain_ ** domainAddr,
            clock_ * clock,
            simInfo_ * info,
            flags_ * flags)
```

Initialize the simulation parameters.

set the initial internal fields

5.74 /zfs60/nobackup/stipa/software/TOSCA/src/io.c File Reference

Contains i/o operations function definitions.

```
#include "include/base.h"
#include "include/domain.h"
#include "include/io.h"
#include "include/inline.h"
```

Functions

- PetscErrorCode **InitializeIO** (io_ ∗io)

  Initializes io data structure.
- PetscErrorCode **ReadIO** (domain_ ∗domain)

  Re-read IO write parameters.
- PetscErrorCode **UpdateInput** (io_ ∗io, word &modified)

  Called by ReadIO, triggers updates in each domain.
- PetscErrorCode **readFields** (domain_ ∗domain, PetscReal timeValue)

  read and save the different fields stored in the fields/meshName/time folder
- void **writeBinaryField** (MPI_Comm comm, Vec V, const char ∗file)
- PetscErrorCode **VecScalarLocalToGlobalCopy** (mesh_ ∗mesh, Vec &lV, Vec &V)
- PetscErrorCode **VecVectorLocalToGlobalCopy** (mesh_ ∗mesh, Vec &lV, Vec &V)
- PetscErrorCode **VecSymmTensorLocalToGlobalCopy** (mesh_ ∗mesh, Vec &lV, Vec &V)
- PetscErrorCode **VecTensorLocalToGlobalCopy** (mesh_ ∗mesh, Vec &lV, Vec &V)
- PetscErrorCode **initializeTimeDir** (io_ ∗io)

  Creates the checkpoint directory with a trailing _tmp (indicates that it is not finalized)
- PetscErrorCode **finalizeTimeDir** (io_ ∗io)

  Renames the temporary time directory to the final name (without _tmp) and deletes prev chekpoints if purge is active.
- PetscErrorCode **writeFields** (io_ ∗io)

  Write output fields.
- void **fatalErrorInFunction** (const char ∗functionName, const char ∗errorMsg)

  Calls fatal error and exits.
- void **warningInFunction** (const char ∗functionName, const char ∗wrngMsg)

  Calls warning.
- void **createDir** (MPI_Comm comm, const char ∗path)

  Creat directory (remove stuff it exists)
- void **createDirNoRemove** (MPI_Comm comm, const char ∗path)

  Creat directory (leave stuff it exists)
- word **thisCaseName** ()

  Retrieves this case name.
- PetscErrorCode **getFileList** (const char ∗dataLoc, std::vector< word > &fileSeries, PetscInt &nfiles)

  Returns the file list (from folder names) and number of files contained in a folder.
- PetscErrorCode getTimeList (const char ∗dataLoc, std::vector< PetscReal > &timeSeries, PetscInt &ntimes)

  Returns the time list (from folder names) and size contained in a folder.
- PetscInt **foundInString** (const char ∗str, word keyword)

  Finds a word inside a string.
- PetscInt **file_exist** (const char ∗str)

  Checks if file exists.
- PetscInt **dir_exist** (const char ∗str)

  Checks if directory exists.

- PetscInt **count_files** (const char ∗path)

  Count number of files.
- void **remove_dir** (MPI_Comm comm, const char ∗path2dir)

  Removes directory.
- void **remove_subdirs** (MPI_Comm comm, const char ∗path2dir)

  Removes all subdirectory of a given directory.
- void **remove_subdirs_except** (MPI_Comm comm, const char ∗path2dir, const word name)

  Removes all subdirectory of a given directory except the name provided.
- void **remove_subdirs_except_keep_n** (MPI_Comm comm, const char ∗path2dir, const word name, PetscInt nKeep)

  Removes all subdirectory except the latest n times inside a given directory and the name provided.
- void **remove_subdirs_except2** (MPI_Comm comm, const char ∗path2dir, const word name1, const word name2)

  Removes all subdirectory of a given directory except the 2 names provided.
- void **remove_subdirs_except3** (MPI_Comm comm, const char ∗path2dir, const word name1, const word name2, const word name3)

  Removes all subdirectory of a given directory except the 3 names provided.
- void **remove_subdirs_except4** (MPI_Comm comm, const char ∗path2dir, const word name1, const word name2, const word name3, const word name4)

  Removes all subdirectory of a given directory except the 4 names provided.
- void **remove_subdirs_except4_keep_n** (MPI_Comm comm, const char ∗path2dir, const word name1, const word name2, const word name3, const word name4, PetscInt nKeep)

  Removes all subdirectory except the latest n times inside a given directory and the 4 names provided.
- word **getTimeName** (clock_ ∗clock)

  Get time name in string format with required digits.
- word **getArbitraryTimeName** (clock_ ∗clock, double timeValue)

  Get time value in string format with required digits.
- word **getStartTimeName** (clock_ ∗clock)

  Get start time name in string format with required digits.
- PetscErrorCode **setRunTimeWrite** (domain_ ∗domain)

  Sets the runTimeWrite flag and creates initial output directory.
- PetscErrorCode **readDictDouble** (const char ∗dictName, const char ∗keyword, PetscReal ∗value)

  Read mandatory PetscReal from dictionary.
- PetscErrorCode **readDictVector2D** (const char ∗dictName, const char ∗keyword, Cpt2D ∗value)

  Read mandatory 2D vector from dictionary.
- PetscErrorCode **readDictVector** (const char ∗dictName, const char ∗keyword, Cmpnts ∗value)

  Read mandatory vector from dictionary.
- PetscErrorCode **readDictInt** (const char ∗dictName, const char ∗keyword, PetscInt ∗value)

  Read mandatory PetscInt from dictionary.
- PetscErrorCode **readDictWord** (const char ∗dictName, const char ∗keyword, word ∗value)

  Read mandatory word from dictionary.
- PetscErrorCode **readDictWordAndDouble** (const char ∗dictName, const char ∗keyword, word ∗value1, PetscReal ∗value2)

  Read mandatory word and PetscReal from dictionary.
- PetscErrorCode **readDictWordAndVector** (const char ∗dictName, const char ∗keyword, word ∗value1, Cmpnts ∗value2)

  Read mandatory word and vector from dictionary.
- PetscErrorCode **readSubDictDouble** (const char ∗dictName, const char ∗subdict, const char ∗keyword, PetscReal ∗value)

  Read mandatory PetscReal from sub-dictionary.
- PetscErrorCode **readSubDictInt** (const char ∗dictName, const char ∗subdict, const char ∗keyword, PetscInt ∗value)

Read mandatory PetscInt from sub-dictionary.

- PetscErrorCode **readSubDictWord** (const char ∗dictName, const char ∗subdict, const char ∗keyword, word ∗value)

  Read mandatory word from sub-dictionary.

- PetscErrorCode **readSubDictVector** (const char ∗dictName, const char ∗subdict, const char ∗keyword, Cmpnts ∗value)

  Read mandatory vector from sub-dictionary.

- PetscErrorCode **readSubDictIntArray** (const char ∗dictName, const char ∗subdict, const char ∗keyword, labelList &value)

  Read mandatory array of PetscInt from sub-dictionary.

- std::string ∗ **readSubDictWordArray** (const char ∗dictName, const char ∗subdict, const char ∗keyword, PetscInt numW)
- word **trim** (const word &str)

  Trims a string removing pre and trail spaces.

- bool **isNumber** (const word &str)

  Check if is a number.

5.74.1 Detailed Description

Contains i/o operations function definitions.

5.74.2 Function Documentation

5.74.2.1 getTimeList()

```
PetscErrorCode getTimeList (
            const char * dataLoc,
            std::vector< PetscReal > & timeSeries,
            PetscInt & ntimes)
```

Returns the time list (from folder names) and size contained in a folder.

gets list of time folders contained in a directory

5.75 /zfs60/nobackup/stipa/software/TOSCA/src/les.c File Reference

Contains LES model function definitions.

```
#include "include/base.h"
#include "include/domain.h"
#include "include/io.h"
#include "include/inline.h"
```

Functions

- PetscErrorCode **InitializeLES** (les_ *les)

  Initializes LES environment.
- PetscErrorCode UpdateCs (les_ *les)

  Update Cs Smagorinky coefficient.
- PetscErrorCode **UpdateNut** (les_ *les)

  Update effective viscosity.
- PetscErrorCode **ComputeTurbulenceStatistics** (les_ *les)

  Compute turbulent statistics based on the les->turbStat flag.
- PetscErrorCode **updateLESStructuralModelCartesianForm** (les_ *les)
- PetscErrorCode **updateLESStructuralModelContravariantForm** (les_ *les)

Variables

- const PetscReal **wall_cs** = 0.001
- const PetscReal **std_cs** = 0.0289
- const PetscReal **amd_cs** = 0.1
- const PetscReal **vreman_cs** = 0.07225
- const PetscReal **sa_cs** = 0.325

5.75.1 Detailed Description

Contains LES model function definitions.

5.75.2 Function Documentation

5.75.2.1 UpdateCs()

```
PetscErrorCode UpdateCs (
            les_ * les)
```

Update Cs Smagorinky coefficient.

isOnCornerCellCenters(I, J, K, mesh->info) <- use also ghost? Not for now

isOnCornerCellCenters(I, J, K, mesh->info) <- use also ghost? Not for now

5.76 /zfs60/nobackup/stipa/software/TOSCA/src/mesh.c File Reference

Contains mesh definition functions.

```
#include "include/base.h"
#include "include/domain.h"
#include "include/io.h"
#include "include/inline.h"
```

Functions

- PetscErrorCode **InitializeMesh** (mesh_ *mesh)

  Initialize mesh.
- PetscErrorCode **SetDistributedArrays** (mesh_ *mesh)

  Set distributed arrays.
- PetscErrorCode **DeformMeshBasedOnBLDisp** (mesh_ *mesh)

  Deform the mesh according to prescribed BL Disp.
- PetscErrorCode **SetMeshMetrics** (mesh_ *mesh)

  Set curvilinear coordinates metrics.
- PetscErrorCode **SetBoundingBox** (mesh_ *mesh)

  Set bounding box.
- PetscErrorCode **ghostnodesCellcenter** (mesh_ *mesh)

  Find the cell center for the ghost nodes.

5.76.1 Detailed Description

Contains mesh definition functions.

5.77 /zfs60/nobackup/stipa/software/TOSCA/src/overset.c File Reference

overset function definitions

```
#include "include/base.h"
#include "include/domain.h"
#include "include/ibm.h"
#include "include/io.h"
#include "include/inline.h"
#include "include/initialField.h"
#include "include/overset.h"
#include "include/ibmInput.h"
```

Classes

- struct OctreeNode

Functions

- PetscErrorCode **InitializeOverset** (domain_ *domain)

  Initialize overset coupling.
- PetscErrorCode **UpdateOversetInterpolation** (domain_ *domain)
- PetscErrorCode **UpdateDomainInterpolation** (PetscInt d, domain_ *domain, PetscInt level)

  Update overset interpolation for all domains.
- PetscErrorCode **findAcceptorCells** (PetscInt d, domain_ *domain, PetscInt level, const std::vector< HoleObject > &holeObjects)
- PetscErrorCode **findClosestDomainDonors** (PetscInt d, domain_ *domain, PetscInt level, const std↩ ::vector< HoleObject > &holeObjects)

- PetscErrorCode **FindHoleObject** (const std::vector< HoleObject > &holeObjects, PetscInt parentId, PetscInt childId, char **holeObjectName)
- PetscErrorCode **readHoleObjects** (std::vector< HoleObject > &holeObjects, PetscInt numHoleObjects)
- PetscErrorCode **readBlankingIBMObject** (overset_ *os, domain_ *domain, char *holeObjectName, const std::vector< HoleObject > &holeObjects)
- PetscErrorCode **readOversetProperties** (overset_ *os)
- PetscErrorCode **interpolateACellTrilinearP2C** (mesh_ *meshD, mesh_ *meshA)
- PetscErrorCode **interpolateACellTrilinearC2P** (mesh_ *meshD, mesh_ *meshA, PetscInt donorId)
- PetscErrorCode **setBackgroundBC** (mesh_ *meshA)
- PetscErrorCode **createAcceptorCellOverset** (overset_ *os)
- PetscErrorCode **createAcceptorCellBackground** (overset_ *os, PetscInt donorMeshId)

  Create the list of background acceptor cells.
- void **buildOctree** (OctreeNode *node, Cmpnts ***donorCells, PetscInt lxs, PetscInt lxe, PetscInt lys, PetscInt lye, PetscInt lzs, PetscInt lze, PetscInt maxDepth, PetscInt maxCellsPerNode)
- Dcell **searchOctree** (OctreeNode *node, PetscReal procContrib, const Cmpnts &acceptorCoord, Cmpnts ***centroids, PetscReal &minDist, PetscInt lxs, PetscInt lxe, PetscInt lys, PetscInt lye, PetscInt lzs, PetscInt lze)
- PetscErrorCode **findClosestDonorC2P_Bins** (mesh_ *meshDonor, mesh_ *meshAcceptor, PetscInt donorId)
- PetscErrorCode **findClosestDonorC2P** (mesh_ *meshDonor, mesh_ *meshAcceptor, PetscInt donorId)
- PetscErrorCode **findClosestDonorP2C** (mesh_ *meshDonor, mesh_ *meshAcceptor)
- PetscErrorCode **findClosestDonorP2C_Bins** (mesh_ *meshDonor, mesh_ *meshAcceptor)
- PetscErrorCode **updateAcceptorCoordinates** (overset_ *os)
- PetscErrorCode **oversetMeshTranslation** (overset_ *os)
- PetscErrorCode **oversetIbmSearch** (ibm_ *ibm)
- PetscErrorCode **computeOversetIBMElementNormal** (ibm_ *ibm)
- void **defineStruct_AcellI** (MPI_Datatype *tstype)
- void **sum_struct_AcellI** (void *in, void *inout, int *len, MPI_Datatype *type)

5.77.1 Detailed Description

overset function definitions

5.78 /zfs60/nobackup/stipa/software/TOSCA/src/peqn.c File Reference

Contains P equation function definitions.

```
#include "include/base.h"
#include "include/domain.h"
#include "include/io.h"
#include "include/inline.h"
```

Macros

- #define matID(i, j, k)
- #define **CP** 0
- #define **EP** 1
- #define **WP** 2
- #define **NP** 3
- #define **SP** 4
- #define **TP** 5
- #define **BP** 6
- #define **NE** 7
- #define **SE** 8
- #define **NW** 9
- #define **SW** 10
- #define **TN** 11
- #define **BN** 12
- #define **TS** 13
- #define **BS** 14
- #define **TE** 15
- #define **BE** 16
- #define **TW** 17
- #define **BW** 18

Functions

- PetscErrorCode **InitializePEqn** (peqn_ *peqn)

  Initializes Peqn environment.
- PetscErrorCode **SetPoissonConnectivity** (peqn_ *peqn)

  Compute cell-matrix connectivity.
- PetscErrorCode **CreateHypreMatrix** (peqn_ *peqn)

  Initializes HYPRE matrix.
- PetscErrorCode **CreatePETScMatrix** (peqn_ *peqn)

  Initializes PETSc matrix.
- PetscErrorCode **CreateHypreVector** (peqn_ *peqn)

  Initializes HYPRE vector.
- PetscErrorCode **CreatePETScVector** (peqn_ *peqn)

  Initializes PETSc vector.
- PetscErrorCode **CreateHypreSolver** (peqn_ *peqn)

  Initializes HYPRE solver.
- PetscErrorCode **MyKSPMonitorPoisson** (KSP ksp, PetscInt iter, PetscReal rnorm, void *dummy)
- PetscErrorCode **CreatePETScSolver** (peqn_ *peqn)

  Initializes PETSc solver.
- PetscErrorCode **DestroyHypreMatrix** (peqn_ *peqn)

  Destroys HYPRE matrix.
- PetscErrorCode **DestroyHypreVector** (peqn_ *peqn)

  Destroys HYPRE vector.
- PetscErrorCode **DestroyPETScVector** (peqn_ *peqn)

  Destroys PETSc vector.
- PetscErrorCode **DestroyHypreSolver** (peqn_ *peqn)

  Destroys HYPRE solver.
- PetscErrorCode **DestroyPETScSolver** (peqn_ *peqn)

  Destroys PETSc solver.

- cellIds **GetIdFromStencil** (int stencil, int k, int j, int i)

  get the cell id from stencil position
- PetscErrorCode **SetCoeffMatrix** (peqn_ *peqn)

  Compute coefficient matrix.
- PetscErrorCode **Petsc2HypreVector** (Vec &A, HYPRE_IJVector &B, HYPRE_Int startID)

  Transfer vector values from Petsc 2 Hypre.
- PetscErrorCode **Hypre2PetscVector** (HYPRE_IJVector &B, Vec &A, HYPRE_Int startID)

  Transfer vector values from Hypre 2 Petsc.
- PetscErrorCode **phiToPhi** (peqn_ *peqn)

  Convert phi to Phi (1D to 3D vector)
- PetscReal **L2NormHypre** (peqn_ *peqn, HYPRE_IJMatrix &A, HYPRE_IJVector &X, HYPRE_IJVector &B)

  Compute L2 norm of system residual.
- PetscErrorCode **SubtractAverageHypre** (peqn_ *peqn, HYPRE_IJVector &B)

  Subtract average from the solution.
- PetscErrorCode **SubtractAveragePETSc** (peqn_ *peqn, Vec &B)

  Subtract average from the solution.
- PetscErrorCode **SetRHS** (peqn_ *peqn)

  Set RHS of the pressure equation.
- PetscErrorCode **AdjustIBMFlux** (peqn_ *peqn)

  Correct IBM volume flux.
- PetscErrorCode **ProjectVelocity** (peqn_ *peqn)

  Project Ucont into an incompressible space.
- PetscErrorCode **UpdatePressure** (peqn_ *peqn)

  Update pressure and subtract average.
- PetscErrorCode **GradP4thOrder** (peqn_ *peqn)

  Compute 4th order pressure gradient term.
- PetscErrorCode **GradP** (peqn_ *peqn)

  Compute pressure gradient term.
- PetscErrorCode **SolvePEqn** (peqn_ *peqn)

  Compute pressure gradient term.
- PetscErrorCode **SetPressureReference** (peqn_ *peqn)

  Set pressure = 0 at k = 0, j = 0, i = 0.
- PetscErrorCode **ContinuityErrors** (peqn_ *peqn)

  Compute continuity errors (also calculates which cell and processor has the max)
- PetscErrorCode **ContinuityErrorsOptimized** (peqn_ *peqn)

  Compute continuity errors (only prints the max)

5.78.1 Detailed Description

Contains P equation function definitions.

5.78.2 Macro Definition Documentation

5.78.2.1 matID

```
#define matID(
            i,
            j,
            k)
```

Value:

```
(HYPRE_Int)(gid[k][j][i])
```

5.79 /zfs60/nobackup/stipa/software/TOSCA/src/precursor.c File Reference

Contains top to bottom level routines for the concurrent precursor method.

```
#include "include/base.h"
#include "include/domain.h"
#include "include/io.h"
#include "include/inline.h"
#include "include/inflow.h"
#include "sources/ueqn_sources.h"
#include "sources/teqn_sources.h"
#include "solvers/ueqn_solvers.h"
#include "include/initialization.h"
#include "include/initialField.h"
```

Functions

- PetscErrorCode **SetSolutionFlagsPrecursor** (domain_ *domain)

  Precursor solution flags definition.
- PetscErrorCode **concurrentPrecursorInitialize** (abl_ *abl)

  initialize concurrent precursor
- PetscErrorCode **InitializePrecursorIBM** (ibm_ *ibm, ibm_ *succIbm)
- PetscErrorCode **SetStartTimePrecursor** (domain_ *domain, abl_ *abl)

  Checks that start time is available to read if spinUp == 0.
- PetscErrorCode **InitializeMeshPrecursor** (abl_ *abl)

  Precursor mesh initialization.
- PetscErrorCode **concurrentPrecursorSolve** (abl_ *abl)

  Solve concurrent precursor.
- PetscErrorCode **SetBoundaryConditionsPrecursor** (mesh_ *mesh)

  Precursor boundary conditions initialization function.
- PetscErrorCode **SetInflowFunctionsPrecursor** (mesh_ *mesh)

  Sets the inflow function on the precursor.
- PetscErrorCode **MapInitialConditionPrecursor** (abl_ *abl)

  Map fields from successor to precursor.
- PetscErrorCode **successorPrecursorMapVectorField** (abl_ *abl, Vec &Source, Vec &Target, Vec &ITarget)

  Map vector field from successor to precursor.
- PetscErrorCode **successorPrecursorMapScalarField** (abl_ *abl, Vec &Source, Vec &Target, Vec &ITarget)

  Map scalar tor field from successor to precursor.
- PetscErrorCode **ABLInitializePrecursor** (domain_ *domain)

  Initialize abl for precursor (x damping layer is not set)
- PetscErrorCode **saveSourcePrecursor2Successor** (abl_ *precursorABL, abl_ *successorABL)

5.79.1 Detailed Description

Contains top to bottom level routines for the concurrent precursor method.

5.80 /zfs60/nobackup/stipa/software/TOSCA/src/solvers/teqn_solvers.c File Reference

Temperature-equation solver function implementations.

```
#include "teqn_solvers.h"
```

Functions

- PetscErrorCode **FormT** (teqn_ ∗teqn, Vec &Rhs, PetscReal scale, PetscInt formMode)

  Viscous and divergence terms.
- PetscErrorCode **TeqnSNES** (teqn_ ∗teqn)

  Implicit SNES solver for T equation.
- PetscErrorCode **SNESFuncEvalT** (SNES snes, Vec T, Vec Rhs, void ∗ptr)

  SNES evaluation function.
- PetscErrorCode **FormExplicitRhsT** (teqn_ ∗teqn)

  Compute RHS of temperature equation using current ITmprt.
- PetscErrorCode **TeqnRK4** (teqn_ ∗teqn)

  Solve Teqn using RungeKutta 4.
- PetscErrorCode **TeqnABBE** (teqn_ ∗teqn)

  IMEX time step for T equation (AB2 convection + backward-Euler BDF1 diffusion)
- PetscErrorCode **ABBEMatVec** (Mat A, Vec v, Vec Av)

  MatShell operator for ABBE linear system: A∗v = v - dt∗D(v)

5.80.1 Detailed Description

Temperature-equation solver function implementations.

5.81 /zfs60/nobackup/stipa/software/TOSCA/src/solvers/teqn_solvers.h File Reference

Private temperature-equation solver function declarations.

```
#include "../include/base.h"
#include "../include/domain.h"
#include "../include/io.h"
#include "../include/inline.h"
#include "../sources/teqn_sources.h"
```

Functions

- PetscErrorCode **FormT** (teqn_ ∗teqn, Vec &Rhs, PetscReal scale, PetscInt formMode=0)

  Viscous and divergence terms.
- PetscErrorCode **TeqnSNES** (teqn_ ∗teqn)

  Implicit SNES solver for T equation.
- PetscErrorCode **SNESFuncEvalT** (SNES snes, Vec T, Vec Rhs, void ∗ptr)

  SNES evaluation function.
- PetscErrorCode **FormExplicitRhsT** (teqn_ ∗teqn)

  Compute RHS of temperature equation using current ITmprt.
- PetscErrorCode **TeqnRK4** (teqn_ ∗teqn)

  Solve Teqn using RungeKutta 4.
- PetscErrorCode **TeqnABBE** (teqn_ ∗teqn)

  IMEX time step for T equation (AB2 convection + backward-Euler BDF1 diffusion)
- PetscErrorCode **ABBEMatVec** (Mat A, Vec v, Vec Av)

  MatShell operator for ABBE linear system: A∗v = v - dt∗D(v)

5.81.1 Detailed Description

Private temperature-equation solver function declarations.

5.82 teqn_solvers.h

Go to the documentation of this file.

```
00001 #include "../include/base.h"
00002 #include "../include/domain.h"
00003 #include "../include/io.h"
00004 #include "../include/inline.h"
00005 #include "../sources/teqn_sources.h"
00006
00009
00010 #ifndef TEQN_SOLVERS_H
00011 #define TEQN_SOLVERS_H
00012
00013
00015 PetscErrorCode FormT(teqn_ *teqn, Vec &Rhs, PetscReal scale, PetscInt formMode = 0);
00016
00018 PetscErrorCode TeqnSNES(teqn_ *teqn);
00019
00021 PetscErrorCode SNESFuncEvalT(SNES snes, Vec T, Vec Rhs, void *ptr);
00022
00024 PetscErrorCode FormExplicitRhsT(teqn_ *teqn);
00025
00027 PetscErrorCode TeqnRK4(teqn_ *teqn);
00028
00030 PetscErrorCode TeqnABBE(teqn_ *teqn);
00031
00033 PetscErrorCode ABBEMatVec(Mat A, Vec v, Vec Av);
00034
00035 #endif
```

5.83 /zfs60/nobackup/stipa/software/TOSCA/src/solvers/ueqn_solvers.c File Reference

Momentum-equation solver function implementations.

```
#include "ueqn_solvers.h"
```

Functions

- PetscErrorCode **FormU** (ueqn_ *ueqn, Vec &Rhs, PetscReal scale, PetscInt formMode)
- PetscErrorCode **UeqnSNES** (ueqn_ *ueqn)
- PetscErrorCode **SNESFuncEval** (SNES snes, Vec Ucont, Vec Rhs, void *ptr)
- PetscErrorCode **ExplicitRhsU** (ueqn_ *ueqn, PetscInt formMode)
- PetscErrorCode **UeqnFE** (ueqn_ *ueqn)
- PetscErrorCode **UeqnRK4** (ueqn_ *ueqn)
- PetscErrorCode **UeqnRK3CN_W** (ueqn_ *ueqn)
- PetscErrorCode **UeqnRK3CN_SO** (ueqn_ *ueqn)
- PetscErrorCode **UeqnABCN** (ueqn_ *ueqn)
- PetscErrorCode **IMEXMatVec** (Mat A, Vec v, Vec Av)
- PetscErrorCode **IMEXMatGetDiagonal** (Mat A, Vec diag)

5.83.1 Detailed Description

Momentum-equation solver function implementations.

5.84 /zfs60/nobackup/stipa/software/TOSCA/src/solvers/ueqn_solvers.h File Reference

Private momentum-equation solver function declarations.

```
#include "../include/base.h"
#include "../include/domain.h"
#include "../include/io.h"
#include "../include/inline.h"
#include "../include/inflow.h"
#include "../sources/ueqn_sources.h"
```

5.84.1 Detailed Description

Private momentum-equation solver function declarations.

5.85 ueqn_solvers.h

Go to the documentation of this file.

```
00001 #include "../include/base.h"
00002 #include "../include/domain.h"
00003 #include "../include/io.h"
00004 #include "../include/inline.h"
00005 #include "../include/inflow.h"
00006 #include "../sources/ueqn_sources.h"
00007
00010
00011 #ifndef UEQN_SOLVERS_H
00012 #define UEQN_SOLVERS_H
00013
00015 PetscErrorCode FormU(ueqn_ *ueqn, Vec &Rhs, PetscReal scale, PetscInt formMode = 0);
00016
00018 PetscErrorCode UeqnSNES(ueqn_ *ueqn);
00019
```

```
PetscErrorCode SNESFuncEval(SNES snes, Vec Ucont, Vec Rhs, void *ptr);

PetscErrorCode ExplicitRhsU(ueqn_ *ueqn, PetscInt formMode = 0);

PetscErrorCode UeqnFE(ueqn_ *ueqn);

PetscErrorCode UeqnRK4(ueqn_ *ueqn);

PetscErrorCode UeqnABCN(ueqn_ *ueqn);

PetscErrorCode UeqnRK3CN_SO(ueqn_ *ueqn);
PetscErrorCode UeqnRK3CN_W(ueqn_ *ueqn);

PetscErrorCode IMEXMatVec(Mat A, Vec v, Vec Av);

#endif
```

5.86 /zfs60/nobackup/stipa/software/TOSCA/src/sources/teqn_sources.c File Reference

Temperature-equation source-term function implementations.

```
#include "teqn_sources.h"
```

Functions

- PetscErrorCode **CorrectSourceTermsT** (teqn_ *teqn, PetscInt print)

  Compute temperature control source term.
- PetscErrorCode **correctDampingSourcesT** (teqn_ *teqn)

  Compute tBar state for lateral damping region.
- PetscErrorCode **dampingSourceT** (teqn_ *teqn, Vec &Rhs, PetscReal scale)

  Apply fringe region damping.
- PetscErrorCode **sourceT** (teqn_ *teqn, Vec &Rhs, PetscReal scale)

  Apply temperature control.
- PetscErrorCode **hyperViscosityT** (teqn_ *teqn, Vec &Rhs, PetscReal scale)

  Apply biharmonic hyperviscosity to the T equation RHS.

5.86.1 Detailed Description

Temperature-equation source-term function implementations.

5.87 /zfs60/nobackup/stipa/software/TOSCA/src/sources/teqn_sources.h File Reference

Private temperature-equation source-term function declarations.

```
#include "../include/base.h"
#include "../include/domain.h"
#include "../include/io.h"
#include "../include/inline.h"
```

Functions

- PetscErrorCode **CorrectSourceTermsT** (teqn_ *teqn, PetscInt print)

  Compute temperature control source term.
- PetscErrorCode **correctDampingSourcesT** (teqn_ *teqn)

  Compute tBar state for lateral damping region.
- PetscErrorCode **dampingSourceT** (teqn_ *teqn, Vec &Rhs, PetscReal scale)

  Apply fringe region damping.
- PetscErrorCode **sourceT** (teqn_ *teqn, Vec &Rhs, PetscReal scale)

  Apply temperature control.
- PetscErrorCode **hyperViscosityT** (teqn_ *teqn, Vec &Rhs, PetscReal scale)

  Apply biharmonic hyperviscosity to the T equation RHS.

5.87.1 Detailed Description

Private temperature-equation source-term function declarations.

5.88 teqn_sources.h

Go to the documentation of this file.

```
#include "../include/base.h"
#include "../include/domain.h"
#include "../include/io.h"
#include "../include/inline.h"


#ifndef TEQN_SOURCES_H
#define TEQN_SOURCES_H

PetscErrorCode CorrectSourceTermsT(teqn_ *teqn, PetscInt print);

PetscErrorCode correctDampingSourcesT(teqn_ *teqn);

PetscErrorCode dampingSourceT(teqn_ *teqn, Vec &Rhs, PetscReal scale);

PetscErrorCode sourceT(teqn_ *teqn, Vec &Rhs, PetscReal scale);

PetscErrorCode hyperViscosityT(teqn_ *teqn, Vec &Rhs, PetscReal scale);

#endif
```

5.89 /zfs60/nobackup/stipa/software/TOSCA/src/sources/ueqn_↩ sources.c File Reference

Momentum-equation source-term and auxiliary-flux function implementations.

```
#include "ueqn_sources.h"
```

Functions

- PetscErrorCode **CorrectSourceTerms** (ueqn_ *ueqn, PetscInt print)
- PetscErrorCode **mapYDamping** (ueqn_ *ueqn)
- PetscErrorCode **correctDampingSources** (ueqn_ *ueqn)
- PetscErrorCode **Buoyancy** (ueqn_ *ueqn, PetscReal scale)
- PetscErrorCode **sourceU** (ueqn_ *ueqn, Vec &Rhs, PetscReal scale)
- PetscErrorCode **dampingSourceU** (ueqn_ *ueqn, Vec &Rhs, PetscReal scale)
- PetscErrorCode **Coriolis** (ueqn_ *ueqn, Vec &Rhs, PetscReal scale)
- PetscErrorCode **CanopyForce** (ueqn_ *ueqn, Vec &Rhs, PetscReal scale)
- PetscErrorCode **meanGradPForcing** (ueqn_ *ueqn, Vec &Rhs, PetscReal scale)
- PetscErrorCode **hyperViscosityU** (ueqn_ *ueqn, Vec &Rhs, PetscReal scale)

5.89.1 Detailed Description

Momentum-equation source-term and auxiliary-flux function implementations.

5.90 /zfs60/nobackup/stipa/software/TOSCA/src/sources/ueqn_↩ sources.h File Reference

Private momentum-equation source-term function declarations.

```
#include "../include/base.h"
#include "../include/domain.h"
#include "../include/io.h"
#include "../include/inline.h"
#include "../include/inflow.h"
```

5.90.1 Detailed Description

Private momentum-equation source-term function declarations.

5.91 ueqn_sources.h

Go to the documentation of this file.

```
#include "../include/base.h"
#include "../include/domain.h"
#include "../include/io.h"
#include "../include/inline.h"
#include "../include/inflow.h"


#ifndef UEQN_SOURCES_H
#define UEQN_SOURCES_H

PetscErrorCode CorrectSourceTerms(ueqn_ *ueqn, PetscInt print);

PetscErrorCode mapYDamping(ueqn_ *ueqn);

PetscErrorCode correctDampingSources(ueqn_ *ueqn);

PetscErrorCode Buoyancy(ueqn_ *ueqn, PetscReal scale);

```

```
PetscErrorCode sourceU(ueqn_ *ueqn, Vec &Rhs, PetscReal scale);

PetscErrorCode dampingSourceU(ueqn_ *ueqn, Vec &Rhs, PetscReal scale);

PetscErrorCode Coriolis(ueqn_ *ueqn, Vec &Rhs, PetscReal scale);

PetscErrorCode CanopyForce(ueqn_ *ueqn, Vec &Rhs, PetscReal scale);

PetscErrorCode meanGradPForcing(ueqn_ *ueqn, Vec &Rhs, PetscReal scale);

PetscErrorCode hyperViscosityU(ueqn_ *ueqn, Vec &Rhs, PetscReal scale);


#endif
```

5.92 /zfs60/nobackup/stipa/software/TOSCA/src/teqn.c File Reference

Contains T equation function definitions.

```
#include "include/base.h"
#include "include/domain.h"
#include "include/io.h"
#include "include/inline.h"
#include "sources/teqn_sources.h"
#include "solvers/teqn_solvers.h"
```

Functions

- PetscErrorCode **SNESMonitorT** (SNES snes, PetscInt iter, PetscReal rnorm, void ∗comm)
- PetscErrorCode **InitializeTEqn** (teqn_ ∗teqn)

  Initializes Teqn environment.
- PetscErrorCode **SolveTEqn** (teqn_ ∗teqn)

  Solve T equation.
- PetscErrorCode **ghGradRhoK** (teqn_ ∗teqn)

  Computes g∗h times gradient of rho_k / rho_0.
- PetscErrorCode **TmprtPredictor** (teqn_ ∗teqn)

  Forward-Euler conv-only T predictor: T∗ = T^n + dt∗N(T^n) -> ITmprt (for buoyancy in SolveUEqn)
- PetscErrorCode **TmprtRestoreFromOld** (teqn_ ∗teqn)

  Restore Tmprt/ITmprt to T^n from Tmprt_o (undo TmprtPredictor before SolveTEqn)

5.92.1 Detailed Description

Contains T equation function definitions.

5.93 /zfs60/nobackup/stipa/software/TOSCA/src/turbines.c File Reference

Contains top to bottom level routines for the wind farm modeling.

```
#include "include/turbines.h"
#include "turbines/turbines_io.h"
#include "turbines/turbines_openfast.h"
#include "turbines/turbines_parallel.h"
#include "turbines/turbines_sampling.h"
#include "turbines/turbines_force.h"
#include "turbines/turbines_phys.h"
```

Functions

- PetscErrorCode **UpdateWindTurbines** (farm_ ∗farm)
- PetscErrorCode **InitializeWindFarm** (farm_ ∗farm)
- PetscErrorCode **computeMaxTipSpeed** (farm_ ∗farm)

5.93.1 Detailed Description

Contains top to bottom level routines for the wind farm modeling.

5.94 /zfs60/nobackup/stipa/software/TOSCA/src/turbines/turbines_force.h File Reference

Turbine force computation and projection routines.

```
#include "../include/base.h"
#include "../include/domain.h"
#include "../include/inline.h"
#include "../include/turbines.h"
```

Functions

- PetscErrorCode **computeBladeForce** (farm_ ∗farm)

  Compute aerodynamic forces at the turbine mesh points.
- PetscErrorCode **projectBladeForce** (farm_ ∗farm)

  Project the wind turbine forces on the background mesh.
- PetscErrorCode **computeTowerForce** (farm_ ∗farm)

  Compute the tower forces on the background mesh.
- PetscErrorCode **projectTowerForce** (farm_ ∗farm)

  Compute project the tower forces on the background mesh.
- PetscErrorCode **projectNacelleForce** (farm_ ∗farm)

  Compute and project the nacelle forces on the background mesh.
- PetscErrorCode **bodyForceCartesian2Contravariant** (farm_ ∗farm)

  Transform the cartesian body force to contravariant.

5.94.1 Detailed Description

Turbine force computation and projection routines.

5.95 turbines_force.h

Go to the documentation of this file.

```
#include "../include/base.h"
#include "../include/domain.h"
#include "../include/inline.h"
#include "../include/turbines.h"


#ifndef TRB_FORCE_H
#define TRB_FORCE_H

PetscErrorCode computeBladeForce(farm_ *farm);

PetscErrorCode projectBladeForce(farm_ *farm);

PetscErrorCode computeTowerForce(farm_ *farm);

PetscErrorCode projectTowerForce(farm_ *farm);

PetscErrorCode projectNacelleForce(farm_ *farm);

PetscErrorCode bodyForceCartesian2Contravariant(farm_ *farm);

#endif
```

5.96 /zfs60/nobackup/stipa/software/TOSCA/src/turbines/turbines_io.h File Reference

Turbine IO routines.

```
#include "../include/base.h"
#include "../include/domain.h"
#include "../include/inline.h"
#include "../include/turbines.h"
```

Functions

- PetscErrorCode **windTurbinesWrite** (farm_ ∗farm)

  Write output if applicable.
- PetscErrorCode **windTurbinesWriteCheckpoint** (farm_ ∗farm)

  Write checkpoint file.
- PetscErrorCode **windTurbinesReadCheckpoint** (farm_ ∗farm)

  Read checkpoint file and prepare wind turbines.
- PetscErrorCode **printFarmProperties** (farm_ ∗farm)

  Print wind farm information.
- PetscErrorCode **writeFarmADMesh** (farm_ ∗wf)

  Write the wind farm AD mesh to ucd file.
- PetscErrorCode **writeFarmALMesh** (farm_ ∗farm)

  Write the wind farm AL mesh to ucd file.
- PetscErrorCode **writeFarmTwrMesh** (farm_ ∗wf)

  Write the wind farm tower mesh to ucd file.
- PetscErrorCode **readActuatorModelParameters** (const word actuatorModel, windTurbine ∗wt, const word meshName)

  Read actuator model parameters and allocate memory.
- PetscErrorCode **readADM** (windTurbine ∗wt, const word meshName)

Initializes the ADM reading from files and allocating memory.

- PetscErrorCode **meshADM** (windTurbine ∗wt)

  Creates the ADM mesh.
- PetscErrorCode **readUADM** (windTurbine ∗wt, const word meshName)

  Initialize the UADM Model.
- PetscErrorCode **meshUADM** (windTurbine ∗wt)

  Creates the UADM mesh.
- PetscErrorCode **readALM** (windTurbine ∗wt, const word meshName)

  Initializes the ALM reading from files and allocating memory.
- PetscErrorCode **meshALM** (windTurbine ∗wt)

  Creates the ALM mesh.
- PetscErrorCode **readAFM** (windTurbine ∗wt, const word meshName)

  Initializes the AFM reading from files and allocating memory.
- PetscErrorCode **meshAFM** (windTurbine ∗wt)

  Creates the AFM mesh.
- PetscErrorCode **initSamplePoints** (windTurbine ∗wt, const word meshName)

  Initialize the upstream sample points data structure.
- PetscErrorCode **initTwrModel** (windTurbine ∗wt, Cmpnts &base)

  Initializes the tower model.
- PetscErrorCode **initNacModel** (windTurbine ∗wt, Cmpnts &base)

  Initializes the nacelle model.
- PetscErrorCode **readFarmProperties** (farm_ ∗farm)

  Read the farm_Properties file and fill the structs.
- PetscErrorCode **readTurbineArray** (farm_ ∗wf)

  Read the wind turbines inside the farm_Properties file.
- PetscErrorCode **readTurbineProperties** (windTurbine ∗wt, const char ∗dictName, const word meshName, const word modelName, Cmpnts &base)

  Fill the windTurbine struct reading in the file named as the wind turbine type.
- PetscErrorCode **readAirfoilProperties** (windTurbine ∗wt, const char ∗dictName)

  Reads the names of the airfoil used in the turbine (given in the airfoils subdict inside the file named as the wind turbine type)
- PetscErrorCode **readBladeProperties** (windTurbine ∗wt, const char ∗dictName, const PetscInt read↩Thickness)

  Reads the blades aero properties used in the turbine (given in the bladeData subdict inside the file named as the wind turbine type)
- PetscErrorCode **readCtTable** (windTurbine ∗wt, const char ∗dictName)

  Reads the turbine Ct curve used in the turbine (given in the CtTable subdict inside the file named as the wind turbine type)
- PetscErrorCode **readPitchTable** (windTurbine ∗wt, const char ∗dictName)

  Reads the turbine blade pitch curve used in the turbine (given in the pitchTable subdict inside the file /turbines/control/bladePitchCurve)
- PetscErrorCode **readRpmTable** (windTurbine ∗wt, const char ∗dictName)

  Reads the turbine rpm curve used in the turbine (given in the rpmTable subdict inside the file /turbines/control/rotor↩RpmCurve)
- PetscErrorCode **readTowerProperties** (windTurbine ∗wt, const char ∗dictName)

  Reads the tower properties used in the turbine (given in the towerData subdict inside the file named as the wind turbine)
- PetscErrorCode **readNacelleProperties** (windTurbine ∗wt, const char ∗dictName)

  Reads the nacelle properties used in the turbine (given in the nacelleData subdict inside the file named as the wind turbine)
- PetscErrorCode **readAirfoilTable** (foilInfo ∗af, const char ∗tableName)

  Reads the 2D airfoil tables given in the file named as the airfoil.

- PetscErrorCode **readGenControllerParameters** (windTurbine ∗wt, const char ∗dictName, const char ∗meshName)

  Reads generator torque controller parameters.
- PetscErrorCode **readPitchControllerParameters** (windTurbine ∗wt, const char ∗dictName, const char ∗meshName)

  Reads blade pitch controller parameters.
- PetscErrorCode **readYawControllerParameters** (windTurbine ∗wt, const char ∗dictName, const char ∗meshName)

  Reads nacelle yaw controller parameters.
- PetscErrorCode **readWindFarmControlTable** (windTurbine ∗wt)

  Read wind farm controller table (1 header and time - value list)
- PetscErrorCode **readDipcControllerParameters** (windTurbine ∗wt, const char ∗dictName, const char ∗meshName)

  Read discrete induction controller parameters.
- PetscErrorCode **checkTurbineMesh** (farm_ ∗farm)

  Check that the mesh is resolved around the turbine.

5.96.1 Detailed Description

Turbine IO routines.

5.97 turbines_io.h

Go to the documentation of this file.

```
#include "../include/base.h"
#include "../include/domain.h"
#include "../include/inline.h"
#include "../include/turbines.h"


#ifndef TRB_IO_H
#define TRB_IO_H

PetscErrorCode windTurbinesWrite(farm_ *farm);

PetscErrorCode windTurbinesWriteCheckpoint(farm_ *farm);

PetscErrorCode windTurbinesReadCheckpoint(farm_ *farm);

PetscErrorCode printFarmProperties(farm_ *farm);

PetscErrorCode writeFarmADMesh(farm_ *wf);

PetscErrorCode writeFarmALMesh(farm_ *farm);

PetscErrorCode writeFarmTwrMesh(farm_ *wf);

PetscErrorCode readActuatorModelParameters(const word actuatorModel, windTurbine *wt, const word meshName);

PetscErrorCode readADM(windTurbine *wt, const word meshName);

PetscErrorCode meshADM(windTurbine *wt);

PetscErrorCode readUADM(windTurbine *wt, const word meshName);

PetscErrorCode meshUADM(windTurbine *wt);

PetscErrorCode readALM(windTurbine *wt, const word meshName);

PetscErrorCode meshALM(windTurbine *wt);

PetscErrorCode readAFM(windTurbine *wt, const word meshName);

PetscErrorCode meshAFM(windTurbine *wt);

```

```
PetscErrorCode initSamplePoints(windTurbine *wt, const word meshName);

PetscErrorCode initTwrModel(windTurbine *wt, Cmpnts &base);

PetscErrorCode initNacModel(windTurbine *wt, Cmpnts &base);

PetscErrorCode readFarmProperties(farm_ *farm);

PetscErrorCode readTurbineArray(farm_ *wf);

PetscErrorCode readTurbineProperties(windTurbine *wt, const char *dictName, const word meshName, const
    word modelName, Cmpnts &base);

PetscErrorCode readAirfoilProperties(windTurbine *wt, const char *dictName);

PetscErrorCode readBladeProperties(windTurbine *wt, const char *dictName, const PetscInt
    readThickness);

PetscErrorCode readCtTable(windTurbine *wt, const char *dictName);

PetscErrorCode readPitchTable(windTurbine *wt, const char *dictName);

PetscErrorCode readRpmTable(windTurbine *wt, const char *dictName);

PetscErrorCode readTowerProperties(windTurbine *wt, const char *dictName);

PetscErrorCode readNacelleProperties(windTurbine *wt, const char *dictName);

PetscErrorCode readAirfoilTable(foilInfo *af, const char *tableName);

PetscErrorCode readGenControllerParameters(windTurbine *wt, const char *dictName, const char
    *meshName);

PetscErrorCode readPitchControllerParameters(windTurbine *wt, const char *dictName, const char
    *meshName);

PetscErrorCode readYawControllerParameters(windTurbine *wt, const char *dictName, const char
    *meshName);

PetscErrorCode readWindFarmControlTable(windTurbine *wt);

PetscErrorCode readDipcControllerParameters(windTurbine *wt, const char *dictName, const char
    *meshName);

PetscErrorCode checkTurbineMesh(farm_ *farm);

#endif
```

5.98 /zfs60/nobackup/stipa/software/TOSCA/src/turbines/turbines_↩ openfast.h File Reference

```
#include "../include/base.h"
#include "../include/domain.h"
#include "../include/inline.h"
#include "../include/turbines.h"
```

5.99 turbines_openfast.h

Go to the documentation of this file.

```
#include "../include/base.h"
#include "../include/domain.h"
#include "../include/inline.h"
#include "../include/turbines.h"


#ifndef TRB_OF_H
#define TRB_OF_H

#if USE_OPENFAST

```

```
#include "OpenFAST.H"

PetscErrorCode initOpenFAST(farm_ *farm);

PetscErrorCode stepZeroOpenFAST(farm_ *farm);

PetscErrorCode stepOpenFAST(farm_ *farm);

PetscErrorCode computeWindVectorsRotorOpenFAST(farm_ *farm);

PetscErrorCode computeWindVectorsTowerOpenFAST(farm_ *farm);

PetscErrorCode computeBladeForceOpenFAST(farm_ *farm);

PetscErrorCode computeTowerForceOpenFAST(farm_ *farm);

PetscErrorCode findControlledPointsRotorOpenFAST(farm_ *farm);

PetscErrorCode findControlledPointsTowerOpenFAST(farm_ *farm);

PetscErrorCode getDataFromOpenFAST(farm_ *farm);

PetscErrorCode getGlobParamsOpenFAST(windTurbine *wt, const word actuatorModel);

PetscErrorCode getVelPtsBladeOpenFAST(windTurbine *wt);

PetscErrorCode getForcePtsBladeOpenFAST(windTurbine *wt);

PetscErrorCode getVelPtsTwrOpenFAST(windTurbine *wt);

PetscErrorCode getForcePtsTwrOpenFAST(windTurbine *wt);

PetscErrorCode mapOFDisplToActPts(farm_ *farm);

PetscErrorCode mapOFForcesToActPts(farm_ *farm);

#endif

#endif
```

5.100 /zfs60/nobackup/stipa/software/TOSCA/src/turbines/turbines_parallel.h File Reference

Turbine parallel computation routines.

```
#include "../include/base.h"
#include "../include/domain.h"
#include "../include/inline.h"
#include "../include/turbines.h"
```

Functions

- PetscErrorCode **initControlledCells** (farm_ *farm)

  Initialize sphere cells and see what proc controls which turbine.
- PetscErrorCode **initSampleControlledCells** (farm_ *farm)

  Initialize sphere cells and see what proc controls which sample point.
- PetscErrorCode **findControlledPointsRotor** (farm_ *farm)

  Discrimination algorithm: find out which points of each rotor are controlled by this processor.
- PetscErrorCode **findControlledPointsTower** (farm_ *farm)

  Discrimination algorithm: find out which points of each tower are controlled by this processor.
- PetscErrorCode **findControlledPointsNacelle** (farm_ *farm)

  Discrimination algorithm: find out which points of each nacelle are controlled by this processor.
- PetscErrorCode **findControlledPointsSample** (farm_ *farm)

  Discrimination algorithm: find out which points of each sample rig are controlled by this processor.

- PetscErrorCode **checkPointDiscriminationRotor** (farm_ ∗farm)

  Debug check for the discrimination algorithm on the rotor.
- PetscErrorCode **checkPointDiscriminationTower** (farm_ ∗farm)

  Debug check for the discrimination algorithm on the tower.
- PetscErrorCode **checkPointDiscriminationNacelle** (farm_ ∗farm)

  Debug check for the discrimination algorithm on the nacelle.
- PetscErrorCode **checkPointDiscriminationSample** (farm_ ∗farm)

  Debug check for the discrimination algorithm on sample points.

5.100.1 Detailed Description

Turbine parallel computation routines.

5.101 turbines_parallel.h

Go to the documentation of this file.

```
#include "../include/base.h"
#include "../include/domain.h"
#include "../include/inline.h"
#include "../include/turbines.h"


#ifndef TRB_PAR_H
#define TRB_PAR_H

PetscErrorCode initControlledCells(farm_ *farm);

PetscErrorCode initSampleControlledCells(farm_ *farm);

PetscErrorCode findControlledPointsRotor(farm_ *farm);

PetscErrorCode findControlledPointsTower(farm_ *farm);

PetscErrorCode findControlledPointsNacelle(farm_ *farm);

PetscErrorCode findControlledPointsSample(farm_ *farm);

PetscErrorCode checkPointDiscriminationRotor(farm_ *farm);

PetscErrorCode checkPointDiscriminationTower(farm_ *farm);

PetscErrorCode checkPointDiscriminationNacelle(farm_ *farm);

PetscErrorCode checkPointDiscriminationSample(farm_ *farm);

#endif
```

5.102 /zfs60/nobackup/stipa/software/TOSCA/src/turbines/turbines_phys.h File Reference

Turbine physics module and control (substituted by OpenFAST if the latter is used)

```
#include "../include/base.h"
#include "../include/domain.h"
#include "../include/inline.h"
#include "../include/turbines.h"
```

Functions

- PetscErrorCode **computeRotSpeed** (farm_ ∗farm)

  Solve rotor dynamics and compute filtered rot speed.
- PetscErrorCode **rotateBlades** (windTurbine ∗wt, PetscReal angle, PetscInt updateAzimuth)

  Rotate blades (only for ALM)
- PetscErrorCode **controlGenSpeed** (farm_ ∗farm)

  Compute generator torque with 5-regions control system model.
- PetscErrorCode **controlBldPitch** (farm_ ∗farm)

  Compute blade pitch using the PID control.
- PetscErrorCode **controlNacYaw** (farm_ ∗farm)

  Compute nacelle yaw.
- PetscErrorCode **windFarmControl** (farm_ ∗farm)

  Compute wind turbine control based on wind farm controller.

5.102.1 Detailed Description

Turbine physics module and control (substituted by OpenFAST if the latter is used)

5.103 turbines_phys.h

Go to the documentation of this file.

```
#include "../include/base.h"
#include "../include/domain.h"
#include "../include/inline.h"
#include "../include/turbines.h"


#ifndef TRB_PHYS_H
#define TRB_PHYS_H

PetscErrorCode computeRotSpeed(farm_ *farm);

PetscErrorCode rotateBlades(windTurbine *wt, PetscReal angle, PetscInt updateAzimuth);

PetscErrorCode controlGenSpeed(farm_ *farm);

PetscErrorCode controlBldPitch(farm_ *farm);

PetscErrorCode controlNacYaw(farm_ *farm);

PetscErrorCode windFarmControl(farm_ *farm);

#endif
```

5.104 /zfs60/nobackup/stipa/software/TOSCA/src/turbines/turbines_↩ sampling.h File Reference

Turbine velocity sampling routines.

```
#include "../include/base.h"
#include "../include/domain.h"
#include "../include/inline.h"
#include "../include/turbines.h"
```

Functions

- PetscErrorCode **computeWindVectorsRotor** (farm_ *farm)

  Compute wind velocity at the rotor mesh points.
- PetscErrorCode **computeWindVectorsTower** (farm_ *farm)

  Compute wind velocity at the tower mesh points.
- PetscErrorCode **computeWindVectorsNacelle** (farm_ *farm)

  Compute wind velocity at the nacelle mesh points.
- PetscErrorCode **computeWindVectorsSample** (farm_ *farm)

  Compute wind velocity at the sample mesh points.

5.104.1 Detailed Description

Turbine velocity sampling routines.

5.105 turbines_sampling.h

Go to the documentation of this file.

```
00001 #include "../include/base.h"
00002 #include "../include/domain.h"
00003 #include "../include/inline.h"
00004 #include "../include/turbines.h"
00005
00008
00009 #ifndef TRB_SAMPLING_H
00010 #define TRB_SAMPLING_H
00011
00013 PetscErrorCode computeWindVectorsRotor(farm_ *farm);
00014
00016 PetscErrorCode computeWindVectorsTower(farm_ *farm);
00017
00019 PetscErrorCode computeWindVectorsNacelle(farm_ *farm);
00020
00022 PetscErrorCode computeWindVectorsSample(farm_ *farm);
00023
00024 #endif
```

5.106 /zfs60/nobackup/stipa/software/TOSCA/src/ueqn.c File Reference

Contains U equation function definitions.

```
#include "include/base.h"
#include "include/domain.h"
#include "include/io.h"
#include "include/inline.h"
#include "include/inflow.h"
#include "sources/ueqn_sources.h"
#include "solvers/ueqn_solvers.h"
```

Functions

- PetscErrorCode **SNESMonitorU** (SNES snes, PetscInt iter, PetscReal rnorm, void ∗comm)
- PetscErrorCode **InitializeUEqn** (ueqn_ ∗ueqn)

  Initializes Ueqn environment.
- PetscErrorCode **SolveUEqn** (ueqn_ ∗ueqn)

  Solve the momentum equation.
- PetscErrorCode **UpdateFluxLimiter** (ueqn_ ∗ueqn)

  Updates flux limiter.
- PetscErrorCode **contravariantToCartesian** (ueqn_ ∗ueqn)

  Transform velocity from contravariant to cartesian.
- PetscErrorCode **contravariantToCartesianGeneric** (mesh_ ∗mesh, Vec &lCont, Vec &lCat)

  Transform generic local vector from contravariant to cartesian.
- PetscErrorCode **adjustFluxesLocal** (ueqn_ ∗ueqn)

  Adjust fluxes to obey mass conservation (on a per-cell basis)

5.106.1 Detailed Description

Contains U equation function definitions.

Index

/zfs60/nobackup/stipa/software/TOSCA/src/abl.c, 73
/zfs60/nobackup/stipa/software/TOSCA/src/acquisition.c, 74
/zfs60/nobackup/stipa/software/TOSCA/src/boundary.c, 76
/zfs60/nobackup/stipa/software/TOSCA/src/clock.c, 77
/zfs60/nobackup/stipa/software/TOSCA/src/ibm.c, 78
/zfs60/nobackup/stipa/software/TOSCA/src/ibmInput.c, 80
/zfs60/nobackup/stipa/software/TOSCA/src/include/abl.h, 81, 82
/zfs60/nobackup/stipa/software/TOSCA/src/include/access.h, 86
/zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition.h, 86, 89
/zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/ablAcquisition.h, 91, 92
/zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/averageAcquisition.h, 93
/zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/keAcquisition.h, 94
/zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/perturbAcquisition.h, 96
/zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/probeAcquisition.h, 96, 97
/zfs60/nobackup/stipa/software/TOSCA/src/include/acquisition/sectionAcquisition.h, 97, 98
/zfs60/nobackup/stipa/software/TOSCA/src/include/base.h, 98, 100
/zfs60/nobackup/stipa/software/TOSCA/src/include/boundary.h, 101, 103
/zfs60/nobackup/stipa/software/TOSCA/src/include/catalystAdaptor.h, 105
/zfs60/nobackup/stipa/software/TOSCA/src/include/clock.h, 109, 110
/zfs60/nobackup/stipa/software/TOSCA/src/include/domain.h, 110, 111
/zfs60/nobackup/stipa/software/TOSCA/src/include/flags.h, 112
/zfs60/nobackup/stipa/software/TOSCA/src/include/ibm.h, 112, 115
/zfs60/nobackup/stipa/software/TOSCA/src/include/ibmInput.h, 120
/zfs60/nobackup/stipa/software/TOSCA/src/include/inflow.h, 120, 121
/zfs60/nobackup/stipa/software/TOSCA/src/include/initialField.h, 121, 123
/zfs60/nobackup/stipa/software/TOSCA/src/include/initialization.h, 123, 125
/zfs60/nobackup/stipa/software/TOSCA/src/include/inline.h, 125, 132
/zfs60/nobackup/stipa/software/TOSCA/src/include/io.h, 230, 233
/zfs60/nobackup/stipa/software/TOSCA/src/include/les.h, 234
/zfs60/nobackup/stipa/software/TOSCA/src/include/lesScalar.h, 235, 236
/zfs60/nobackup/stipa/software/TOSCA/src/include/mesh.h, 236, 237
/zfs60/nobackup/stipa/software/TOSCA/src/include/objects.h, 238
/zfs60/nobackup/stipa/software/TOSCA/src/include/overset.h, 239, 240
/zfs60/nobackup/stipa/software/TOSCA/src/include/peqn.h, 243, 244
/zfs60/nobackup/stipa/software/TOSCA/src/include/precursor.h, 246, 247
/zfs60/nobackup/stipa/software/TOSCA/src/include/teqn.h, 247, 248
/zfs60/nobackup/stipa/software/TOSCA/src/include/tosca2PV.h, 249, 253
/zfs60/nobackup/stipa/software/TOSCA/src/include/turbines.h, 258
/zfs60/nobackup/stipa/software/TOSCA/src/include/ueqn.h, 265
/zfs60/nobackup/stipa/software/TOSCA/src/include/wallfunctions.h, 267, 268
/zfs60/nobackup/stipa/software/TOSCA/src/include/wallmodel.h, 269
/zfs60/nobackup/stipa/software/TOSCA/src/initialField.c, 270
/zfs60/nobackup/stipa/software/TOSCA/src/initialization.c, 271
/zfs60/nobackup/stipa/software/TOSCA/src/io.c, 273
/zfs60/nobackup/stipa/software/TOSCA/src/les.c, 275
/zfs60/nobackup/stipa/software/TOSCA/src/mesh.c, 276
/zfs60/nobackup/stipa/software/TOSCA/src/overset.c, 277
/zfs60/nobackup/stipa/software/TOSCA/src/peqn.c, 278
/zfs60/nobackup/stipa/software/TOSCA/src/precursor.c, 281
/zfs60/nobackup/stipa/software/TOSCA/src/solvers/teqn_solvers.c, 282
/zfs60/nobackup/stipa/software/TOSCA/src/solvers/teqn_solvers.h, 282, 283
/zfs60/nobackup/stipa/software/TOSCA/src/solvers/ueqn_solvers.c, 283

/zfs60/nobackup/stipa/software/TOSCA/src/solvers/ueqn_solvers.h, 284
/zfs60/nobackup/stipa/software/TOSCA/src/sources/teqn_sources.c, 285
/zfs60/nobackup/stipa/software/TOSCA/src/sources/teqn_sources.h, 285, 286
/zfs60/nobackup/stipa/software/TOSCA/src/sources/ueqn_sources.c, 286
/zfs60/nobackup/stipa/software/TOSCA/src/sources/ueqn_sources.h, 287
/zfs60/nobackup/stipa/software/TOSCA/src/teqn.c, 288
/zfs60/nobackup/stipa/software/TOSCA/src/turbines.c, 288
/zfs60/nobackup/stipa/software/TOSCA/src/turbines/turbines_force.h, 289, 290
/zfs60/nobackup/stipa/software/TOSCA/src/turbines/turbines_io.h, 290, 292
/zfs60/nobackup/stipa/software/TOSCA/src/turbines/turbines_openfast.h, 293
/zfs60/nobackup/stipa/software/TOSCA/src/turbines/turbines_parallel.h, 294, 295
/zfs60/nobackup/stipa/software/TOSCA/src/turbines/turbines_phys.h, 295, 296
/zfs60/nobackup/stipa/software/TOSCA/src/turbines/turbines_sampling.h, 296, 297
/zfs60/nobackup/stipa/software/TOSCA/src/ueqn.c, 297

abl_, 11
access_, 18
Acell, 18
acquisition.c
read3LMFields, 76
acquisition.h
read3LMFields, 89
acquisition_, 19
avgFields, 20

BinIndex, 22
boundingBox, 22

Cabot, 23
cellIds, 23
cellList, 24
cellNode, 24
clock_, 24
Cmpnts, 25
computeEm
inline.h, 132
constants_, 26
Cpt2D, 26

data3LM, 26
dataABL, 28
Dcell, 29
domain_, 30

elementBox, 31

flags_, 31

getTimeList
io.c, 275
tosca2PV.h, 253

HoleObject, 32

ibm, 32
ibmFluidCell, 34
ibmInput.c
readIBMSurfaceFileAbaqusInp, 81
ibmMesh, 35
ibmObject, 35
ibmPitchMotion, 37
ibmRotation, 37
ibmSineMotion, 37
inflowData, 38
initialField.c
SetInitialField, 271
initialField.h
SetInitialField, 122
initialization.c
simulationInitialize, 272
initialization.h
SetInitialField, 124
simulationInitialize, 124
inletFunctions, 38
inletFunctionTypes, 39
inline.h
computeEm, 132
io.c
getTimeList, 275
io_, 41

keBox, 42
keFields, 44

les.c
UpdateCs, 276
les_, 45
lesScalar.h
UpdateCsk, 236
level3LM, 46
list, 47
LogLawAPG, 47

mapInfo, 47
matID
peqn.c, 280
Matrix3, 48
mesh_, 48
ModeData, 50

node, 51

OctreeNode, 51
overset_, 52
oversetMotion, 53

patchVectorField, 53
peqn.c

matID, 280
peqn_, 54
perturbFields, 55
postProcess, 56
PowerLawAPG, 57
precursor_, 57
probeRake, 58

rakes, 59
read3LMFields
acquisition.c, 76
acquisition.h, 89
readIBMSurfaceFileAbaqusInp
ibmInput.c, 81

scalarBC, 59
searchBox, 60
sections, 60
SetInitialField
initialField.c, 271
initialField.h, 122
initialization.h, 124
Shumann, 61
simInfo_, 62
simulationInitialize
initialization.c, 272
initialization.h, 124
skewSymmTensor, 63
std::hash< BinIndex >, 32
surface, 63
symmTensor, 63

Tensor, 64
TensorInvariants, 64
teqn_, 64
TOSCA, 1
tosca2PV.h
getTimeList, 253

udSections, 66
ueqn_, 67
UpdateCs
les.c, 276
UpdateCsk
lesScalar.h, 236
uSections, 69

vectorBC, 70

wallModel, 71